

Maestría en Ingeniería de Software y Sistemas
Informáticos

Plataforma de Desarrollo de Software

Plataformas de Desarrollo de Software

Tema 1. Introducción a las plataformas de desarrollo de software

Índice

Esquema

Ideas clave

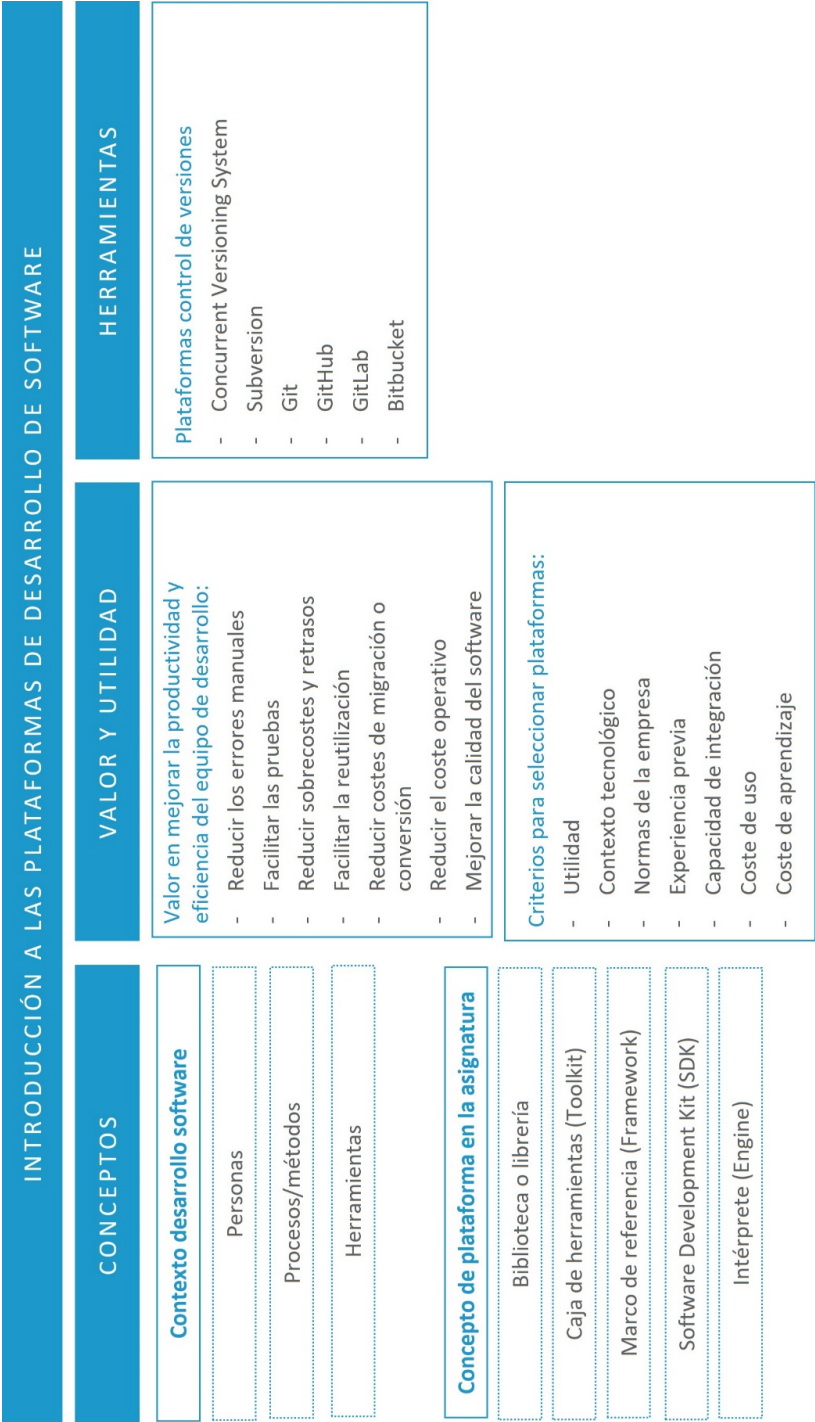
- 1.1. Introducción y objetivos
- 1.2. Contexto de las plataformas de desarrollo de software
- 1.3. Valor y utilidad de las plataformas de desarrollo de software
- 1.4. Plataformas para el control de versiones
- 1.5. Referencias bibliográficas

A fondo

Comenzando con Git y GitHub: una guía completa de control de versiones, gestión de proyectos y trabajo en equipo para el nuevo desarrollador

Git para equipos

Test



1.1. Introducción y objetivos

En este tema se introduce el concepto de plataforma de apoyo al desarrollo de software junto con el equipo y el proceso o metodología. En primer lugar, veremos la **terminología** y aclararemos el concepto de plataforma de desarrollo de software que usaremos en la asignatura frente a términos como librería, caja de herramienta, marco de referencia, kit de desarrollo o intérprete, entre otros.

Seguidamente, veremos que existen múltiples formas de **organizar y clasificar** las herramientas, ya sea por utilidad, base tecnológica, parte del proceso que apoya o capacidades de integración. Definiremos la clasificación que vamos a usar durante la asignatura y que motiva la descomposición en temas.

Posteriormente, veremos las principales **ventajas** del uso de plataformas de apoyo al desarrollo de software junto con algunos criterios para seleccionarlas. Finalmente, conoceremos las **principales herramientas** de apoyo al control de versiones.

Con el estudio de este tema pretendemos alcanzar los siguientes objetivos:

- ▶ Entender el **contexto** de las plataformas de apoyo al desarrollo de software frente a metodologías y organización de equipos y personas.
- ▶ Comprender el **valor y la utilidad** que tienen las plataformas de desarrollo de software.
- ▶ Identificar los **criterios principales** que nos ayuden a seleccionar las plataformas en cada caso.
- ▶ Conocer algunas plataformas de apoyo al **control de versiones**.

1.2. Contexto de las plataformas de desarrollo de software

El desarrollo de software es una tarea compleja que requiere la **interacción** entre múltiples equipos y personas para la construcción y evolución de soluciones que resuelven problemas con base en una correcta extracción de necesidades y traducción a requisitos técnicos. Para llevar a cabo la construcción del software, se requiere de:

- ▶ Un equipo de **personas** formadas y especializadas en roles y que trabajen en equipo.
- ▶ Un **método y procesos** que guíen en la construcción del software. Si partimos del trabajo realizado por Watts Humphrey en la década de los 90 (Humphrey, 1989), esta disciplina ha estado liderada por el Software Engineering Institute bajo el término «mejora de procesos de software» (*software process improvement*, SPI). Creó marcos metodológicos para organizaciones (*capability maturity model integration*, CMMI), equipos (*team software process*, TSP) (Humphrey, 2000b) o programadores (*personal software process*, PSP), entre otros (Humphrey, 2000a).
- ▶ Un conjunto de **herramientas** que facilitan y aceleran el desarrollo. Esta asignatura tiene como objetivo dotar al estudiante de una visión global y eminentemente práctica de herramientas, *toolkits* y plataformas que ayuden en todo el ciclo de vida del desarrollo de software. Por ello nos centraremos en este último punto de la construcción del software, pero siempre con la perspectiva de que son herramientas que facilitan y aceleran para dar soporte a una forma de abordar el desarrollo de software (metodología), que viene dirigida y ejecutada por un equipo (personas).

Antes de seleccionar las herramientas que pueden ayudar a desarrollar software, elige el proceso. Primero el proceso, después las

herramientas.

Una herramienta de desarrollo de software puede ser **cualquier programa** de software o utilidad que ayude a los desarrolladores o programadores de software a crear, editar, depurar, mantener y/o realizar alguna tarea específica de programación o desarrollo. Las herramientas se diseñaron inicialmente para extender o complementar los lenguajes de programación al proporcionar la funcionalidad y las características que estos lenguajes no tenían. Por lo general, son utilidades independientes que **brindan o respaldan** una tarea particular dentro de cualquier fase del ciclo de desarrollo/programación.

En la actualidad existen múltiples conceptos y términos asociados a herramientas que pueden ayudar al desarrollo de software, como:

- ▶ **Biblioteca o librería:** es una colección de código relacionado con una tarea específica o un conjunto de tareas estrechamente relacionadas que operan aproximadamente al mismo nivel de abstracción. Por lo general, carece de cualquier propósito o intención propia y está destinada a ser utilizada e integrada con el código del cliente para ayudar a ejecutar sus tareas.
- ▶ **Caja de herramientas (*toolkit*):** históricamente, un kit de herramientas es una biblioteca más enfocada, con un propósito definido y específico. Actualmente, este término ha caído en desuso y se usa casi exclusivamente para *widgets* gráficos y componentes de interfaz gráfica de usuario (GUI). La mayoría de las veces, un conjunto de herramientas operará en una capa más alta de abstracción que una biblioteca y, a menudo, consumirá y usará las bibliotecas por sí mismo. A diferencia de las bibliotecas, el código del kit de herramientas se suele usar para ejecutar la tarea del código del cliente, como construir una ventana, cambiar su tamaño, etc.
- ▶ **Marco de referencia (*framework*):** históricamente, un marco era un conjunto de bibliotecas y módulos interrelacionados que se separaban en categorías generales o específicas. Los marcos generales estaban destinados a ofrecer una plataforma

completa e integrada para crear aplicaciones al ofrecer funcionalidad general, como gestión de memoria multiplataforma, abstracciones de subprocesos múltiples, estructuras dinámicas y estructuras genéricas en general. Los marcos específicos se desarrollaron para tareas únicas como los sistemas de comando y control para sistemas industriales y las primeras pilas de redes y operaron a un alto nivel de abstracción. Se usaron juegos de herramientas similares para llevar a cabo la ejecución de las tareas de códigos de cliente. Actualmente, la definición de un marco se ha enfocado más y adopta el principio de **inversión de control**, por lo que el marco lleva a cabo el flujo del programa, así como la ejecución.

- ▶ **Kit de desarrollo de software (*software development kit*, SDK):** es una colección de herramientas para ayudar al programador a crear e implementar código y/o contenido que está específicamente diseñado para ejecutarse en una plataforma muy particular o de una manera muy particular. Un SDK puede consistir simplemente en un conjunto de bibliotecas que deben ser utilizadas de una manera específica solo por el código del cliente y que pueden compilarse normalmente, hasta un conjunto de herramientas binarias que crean o adaptan activos binarios para producir su aplicación en producción.
- ▶ **Intérprete (*engine*):** un motor (en términos de interpretación de código) es un binario que ejecutará contenido personalizado o procesará datos de entrada de alguna manera. Los motores de juegos y gráficos son quizás los usuarios más frecuentes de este término y se utilizan casi universalmente con un SDK para apuntar al propio intérprete, como el *unreal development kit* (UDK), pero también existen otros motores, como los motores de búsqueda y los motores de gestión de bases de datos.

- ▶ **Plataforma:** el término está más orientado a las infraestructuras físicas (máquinas o *cloud*) o lógicas (sistemas operativos o entornos virtuales) sobre las que se ejecutan las aplicaciones.

En el contexto de la asignatura, hablaremos de plataformas de desarrollo de software para hacer referencia a cualquier solución informática que acelera las labores de un equipo de desarrollo de software en cualquiera de las fases del ciclo de vida.

Las **clasificaciones** de las herramientas de software nos ayudan a comprender los tipos de herramientas de software y su papel en el soporte de las actividades del proceso de software. Hay varias formas de clasificar las herramientas de software, cada una de las cuales nos brinda una perspectiva diferente sobre estas.

- ▶ Una perspectiva **funcional** donde las herramientas se clasifican según su función específica.
- ▶ Una perspectiva de **proceso** donde las herramientas se clasifican de acuerdo con las actividades de proceso que soportan.
- ▶ Una perspectiva de **integración** en la que las herramientas se clasifican según cómo se organizan en unidades integradas que brindan soporte para una o más actividades de proceso.

Por ejemplo, Sommerville (2015) clasifica las soluciones por su perspectiva funcional de esta forma:

Clasificación de herramientas de desarrollo de software	
Herramientas de planificación	Herramientas de técnicas de revisión y evaluación de programas (PERT), herramientas de estimación, hojas de cálculo
Herramientas de edición	Editores de texto, editores de diagramas, procesadores de texto
Herramientas de gestión de cambios	Herramientas de trazabilidad de requisitos, sistemas de control de cambios
Herramientas de gestión de configuración	Sistemas de gestión de versiones, herramientas de creación de sistemas
Herramientas de creación de prototipos	Lenguajes de muy alto nivel, generadores de interfaz de usuario
Herramientas de soporte de métodos	Editores de diseño, diccionarios de datos, generadores de código
Herramientas de procesamiento del lenguaje	Compiladores, intérpretes
Herramientas de análisis de programas	Generadores de referencias cruzadas, analizadores estáticos, analizadores dinámicos
Herramientas de prueba	Generadores de datos de prueba, comparadores de archivos
Herramientas de depuración	Sistemas de depuración interactivos
Herramientas de documentación	Programas de maquetación de páginas, editores de imágenes
Herramientas de reingeniería	Sistemas de referencia cruzada, sistemas de reestructuración de programas

Tabla 1. Clasificación funcional de herramientas. Fuente: basado en Sommerville, 2015.

El cuerpo de conocimiento de ingeniería de software (*software engineering body of knowledge*, SWEBOK) del Instituto de Ingenieros Eléctricos y Electrónicos (IEEE) define esta clasificación basada en tipos y subtipos de herramientas (Roongkaew, 2013):

Clasificación de herramientas de desarrollo de software según SWEBOK	
Herramientas de requisitos software	▶ Modelado de requisitos ▶ Trazabilidad de requisitos
Herramientas de diseño	▶ Diseño de arquitectura o componentes
Herramientas de construcción de software	▶ Editores de programas ▶ Compiladores y generadores de código ▶ Intérpretes ▶ Depuradores
Herramientas de prueba de software	▶ Pruebas de software ▶ Generadores de pruebas ▶ Marcos de ejecución de pruebas ▶ Evaluación de las pruebas ▶ Gestión de pruebas ▶ Análisis de rendimiento
Herramientas de mantenimiento de software	▶ Comprensión ▶ Reingeniería
Herramientas de gestión de configuración de software	▶ Defectos, mejoras, problemas, seguimiento de problemas ▶ Gestión de versiones ▶ Liberar y compilar
Herramientas de gestión de ingeniería de software	▶ Planificación y seguimiento de proyectos ▶ Gestión de riesgos ▶ Medición
Herramientas de proceso de ingeniería de software	▶ Modelado de procesos ▶ Gestión de procesos ▶ Entornos de ingeniería de software asistida por computadora (CASE) integrados ▶ Software centrado en procesos
Herramientas de calidad de software	▶ Revisión y auditoría ▶ Análisis estático
Otras herramientas	

Tabla 2. Clasificación funcional de herramientas. Fuente: basado en Roongkaew, 2013.

Para estructurar las diversas plataformas de apoyo al desarrollo de software, hemos realizado una clasificación propia que combina, en ocasiones, una vista funcional, en otras una perspectiva tecnológica o de arquitectura del sistema y, por último, una vista metodológica o de proceso.

De esta forma, la clasificación de plataformas de desarrollo de software que se ha

seguido para la asignatura se estructura en base a las siguientes categorías, estructuradas en diversos temas:

Clasificación de plataformas de desarrollo de software
Plataformas para requisitos
Plataformas de ingeniería de software
Plataformas low-code y no-code
Plataformas para el desarrollo en Java
Plataformas para el desarrollo en .NET
Plataformas para el desarrollo de servicios
Plataformas para el desarrollo móvil
Plataformas para pruebas
Plataformas para el despliegue de sistemas distribuidos

Tabla 3. Clasificación de plataformas de software en la asignatura. Fuente: elaboración propia.

En el contexto de la asignatura, las plataformas de desarrollo de software que analicemos serán lo más independientes posibles al proceso o metodología seguido para la construcción. Perseguiremos el mismo objetivo a nivel tecnológico y de arquitectura de la solución, aunque aquellas específicas de Java, .NET, móvil o arquitecturas de sistemas distribuidos basadas en servicio tendrán un contexto tecnológico concreto y acotado.

1.3. Valor y utilidad de las plataformas de desarrollo de software

Las herramientas de apoyo al desarrollo de software tienen por objetivo mejorar la **productividad y eficiencia** del equipo de desarrollo, automatizando la ejecución de procesos dentro del ciclo de vida de la construcción del software, simplificando la comunicación y reduciendo errores manuales. En general, alguno de los principales beneficios del uso de plataformas de apoyo al desarrollo de software son:

- ▶ **Reducir los errores manuales:** la programación estructurada y apoyada en herramientas produce programas que son más fáciles de probar y, una vez probados, más sencillos de mantener.
- ▶ **Facilitar las pruebas:** con herramientas apropiadas, se simplifica el trabajo de validar y verificar un desarrollo antes de su salida a producción.
- ▶ **Reducir sobrecostos y retrasos:** las técnicas de diseño y desarrollo actuales pueden hacer que el proceso de desarrollo sea más fácil de controlar, reduciendo riesgos tecnológicos y facilitando la toma de decisiones durante el seguimiento del proyecto.
- ▶ **Facilitar la reutilización:** mediante las técnicas y soluciones existentes, se maximiza y facilita que las organizaciones reutilicen el software existente.
- ▶ **Reducir costes de migración o conversión:** una necesidad frecuente es migrar tecnológicamente sistemas legados. Con las herramientas apropiadas, se puede reducir significativamente el esfuerzo y coste asociado para evolucionar un software existente a otro entorno o lenguaje.

- ▶ **Reducir el coste operativo:** con plataformas de software, podemos optimizar nuestro sistema para reducir la utilización de los recursos de la máquina necesarios.
- ▶ **Mejorar la calidad del software:** las herramientas nos pueden ayudar a escribir mejor código, detectar antes los errores y facilitar su futuro mantenimiento.

Aunque las plataformas de apoyo al desarrollo de software pueden suponer una mejora en la eficiencia del desarrollo, entender los requisitos, plantear una solución técnica, diseñar la arquitectura y definir cómo probar son tareas clave que hay que realizar con esfuerzo cognitivo y sin apoyo de herramientas.

A la hora de seleccionar las posibles plataformas de apoyo al desarrollo de software en una organización, existe una **serie de factores** a considerar. No todas las herramientas son necesariamente apropiadas para un proyecto determinado, mientras que otras pueden ser críticas. Cada equipo debe considerar una serie de factores al momento de decidir sobre las herramientas que se pueden utilizar en el desarrollo de software de cada proyecto. Alguno de los factores a considerar son:

- ▶ **Utilidad:** es el factor principal al decidir si usar un tipo de herramienta y qué implementación de esa herramienta es la utilidad que proporcionará para la finalización general del proyecto.
- ▶ **Aplicabilidad a nuestro contexto tecnológico:** no todas las herramientas se aplican a todos los entornos o contextos tecnológicos. Por ejemplo, existen soluciones más adecuadas en función de si vamos a crear una solución de escritorio, web, móvil o *serverless* en la nube.
- ▶ **Normas de la empresa:** en organizaciones más grandes y a menudo también en las más pequeñas, el uso de ciertas herramientas será obligatorio para lograr objetivos o cumplir con la política establecida. La estandarización de las herramientas puede

ayudar a una organización a mover a los desarrolladores fácilmente entre proyectos según sea necesario y brinda a la gerencia la seguridad de que se siguen procesos similares entre diferentes proyectos y equipos de proyectos, lo que da como resultado una calidad de producto homogénea.

- ▶ **Experiencia previa del equipo** con la herramienta: hasta cierto punto, casi todo el software tiene una curva de aprendizaje. La selección de herramientas específicas puede verse influenciada por el nivel de experiencia que los desarrolladores ya tengan con ellas. Esa experiencia específica también se puede utilizar como recurso para decidir si una herramienta puede ser útil en el proyecto o no, ya que los desarrolladores suelen tener opiniones sólidas sobre estos asuntos y no suelen tener vergüenza de expresarlas.
- ▶ **Capacidad de integración:** la integración de una herramienta con otras herramientas puede tener un gran impacto en el valor que agrega al equipo y al proyecto. Algunas integraciones toman la forma de integración de «conveniencia» — es decir, la utilidad de control de código fuente se integra con el IDE de modo que un desarrollador hace que un archivo se desproteja automáticamente una vez que comienza a editarlo—. Otra integración más profunda fusiona información y responde a eventos entre herramientas para brindar un alto valor al equipo y a otros grupos dentro de la organización. Un ejemplo de esta integración más profunda es cuando el control de código fuente se integra con el seguimiento de errores, que a su vez se integra con el seguimiento de incidentes (o problemas del cliente). Un cambio de código registrado en el control de fuente puede indicar al rastreador de errores que hay una solución disponible para un error informado. Ese error puede estar asociado con un informe de incidente, que luego se puede actualizar con el estado del trabajo que se ha realizado que podría, eventualmente, resolver ese incidente.
- ▶ **Coste de uso:** existen herramientas comerciales y una gran base de herramientas *open source* y de uso gratuito, que debemos considerar en función del contexto, de la utilidad y del presupuesto disponible.

- ▶ **Coste de aprendizaje:** todo software tiene una curva de aprendizaje, en un grado u otro. Las herramientas complejas también pueden requerir tiempo y esfuerzo para implementarlas con el equipo e integrarlas en el software de desarrollo existente. Además de la implementación inicial y la curva de aprendizaje, muchas herramientas requieren cierto tiempo y esfuerzo para usarlas. Esta sobrecarga debe tenerse en cuenta al evaluar el valor general de la herramienta.

1.4. Plataformas para el control de versiones

Vamos a presentar algunas plataformas de apoyo al control de versiones, por ser una **práctica común y general** de aplicación a todo tipo de desarrollo de software.

En ingeniería de software, el **control de versiones** (también conocido como control de revisión, control de código fuente o gestión de código fuente) es una práctica, integrada en la gestión de la configuración del software, responsable de gestionar cambios en el código de programas informáticos, su documentación o cualquier colección de información relevante para su desarrollo o evolución.

Los cambios generalmente se identifican mediante un **código de número o letra** denominado número de revisión, nivel de revisión o simplemente revisión. Por ejemplo, un conjunto inicial de archivos es «revisión 1». Cuando se realiza el primer cambio, el conjunto resultante es «revisión 2» y así sucesivamente. Cada revisión está asociada con una marca de tiempo y la persona que realiza el cambio. Las revisiones se pueden comparar, restaurar y, con algunos tipos de archivos, fusionar.

La necesidad de una **forma lógica de organizar y controlar** las revisiones ha existido casi desde que existe la escritura, pero el control de revisiones se volvió mucho más importante y complicado cuando comenzó la era de la informática. La numeración de ediciones de libros y de revisiones de especificaciones son ejemplos que se remontan únicamente a la era de la impresión. Hoy en día, los sistemas de control de revisión más capaces (así como complejos) son los que se utilizan en el desarrollo de software, en los que un equipo de personas puede realizar cambios en los mismos archivos al mismo tiempo. Los sistemas de control de versiones suelen ejecutarse como aplicaciones independientes, pero el control de revisiones también está integrado en varios tipos de software, como procesadores de texto y hojas de cálculo, documentos web colaborativos y sistemas de gestión de contenido.

Los principales propósitos y valores del control de versiones son los siguientes:

- ▶ Varias personas pueden **trabajar simultáneamente** en un solo proyecto. Cada uno trabaja y edita su propia copia de los archivos y depende de ellos compartir los cambios realizados por ellos con el resto del equipo.
- ▶ También permite que una persona use **varias computadoras** para trabajar en un proyecto, por lo que es valioso incluso si está trabajando sola.
- ▶ **Integra el trabajo** que se realiza simultáneamente por diferentes miembros del equipo. En algunos casos raros, cuando dos personas realizan ediciones conflictivas en la misma línea de un archivo, el sistema de control de versiones solicita la asistencia humana para decidir qué se debe hacer.
- ▶ Proporciona acceso a las **versiones históricas** de un proyecto. Este es un seguro contra fallas en la computadora o pérdida de datos. Si se comete algún error, puede retroceder fácilmente a una versión anterior.
- ▶ También es posible **deshacer ediciones** específicas sin perder el trabajo realizado mientras tanto. Se puede saber fácilmente cuándo, por qué y quién editó cualquier parte de un archivo.
- ▶ Adicionalmente, simplifica la **trazabilidad** del código fuente respecto a otra documentación del proyecto, como requisitos, incidencias, diseños, etc.

Estas son las consideraciones básicas para tener en cuenta antes de usar o comprar un sistema de control de versiones:

- ▶ **Tipología:** existen repositorios centralizados o distribuidos.
- ▶ **Comercial contra abierto:** evalúe su proyecto, ya sea que utilice un control de versiones comercial o de código abierto.
- ▶ **Tamaño del equipo:** compruebe el número máximo de usuarios por cuenta o repositorio.

- ▶ **Tamaño del almacenamiento:** determine el espacio de almacenamiento disponible.
- ▶ **Múltiples proyectos:** evalúe la cantidad de proyectos permitidos por cuenta.
- ▶ **Funciones:** verifique la disponibilidad de herramientas adicionales, como rastreadores de errores, seguimiento de tiempo, documentación, etc.

En el ámbito del control de versiones, la **bifurcación** (*branching*) es la duplicación de un objeto bajo el control de versiones —como un archivo de código fuente o un árbol de directorios—. A partir de entonces, cada objeto se puede modificar por separado y en paralelo para que los objetos se vuelvan diferentes. En este contexto, los objetos se denominan **ramas**. Los usuarios del sistema de control de versiones pueden ramificar cualquier rama.

Las ramas también se conocen como árboles, arroyos o líneas de código. La rama de origen a veces se la llama rama principal, rama ascendente (o simplemente ascendente, especialmente si las sucursales son mantenidas por diferentes organizaciones o individuos) o corriente de respaldo. Las ramas secundarias son ramas que tienen un padre: una rama sin un padre se conoce como troncal o línea principal.

Las principales plataformas para el control de versiones son:

Plataformas para el control de versiones
Concurrent Versioning System
Subversion
Git
GitHub
GitLab
Bitbucket

Tabla 4. Plataformas para el control de versiones. Fuente: elaboración propia.

Concurrent Versions System (CVS)

Es un sistema de control de versiones, un componente importante de la gestión de configuración de software (*source configuration management*, SCM). Utilizándolo, puede **registrar el historial** de fuentes, archivos y documentos. Cumple una función similar a los paquetes de software libre RCS, PRCS y Aegis. Fue uno de los primeros sistemas de control de versiones más ampliamente utilizado, especialmente en muchos proyectos de software libre (CVS, 2022).

Si bien CVS almacena el historial de archivos individuales en el mismo formato que RCS, ofrece las siguientes **ventajas significativas** por sobre este último:

- Puede ejecutar **secuencias de comandos** para registrar operaciones de CVS o aplicar políticas específicas del sitio.

- ▶ El cliente/servidor CVS permite que los desarrolladores dispersos por geografía o módems lentos funcionen como un solo equipo. El historial de versiones se almacena en **un único servidor central** y las máquinas cliente tienen una copia de todos los archivos en los que están trabajando los desarrolladores. Por lo tanto, la red entre el cliente y el servidor debe estar activa para realizar operaciones CVS (como registros o actualizaciones), pero no para editar o manipular las versiones actuales de los archivos. Los clientes pueden realizar todas las mismas operaciones que están disponibles localmente.
- ▶ En los casos en que varios desarrolladores o equipos quieran mantener cada uno su propia versión de los archivos, debido a la geografía y/o la política, las sucursales de proveedores de CVS pueden **importar** una versión de otro equipo (incluso si no usan CVS) y luego CVS pueden fusionar los cambios de la rama del proveedor con los archivos más recientes si eso es lo que se desea.
- ▶ Permite **cambios concurrentes**, por lo que más de un desarrollador puede trabajar en los mismos archivos al mismo tiempo.
- ▶ CVS provee una base de datos de **módulos flexibles** que proporciona un mapeo simbólico de nombres a componentes de una distribución de software más grande. Aplica nombres a colecciones de directorios y archivos. Un solo comando puede manipular toda la colección.
- ▶ Los servidores CVS se ejecutan en la **mayoría** de las variantes de Unix y también hay disponibles clientes para Windows NT/95, OS/2 y VMS. CVS también funcionará en lo que a veces se denomina modo de servidor en repositorios locales en Windows 95/NT.

Apache Subversion

También conocido como **Subversion** o **SVN**, es un sistema de control de versiones de código abierto bajo la licencia de Apache, que nació como evolución de CVS para mejorar algunas de sus funcionalidades (Blokdyk, 2018).

CVS solo rastrea la modificación archivo por archivo, mientras que SVN rastrea una **confirmación completa** como una nueva revisión, lo que significa que es más fácil seguir el historial de su proyecto. A su vez, todo el software de control de fuente moderno utiliza el concepto de revisión, por lo que es mucho más fácil migrar desde SVN que desde CVS.

También está el problema del **compromiso atómico**. Es posible que dos personas que trabajen juntas en CVS puedan entrar en conflicto entre sí y se pierdan algunos datos, lo que pondría a su cliente en un estado inconsistente. Cuando se detectan a tiempo, estos problemas no son importantes porque sus datos todavía están en algún lugar, pero pueden ser una molestia en un entorno estresante.

Entre las **características clave** de SVN, se incluyen: gestión de inventario, gestión de seguridad, seguimiento de historial, controles de acceso de usuarios, recuperación de datos y gestión de flujo de trabajo. SVN es fácil de implementar con cualquier lenguaje de programación. Además, ofrece almacenamiento consistente para manejar archivos binarios y de texto.

Git

Es la base de los controladores de versiones actuales. La principal diferencia entre Git y cualquier otro anterior es la forma en la que Git estructura sus datos. Conceptualmente, la mayoría de los otros sistemas almacenan información como una lista de cambios basados en archivos (Chacon y Straub, 2014). Estos otros sistemas (CVS, Subversion, Perforce, Bazaar, etc.) estructuran la información que almacenan como un conjunto de archivos y los cambios realizados en cada archivo a lo largo del tiempo (esto se describe comúnmente como control de versión basado en delta).

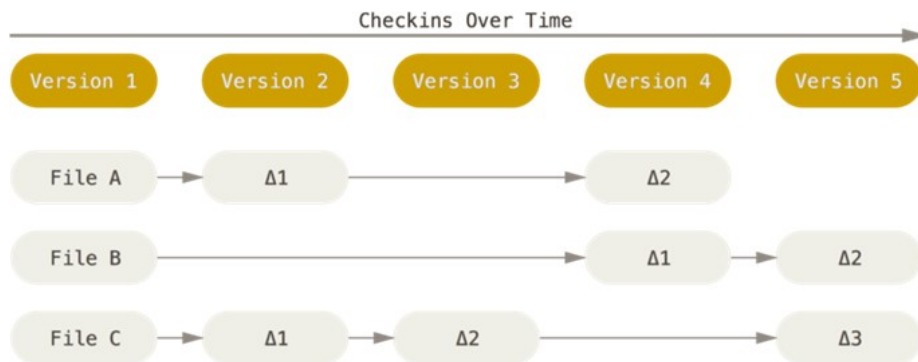


Figura 1. Modelo de gestión de cambios por deltas. Fuente: Chacon y Straub, 2014.

Git no estructura ni almacena sus datos de esta manera, sino que lo hace como una **serie de instantáneas** de un sistema de archivos en miniatura. Con Git, cada vez que confirma o guarda el estado de su proyecto, básicamente se toma una imagen de cómo se ven todos sus archivos en ese momento y almacena una referencia a esa instantánea. Para ser eficiente, si los archivos no han cambiado, Git no vuelve a almacenar el archivo, solo un enlace al archivo idéntico anterior que ya ha almacenado.

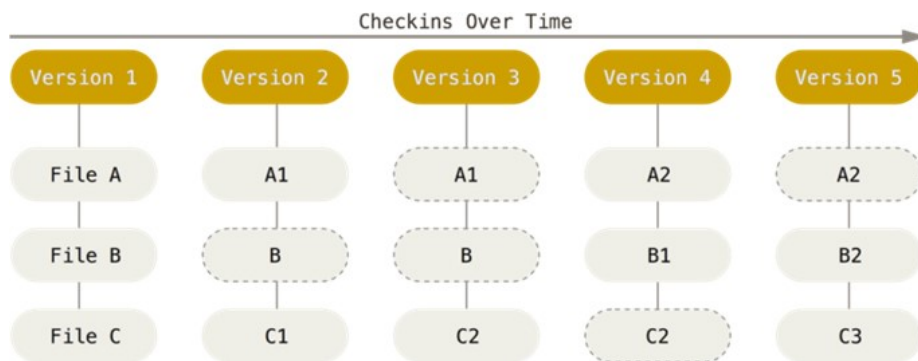


Figura 2. Modelo de gestión de cambios por instantáneas. Fuente: Chacon y Straub, 2014.

Git tiene tres estados principales en los que pueden residir sus archivos: modificado, preparado y confirmado:

- ▶ **Modificado (*modified*):** significa que ha cambiado el archivo pero aún no lo ha confirmado en su base de datos.
- ▶ **Preparado (*staged*):** significa que ha marcado un archivo modificado en su versión actual para pasar a su próxima instantánea de confirmación.
- ▶ **Comprometido (*committed*):** significa que los datos se almacenan de forma segura en su base de datos local.

El **directorio** de Git es donde almacena los metadatos y la base de datos de objetos para su proyecto. Esta es la parte más importante de Git y es lo que se copia cuando se **clona** un repositorio desde otra computadora. El flujo de trabajo básico de Git es algo así:

- ▶ Modificas archivos en tu árbol de trabajo.
- ▶ Organizas selectivamente solo aquellos cambios que desea que sean parte de su próxima confirmación, que agrega solo esos cambios al área de preparación.
- ▶ Realizas una confirmación, que toma los archivos tal como están en el área de preparación y almacena esa instantánea de forma permanente en su directorio de Git.

GitHub

Es una plataforma como servicio en la nube para el control de versiones de nuestros proyectos basada en Git. Ha sido una de las referencias para proyectos *open source*, con millones de proyectos accesibles a través de sus repositorios. Adquirida por Microsoft en 2018, en la actualidad tiene versiones gratuitas para equipos limitados y soluciones *open source* y versiones de pago para empresas y equipos grandes (GitHub, s. f.).

Algunas de sus características son:

- ▶ **Codificación colaborativa:** contribuya a los proyectos rápidamente con la configuración automática del entorno. Asegúrese de ver los cambios que le interesan y cree una comunidad en torno a su código.
- ▶ **Automatización y CI/CD:** automatice la integración y distribución continuas (CI/CD), pruebas, planificación, gestión de proyectos, etiquetado de problemas, aprobaciones, incorporación y más.
- ▶ **Seguridad:** asegure el código mientras lo escribe. Revise automáticamente cada cambio en su base de código e identifique vulnerabilidades antes de que lleguen a producción.
- ▶ **Múltiples aplicaciones cliente:** integre GitHub en cualquier dispositivo conectado. Omita la interfaz de usuario visual con una interfaz de texto rápida y potente. Acceda a GitHub desde su escritorio macOS o Windows.
- ▶ **Administración del proyecto:** gestione las solicitudes de funciones y los errores. Coordine iniciativas grandes y pequeñas con tablas de proyectos, tableros y listas de tareas. Realice un seguimiento de lo que entrega hasta el despliegue.
- ▶ **Administración del equipo:** simplifique la gestión de accesos y permisos en sus proyectos y equipos. Actualice los permisos, agregue nuevos usuarios a medida que crece y otorgue a todos los permisos exactos que necesitan. Sincroniza con Okta y Azure Active Directory.

OctoArcade Invaders

The Plan | Game loop Backlog | Standup | + New view

Title	Team	Status	Assignees	Milestones
Prototype 3				
1 Game brief and go-no-go	Producers	Complete	preciselyalys	Prot
2 Engine prototype (physics, rendering)	Engine	Complete	marlorod and pm	Prot
3 Initial concept art	Art	Complete	pmarscell	Prot
Beta 5				
4 Integrate with Leaderboard Service	Game Loop	Not Started	chiedo	Beta
5 Creative design update to aliens for variety	Art	Planning	ajashams	Beta
6 Updates to alien, beam, and cannon sprites	Art	Building	mikwing	Beta
7 Update to collision logic	Engine	Building	mdo	Beta
8 Improve alien respawn rate	Game Loop	Behind	mattjohnlee	Beta
Launch 6				
9 Interviews with media outlets	Producers	Not Started	marlorod	Launch
10 Save score across levels	Game Loop	Not Started	pmarscell	Launch

Context menu for Milestones:

- Sort ascending (first...last)
- Sort descending (last...first)
- Edit values...
- Filter by values
- Group by values
- Hide field
- Delete field

Figura 3. Vista de GitHub *issues*. Fuente: GitHub, s. f.

GitLab

Es un servicio de **alojamiento de repositorio Git** basado en la web. Ofrece toda la funcionalidad de control de revisión distribuida y gestión de código fuente (SCM) de Git, además de agregar sus propias características. GitLab Enterprise Edition se basa en Git e incluye funciones adicionales (GitLab, 2022).

El software GitLab se escribió originalmente en el lenguaje de programación Ruby y algunas partes se reescribieron posteriormente en el lenguaje de programación Go. Inicialmente, era una solución de administración de código fuente para colaborar dentro de un equipo en el desarrollo de software que evolucionó a una solución integrada que cubría el ciclo de vida del desarrollo de software y luego a todo el ciclo de vida de desarrollo y operaciones (DevOps). La tecnología de software actual utilizada incluye Go, Ruby on Rails y Vue.js.

Sigue un modelo de desarrollo de **núcleo abierto** en el que la funcionalidad principal se publica bajo una licencia de código abierto (MIT), mientras que las funcionalidades adicionales están bajo una licencia propietaria.

Algunas de sus características son:

- ▶ **Planificación:** permite la planificación y gestión de carteras a través de épicas, grupos (programas) e hitos para organizar y realizar un seguimiento del progreso. Independientemente de su metodología, desde Waterfall hasta DevOps, el enfoque simple y flexible de planificación de GitLab satisface las necesidades de pequeños equipos y grandes empresas.
- ▶ **Creación:** cree, vea y administre código y datos de proyectos a través de poderosas herramientas de bifurcación.
- ▶ **Verificación:** mantenga estrictos estándares de calidad para el código de producción con pruebas e informes automáticos.
- ▶ **Empaquetar:** cree una cadena de suministro de software coherente y fiable con la gestión de paquetes integrada.
- ▶ **Seguridad:** capacidades de seguridad, integradas en su ciclo de vida de desarrollo. GitLab proporciona pruebas de seguridad de aplicaciones estáticas (SAST), pruebas de seguridad de aplicaciones dinámicas (DAST), escaneo de contenedores y escaneo de dependencias para ayudarlo a entregar aplicaciones seguras junto con el cumplimiento de licencias.
- ▶ **Generación de versiones:** la solución de CD integrada de GitLab le permite enviar código sin intervención, ya sea en uno o mil servidores.
- ▶ **Monitorización:** ayude a reducir la gravedad y la frecuencia de los incidentes. Obtenga comentarios y las herramientas que lo ayudarán a reducir la gravedad y la frecuencia de los incidentes para que pueda lanzar software con frecuencia y confianza.
- ▶ **Gestión del desarrollo:** ayuda a los equipos a administrar y optimizar su ciclo de vida de entrega de software con métricas y conocimientos de flujo de valor para optimizar y aumentar su velocidad de entrega.

GitHub vs GitLab: GitHub tiene una mayor disponibilidad y se focaliza más en el rendimiento de la infraestructura, mientras que GitLab se centra más en ofrecer un sistema basado en funciones con una plataforma integrada y centralizada. Adicionalmente, GitLab puede descargarse e instalarse en los servidores corporativos.

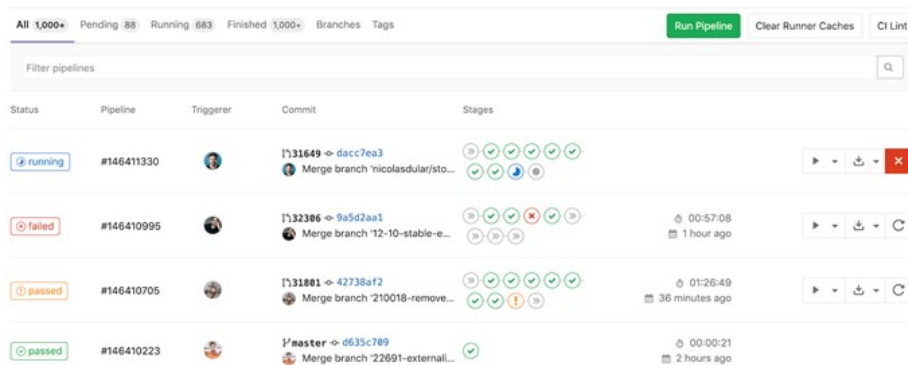


Figura 8. Vista de GitLab *pipelines*. Fuente: GitLab, s. f.

Bitbucket

Es la solución de Atlassian de gestión de repositorios Git diseñada para equipos profesionales. Brinda un **lugar central** para administrar los repositorios de Git, colaborar en su código fuente y guiarlo a través del flujo de desarrollo (Bitbucket,s. f.). Proporciona características como:

- ▶ Control de acceso para **restringir** el acceso a su código fuente.
- ▶ Control de flujo de trabajo para **hacer cumplir** un flujo de trabajo de proyecto o equipo.
- ▶ Solicitudes de **extracción con comentarios** en línea para colaborar en la revisión del código.
- ▶ Integración de Jira para una **trazabilidad completa** del desarrollo.
- ▶ API de descanso completo para crear **funciones personalizadas** para su flujo de

trabajo si aún no están disponibles en nuestro *marketplace*.

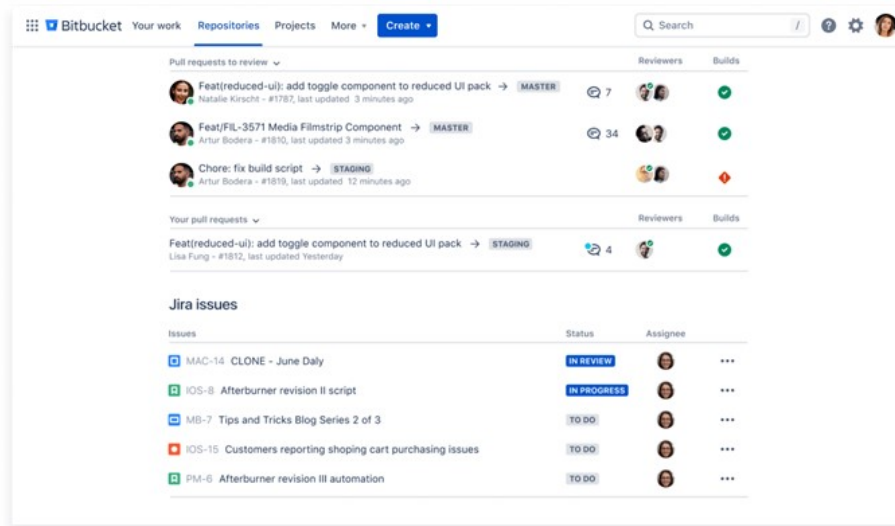


Figura 9. Vista de BitBucket. Fuente: Bitbucket, s. f.

1.5. Referencias bibliográficas

Bitbucket. (s. f.) *Code & CI/CD, optimized for teams using Jira*. <https://bitbucket.org/>

Blokdyk, G. (2018). *Apache subversion: The ultimate step-by-step guide*. Createspace Independent Publishing Platform.

Chacon, S. y Straub, B. (2014). *Pro Git*. A press. <https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F>

Nongnu. (s. f.). *CVS - Concurrent Versions System*. <https://www.nongnu.org/cvs/>

GitHub. (s. f.). *Project planning for developers*. <https://github.com/features/issues>

GitLab. (2022). *The one DevOps platform*. <https://about.gitlab.com/>

GitLab. (s. f.) *CI/CD pipelines*. <https://docs.gitlab.com/ee/ci/pipelines/>

Humphrey, W. (1989). *Managing the Software Process*. Pearson Education.

Humphrey, W. S. (2000a). *The Personal Software Process (PSP)*. Carnegie Mellon Software Engineering Institute. <https://doi.org/10.1184/R1/6585197.V1>

Humphrey, W. S. (2000b). *The Team Software Process (TSP)*. Caregie Mellon Software Engineering Institute. https://resources.sei.cmu.edu/asset_files/TechnicalReport/2000_005_001_13754.pdf

Roongkaew, W. y Prompoon, N. (2013). Software engineering tools classification based on SWEBOK taxonomy and software profile. *2013 Second International Conference on Informatics & Applications (ICIA)*, pp. 122-128. <https://ieeexplore.ieee.org/document/6650241>

Sommerville, I. (2015). *Software Engineering* (10ª ed.). Pearson.

Comenzando con Git y GitHub: una guía completa de control de versiones, gestión de proyectos y trabajo en equipo para el nuevo desarrollador

Tsitoara, M. (2019). *Beginning Git and GitHub: A comprehensive guide to version control, project management, and teamwork for the new developer* (1ª ed.). Apress.

Libro que proporciona una explicación del control de versiones de software utilizando Git y el uso de la plataforma GitHub.

Git para equipos

Westby, H. y Jane, E. (2015). *Git for Teams*. O'Reilly Media.

Esta guía práctica ofrece un enfoque único del control de versiones que prioriza a las personas y también explica cómo el uso de Git como punto focal puede ayudar a su equipo a trabajar mejor en conjunto. Aprenderá a planificar y seguir un flujo de trabajo de Git que no solo garantice que logre los objetivos del proyecto, sino que también se adapte a las necesidades inmediatas y al crecimiento futuro de su equipo.

1. Una colección de herramientas para ayudar al programador a crear e implementar código/contenido que está específicamente diseñado para ejecutarse en una plataforma muy particular o de una manera muy particular se denomina:

- A. Marco de referencia (*framework*).
- B. Kit de desarrollo de software (*software development kit*, SDK).
- C. Caja de herramientas (*toolkit*).
- D. Biblioteca.

2. Relaciona las características con las diversas plataformas *low-code*:

GitLab	1	A	Git de Atlassian
Bitbucket	2	B	Git de Microsoft
Subversion	3	C	Git instalable <i>on premises</i>
GitHub	4	D	No se basa en Git

3. Selecciona cuál es el principal valor de las plataformas de desarrollo de software:

- A. Facilitar las pruebas.
- B. Mejorar la productividad y eficiencia del equipo de desarrollo.
- C. Reducir costes de migración o conversión.
- D. Reducir el coste operativo.

4. Selecciona cuál de las siguientes funcionalidades debe realizarse primero de forma manual y sin apoyo de herramientas:

- A. Plantear una solución técnica.
- B. Entender los requisitos.
- C. Definir cómo probar.
- D. Todas las anteriores.

5. Ordena, del uno (más antigua) al cuatro (más reciente), el orden en el que se crearon las plataformas de apoyo a la gestión de la configuración:

GitHub	1	A	Uno (más antigua)
Git	2	B	Dos
CVS	3	C	Tres
SVN	4	D	Cuatro (más reciente)

6. El control de versiones es una práctica, integrada en la gestión de la configuración del software, responsable de gestionar cambios en:

- A. Solo en el código fuente.
- B. En el código y en los requisitos
- C. En el código, su documentación o cualquier colección de información relevante para su desarrollo o evolución.
- D. Solo en la documentación.

7. ¿Qué estado no forma parte de un fichero bajo un control de versiones con Git?

- A. Modificado (*modified*).
- B. Bifurcado (*branched*).
- C. Preparado (*staged*).
- D. Comprometido (*committed*).

8. ¿Qué funcionalidad no forma parte de un control de versiones?
- A. Deshacer ediciones específicas sin perder el trabajo realizado mientras tanto.
 - B. Proporcionar acceso a las versiones históricas de un proyecto.
 - C. Garantizar la trazabilidad del código fuente respecto a otra documentación del proyecto, como requisitos, incidencias, diseños, etc.
 - D. Integrar el trabajo que se realiza simultáneamente por diferentes miembros del equipo.
9. En el ámbito del control de versiones, si utilizamos bifurcaciones, los objetos se gestionan en:
- A. Deltas.
 - B. *Releases*.
 - C. Ramas.
 - D. Instantes.
10. Para llevar a cabo la construcción del software, se requiere de:
- A. Un equipo de personas formadas y especializadas en roles que trabajan en equipo.
 - B. Un método y procesos que guíen en la construcción del software.
 - C. Un conjunto de herramientas que facilitan y aceleran el desarrollo.
 - D. Todas las anteriores.

Plataformas de Desarrollo de Software

Tema 2. Plataformas para requisitos

Índice

Esquema

Ideas clave

- 2.1. Introducción y objetivos
- 2.2. Contexto de plataformas de requisitos
- 2.3. Valor y utilidad de las plataformas de requisitos
- 2.4. Principales plataformas de requisitos
- 2.5. Plataformas para historias de usuario y prototipos de interfaz gráfico
- 2.6. Referencias bibliográficas

A fondo

User story mapping: discover the whole story, build the right product

User stories applied: For agile software development

Requirements management: The interface between requirements development and all other systems engineering processes

Requirements engineering and management for software development projects.

Test

PLATAFORMAS PARA REQUISITOS			
CONCEPTOS	VALOR Y UTILIDAD	HERRAMIENTAS	
Requisito Capacidad para resolver un problema Capacidad para satisfacer un contrato Representación documentada	Características ISO/IEC TR 24766:2009: - Elicitación de requisitos - Análisis de requisitos - Especificación de requisitos - Verificación y validación de requisitos - Gestión de requisitos - Otras capacidades	Plataformas de requisitos: - IBM DOORS - Jama Software - Modern Requirements4DevOps - Visure Requirements - ReqSuite RM	
Requisitos y CMMI Gestión de requisitos (<i>requirements management</i>, REQM) Desarrollo de Requisitos (<i>requirements development</i>, RD)	Criterios para seleccionar plataformas: - Organización en base a atributos o metadatos - Dependencias y consistencia - Gestión de versiones y variantes - Soporte a trazabilidad - Generación de informes - Colaboración y gestión del proceso - Facilidad de integración - Soporte para Agile y DevOps	Plataformas <i>user story mapping</i>: - Agile User Story Map para Jira - Avion - Cardboard - Craft.io - Easy Agile - Featmap - Featuremap - Miro - Productboard - StoriesOnBoard - Trello	
Otras formas de establecer requisitos <i>User story mapping</i> <i>Wireframes</i>		Plataformas <i>wireframes</i>: - Adobe XD - Balsamiq - Figma - InVision Freehand - Justinmind - MockupPlus - Sketch - UXPin	

2.1. Introducción y objetivos

En este tema se estudian las principales plataformas que nos ayudan a **definir, documentar y gestionar** los requisitos en un proyecto de software.

Todo proyecto de software tiene un punto de entrada común: la **captura de requisitos**. Conocer las necesidades del cliente, documentarlas y transformarlas a requisitos de desarrollo, tanto funcionales como no funcionales, y la gestión de estos a lo largo del proyecto son tareas necesarias y fundamentales en cualquier desarrollo.

Las herramientas de requisitos no son nuevas y han ido **evolucionando** en paralelo a las metodologías, las herramientas, los lenguajes de desarrollo y las técnicas de ingeniería de software.

En este tema posicionaremos las diferentes herramientas de requisitos, detallaremos su relevancia actual y los criterios que podrían ser de utilidad para tomar la decisión de su adopción en una organización.

Con el estudio de este tema pretendemos alcanzar los siguientes objetivos:

- ▶ Comprender el **valor y la utilidad** que tienen las plataformas para la definición y gestión de los requisitos.
- ▶ Identificar los **criterios principales** que nos ayuden a seleccionar las plataformas en cada caso.
- ▶ Conocer las principales plataformas de **definición de requisitos**.
- ▶ Conocer las principales plataformas de **gestión de requisitos**.
- ▶ Saber utilizar algunas de las principales **plataformas de requisitos**

2.2. Contexto de plataformas de requisitos

El software que se produce es cada día más complejo. En las últimas décadas, se han introducido importantes mejoras en la forma en la que se lleva a cabo el desarrollo de software, especialmente en el ámbito de la gestión de proyectos, con la aparición de nuevas metodologías más eficientes, la codificación y las pruebas, mediante la aplicación de técnicas que permite la reutilización, la composición e incluso la automatización. Sin embargo, la principal fuente de defectos en cualquier desarrollo sigue siendo la actividad de **captura y gestión de requisitos**, puesto que es el punto en el que debe establecerse la comunicación entre negocio y tecnología.

Los requisitos son el **preámbulo** de cualquier proyecto de desarrollo de software. Ningún proyecto de software debería comenzar sin saber en qué va a consistir, qué es lo que se quiere desarrollar y cómo se podrá decir que se ha alcanzado con éxito su objetivo. Al ser el punto de partida de cualquier desarrollo, son considerados el principal criterio de éxito del equipo de proyecto, el fundamento de la definición de tareas del equipo y la base para el resto de las fases de desarrollo: diseño, codificación y pruebas.

Podemos encontrar **diferentes definiciones** de requisitos, desde la más básica en la que se establece que los requisitos son una especificación de lo que debería ser implementado (Sommerville y Sawyer, 1997), hasta otras más estandarizadas, como la de IEEE, que ofrece tres posibles definiciones de requisitos (IEEE, 2002):

- ▶ Una condición o capacidad que un usuario necesita para resolver un problema o alcanzar un objetivo.
- ▶ Una condición o capacidad que un sistema o componente de sistema debe poseer para satisfacer un contrato, estándar, especificación u otros documentos formalmente impuestos.

- Una representación documentada de una condición o una capacidad que cumple alguna de las dos definiciones anteriores.

En muchas ocasiones, requisitos se considera como sinónimo de **requerimiento** o de **necesidad** y, aunque tienen connotaciones algo diferentes, se puede considerar válida dicha consideración.

Aunque la industria del software suele utilizar estas definiciones como guía, es necesario destacar que hay aspectos y características propias de los requisitos que no quedan completamente descritas. Así, se tiene en cuenta las necesidades del usuario, ¿pero las de los desarrolladores?, ¿un cliente y un usuario son considerados lo mismo y tienen las mismas necesidades? o ¿tiene en cuenta también las restricciones? Estas son solo un pequeño ejemplo de lo que quedaría al margen de dicha definición. Por ello, Chemuturi (2013) establece:

«Un requisito es una necesidad, expectativa, restricción o interfaz establecida por cualquiera de los actores involucrados (*stakeholders*) en un desarrollo de software que debe ser satisfecha por el producto de software resultante en algún momento de su desarrollo» (Chemuturi, 2013).

De esta forma, los requisitos permiten alinear las **diferentes visiones** entre las personas involucradas en el proyecto para establecer una común y compartida, lo que define objetivos y expectativas.

Además de ser el inicio de un proyecto, los requisitos y las actividades relacionadas con ellos se extienden a lo largo del ciclo de vida de este, ya que deben ser elicitados, analizados, establecidos, documentados, modificados y gestionados a lo largo de todo el proyecto.

La integración de modelos de madurez de capacidades o CMMI (Chrissis et al., 2011), un marco de referencia fundamental en el ámbito de las buenas prácticas en el proceso de desarrollo del software, hace alusión a los requisitos en dos áreas de proceso diferenciadas: la primera, que pertenece al nivel dos, se denomina gestión

de requisitos (*requirements management*, REQM) y su propósito consiste en **gestionar los requisitos** del producto y componentes del producto que tiene un proyecto, así como **identificar las inconsistencias existentes** entre dichos requisitos y los diferentes elementos de trabajo y planes dentro del proyecto. La segunda, denominada desarrollo de requisitos (*requirement development*, RD), pertenece al nivel tres y tiene como objetivo **establecer y analizar todos los requisitos**: de cliente, de producto y de componente de producto. Así se pone de manifiesto, por un lado, la importancia de los requisitos a la hora de llevar a cabo el desarrollo de un proyecto y, por otro, que los requisitos son elementos vivos a lo largo de este, por lo que hay que establecer los mecanismos adecuados para su gestión de forma sistemática y disciplinada. En ocasiones, el conjunto de todas estas tareas es denominada como ingeniería de requisitos.

Las organizaciones de desarrollo tienen claro que los requisitos importan. Sin embargo, siguen siendo uno de los grandes retos para cualquier proyecto de software y uno de los procesos de negocio más desafiantes para llevarlos a cabo correctamente a pesar de los años en los que lleva siendo objeto de estudio (Lawrence et al., 2001). Unos requisitos mal definidos, inexistentes o cambiantes pueden hacer de un proyecto un **problema**, con un alto coste en dinero y tiempo. El éxito de un proyecto depende de una buena estructura para la ingeniería de requisitos puesto que la buena comunicación y entendimiento entre clientes, proveedores, actores implicados, organización interna y equipo de desarrollo hace que los requisitos sean algo más que un conjunto de documentos que recogen funcionalidades o diagramas del lenguaje unificado de modelado (UML).

2.3. Valor y utilidad de las plataformas de requisitos

Una vez establecida la necesidad de **documentar y mantener los requisitos** de los proyectos, las diferentes organizaciones comienzan a plantearse la necesidad de tener herramientas que les permitan hacer dichas actividades de forma más eficaz. En un momento inicial, la primera decisión pasa por tener algo de fácil acceso y posible utilización por cualquier persona. Por ello, las herramientas más utilizadas en la fase de requisitos suelen ser los editores de texto, como MS Word, o las hojas de cálculo, como MS Excel. En el ámbito de la ingeniería de requisitos, esto se conoce como **actividades manuales** que, a pesar de tener buena voluntad, acaban siendo rechazadas puesto que no son efectivas ni eficientes, por lo que no suelen ser suficientes.

Diferentes herramientas y plataformas de gestión de requisitos han ido surgiendo a lo largo de los últimos años, más o menos complejas, que permiten llevar a cabo acciones como la administración de las **relaciones** entre los requisitos, **la gestión de los cambios** o **análisis del impacto** a un cambio de requisitos. Además, gran parte de las herramientas de desarrollo de software ofrecen funcionalidades para la **gestión de requisitos**. Esto, que puede ser considerado como una gran ventaja por los equipos de desarrollo, ya que se obtiene por defecto la interoperabilidad con el resto de las fases de desarrollo, no siempre dispone de toda la potencia que una buena definición y gestión de requisitos necesita.

A la hora de seleccionar la plataforma de requisitos que más se ajusta a una organización, podemos hacer uso de la **ISO/IEC TR 24766:2009**, un marco que sirve como guía para la evaluación de las capacidades de las herramientas de ingeniería de requisitos. Para ello divide las capacidades de una herramienta en seis categorías principales (International Organization for Standardization, 2009):

- ▶ **Elicitación de requisitos**, es decir, si la plataforma de requisitos es capaz de soportar la identificación de los *stakeholders* (actores asociados), la captura y traza con los requerimientos de negocio y usuario, los requerimientos funcionales y los requerimientos no funcionales durante las tareas de descubrimiento de requisitos.
- ▶ **Análisis de requisitos**, es decir, si la plataforma de requisitos es capaz de soportar la descomposición de requerimientos en requisitos funcionales y no funcionales y de analizar la viabilidad y el riesgo de los requisitos.
- ▶ **Especificación de requisitos**, es decir, si la plataforma es capaz de soportar la especificación de los requisitos de una forma consistente, accesible y revisable.
- ▶ **Verificación y validación de requisitos**, es decir, si la herramienta es capaz de soportar diferentes formas evaluar y testear la validación y verificación de los requisitos.
- ▶ **Gestión de requisitos**, es decir, si la herramienta es capaz de soportar la monitorización de los cambios y el mantenimiento de los requisitos para asegurar que están reflejados en el producto resultante.
- ▶ **Otras capacidades**, entre las que podemos incluir la trazabilidad de los requisitos con los resultados de otras partes del proyecto, información más administrativa de la plataforma, integración con otras herramientas o el uso de interfaces de usuarios más gráficos.

Durante los últimos años, se han realizado diferentes estudios y revisiones que intentan recoger los requisitos que deben cumplir las herramientas de gestión de requisitos. Entre los más importantes destacamos los trabajos del INCOSE Tool Survey y el informe de Business Industry Reports.

En esta misma línea, Carrillo de Gea et al. (2021), en su trabajo, han actualizado una lista de criterios para evaluar las herramientas de ingeniería de requisitos, al incorporar nuevos criterios aparecidos debido a la evolución de las diferentes metodologías y tecnologías junto aquellos a los que hacía referencia la ISO/IEC TR 24766:2009 y que siguen siendo aspectos clave de cualquier herramienta (International Organization for Standardization, 2009).

La lista, en la que se destacan los diferentes criterios, su descripción y su identificación, es la siguiente, en la que se detalla la forma de evaluar como (+), (++) o (+++) el grado de cobertura de cada herramienta respecto a cada criterio:

- ▶ **Organización de los requerimientos con metadatos, atributos y reutilización (OR):** la herramienta permite al usuario agregar metadatos a los requerimientos en diferentes formatos (+), incluye atributos de requisitos predefinidos y atributos definidos por el usuario (++) y proporciona requisitos definidos por el usuario y plantillas de documentos de requisitos (+++).
- ▶ **Informes, consultas a bases de datos y lenguaje de interfaz abierta (RDOIL):** la herramienta es capaz de generar informes estructurados, sofisticados y/o estandarizados a partir de los datos (+), su funcionalidad se puede ampliar fácilmente (++) y ofrece un lenguaje de extensión, API y/o API REST (+++).
- ▶ **Controles internos, es decir, consistencia, dependencias e historia (IC):** la herramienta ofrece mecanismos para la detección y resolución de conflictos entre requisitos y/o verificaciones de consistencia (+), admite análisis de dependencia (++) e incluye funcionalidad de control de versión de requisitos y gestión de cambios de requisitos (+++).

- ▶ **Apoyo a la trazabilidad, es decir, arrastrar y soltar (horizontal y verticales) (TS):** la herramienta ofrece soporte a trazabilidad visual y/o diferentes tipos de trazas (+), soportes a la trazabilidad de los requisitos con otros artefactos (++) y ofrece una trazabilidad completa para gestionar los requisitos a lo largo del ciclo de vida del desarrollo, incluida la trazabilidad externa a otras herramientas (+++).
- ▶ **Gestión de variantes (VM):** la herramienta incluye soporte explícito para reutilización (+), creación y gestión de variantes (++) y líneas de productos de software y/o catálogos de requisitos reutilizables (+++).
- ▶ **Trabajo remoto, solo en la nube (RWC):** la herramienta puede funcionar sin conectividad (+), su funcionalidad cuando no está conectada es completa (++) y permite al usuario trabajar sin conexión y sincronizar cuando la conectividad está disponible (+++).
- ▶ **Múltiples vistas (MV):** la herramienta incluye bases de datos y vistas orientadas a documentos de los requisitos (es decir, Word especificación, tabla, etc.) (+), ofrece diferentes vistas de las relaciones de trazabilidad (textual, árbol, matriz) (++) y permite al usuario visualizar las dependencias de requisitos y su evolución (+++).
- ▶ **Rendimiento (Perf):** la herramienta puede gestionar grandes proyectos con grandes cantidades de datos (+), está optimizada para cargar grandes modelos (++) y ofrece un rendimiento adecuado y es receptivo, incluso si hay trabajo remoto sin conectividad (+++).
- ▶ **Colaboración, gestión del workflow (CWM):** la herramienta brinda soporte para el trabajo colaborativo y/o distribuido, ofrece funcionalidad de revisión, discusión y/o voto —así como comunicación asíncrona y/o síncrona—, incluye control de acceso y seguridad granular (+), proporciona flujos de trabajo (*workflows*) predefinidos, flexibles y personalizables (++) y admite múltiples dispositivos (por ejemplo, navegador web, aplicación móvil) (+++).

- ▶ **Fácilmente adaptable e integrable en los procesos de negocio (IBP):** la herramienta es personalizable y flexible para adaptarse a las condiciones del proyecto, los *stakeholders* o la naturaleza. de los requisitos (+), puede adaptarse fácilmente a cualquier tipo de proceso de negocio o industria (++) y puede integrarse o adaptarse fácilmente a herramientas y procesos preexistentes dentro del organización (+++).
- ▶ **Federación y notificación con herramientas *application and product lifecycle management* (ALM/PLM) (FN):** la herramienta ofrece funcionalidad ALM/PLM (+), ofrece la posibilidad de integrarse con soluciones ALM/PLM de terceros (++) y tiene soporte Open Services for Lifecycle Collaboration (+++).
- ▶ **Exportación/Importación con formatos estándar (EI):** la herramienta ofrece una funcionalidad completa de importación/exportación y soporte para formatos estándar: CSV (+), otros (excepto el *requirements interchange format* [ReqIF] estándar) (++) y ReqIF estándar (+++).
- ▶ **Macros para repetir comandos (MaC):** la herramienta incluye la funcionalidad de creación y gestión de macros (+), proporciona un lenguaje de scripting (++) y/u otros mecanismos de automatización (+++).
- ▶ **Esfuerzo de formación y curva de aprendizaje (TLC):** la herramienta incluye documentación adecuada, tutoriales, materiales de capacitación y recursos (+) y cursos y/o certificaciones (++) y es fácil de usar y/o aprender (+++).
- ▶ **Agile, integración y distribución continuas (CI/CD) y desarrollo y operaciones (DevOps):** la herramienta está concebida o es fácilmente adaptable a metodologías ágiles y *pipelines* DevOps (+), permite la gestión de requisitos en marcos ágiles, *lean* o *scaled agile* (++) e integración continua (CI)/ despliegue continuo (CD) y DevOps (+++).

- **Soporte inteligente (IS):** la herramienta el procesado, agrupado o clasificación de requisitos mediante el uso de procesamiento de lenguaje u otros métodos automáticos basados en inteligencia artificial y analíticas de datos (+), ofrece funcionalidad de generación de código (++) y proporciona recomendaciones sobre los activos del proyecto (por ejemplo, *stakeholders* y requisitos) para facilitar el seguimiento y la toma de decisiones (+++).
- **Escalabilidad (Scal):** la herramienta ofrece soporte básico a grandes proyectos (+), puede escalar adecuadamente con el tamaño del proyecto y del equipo (++) y se ofrece dentro de un modelo de servicio en la nube (SaaS) (+++).

Siguiendo estos criterios, Carrillo de Gea et al. (2021) han llevado a cabo una evaluación de trece herramientas seleccionadas entre más de las 200 disponibles en el mercado. En la Figura 1 se observa el resumen del estado del uso de las trece herramientas de ingeniería de requisitos y en la Tabla 1, la evaluación de dichas herramientas en cuanto a los criterios definidos.

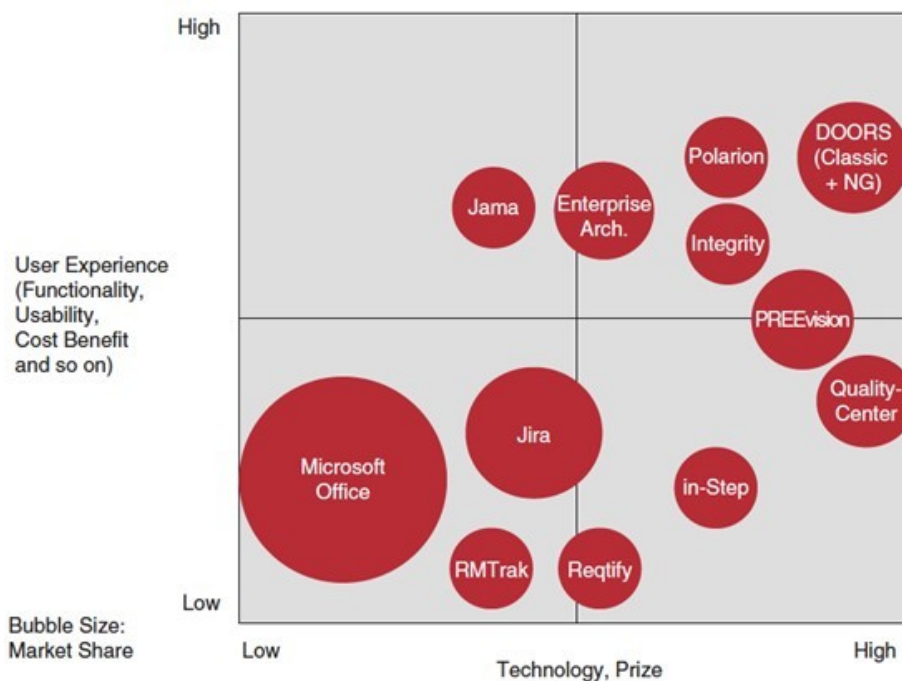


Figura 1. Herramientas de ingeniería de requisitos y estado de su práctica. Fuente: Carrillo de Gea et al., 2021.

Tool name	DOORS DNG	DOORS	Jira	Siemens Polarion	PTC Integrity	Regify	Quality Center	in-STEP Blue	PREBvision	Enterprise Architect	Jama Connect	RMTrak	Microsoft Office
Version, date	7.0.3, December 2020	9.7.2, July 2020	8.14.0, November 2020	20R2, October 2020	12.2.1.0, October 2019	2020x, November 2019	15.5, September 2020	6.5, January 2020	9.5, 2021	15.2, October 2020	8.561, November 2020	5.3.22, 2017	365, January 2021
Vendor name	IBM	IBM	Atlassian	Siemens	PTC	Dassault	Micro Focus	Micro-TOOL	Vector	Sparx Systems	Jama Software	Pro-netes	Microsoft
Cost	High	High	Low	High	High	Middle	High	High	High	Middle	Middle	Middle	Low
OR	+++	+++	0	+++	++	+	+	+	++	++	+++	+	++
RDOIL	++	+	+	++	++	+	++	+	++	++	++	+	+
IC	++	++	+	++	++	+	+	+	++	++	++	+	0
TS	+++	+++	+	+++	+++	+	+	+	++	+++	+++	+	0
VM	+++	++	0	+++	++	0	0	+	+++	+	+++	0	0
RWC	++	+	+	++	++	0	+++	++	+++	+++	++	0	++
MV	+++	++	0	+++	+++	+	+	+	+++	+++	+++	+	+
Perf	++	+	++	++	++	+	+	0	++	++	++	0	+
CWM	+++	+	++	+++	++	+	++	+	+++	+++	+++	+	++
IBP	++	+	++	++	++	+	+	+	++	++	++	0	++
FN	+++	+++	++	+++	+++	0	+	+	+++	+++	+++	0	+
EI	++	++	+	++	++	+	+	++	++	++	++	+	+
MaC	++	++	++	++	++	0	+	0	++	++	++	+	++
TLC	+	+	+	+	+	+	++	+	+	+	+	+	++
DevOps	+	+	++	++	+	+	++	+	+	+	+	0	0
IS	+	0	0	+	0	0	0	0	0	++	0	0	0
Scal	+++	+	+	+++	+++	+	+++	+	++	+++	+++	+	++
Total	38	27	19	39	34	12	23	16	35	37	37	10	19

Values include "0", "+", "++", and "+++" from minimum/no support to full support.

Tabla 1. Evaluación de herramientas de Ingeniería de Requisitos. Fuente: Carrillo de Gea et al., 2021.

2.4. Principales plataformas de requisitos

Algunas de las herramientas comerciales mejor valoradas por los diferentes estudios sobre soluciones de gestión e ingeniería de requisitos son las siguientes:

Plataformas de requisitos
IBM DOORS
Jama Software
Modern Requirements4DevOps
Visure Requirements
ReqSuite RM

Tabla 2. Principales plataformas de requisitos. Fuente: elaboración propia.

IBM DOORS Family

IBM ofrece dos soluciones de gestión de requisitos con muchas capacidades similares: IBM Requirements Management DOORS e IBM Requirements Management DOORS Next. Sin embargo, los productos ofrecen diferentes **implementaciones y oportunidades estratégicas** para los analistas de requisitos y los equipos de diseño, desarrollo y prueba. Ambos productos continúan siendo mejorados y respaldados por sus respectivas bases de clientes.

DOORS continúa siendo, después de años, la solución de gestión de requisitos **líder del mercado** y ya se encuentra en la versión 9. Proporciona una amplia gama de capacidades de gestión de requisitos muy maduras, como módulos de especificación de requisitos estructurados, importación y exportación de datos de ida y vuelta, firmas electrónicas, líneas de base y vistas de requisitos personalizables con trazabilidad de varios niveles y la gestión de cambios de los cambios a los requisitos. Da soporte a grandes proyectos de desarrollo con equipos en programas de ingeniería de sistemas complejos y de alto cumplimiento, que dispongan de miles de objetos o cientos de usuarios simultáneos.

DOORS ofrece una API (interfaz de programación de aplicaciones) extensible programable por el usuario llamada DOORS eXtension Language (DXL). DXL proporciona una forma de personalizar estas actividades:

- ▶ Automatización de la administración
- ▶ Creación de vistas de trazabilidad multinivel
- ▶ Cálculo de métricas
- ▶ Implementación de la lógica de negocios
- ▶ Ampliación de la interfaz de usuario
- ▶ Integración con otras herramientas

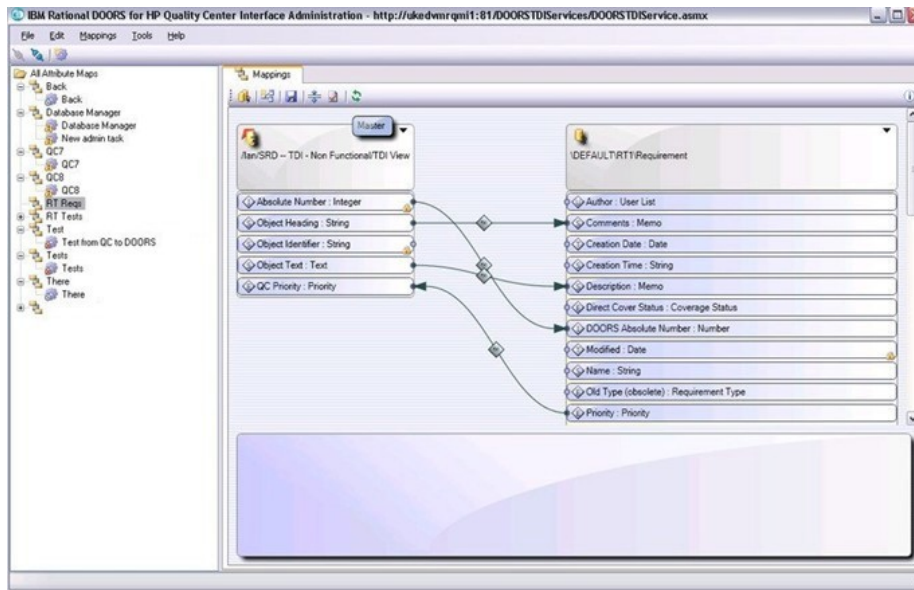


Figura 2. Vista de Project Window en DOORS. Fuente: Software informer, 2022.

DOORS Next se ha desarrollado como una plataforma colaborativa de gestión de requisitos que dispone de un cliente web. Este producto incluye capacidades tales como definición de requisitos visuales, elementos de trabajo e integración de pruebas y planificación. Utiliza las capacidades de Jazz Team Server para respaldar la colaboración en equipo a través de paneles, revisiones y comentarios. Los usuarios, proyectos, tipos de datos, tipos de artefactos, atributos y etiquetas tienen una **administración común**. Estos servicios comunes y sistemas de tipos ayudan a los equipos de proyecto a definir y gestionar los requisitos de forma coherente. Su principal característica es su interoperabilidad con otras herramientas, incluida la gestión del ciclo de vida, la colaboración en equipo y la ingeniería de sistemas/software.

ID	Name	Artifact Type	Modified By	Modified On
319	Dividend allocation by percentage	Feature	Administrator Jazz	Jan 14, 2018, 10:45:14 PM
282	Healthy Customer Ecosystem	Business Goal	Administrator Jazz	Jan 14, 2018, 10:45:14 PM
280	Olive Back	Business Goal	Administrator Jazz	Jan 14, 2018, 10:45:14 PM
275	Service for Allocating Dividends	Feature	Administrator Jazz	Jan 14, 2018, 10:45:14 PM
241	Organization must identify how much money is desired	Feature	Administrator Jazz	Jan 14, 2018, 10:45:14 PM
238	Requests sent in form of hard copy mail	Feature	Administrator Jazz	Jan 14, 2018, 10:45:14 PM
188	Donation by Amount	Feature	Administrator Jazz	Jan 14, 2018, 10:45:14 PM
181	Organizations can apply	Feature	Administrator Jazz	Jan 14, 2018, 10:45:14 PM
178	Donor Dividend Allocation Criteria	Feature	Administrator Jazz	Jan 14, 2018, 10:45:14 PM
166	Donors Deposit Money in a Pooled Assistance Fund	Feature	Administrator Jazz	Jan 14, 2018, 10:45:14 PM
163	Requests sent in form of email	Feature	Administrator Jazz	Jan 14, 2018, 10:45:14 PM
148	Donors will receive confirmation and receipt	Feature	Administrator Jazz	Jan 14, 2018, 10:45:14 PM
138	Allocate dividends by Percentage	User Story Elaboration	Administrator Jazz	Jan 14, 2018, 10:45:14 PM
127	Dividend processing payment in a one time transaction	Feature	Administrator Jazz	Jan 14, 2018, 10:45:14 PM
120	Promote JKE Corporate Image	Business Goal	Administrator Jazz	Jan 14, 2018, 10:45:14 PM
110	Organizations may	Feature	Administrator Jazz	Jan 14, 2018, 10:45:14 PM

Figura 3. Vista de la lista de artefactos en DOORS Next. Fuente: ReqPro, s.f.

DOORS y DOORS Next pueden **utilizarse conjuntamente**. Por ejemplo, puede vincular requisitos y artefactos relacionados entre los dos productos. Los módulos y objetos que se exportan desde DOORS se pueden importar a DOORS Next.

El cliente web de DOORS Next proporciona **herramientas** para definir requisitos en documentos de texto enriquecido y representaciones visuales en diagramas de procesos comerciales, diagramas de casos de uso, guiones gráficos, bocetos de interfaz de usuario y flujos de pantalla. Los artefactos de requisitos se pueden organizar y reutilizar en vistas, colecciones y módulos. El cliente web admite enlaces de trazabilidad a artefactos de requisitos relacionados y en aplicaciones Elm a planes de desarrollo, elementos de trabajo, planes de prueba, casos de prueba, diseños y modelos.

Jama Software

Jama Connect es una plataforma de desarrollo de productos que ayuda a los equipos de hardware y software a **conectar** actividades aisladas de desarrollo, pruebas y gestión de riesgos para proporcionar cumplimiento integral, mitigación de riesgos y

mejora de procesos. La plataforma permite a los usuarios definir, alinear y ejecutar proyectos, lo que reduce los tiempos de ciclos prolongados, el esfuerzo dedicado a demostrar el cumplimiento y el desperdicio de trabajo.

Jama Connect admite **múltiples metodologías** de desarrollo concurrentes e incluye trazabilidad que permite la gestión de procesos complejos, una interfaz intuitiva y fácil de usar. Esto, a su vez, alinea a las partes interesadas en el desarrollo de hardware, firmware y software. Tiene características como la trazabilidad de datos y recursos de extremo a extremo, análisis de impacto en tiempo real, revisiones y aprobaciones que pueden ayudar a impulsar la colaboración en equipo. Además, puede integrar el software con aplicaciones de terceros como Jira, Rally, GitHub y otras herramientas de gestión de requisitos o ALM, como Micro Focus y Perforce para asegurar la trazabilidad, visibilidad y colaboración a lo largo de todo el ciclo de vida.

La herramienta pone a los equipos en funcionamiento rápidamente con marcos específicos de la industria que se alinean con los estándares, lo que genera confianza en el cumplimiento de las normativas, y permite que los equipos muestren fácilmente a los auditores cómo se tomaron las decisiones con informes.

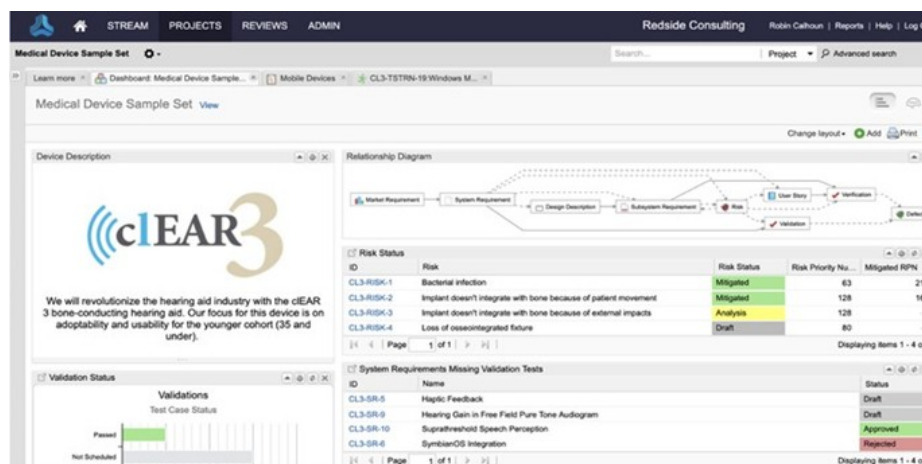


Figura 4. Vista de trazabilidad de requisitos en Jama Software. Fuente: Jama Software, s. f.

Modern Requirements4DevOps

Es la solución de gestión de requisitos **mejor valorada** por los principales analistas. Está integrada en Azure DevOps, lo que lo convierte en una potente solución de gestión de requisitos. Facilita la integración de los equipos bajo una plataforma para facilitar un modelo de fuente única de la verdad. Sus herramientas están diseñadas para acelerar la velocidad de los proyectos, brindar valor comercial, satisfacer las necesidades de calidad y cumplimiento para todas las metodologías e industrias (Requirements Management, 2019).

Da soporte a documentación textual, casos de uso, revisión en línea, integración con firma electrónica, trazabilidad extremo a extremo, creación de línea de base y control de versiones, conexión de requisitos con diagramas y *mockups*, así como la generación automática de informes y auditorías personalizados.

Algunas de sus características o funcionales principales son:

- ▶ **Smart Docs:** una herramienta de creación similar a Word que está disponible en línea. Permite crear elementos de trabajo de Azure DevOps en una vista de documento con texto enriquecido, tablas e imágenes en línea, utilizando un conjunto completo de controles de texto enriquecido.
- ▶ **Baseline:** proporciona una instantánea de un punto en el tiempo de cualquiera o todos los elementos de trabajo de Azure DevOps dentro de un proyecto. Esta herramienta brinda a los usuarios la capacidad de ver y comparar líneas base y comparar entre reversiones más antiguas y recientes de elementos de trabajo. Tener una línea de base en cada proyecto permite a los usuarios monitorear el desempeño actual del proyecto, administrar el alcance de la versión, crear notas de la versión, retroceder a una versión anterior y mejorar la precisión de las estimaciones futuras. Un uso secundario de la función de copia de elementos de trabajo dentro de la herramienta es para la reutilización de elementos de trabajo. La herramienta permite a los usuarios crear una biblioteca de paquetes reutilizables de requisitos —como

requisitos no funcionales (NFR)— y luego aplicarlos según sea necesario a nuevos proyectos, junto con capacidades de informes de reutilización para realizar un seguimiento de las eficiencias.

- ▶ **Análisis de trazas:** la herramienta proporciona enlaces completos de extremo a extremo entre hasta diez niveles de diferentes tipos de elementos de trabajo en una matriz horizontal, incluidos los resultados de las pruebas y los defectos. También le permite personalizar las propiedades que se muestran. La matriz de intersección permite a los usuarios agregar o eliminar relaciones entre elementos de trabajo de manera eficiente, lo que ayuda a los usuarios a planificar, rastrear y analizar la calidad, la cobertura y la integridad del proyecto.
- ▶ **Gestión de la revisión:** permite administrar la función de revisión y aprobación de requisitos o cualquier tipo de elementos de trabajo y solicitar firmas electrónicas. Los usuarios pueden programar una revisión y solicitar una respuesta de uno o más revisores sobre múltiples requisitos. Las partes interesadas pueden ver la lista de requisitos en un paquete de revisión y también ver todas las respuestas proporcionadas en relación con los requisitos individuales. Los revisores pueden responder en «Aceptar/Rechazar» y «Aceptar todo/Rechazar todo».
- ▶ **Alice:** es la primera de una serie de herramientas inspiradas en inteligencia artificial (IA) en la familia de requisitos modernos. Es un asistente de analista de negocios cognitivo que le permite obtener requisitos automáticamente a través de una jerarquía de preguntas estructurada lógicamente.
- ▶ **Informes inteligentes:** permite crear automáticamente informes HTML detallados directamente desde la vista de trabajo pendiente o consulta. Filtra, agrupa, ordena y exporta la información como archivo Word o PDF. Utilice informes para compartir información con partes interesadas externas y generar documentos de requisitos del sistema o del negocio. Con informes avanzados, los usuarios pueden crear y almacenar plantillas de Word definibles por el usuario.
- ▶ **Diagramas:** proporciona una forma visual para que los usuarios creen y gestionen el

flujo de eventos o el funcionamiento interno de la aplicación, de forma gráfica.

Utilizando modelos UML y otros diagramas profesionales, los usuarios pueden visualizar los requisitos del proyecto en un diseño de modelado. También permite a los usuarios conectar un diagrama con elementos de trabajo de Azure DevOps, así como también admite la discusión sobre elementos de trabajo en medio de un grupo de usuarios en el proyecto del equipo conectado. La automatización de análisis adicional en este módulo proporciona la creación de historias de usuario y el escenario de prueba y la generación de pasos de prueba.

- ▶ **Editor de casos de uso:** los usuarios pueden crear diagramas de actividad automáticamente simplemente ingresando un caso de uso textual, lo que proporciona una verificación visual de la narrativa creada. Los usuarios pueden definir los casos de uso usando varias herramientas dentro del editor de casos de uso para crear una estructura consistente, escenarios correctos y enlaces dentro y hacia casos de uso externos; el diagrama de actividad se actualizará en función de los cambios realizados en la narrativa del caso de uso. Los usuarios pueden codificar con colores los diferentes escenarios dentro de un caso de uso para visualizar diferentes aspectos a fin de comprender y validar los escenarios. Los usuarios pueden ver todos los casos de uso junto con sus representaciones visuales (diagramas) en un proyecto de equipo.
- ▶ **Analizador de impacto:** permite resaltar cualquier elemento de trabajo y ver todos sus elementos de trabajo vinculados visualmente en una vista de mapa mental. La vista se puede ampliar para incluir elementos de trabajo dentro de un equipo o para incluir también dependencias entre equipos. Si los cambios son inminentes, los usuarios pueden capturar el impacto de los cambios, estimar el trabajo involucrado y crear un presupuesto por correo electrónico para enviar a las partes interesadas correspondientes. Los usuarios también pueden crear tareas de elementos de trabajo mientras están en esta vista para abordar los cambios.
- ▶ **Simulación y wireframes:** ayuda a los usuarios a organizar sus pensamientos visualmente, creando un flujo de pantallas o *wireframe*. Luego, cuando la

[illegible]

y CSV. Facilita la integración con varias soluciones de terceros, como Accompa, Jira, MS Sharepoint y Salesforce.

Algunas de sus características o funcionales principales son:

- ▶ **Ingeniería de requisitos** impulsada por procesos: la herramienta orquesta el proceso de desarrollo, gestionando toda la información relacionada con los requisitos (como requisitos, pruebas, solicitudes de cambio, riesgos, etc.), sus relaciones e interacciones con los usuarios. Ayuda a estandarizar y hacer cumplir los procesos definidos en toda la organización, formalizar una estructura de especificación de requisitos comunes y manejar los cambios a lo largo del ciclo de vida.
- ▶ **Cobertura del ciclo de vida** de los requisitos: brinda un soporte integral al proceso completo de requisitos, incluida la captura, el análisis, la especificación, la validación y verificación, la trazabilidad, la gestión y la reutilización de requisitos.
- ▶ **Orientación al usuario:** es una herramienta totalmente personalizable que permite a los administradores proporcionar a los usuarios vistas para realizar cada una de sus tareas específicas como: trazabilidad, análisis de impacto, creación de requisitos, validación, especificación, etc. El usuario accederá solo a la información que necesita para trabajar y con las opciones que necesitan.
- ▶ **Colaboración:** está destinada a ser una herramienta multiusuario que permite a varios usuarios trabajar con el mismo conjunto de requisitos al mismo tiempo, manteniendo un seguimiento e informes sobre cada cambio con un sistema completo de control de versiones.
- ▶ **Integraciones:** proporciona una plataforma de integración versátil que se puede utilizar para integrarse con herramientas de terceros, comerciales o propietarias, para extender las funciones de análisis de impacto de cambio a elementos fuera del alcance de la herramienta.

- **Reutilización:** permite compartir requisitos y otros artefactos como pruebas o casos de uso. Los usuarios pueden crear fácilmente familias de productos —que comparten un núcleo de requisitos comunes—, un conjunto de requisitos de estándares o incluso funciones completas junto con sus pruebas y casos de uso. Además, la reutilización va mucho más allá de copiar y pegar, ya que es posible mantener una referencia al proyecto original y actualizarse a través de los cambios realizados en los elementos originales (si es necesario).

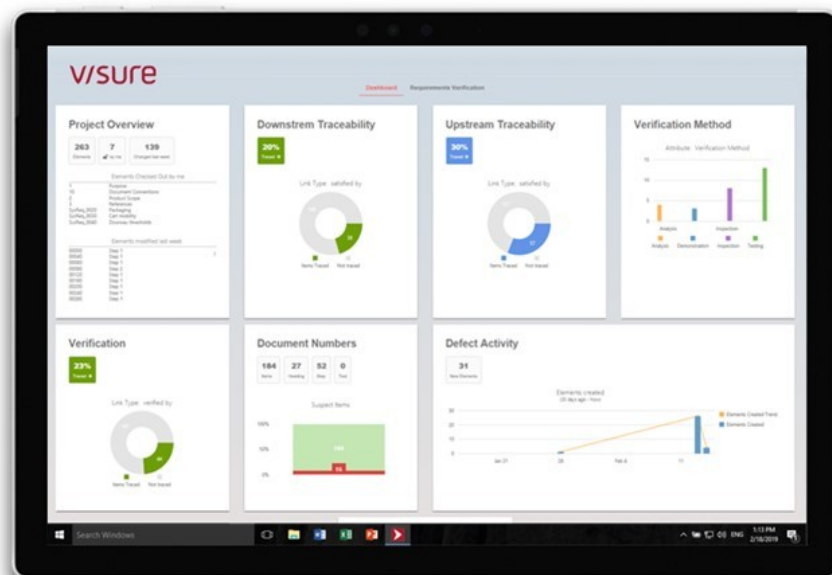


Figura 6. Vista del *dashboard* de Visure Requirements. Fuente: Capterra, s. f.

ReqSuite RM

Es un software de gestión de requisitos totalmente personalizable, intuitivo y alimentado por IA para lograr la máxima eficiencia y facilitación en los proyectos. Con ReqSuite RM, se **minimiza el riesgo** de demoras, errores, reelaboración costosa y esfuerzo manual adicional como resultado de la falta de soporte de herramientas o de un soporte inadecuado.

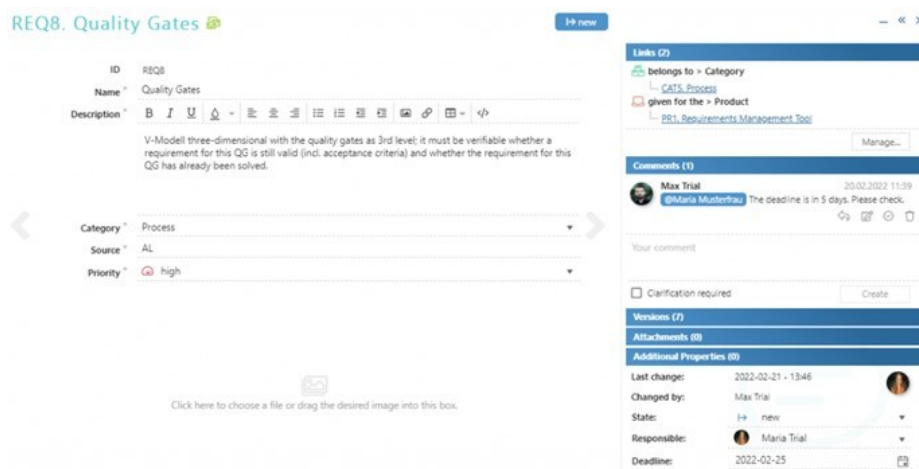


Figura 7. Vista del entorno para especificar requisitos con ReqSuite RM. Fuente: Osseno, 2022.

Algunas de sus características o funcionales principales son:

- **Transparencia y visión general:** permite tener una visión general completa, transparencia y trazabilidad con respecto a sus requisitos. Por lo tanto, siempre conoce el estado actual y mantiene el control sobre su proyecto.
- **Eficiencia y simplicidad:** automatiza tareas que requieren mucho tiempo, aumenta la reutilización y simplifica el intercambio con otras herramientas. Esto hace que todo el trabajo de su proyecto sea más simple, más eficiente y enfocado.
- **Calidad y conformidad:** mejora la calidad de sus requisitos y la conformidad estándar de sus proyectos. Como resultado, logra menos riesgo, reelaboración y necesidad de aclaraciones en sus proyectos.
- **Estructura y uniformidad:** crea estructuras y procesos claros para gestionar sus requisitos de forma centralizada y uniforme. Esto mejora la cooperación y reduce al mínimo las pérdidas de información.

2.5. Plataformas para historias de usuario y prototipos de interfaz gráfico

El **mapeo de historias de usuario** (*user story mapping*, USM) es una técnica utilizada en el descubrimiento de productos para esbozar un nuevo producto o una nueva característica para un producto existente (Patton, 2014). Esta técnica nos permite tener una visión compartida del desarrollo entre los diversos *stakeholders*, a la vez que tenemos un mapa global de la aplicación ordenado por versiones. Permite definir conjunto de historias que conformarán el mínimo producto viable (MPV) a sacar al mercado, que permita a los usuarios llevar a cabo todo el flujo de una manera básica, así como una primera priorización de nuevas funcionalidades ordenadas por versiones y alineada a la funcionalidad que percibe el usuario.

Las principales plataformas para historias de usuario son:



Tabla 3. Principales plataformas que soportan historias de usuario. Fuente: elaboración propia.

Easy Agile

Es un software de historias de usuario que permite a los desarrolladores de productos crear mapas de **historias simples, pero colaborativos** en Jira. El software permite a los usuarios mapear los viajes de los usuarios junto con tableros ágiles en Jira. Los desarrolladores de productos pueden agregar historias nuevas o existentes al *story map* y priorizarlas según el valor de sus usuarios. Es ideal para dividir su historia de usuario en secciones **más pequeñas y manejables** porque tiene una función de planificación de sprint. Le da al propietario del producto o al maestro de scrum una idea más clara de la línea de tiempo del producto para evitar demoras. También tiene una función de planificación de versiones que brinda a los desarrolladores de productos la libertad de experimentar con nuevas funciones sin preocuparse por dañar la versión o el modelo activo (Easy Agile, 2022).

Agile User Story Map para Jira

Es una herramienta de mapeo de historias de usuarios que admite el **refinamiento** de la cartera de productos, lo que permite evaluar rápidamente las tareas que deben completarse para que pueda realizar un seguimiento de los miembros del equipo. También permite a los usuarios ver las dependencias para **priorizar** tareas y promover la responsabilidad. El software utiliza una línea de tiempo similar a un diagrama de Gantt, lo que le brinda una **vista panorámica** de su proyecto, para que sepa si necesita más recursos humanos o si ajusta su presupuesto para cumplir con los plazos (Atlassian Marketplace, 2022).

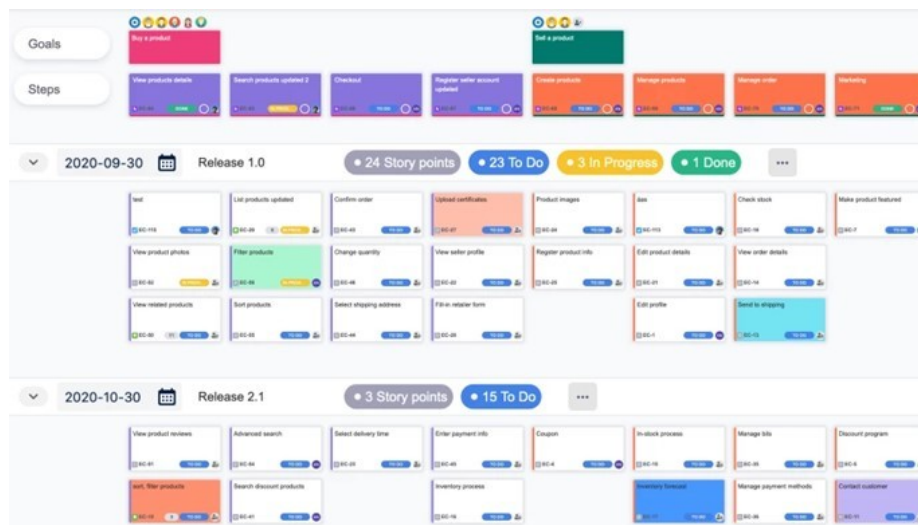


Figura 8. Vista del Agile User Story Map para Jira. Fuente: Atlassian Marketplace, s. f.

Avion

Es una herramienta de mapeo de historias de usuario para el desarrollo ágil de software que le permite visualizar su producto mediante la **creación de recorridos** de usuario detallados. Los usuarios adjuntan archivos y documentos a las historias de usuario, lo que asegura que otros miembros del equipo tengan acceso a documentos críticos. Brinda al equipo **visibilidad** no solo de la hoja de ruta de la historia del usuario, sino también de todos los cambios en los criterios, lo que garantiza que todos estén en la misma página durante todo el proceso de desarrollo del producto. Si maneja con frecuencia proyectos de desarrollo de productos confidenciales, esta plataforma incorpora un control de acceso basado en roles. Esta función permite a los administradores definir el nivel de acceso que tienen los miembros del equipo según sus funciones. Con la función de control de acceso, puede traer talento externo o autónomos para proyectos específicos sin preocuparse de que accedan a información confidencial que no es esencial para sus funciones. Se integra con otras herramientas de desarrollo de productos y software como Jira Cloud, Jira Server, Trello, GitHub, Zenhub, etc. (Avion, s. f.).

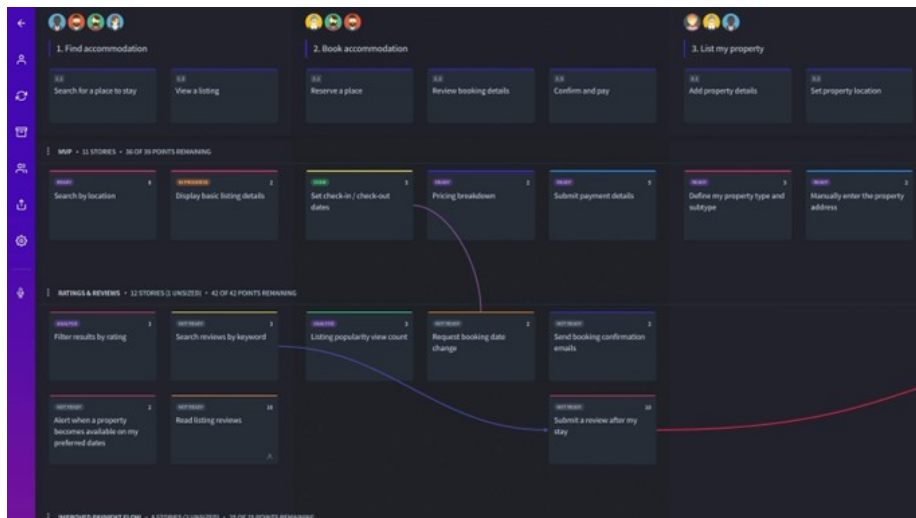


Figura 9. Vista de flujos en USM con Avion. Fuente: Avion, s. f.

Cardboard

Es una **pizarra en línea** que su equipo de desarrollo puede usar para crear historias de usuarios durante el desarrollo del producto. Permite invitar a miembros remotos del equipo a colaborar en tiempo real, lo que garantiza que escribir historias de usuarios se convierta en un **esfuerzo de equipo**. Tiene una funcionalidad de arrastrar y soltar, lo que lo hace fácil de usar y reduce la curva de aprendizaje para los nuevos usuarios. Promueve la responsabilidad porque tiene una función de carriles y divisores que pueden ayudar a los usuarios a distinguir visualmente quién es responsable de cada paso del proyecto, dándoles una mejor idea de con quién colaborar para los pasos o tareas interconectados. Esta característica también puede ayudar a promover la responsabilidad porque los usuarios pueden visualizar cómo los retrasos en ciertos pasos pueden afectar el progreso general de un proyecto. Se integra con otras herramientas de desarrollo de software como Trello, Azure DevOps, VersionOne y Confluence (CardBoard, s. f.).

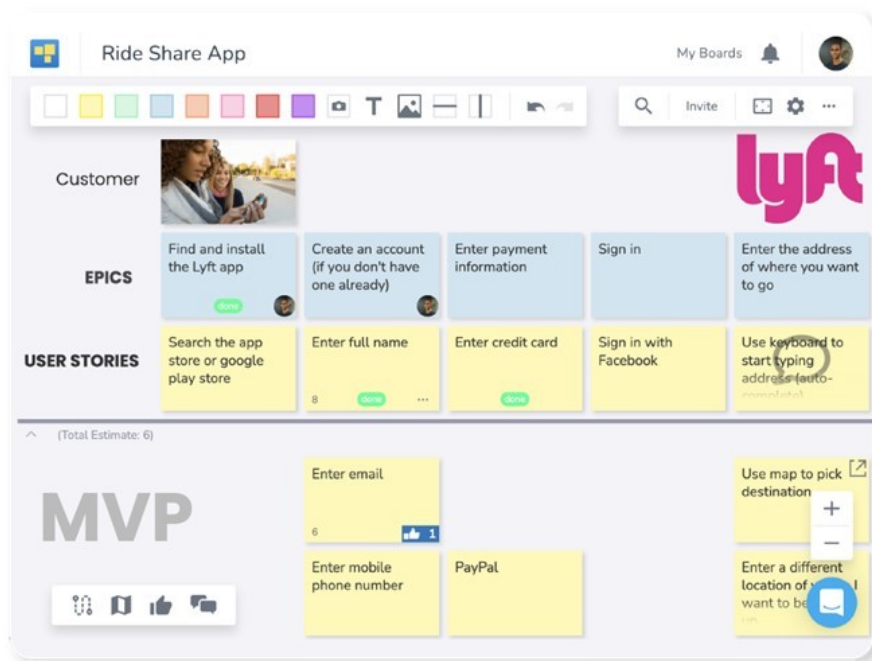


Figura 10. Vista de USM con CardBoard. Fuente: CardBoard, s. f.

Craft.io

Es un software de **administración de productos** en donde los mismos responsables (*product owners*) pueden usarlo para administrar todas las etapas del desarrollo del producto. Esta ágil plataforma de gestión de proyectos permite a los usuarios descubrir nuevas ideas, colaborar y crear prototipos de productos. Se sincroniza automáticamente con Google Drive y Jira para que los desarrolladores puedan concentrarse en sus tareas asignadas en lugar de actualizar documentos constantemente. Incorpora una función de planificación de capacidad que permite evaluar la disponibilidad de recursos y la asignación de miembros del equipo, lo que le brinda una mejor idea sobre cómo redistribuir los recursos para nuevos proyectos.

Puede usar el software para simular cómo el ajuste de los recursos afectará la entrega del proyecto para que pueda adaptarse rápidamente a los cambios inesperados. Se integra con otras herramientas de desarrollo como GitLab, Google Workspace, Dropbox, Jira y Microsoft Active Directory (Craft.io, s. f.).

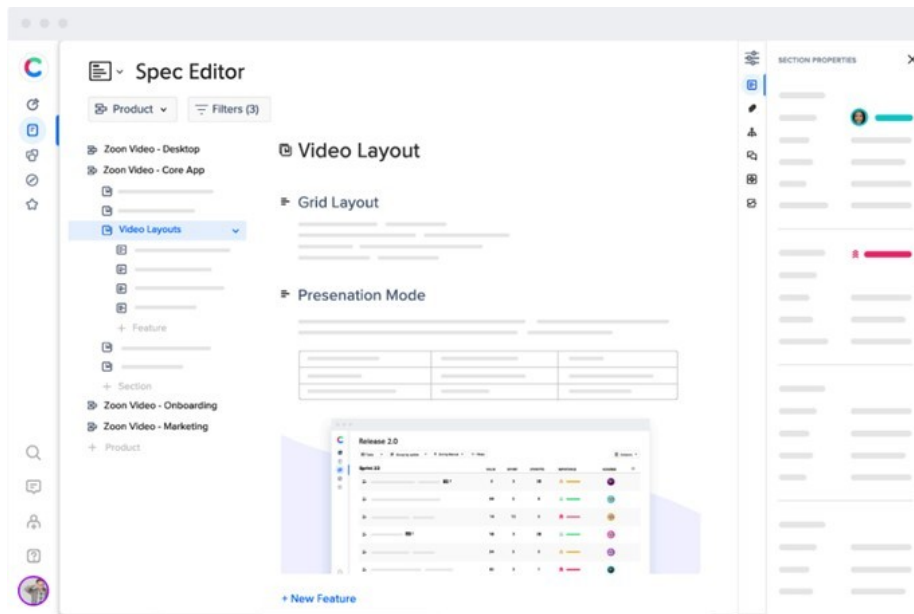


Figura 11. Vista del editor de especificaciones de Craft.io. Fuente: Craft.io, s. f.

Featmap

Es un software de historia de usuario de **código abierto** para ayudar a imaginar qué acciones podría tomar el usuario ideal al interactuar con su producto. Permite asignar personajes de usuario a los miembros del equipo para capturar los comportamientos y las necesidades de sus usuarios finales, lo que le permite crear productos centrados en el usuario. También permite compartir la visión de su producto con los miembros del equipo y las partes interesadas con cuentas de invitados ilimitadas (solo lectura). Como software de código abierto, puede agregar o eliminar funciones según sus necesidades para tener un software de historia de usuario con todas las funciones críticas y ninguna de las adicionales que no necesita. Puede alojar el software en su propio servidor para ahorrar dinero y aumentar el nivel de seguridad

de sus productos (Featmap, s. f.).

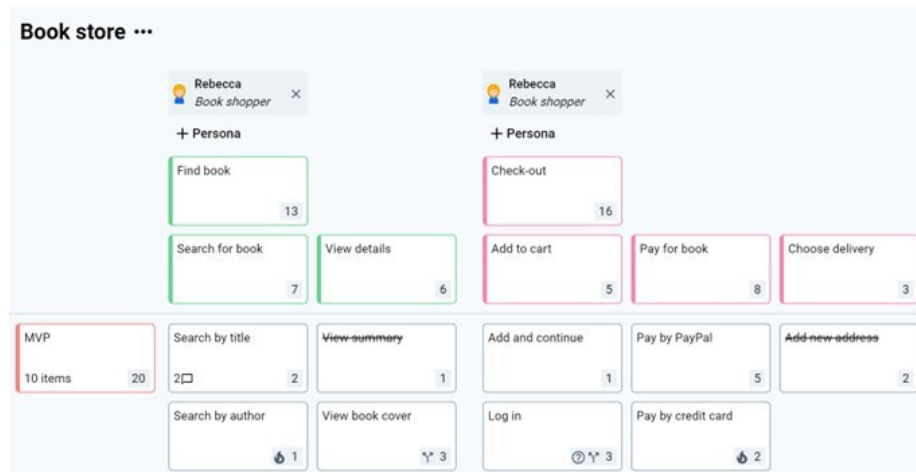


Figura 12. Vista de USM en Featmap. Fuente: Featmap, s. f.

FeatureMap

Es un software de mapeo de historias de usuario para **estructurar el contenido y establecer prioridades**. Puede usar el software para planificar varios pasos por adelantado, lo que le permite anticipar problemas y planificar contingencias. También permite dividir tareas grandes en tareas más pequeñas para que sean más fáciles de completar y evitar abrumar a los miembros del equipo. Es la mejor opción para los usuarios que prefieren una herramienta local que funcione bien incluso en áreas donde la conectividad a Internet es inestable. Ser una herramienta local también significa que la **falta de conexión a internet** no retrasará el desarrollo del producto porque los usuarios aún pueden acceder a archivos y documentos cruciales (FeatureMap, s. f.).

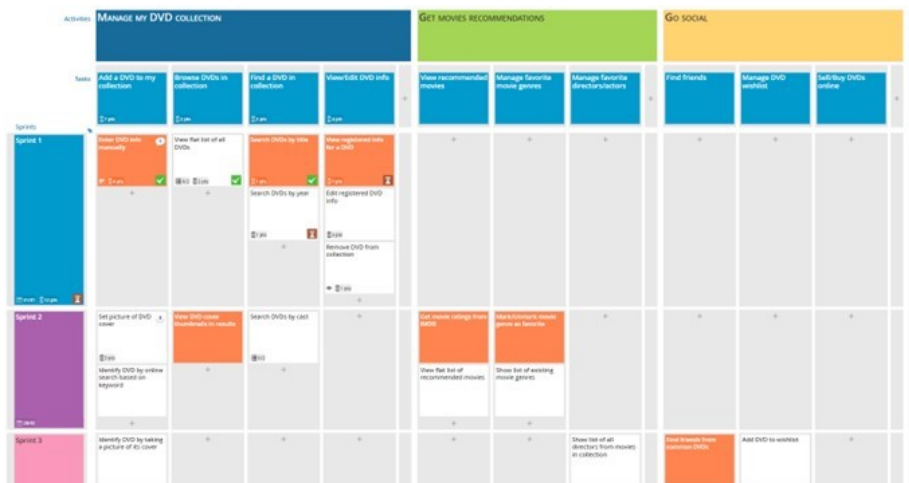
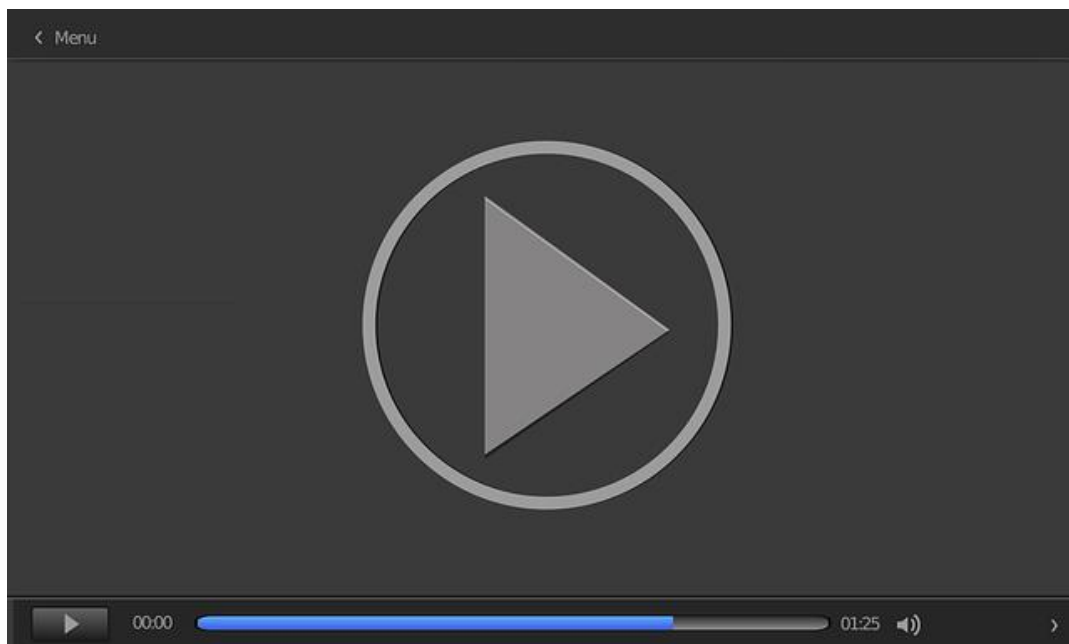


Figura 13. Vista de USM en FeatureMap. Fuente: Feature Map, s. f.

En este vídeo, *Gestionando mapas de historias de usuario con FeatureMap*, aprenderemos a utilizar FeatureMap para crear mapas de historias de usuario y poder priorizar funcionalidades dentro del *roadmap* de un producto.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=e3c41ca2-f50c->

[436c-952f-aec300cd97e8](#)

Miro

Es una **pizarra digital** que podemos usar en varios proyectos, como lluvia de ideas, seminarios web, reuniones remotas y flujos de trabajo ágiles. Tiene funciones de videoconferencia y uso compartido de pantalla para que las reuniones en línea sean más atractivas e informativas. También tiene muchas **plantillas prediseñadas** que puede usar en casi cualquier tipo de proyecto, lo que ayuda a lanzar proyectos rápidamente. Es una de las principales plataformas de historias de usuario para trabajar en múltiples proyectos porque permite una **cantidad ilimitada** de miembros del equipo. Se integra con otras herramientas de productividad y gestión de productos como Jira Cards, Google Drive, Slack, Google Calendar y Microsoft Active Directory (Miro, 2022).

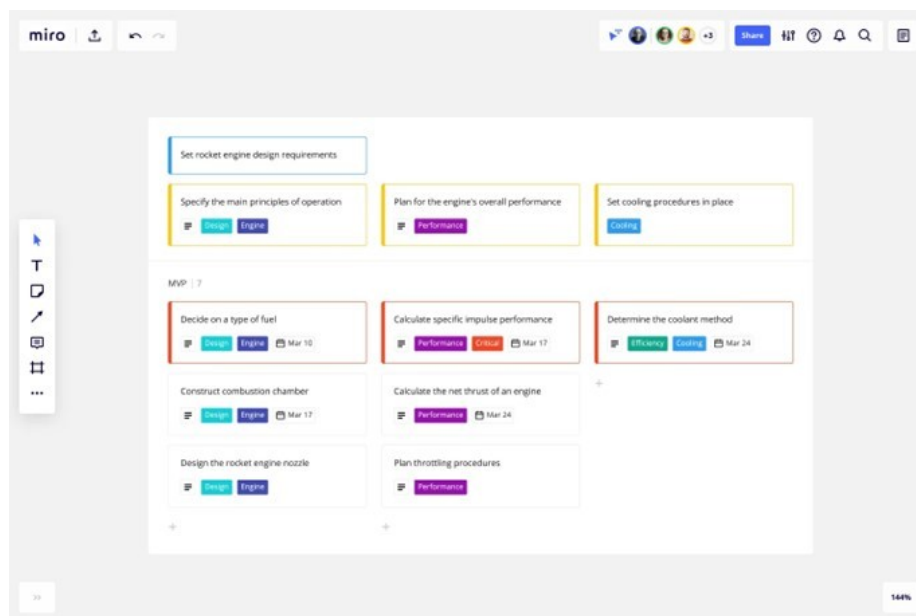


Figura 14. Vista de la plantilla de historias de usuario de Miro. Fuente: Miro, s. f.

Productboard

Es una plataforma de **gestión de productos** para ayudar a los *product owners* a

comprender las necesidades de sus clientes para su consideración durante el desarrollo del producto. Tiene hojas de ruta personalizables con opciones de filtrado que permiten a los usuarios **identificar fácilmente** los objetivos y logros del proyecto. También permite a los usuarios adaptar hojas de ruta a diferentes audiencias, como usuarios finales y partes interesadas, lo que las hace más fáciles de entender. Permite a los usuarios finales asignar puntajes a las características del producto, para que los desarrolladores sepan en qué características concentrarse y cuáles necesitan desarrollar más.

También puede usarse para **identificar características clave** que los usuarios finales consideran críticas para garantizar que las incluya en productos futuros. Se integra con otras herramientas de colaboración y desarrollo de productos como Slack, Microsoft Teams, Trello, Salesforce y Zendesk (Productboard, s. f.).

StoriesOnBoard

Otra herramienta para **identificar y registrar** las necesidades de sus visitantes mediante historias de usuario con multitud de plantillas personalizables. Permite asignar roles, comportamientos y necesidades a los miembros del equipo, lo que le brinda una mejor idea de cómo los futuros usuarios usarán y se beneficiarán del producto. Permite la integración de herramientas como Google Chrome, Zapier, Trello, GitHub, G Suite y otras (StoriesOnBoard, s. f.).

Trello

Llena de listas y tarjetas, esta herramienta parece simple, pero tiene **todas las funciones** que necesita para el mapeo de historias. Puede publicar comentarios para brindar retroalimentación instantánea al equipo, agregar fechas de vencimiento, listas de verificación, etiquetas y cargar archivos usando Google Drive, OneDrive, Box y Dropbox. Tiene un sistema de notificación sólido, que avisa a través de notificaciones de escritorio, notificaciones automáticas móviles y correo electrónico. Trello también

tiene aplicaciones para iPhone, iPad, teléfonos Android, tabletas, relojes, tabletas Kindle Fire y Windows, lo que garantiza que su equipo se mantenga actualizado incluso mientras está en movimiento (Trello, 2022).

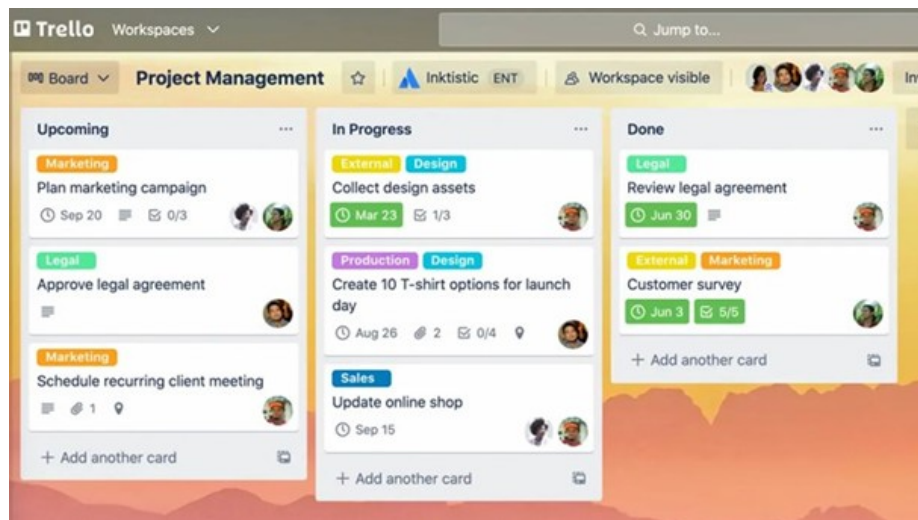
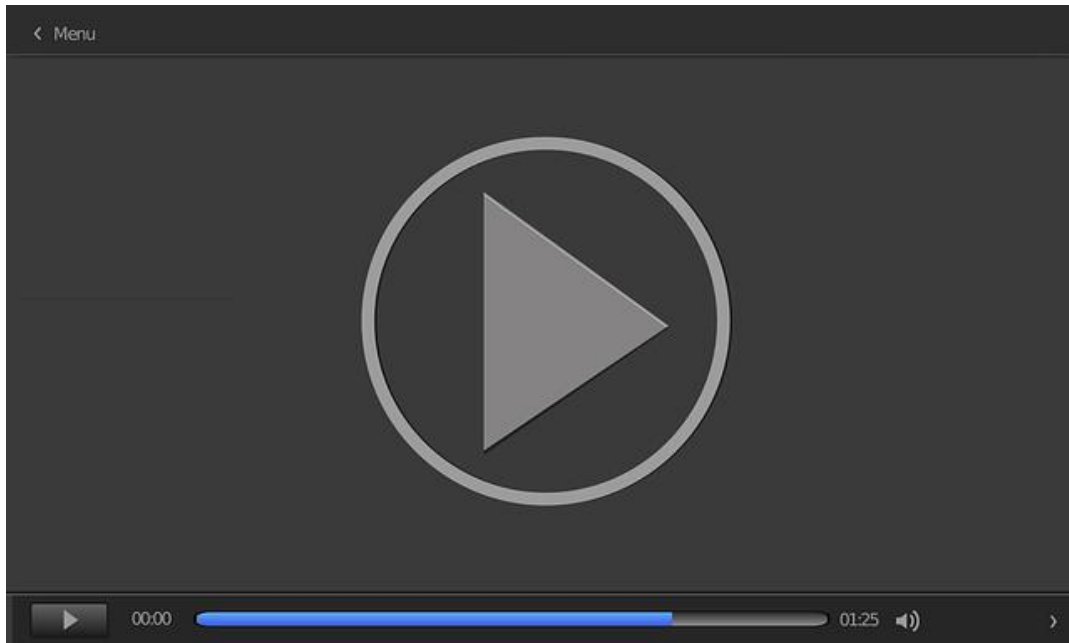


Figura 15. Vista de tareas e historias de usuario en Trello. Fuente: Trello, s. f.

En este vídeo, *Gestionando historias de usuario y tareas con Trello*, aprenderemos a utilizar Trello para gestionar tareas o requisitos de un proyecto de forma visual y colaborativa.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=911a6dbb-9fcf-47d3-939b-aec300cd96b6>

Wireframes

Por otro lado, existen plataformas que ayudan a generar **maquetas o prototipos** de interfaz de usuario. Aunque muchas veces se incluyen dentro de la parte de diseño de la experiencia del usuario, en realidad son también una gran fuente de información para poder obtener requisitos en los que nadie piensa de otra manera y que salen a la luz cuando se muestra un ejemplo rudimentario de la interacción.

Un **wireframe** es un modelo o marco esquelético que describe el diseño básico y las

funciones de una interfaz de usuario (como un sitio web o una aplicación). El objetivo de un *wireframe* es **comunicar rápida y fácilmente** el contenido de la página, su estructura y diseño y las funciones de la aplicación. En ocasiones se confunde con las maquetas (*mockups*), que son estructuras estáticas que incluyen detalles más estilísticos y visuales de la interfaz de usuario para presentar un modelo realista de cómo se verá la página o aplicación final.

Las principales plataformas de apoyo a la creación de prototipos que pueden servirnos como base para extraer requisitos de nuestro sistema son:

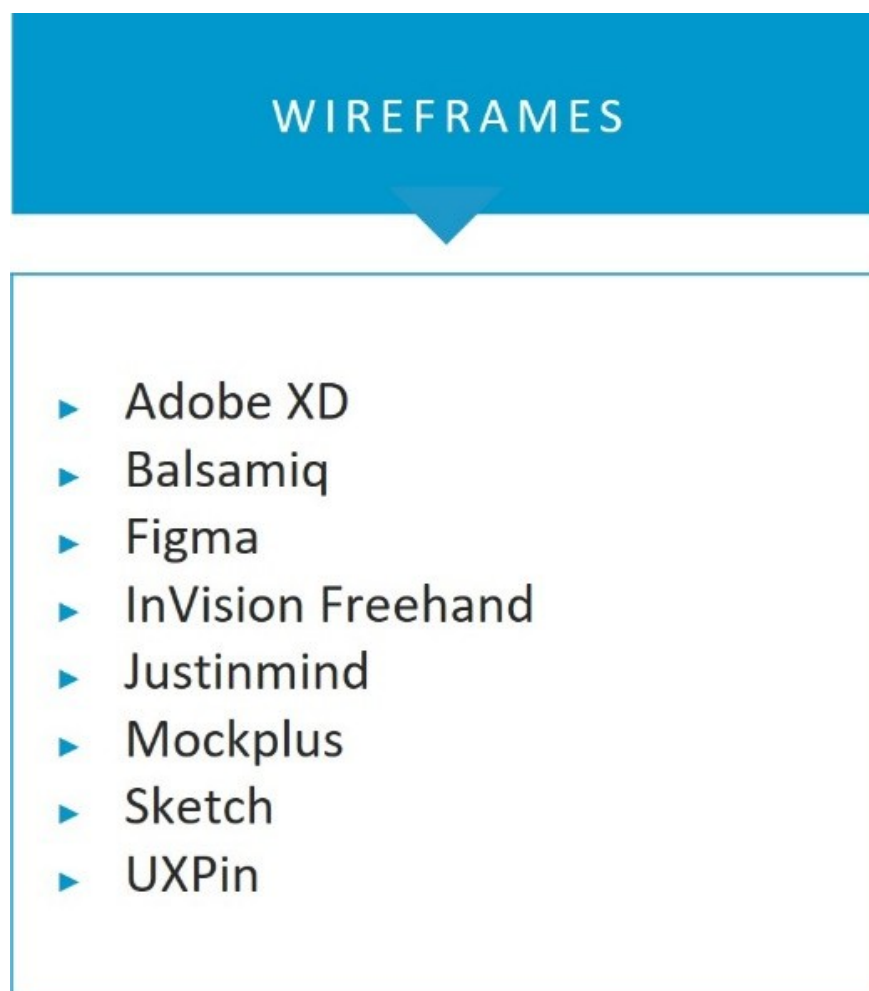


Tabla 4. Principales plataformas para *wireframes*. Fuente: elaboración propia.

Adobe XD

Esta aplicación destaca como una **opción fácil** de usar para proyectos de diseño de interfaces. Todo, desde la creación de esquemas hasta la creación de prototipos básicos, puede suceder dentro de XD. Y, en comparación con otras herramientas de Adobe ricas en funciones, la interfaz mínima de XD es una bocanada de aire fresco. En el momento en que se abre la aplicación, un conjunto de cajas de luz de incorporación interactivas lo ayuda a orientarse rápidamente con las herramientas de diseño al alcance de su mano. La interfaz limpia y la incorporación rápida son **invaluables** para los principiantes que desean usar una herramienta profesional de estructura alámbrica sin perder demasiado tiempo tratando de aprender el software. Las herramientas básicas de diseño de vectores de *wireframe* se encuentran fácilmente en la barra de navegación de la izquierda. Los elementos de la interfaz de usuario no están incluidos, pero puede encontrar una variedad de opciones gratuitas con una búsqueda rápida en Google (Adobe, s. f.).

Balsamiq

Es una herramienta rápida de creación de tramas de interfaz de usuario de baja fidelidad que **reproduce la experiencia** de dibujar en un bloc de notas o pizarra, pero usando una computadora. Realmente obliga a concentrarte en la **estructura y el contenido**, lo que evita largas discusiones sobre colores y detalles que deberían venir más adelante en el proceso. Puede usarlo para generar bocetos digitales de su idea o concepto para una aplicación o sitio web, para facilitar la discusión y la comprensión antes de escribir cualquier código.



Figura 16. Vista de un *wireframe* creado con Balsamiq. Fuente: Balsamiq, s. f.

Figma

Destaca como una poderosa plataforma de *wireframes* **basada en la nube** con un proceso sencillo y rápido. Tendrá que diseñar sus propios componentes de interfaz de usuario (o agregarlos desde un kit prediseñado por separado), pero es fácil crear mesas de trabajo, agregar formas y texto e incluso agregar algunos prototipos para que pueda tener una mejor idea del flujo. El panel de la izquierda ayuda a mantener todo organizado, desde capas hasta mesas de trabajo y páginas separadas dentro del mismo documento de diseño. Para un diseño receptivo, se puede aplicar una superposición de columnas (si se prefiere, el sistema de cuadrícula Bootstrap) o se puede usar las restricciones de Figma, que le indican a cada elemento cómo debe

responder cuando se cambia el tamaño del diseño para imitar varios tamaños de pantalla. Otra área en la que destaca es en sus oportunidades de colaboración en equipo. Debido a que es una aplicación basada en la web, varios miembros del equipo pueden iniciar sesión y acceder al archivo de diseño simultáneamente, ya sea para modificar el diseño o agregar contenido. La característica más atractiva es la capacidad de un equipo de **mantener una conversación** completa dentro del archivo de diseño, dejando comentarios similares a notas adhesivas que otros compañeros de equipo pueden responder o marcar como completos. Cuando llega el momento de que su equipo de desarrollo se haga cargo, los desarrolladores pueden tomar el código CSS del interior del archivo de diseño y exportar elementos individuales para usarlos según sea necesario (Figma, s. f.).

InVision Freehand

Es una aplicación abierta para crear o marcar *wireframes* y maquetas en una pizarra donde todos pueden contribuir y revisar. Invite a su equipo a una colaboración de *wireframes*, donde puede agregar imágenes y formas básicas o generar la maqueta con el cursor del mouse. Dado que es una aplicación en la nube, varios diseñadores y partes interesadas pueden iniciar sesión y escribir al mismo tiempo. Por sí mismo, es una buena herramienta de *wireframes*, aunque simplista, que se parece a cualquier otra aplicación de pizarra. Pero dado que ofrece sincronización en tiempo real con Sketch y Photoshop, su equipo puede optar por dibujar y anotar directamente en prototipos pulidos, lo que facilita mucho la etapa de diseño de la iteración. Y, dado que es parte de la *suite* InVision, sus bocetos de colaboración se almacenan junto con prototipos de trabajo y tableros de ideas para una referencia rápida a medida que avanza en su proyecto de diseño. Funciona bien para los diseñadores que desean colaborar con los miembros del equipo en un boceto de *wireframes* y/o comentar sobre prototipos más refinados (InVision, 2021).

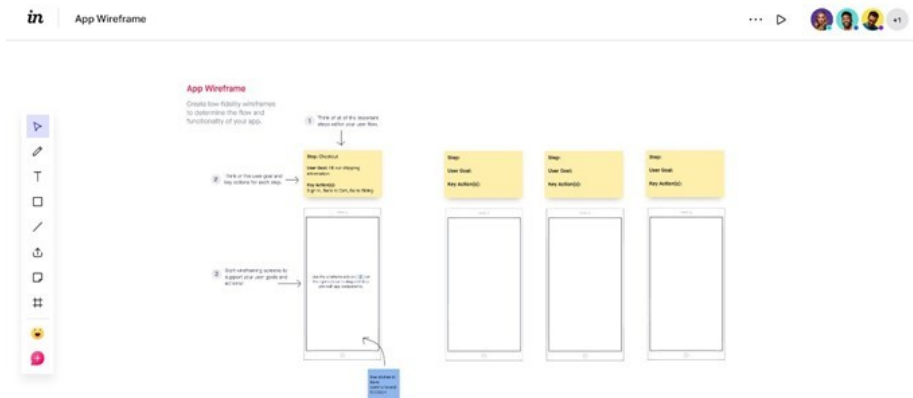


Figura 17. Vista de *wireframe* de la aplicación Freehand. Fuente: InVisión, s. f.

Justinmind

No solo es fácil de aprender y divertido de usar, también mantiene un **enfoque en la capacitación** para crear un *wireframe* que pueda probarse como un prototipo funcional desde el principio. Lo hace al incluir algo que otras aplicaciones simplemente no se han molestado en desarrollar: elementos prototipo interactivos (entradas de texto, botones de radio, menús desplegables, etc.). Incluso a un nivel básico de *wireframe*, un menú desplegable funcional requeriría tres o más pantallas para configurarse en una herramienta como Sketch o Figma. Justinmind te permite agregarlo a tu *wireframe* con un solo clic. De hecho, la facilidad con la que puede crear y compartir *wireframes* realistas puede ahorrarle horas o días de trabajo en cualquier proyecto. Esto convierte a Justinmind en una gran herramienta para cualquier persona que quiera obtener comentarios reales y detallados sobre los diseños mucho antes en la etapa de estructura metálica (con un esfuerzo mínimo). Aparte del contenido de los elementos en sí, el diseño de la herramienta es muy fácil de entender: todos los elementos de diseño están en el lado izquierdo; los elementos de organización (como carpetas, lista de pantallas, etc.) están a la derecha (Justinmind, 2019).

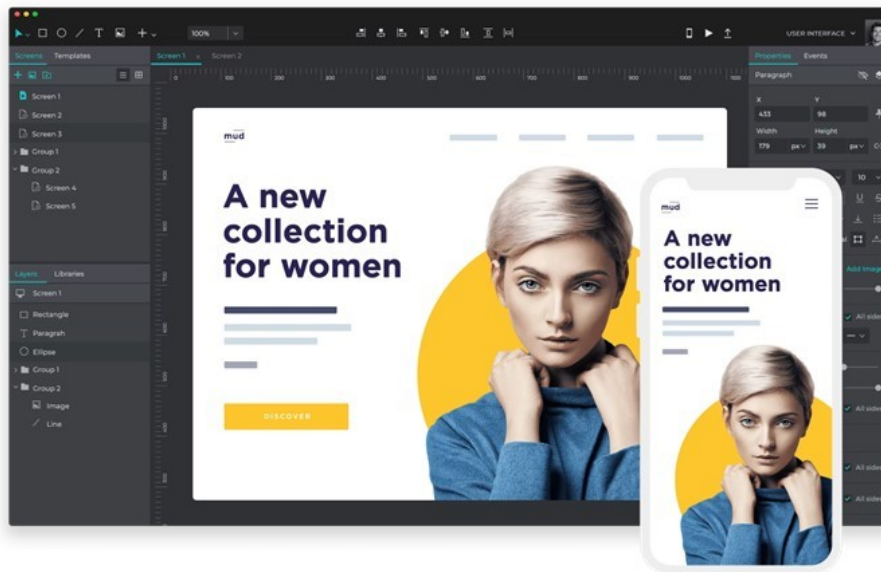


Figura 18. Vista del editor de pantallas de Justinmind. Fuente: Justinmind, s. f.

Mockplus

Además de poder modelar un *wireframe* y agregar notas y documentación a cada elemento de su diseño, el área de proyecto también tiene espacio para que crear y guardar documentación en un editor de texto, guardar la guía de estilo (para que todos estén en la misma página) e incluso crear y asignar tareas al equipo. Dado que las tareas pueden vincularse a un proyecto (e incluso reducirse a un diseño individual dentro del proyecto), realmente puede marcar su **proceso de retroalimentación** de revisión a la vez que evoluciona con el desarrollo (Mockplus, 2022).

Sketch

Desde su lanzamiento en 2010, se ha mantenido en un lugar destacado como una herramienta de diseño vectorial poderosa pero liviana para usuarios de macOS. Por sí mismo, se puede usar para **cualquier cosa**, desde *wireframes* hasta una interfaz de usuario moderna y diseño de vectores de iconos y algunos diseños de interacción. Gracias a esta simplicidad, Sketch se puede usar para crear *wireframes* rápidamente

con una combinación de mesas de trabajo y formas de diseño vectorial. Cuando se descarga la aplicación Sketch en una Mac, se puede notar que no hay componentes de interfaz de usuario integrados. Si bien podría diseñar sus propios componentes para usarlos como parte de su proceso de estructura alámbrica, existe una gran **comunidad** en línea de diseñadores que han creado y compartido muchos kits de diseño de *wireframe* gratuitos. Con un solo clic para descargar, tendrá una gran cantidad de botones, íconos y otros elementos de diseño que se pueden usar dentro de su archivo de Sketch. Como la mayoría de las aplicaciones de escritorio, Sketch hace que la colaboración sea **un poco complicada**, ya que no hay forma de permitir que varios diseñadores trabajen simultáneamente dentro de un diseño (Sketch, 2022).

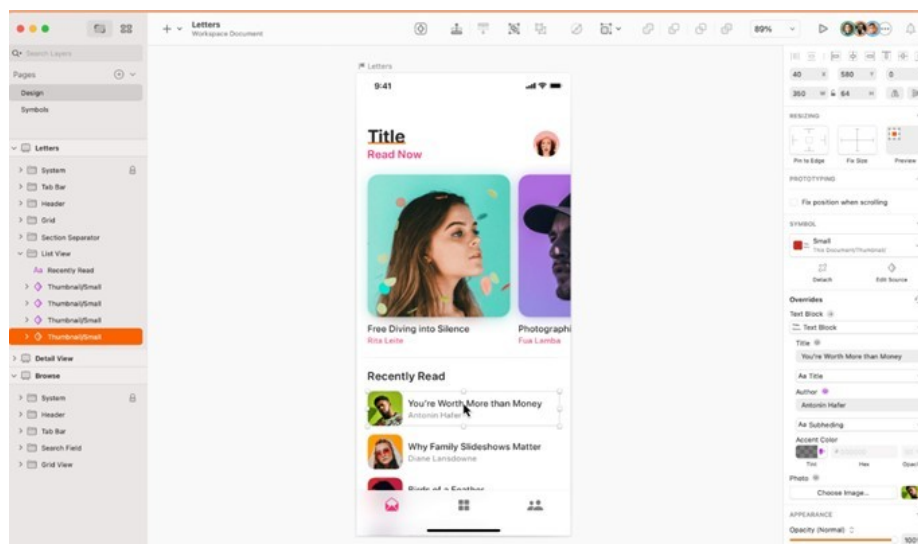


Figura 19. Vista del editor de pantallas de Sketch. Fuente: Sketch, s. f.

UXPin

Es una de las primeras herramientas recomendadas para cualquier persona que esté **aprendiendo** a hacer *wireframes*. Si bien su rico conjunto de funciones puede ser un poco difícil de aprender para un nuevo diseñador, el esfuerzo dedicado a aprender la herramienta definitivamente vale la pena. Con UXPin, se pueden comenzar los

esquemas con una **biblioteca integrada** de elementos de la interfaz de usuario que puede arrastrar y soltar directamente en su lienzo. El beneficio de esta *wireframe* de fidelidad ligeramente superior es que puede realizar un ajuste más fino del flujo y la funcionalidad sin perder mucho tiempo rediseñando los componentes de la pantalla. Y, dado que es posible leer archivos de Sketch y Photoshop, siempre se puede usar esas herramientas para convertir sus esquemas básicos en prototipos de alta fidelidad antes de importar el diseño pulido nuevamente a UXPin para aprovechar las otras características de la herramienta: agregar interacciones, presentar a un equipo y entregando las especificaciones de diseño a un desarrollador.

A pesar de las excelentes características de diseño de interfaz y *wireframes*, son las capacidades de presentación y documentación **en vivo** las que hacen que destaque. Si bien muchas aplicaciones le permiten obtener CSS, HTML o incluso JSON de su diseño, UXPin facilita presentar un prototipo funcional de su diseño, recopilar comentarios e incluir especificaciones/documentación, todo a la vez a través del modo de vista previa (UXPin, 2022).

2.6. Referencias bibliográficas

Adobe. (s. f.). *Adobe XD*. <https://www.adobe.com/products/xd.html>

Atlassian Marketplace. (s. f.). *Agile User Story Map & Product Roadmap for Jira*. <https://marketplace.atlassian.com/apps/1222309/agile-user-story-mapping-board-for-jira?tab=overview&hosting=cloud>

Avion. (s. f.). User story mapping for agile product teams. <https://www.avion.io/>

Balsamiq. (s. f.). *Balsamiq Wireframes*. <https://balsamiq.com/wireframes/>

Capterra. (s. f.) *Visure Requirements*. <https://www.capterra.co/software/146581/visure-requirements>

CardBoard. (s. f.). *Simple and Easy to use*. <https://cardboardit.com/tours/>

Chemuturi, M. (2012). *Requirements engineering and management for software development projects*. Springer

Chrissis, M. B., Konrad, M., y Shrum, S. (2011). *CMMI for development: guidelines for process integration and product improvement*. Pearson Education.

Carrillo de Gea, J. M., Ebert, C., Hosni, M., Vizcaíno, A., Nicolás, J., y Fernández-Alemán, J. L. (2021). Requirements engineering tools: An evaluation. *IEEE*, 38(3), 17-24.

Craft.io. (s. f.). *Contextualized spec creation and management*. <https://craft.io/product/spec-editor.html>

Easy Agile. (s. f.). *Easy Agile User Story Maps*. <https://www.easyagile.com/products/user-story-maps/>

Erich, N. (2020, diciembre 15). *Craft.io - Best product management software*.

Craft.io. <https://craft.io/>

Featmap. (s. f.) *Build products without waste.* <https://www.featmap.com>

FeatureMap. (s. f.). *Convierta su Product Backlog en un Story Map.* <https://www.featuremap.co/>

Figma. (s. f.). *Nothing great is made alone.* <https://www.figma.com/>

InVision. (2021, junio 15). *Freehand: Inspiring collaboration for all the work you do* [Video]. YouTube. <https://www.youtube.com/watch?v=xbkDA7kPgYA>

Invision. (s. f.). *App Wireframe Template.* <https://www.invisionapp.com/freehand/templates/detail/app-wireframe-template>

Justinmind. (s. f.). *Design and prototyping tool for web and mobile apps.* <https://www.justinmind.com/>

IEEE Standard Glossary of Software Engineering Terminology. (2002). *IEEE*, 1-84. <https://ieeexplore.ieee.org/document/159342>

International Organization for Standardization. (2009, diciembre). *ISO/IEC TR 24766:2009. Information technology — Systems and software engineering — Guide for requirements engineering tool capabilities.* <https://www.iso.org/standard/51041.html>

Lawrence, B., Wiegers, K., y Ebert, C. (2001). *The top risk of requirements engineering.* *IEEE*, 18(6), 62-63. <https://ieeexplore.ieee.org/document/965804>

Mockplus. (s. f.). *Design Faster. Collaborate Better.* <https://www.mockplus.com/>

Modern Requirements. (2019). *Requirements Management Tools built for Azure DevOps.* <https://www.modernrequirements.com/>

Miro. (s. f.). *The visual collaboration platform for every team.* <https://miro.com/>

Miro. (s. f.). *User Story Map Template*. <https://miro.com/templates/user-story-map/?r=prd-uss>

Osseno. (s. f.). *The Smart Tool for Requirements Management*. <https://www.osseno.com/en/requirements-management-tool/>

Página de [Jama Software](#).

Patton, J. (2014). *User story mapping: discover the whole story, build the right product*. O'Reilly Media.

Productboard. (s. f.). *The customer-centric product management platform*. <https://www.productboard.com/>

ReqPro. (s. f.). *IBM Rational DOORS Next Generation - List of artifacts*. <http://reqpro.com/products/doors-next-generation>

Sommerville, I., Sawyer, P., (1997). *Requirements Engineering: A Good Practice Guide*. John Wiley & Sons.

Sketch. (s. f.). *Everything you need to*. <https://www.sketch.com/>

Software informer. (2022). *IBM Rational DOORS 9.30*. <https://ibm-rational-doors.software.informer.com/>

StoriesOnBoard. (s. f.). *Realize your product vision sprint by sprint*. <https://storiesonboard.com/>

Trello. (s. f.). *Aprende los aspectos básicos del tablero de Trello*. <https://trello.com/es/guide/trello-101>

Trello. (s. f.). *Trello siempre hace avanzar*. <https://trello.com/>

UXPin. (s. f.). *Design powered with code*. <https://www.uxpin.com/>

Visual Studio. (2019, enero 18). *Modern Requirements4DevOps*.
<https://marketplace.visualstudio.com/items?itemName=edevtech-mr.inteGREAT4TFS>

User story mapping: discover the whole story, build the right product

Patton, J. (2014). *User story mapping: discover the whole story, build the right product*. O'Reilly Media.

Libro que muestra cómo los mapas de historias modificables le permiten a su equipo mantener mejores conversaciones sobre el proyecto a lo largo del proceso de desarrollo. Su equipo aprenderá a salir con una comprensión compartida de lo que está intentando construir y por qué.

User stories applied: For agile software development

Cohn, M. (2004). *User stories applied: For agile software development*. Addison-Wesley Educational.

Libro que proporciona un plan de principio a fin para escribir historias de usuario e integrarlas en su ciclo de vida de desarrollo. Aprenderá qué hace que una gran historia de usuario y qué hace que sea mala. Luego, una vez que haya compilado sus historias de usuario, muestra cómo organizarlas, priorizarlas y usarlas.

Requirements management: The interface between requirements development and all other systems engineering processes

Hood, C. (2008). *Requirements management: The interface between requirements development and all other systems engineering processes*. Springer.

Este libro se centra en las interfaces de la gestión de requisitos con las demás disciplinas de la ingeniería de sistemas. Proporciona una introducción a la gestión y desarrollo de requisitos, así como un resumen de la aplicación hacia la ingeniería de sistemas. Al usarlos, se muestra cómo la gestión de requisitos puede apoyar y optimizar las otras disciplinas del proyecto.

Requirements engineering and management for software development projects.

Chemuturi, M. (2012). *Requirements engineering and management for software development projects*. Springer.

El libro detalla una taxonomía útil para la especificación, clasificación y documentación de los requisitos funcionales del proyecto. Aunque el autor diseñó el libro para que fuera un libro de texto útil para los estudiantes, también debería ser útil para los gerentes de proyectos que necesitan comprender que los resultados de calidad no ocurren simplemente; deben diseñarse, y un diseño exitoso se basa en un análisis completo y detallado de los requisitos y especificaciones.

1. Marca la característica que no es correcta dentro de las plataformas de gestión de requisitos:

- A. Análisis del impacto a un cambio de requisitos.
- B. Administración de las relaciones entre los requisitos.
- C. Gestión de los cambios en los requisitos.
- D. Generación de casos de prueba.

2. Marca aquella herramienta que no cubre la gestión de requisitos:

- A. Jama Software.
- B. IBM Rational Rose.
- C. ReqSuite RM.
- D. Modern Requirements4DevOps.

3. Marca aquella definición de requisito que no encaja en la definición de la IEEE:

- A. Una condición para resolver un aspecto funcional de un sistema.
- B. Una condición o capacidad que un usuario necesita para resolver un problema o alcanzar un objetivo.
- C. Una condición o capacidad que un sistema o componente de sistema debe poseer para satisfacer un contrato, estándar, especificación o otros documentos formalmente impuestos.
- D. Una representación documentada de una condición o una capacidad que cumple alguna de los puntos B y C.

4. Marca el objetivo que no forma parte del área de proceso de gestión de requisitos (*requirements management*, REQM) de CMMI:
- A. Gestionar los requisitos del producto.
 - B. Identificar inconsistencias entre requisitos y otros elementos de trabajo.
 - C. Gestionar los planes de prueba.
 - D. Identificar inconsistencias entre requisitos y planes dentro del proyecto.
5. Marca aquella funcionalidad que no forma parte de la característica de control de consistencia, dependencias e historia en las herramientas de gestión de requisitos:
- A. Detección y resolución de conflictos entre requisitos.
 - B. Verificación de consistencia entre requisitos.
 - C. Trazabilidad de los requisitos con otros artefactos.
 - D. Control de versión de requisitos.
6. Un mapeo de historias de usuario es:
- A. Una forma de definir los requisitos para la primera versión de un producto.
 - B. Una técnica utilizada en el descubrimiento de productos para esbozar un nuevo producto o una nueva característica para un producto existente.
 - C. Una técnica de desarrollo de requisitos basada en casos de uso.
 - D. Una forma de especificar necesidades no funcionales.
7. Selecciona aquella herramienta de *wireframes* que obliga a concentrarte en la estructura y el contenido, lo que evita largas discusiones sobre colores y detalles que deberían venir más adelante en el proceso:
- A. InVision Freehand.
 - B. UXPin.
 - C. Balsamiq.
 - D. Mockplus.

8. Una historia de usuario es:

- A. Un caso de uso descrito de forma textual.
- B. Una explicación general e informal de una función de software escrita desde la perspectiva del usuario final.
- C. Un requisito no funcional.
- D. Un requisito especificado en un *post-it*.

9. Un *wireframe* es:

- A. Un diseño detallado de una web.
- B. Un esquema de la arquitectura de un sistema.
- C. Un diseño detallado de una app móvil.
- D. Un modelo o marco esquelético que describe el diseño básico y las funciones de una interfaz de usuario.

10. Une la definición de capacidades y su definición según la ISO/IEC TR 24766:2009:

Elicitación de requisitos	1	A	Soportar la monitorización de los cambios y el mantenimiento de los requisitos para asegurar que están reflejados en el producto resultante
Análisis de requisitos	2	B	Soportar la especificación de los requisitos de una forma consistente, accesible y revisable
Especificación de requisitos	3	C	Soportar la descomposición de requerimientos en requisitos funcionales y no-funcionales y de analizar la viabilidad y el riesgo de los requisitos
Gestión de requisitos	4	D	Soportar la identificación de los <i>stakeholders</i> (actores asociados), la captura y traza con los requerimientos de negocio y usuario, los requerimientos funcionales y los requerimientos no funcionales durante las tareas de descubrimiento de requisitos

Plataformas de Desarrollo de Software

Tema 3. Plataformas de ingeniería de software

Índice

Esquema

Ideas clave

3.1. Introducción y objetivos

3.2. Contexto de las plataformas de ingeniería de software

3.3. Valor y utilidad de las plataformas de ingeniería de software

3.4. Plataformas ALM

3.5. Plataformas CASE

3.6. Referencias bibliográficas

A fondo

Modelado ágil con UML

Aprendiendo UML 2.0

Test

PLATAFORMAS DE INGENIERÍA DE SOFTWARE			
CONCEPTOS	VALOR Y UTILIDAD	HERRAMIENTAS	
Perspectivas Application lifecycle management (ALM) Computer-aided software engineering (CASE)	Valor plataformas CASE: - Elaboración de diagramas - Conceptualización de necesidades - Análisis de modelos - Repositorio central - Generación de documentación - Generación de código	Plataformas CASE: - Altova UModel - Eclipse Papyrus - Enterprise Architect - Gleek.io - Lucidchart - Plant UML - Star UML - Visual Paradigm	
Pilares ALM Trazabilidad Automatización de procesos de alto nivel Visibilidad del progreso del desarrollo	Valor plataformas ALM: - Gestión de requerimientos - Estimación y planificación - Desarrollo de software - Pruebas y garantía de calidad - Gestión de proyectos - Colaboración y comunicación	Plataformas ALM: - Atlassian Jira + Confluence + BitBucket + Bamboo - CA Agile Central - CodeBeamer - IBM Rational CLM - MicroFocus ALM - Microsoft ALM - Polarion ALM - SpiraTeam - SwiftALM	

3.1. Introducción y objetivos

En este tema se introduce el concepto de *application lifecycle management* (ALM), frente al *software development life cycle* (SDLC) y se justifica que abordaremos las plataformas de ingeniería del software con una **perspectiva amplia**, como aquellas herramientas o combinaciones de estas, que dan soporte transversal a los procesos de ALM.

Por otro lado, se introduce el concepto de herramienta *computer-aided software engineering* (CASE) como una **visión específica** de plataformas de ingeniería del software.

Posteriormente, veremos las principales **ventajas** del uso de plataformas de ingeniería del software, en ambas perspectivas, y enumeraremos las principales disponibles con sus características y funcionalidades.

Con el estudio de este tema, pretendemos alcanzar los siguientes objetivos:

- ▶ Entender el **contexto** de las plataformas de ingeniería del software ALM y CASE.
- ▶ Comprender el **valor y la utilidad** que tienen las de ingeniería del software ALM y CASE.
- ▶ Conocer algunas **plataformas** de ingeniería del software ALM y CASE.

3.2. Contexto de las plataformas de ingeniería de software

Las organizaciones son cada vez más modernas y dependen del software en muchas formas. Los procesos de negocio están, a menudo, implementados en un **flujo digital** en el cual, si no disponemos de un software de apoyo, se pueden producir problemas.

Todo desarrollo de software incluye diferentes **actividades** que son ejecutadas por personas que juegan roles específicos y es necesario que se lleve a cabo de **forma colaborativa** para que los proyectos se realicen con éxito.

La gestión del ciclo de vida de las aplicaciones, *application lifecycle management* (ALM), se focaliza en la forma en la que se puede gestionar el ciclo completo de vida de la construcción de un software: cubre **todo el proceso** desde la idea hasta su puesta en marcha, experimentación, implementación, despliegue en sistemas y soporte. También incluye el **seguimiento** de modificaciones y documentación a lo largo de su uso. Engloba diferentes disciplinas que hasta ahora se consideraban de forma aislada, pero que, conjuntamente y con un buen proceso de ALM, se puede llevar a cabo un desarrollo más rápido, más eficiente en coste y con una mayor calidad.

En algunas ocasiones, se confunde con el ciclo de vida de desarrollo de software (**software development life cycle, SDLC**), definido como los procesos o etapas involucradas en la creación de un nuevo sistema de información o la modificación de un sistema existente y las diversas metodologías utilizadas en el proceso. En otras palabras, son todas las actividades involucradas en el desarrollo del software, los procesos involucrados en llevarlo **de la idea a la realidad** (software real).

SDLC y ALM son dos conceptos que no son completamente independientes. Técnicamente, implican un cierto lapso de las diferentes etapas de una aplicación entre su inicio y el final de su vida útil. En realidad, el SDLC es un **subconjunto** de ALM.

En las organizaciones de desarrollo de producto de software es muy habitual que, en función de las personas que la componen, se dé **prioridad** a algunos aspectos frente a otros en el desarrollo de un producto. Habitualmente se dan las siguientes perspectivas:

- ▶ **Perspectiva del SDLC:** es, posiblemente, la forma más común de observar un ALM, quizá porque es la parte central y ocupa un largo espacio de tiempo.
- ▶ **Perspectiva de administración de servicios u operaciones:** las operaciones, a menudo, se han considerado por separado en el ciclo de vida de un producto de software. Esto ha dado como resultado que este aspecto tenga su propia visión de ALM y los problemas en esta área.
- ▶ **Perspectiva de la gestión de la cartera de aplicaciones (APM):** seguramente, debido a la brecha entre negocios y TI (tecnología de la información), hemos visto muchas organizaciones con una estrategia ALM de cartera en la que el desarrollo es solo una pequeña parte. Desde una perspectiva comercial, el enfoque ha estado en cómo manejar la cartera y no en todo el proceso ALM.

Con todo ello, una **vista unificada**, que ya existe en algunas organizaciones, proporciona una forma holística de enfrentar el proceso ALM, al tener en cuenta las tres perspectivas anteriores. Esta es la única manera de tomar el control y optimizar el ALM. Para un director de información (CIO) de una organización es esencial tener esta vista todo el tiempo para que cada una de las perspectivas se vea reforzada.

Los tres pilares fundamentales de ALM son:

- ▶ **Trazabilidad:** en ocasiones se han dejado de llevar a cabo actualizaciones de un producto software en producción al no tener suficiente trazabilidad en sus componentes. Resultaba demasiado caro ese mantenimiento debido a los **efectos inesperados** que incluso un pequeño cambio podría tener: el efecto de un pequeño cambio en una parte del código podría afectar a otra parte por sorpresa, ya que una mala trazabilidad significaba que no se tenía forma de ver dicha conexión previamente. Debe haber una forma de rastrear los requisitos hasta el código entregado, a través de modelos de arquitectura, diseño de modelos, *scripts*, pruebas unitarias, casos de prueba, etc., no solo para implementar correcciones de errores, sino también para demostrar que el producto contiene las cosas que desde negocio se requerían o para lograr el cumplimiento interno y externo de las normas y reglamentos.
- ▶ **Automatización de procesos de alto nivel:** el siguiente pilar de ALM es la automatización de procesos de alto nivel. Todas las organizaciones tienen procesos, aunque sea de forma implícita. En muchas de ellas, el paso de un proceso a otro (por ejemplo, de análisis a diseño o entre implementación y pruebas) se hace manualmente. ALM enfatiza la **importancia de automatizar** estas tareas para un proceso más efectivo, que consuma menos tiempo y con menos errores.

- **Visibilidad del progreso de los esfuerzos de desarrollo:** el tercer y último pilar de ALM es brindar **visibilidad** sobre el progreso de los esfuerzos de desarrollo. Muchos gerentes y las partes interesadas tienen una visibilidad limitada del progreso de los proyectos de desarrollo y, a menudo, viene de las reuniones del grupo directivo, durante las cuales el director del proyecto revisa la situación actual. Si se desea tener un proceso efectivo, es necesario asegurar la visibilidad. También es fundamental que los miembros del equipo de desarrollo dispongan de esa visibilidad, ya que, de forma demasiado frecuente, la tienen sesgada. Esto se debe, habitualmente, al hecho de que la elaboración de informes es difícil y puede implicar mucho trabajo manual tomando demasiado tiempo y esfuerzo para producirlos, especialmente cuando tiene información en muchos repositorios.

En este capítulo abordaremos, por un lado, las plataformas de ingeniería del software con una perspectiva amplia, como aquellas herramientas o combinaciones de estas que dan soporte transversal a los procesos de ALM.

Por otro lado, también cubriremos una visión mucho más concreta, enfocada en plataformas **computer-aided software engineering (CASE)**, que es un paquete de software que brinda soporte para el diseño e implementación de sistemas de información.

Las herramientas CASE son un conjunto de programas de aplicación de software que se utilizan para **automatizar las actividades** de SDLC. Son utilizadas por gerentes de proyectos de software, analistas e ingenieros para desarrollar sistemas de software. Hay varias herramientas CASE disponibles para simplificar varias etapas del ciclo de vida del desarrollo de software, como herramientas de análisis, herramientas de diseño, herramientas de gestión de proyectos, herramientas de gestión de bases de datos, herramientas de documentación, entre otras.

3.3. Valor y utilidad de las plataformas de ingeniería de software

Desde la perspectiva de plataformas de ingeniería del software como aquellas que, de forma transversal, pueden dar soporte a un ALM en una perspectiva amplia, su valor es dar soporte a las siguientes funcionalidades:

- ▶ **Gestión de requerimientos:** las herramientas de ALM deben tener una funcionalidad que lo ayude a dar sentido a sus requisitos e, idealmente, las herramientas deben poder adaptarse a su metodología y procesos, y no al revés. Algunas herramientas de ALM admiten la captura y gestión de casos de uso del sistema como parte del proceso de definición de requisitos. Los **casos de uso** se pueden almacenar en el sistema junto con sus escenarios asociados y enlaces a los demás requisitos que define el caso de uso. Dependiendo de sus necesidades (si tiene procesos complicados de varios pasos o necesidades simples de ingreso de datos CRUD [crear, leer, actualizar, borrar]), esta puede ser una característica importante.
- ▶ **Estimación y planificación:** las herramientas de ALM generalmente brindan funcionalidad para ayudarlo a estimar y planificar sus proyectos. Algunas herramientas están diseñadas para administrar proyectos en cascada tradicionales, otras están diseñadas solo para proyectos de desarrollo ágiles y otras pueden manejar ambos. Dependiendo del nivel de planificación que precise, es posible que necesite una herramienta ALM que le permita **estimar tareas técnicas** individuales que luego se agregan de nuevo contra los requisitos originales que se especificaron. Si tiene requisitos de revisión y supervisión más complejos, es posible que necesite una funcionalidad de flujo de trabajo que garantice que todas las tareas asignadas sigan un conjunto específico de pasos con revisiones y controles de calidad.
- ▶ **Desarrollo de software:** aunque normalmente el acto real de desarrollar y escribir el código no se considera parte de ALM en sí mismo, por lo general necesita tener la

funcionalidad de **administración de código fuente** integrada estrechamente dentro de la *suite* ALM. La mayoría de las herramientas de ALM ofrecerán una funcionalidad de administración de código fuente integrada: Algunas herramientas de ALM proporcionarán soluciones de gestión de código fuente patentadas, otras utilizarán herramientas estándar de la industria, como Git o Subversion. Idealmente, la herramienta ALM debería proporcionar flexibilidad para admitir diferentes modelos de bifurcación y fusión. Idealmente, aunque la escritura de código no es estrictamente parte de la *suite* ALM en sí, las mejores herramientas ALM del mercado brindan integración con las herramientas utilizadas por los desarrolladores para escribir el código.

- ▶ **Pruebas y garantía de calidad:** uno de los atributos clave que distingue a las *suites* ALM de las meras herramientas de gestión de proyectos o los sistemas de seguimiento de problemas es que incluyen la parte de **control de calidad** del proceso de desarrollo de software. Como mínimo, deberían ofrecer una gestión básica de casos de prueba. La herramienta ALM debería permitirle crear y administrar sus casos de prueba en carpetas con capacidades de clasificación y filtrado. Debería permitir a los usuarios mostrar diferentes columnas en su propia vista personalizada y permitir que el administrador defina propiedades y listas personalizadas que se pueden usar en el proyecto. Algunas herramientas de ALM brindan la capacidad de definir pasos de prueba y parámetros específicos para los casos de prueba, mientras que otras solo pueden ofrecer un artefacto de caso de prueba simple.
- ▶ **Despliegue:** aunque la compilación y el empaquetado del software que se está desarrollando a menudo está fuera del alcance de las propias herramientas de ALM, la mayoría de las herramientas de ALM le permitirán, como mínimo, integrarse con **servidores de integración continua** (también conocidos como CI) como Jenkins. Según sus necesidades, debe asegurarse de que la herramienta ALM que elija pueda integrarse con los diferentes servidores de CI que usan sus equipos de desarrollo y que los informes dentro de la herramienta ALM puedan fusionar la información de otras fuentes con la información de compilación. De esa manera,

puede ver todos los cambios en cada compilación, qué funciones se agregaron y qué problemas se resolvieron.

- ▶ **Desarrollo y operaciones (DevOps):** es la combinación de tareas realizadas por los equipos de desarrollo de aplicaciones y operaciones de sistemas de una empresa en un todo coherente. Anteriormente, sus desarrolladores necesitarían la ayuda del grupo de TI para aprovisionar diferentes entornos (desarrollo, prueba, preparación, etc.) y probar la aplicación por etapas en cada entorno. Ahora, con más aplicaciones alojadas en la nube y ciclos de lanzamiento medidos en días, no en meses, no puede desarrollar y probar de forma aislada. Deberá probar **nuevas actualizaciones** con datos reales en entornos listos para producción, DevOps es la clave para que esto sea una realidad.
- ▶ **Soporte y mantenimiento:** lamentablemente, la parte que más se pasa por alto del ciclo de vida del software es el soporte y el mantenimiento del producto después de su lanzamiento. Tradicionalmente, esto se hacía justo al final, cuando el equipo de desarrollo entregaba el producto terminado al equipo de soporte y luego se lavaba las manos del sistema hasta que se desarrollaba la siguiente versión. Con el paso al desarrollo ágil, la entrega continua y DevOps, el soporte ahora es una **parte integral del proceso**. La atención al cliente es el mecanismo de retroalimentación que guía las nuevas historias de usuario en el próximo sprint. Por lo tanto, no olvide asegurarse de que la *suite* ALM tenga una capacidad de atención al cliente o, al menos, pueda integrarse con otras herramientas de soporte al cliente o de mesa de ayuda.

- ▶ **Gestión de proyectos y carteras:** dado que las herramientas ALM brindan una funcionalidad tan rica y poderosa, es importante asegurarse de que también brinden buenas herramientas y funciones para consumir la información y poder generar informes en todo el sistema con información resumida y fusionada para permitir la toma de decisiones. Una característica común en muchas buenas herramientas de ALM es la capacidad de **mostrar paneles de control** o gráficos y tablas de acceso común que cubren las métricas clave del proyecto. Además, si se necesita administrar programas más grandes compuestos por varios proyectos diferentes, cada uno con sus propios requisitos, características e hitos, deberá asegurarse de que la herramienta ALM tenga la funcionalidad de administración de cartera de proyectos (PPM). Tradicionalmente, las herramientas de PPM eran una categoría separada, pero cada vez más proveedores de ALM incluyen funciones de PPM en sus ofertas principales. Pueden permitirle visualizar el estado actual de sus proyectos y, en algunos de los ejemplos más potentes, administrar las dependencias y tareas en los diferentes proyectos.
- ▶ **Colaboración y comunicación:** es altamente ineficiente depender del correo electrónico, mensajes de texto o, peor aún, reuniones grupales, para facilitar la comunicación de los miembros del equipo en tiempo real. La información puede estar fuera de contexto y, posteriormente, no existe un **registro confiable** del intercambio, lo que deja a otros miembros del equipo en el vacío sin una buena forma de descubrir lo que está sucediendo.

Desde la perspectiva de herramientas CASE, su uso acelera el desarrollo del proyecto para producir el resultado deseado y ayuda a descubrir fallas antes de continuar con la siguiente etapa en el desarrollo de software. Dentro de esta categoría sus principales funcionalidades son:

- ▶ **Elaboración de diagramas:** ayuda en las representaciones esquemáticas y gráficas de los datos y procesos del sistema. Representa los elementos del sistema, el flujo de control y el flujo de datos entre diferentes componentes de software y la estructura del sistema en forma gráfica.

- ▶ **Conceptualización de necesidades:** ayuda a comprender los requisitos y las relaciones e interdependencias.
- ▶ **Análisis de modelos:** automatización en el análisis de la información modelada para buscar inconsistencias, redundancias u omisiones erróneas.
- ▶ **Repositorio central:** proporciona el único punto de almacenamiento para diagramas de datos, informes y documentos relacionados con la gestión de proyectos.
- ▶ **Generación de documentación:** ayuda a generar documentación técnica y de usuario según los estándares. Crea documentos para usuarios técnicos y usuarios finales.
- ▶ **Generación de código:** ayuda en la generación automática de código a partir de los modelos.

3.4. Plataformas ALM

Las principales plataformas para el control de versiones son:



Tabla 1. Plataformas ALM. Fuente: elaboración propia.

Atlassian Jira

Al combinar algunos productos de la *suite Atlassian* como Jira Software, Confluence, Stash (Bitbucket Server) y Bamboo, se diseñó una herramienta ALM competente y de servicio completo.

- ▶ El software Jira se utiliza para planificar y realizar un seguimiento de los proyectos y sus problemas correspondientes.

- ▶ Confluence es una herramienta de colaboración para la gestión de documentos y conocimientos.
- ▶ Bitbucket Server es un repositorio de código con tecnología Git que colabora mediante solicitudes y comentarios en línea.
- ▶ Bamboo Server se utiliza para la integración, la creación y las pruebas programadas, la implementación y la entrega.

CA Agile Central

Anteriormente conocida como Rally, es la principal herramienta ALM para empresas que utilizan métodos Agile. Sus funciones de gestión de proyectos, incluida la planificación de recursos, están diseñadas con Agile en mente. También es una excelente herramienta ALM para **realizar pruebas**. Proporciona funcionalidad de colaboración a través de Flowdock, que tiene funciones de chat y correo electrónico. Es compatible con Scaled Agile Framework (SAFe) y ofrece análisis y métricas específicas de Agile basadas en el índice de rendimiento de desarrollo de software. Proporciona opciones locales y SaaS.

CA Agile Central se utiliza para **desarrollar y entregar** software preciso con alta calidad y velocidad con una plataforma eficiente para soluciones ágiles.

- ▶ Ofrece una plataforma ALM basada en la web a la que se puede acceder desde un navegador como Chrome o Safari, etc. a través de un sistema operativo como Windows o Linux.
- ▶ Esta herramienta es utilizada principalmente por organizaciones de todos los tamaños, como pequeñas, medianas o grandes, que desarrollan aplicaciones con metodología ágil.

- ▶ Es una plataforma de nivel empresarial que se utiliza para escalar metodologías de desarrollo ágiles en un proyecto.
- ▶ Se pueden medir las métricas centrales de proyectos en tiempo real, como las métricas de rendimiento, la productividad, la calidad y la apertura de las aplicaciones.

CodeBeamer

Es una plataforma de ALM para el desarrollo avanzado de productos y software. La plataforma abierta **amplía las funcionalidades** de ALM con capacidades de configuración de línea de productos y proporciona una capacidad de configuración única para procesos complejos.

CodeBeamer proporciona gestión de proyectos, tareas, requisitos, pruebas, cambios, configuración, creación, conocimientos y documentos en un único entorno seguro. Permite que el desarrollo de software y hardware sea más **colaborativo, transparente y productivo**.

CodeBeamer ofrece **flujos de trabajo digitales** únicos para el desarrollo colaborativo, la eficiencia en la ingeniería de líneas de productos y el cumplimiento normativo. Es posible conectar todas las herramientas de desarrollo para proporcionar a sus equipos un centro de desarrollo central y se puede adaptar fácilmente la solución a las necesidades especiales de ingeniería y automatizar el control de procesos para los requisitos reglamentarios.

Alguna de las funcionalidades de CodeBeamer son:

- ▶ Gestión de **requerimientos**: convierta los conocimientos del mercado en productos de calidad más rápido que nunca. Obtenga claridad y alinee a todos los colaboradores para reducir costos y tiempos de ciclo. Seguimiento de elementos y acciones a lo largo de la entrega.

- **Desarrollo de software:** reduzca la complejidad e incorpore calidad a sus productos de software. Garantice una colaboración fluida entre las disciplinas de innovación de hardware, software y servicios. Use procesos maduros, ya sea que esté trabajando con en cascada, ágiles (a escala), híbrido o un método personalizado.
- Garantía de **calidad y pruebas:** integre la gestión de calidad en sus procesos de desarrollo y genere confianza en sus productos. Rastree y gestione las actividades de control de calidad a lo largo del ciclo de vida y acelere la verificación del producto.

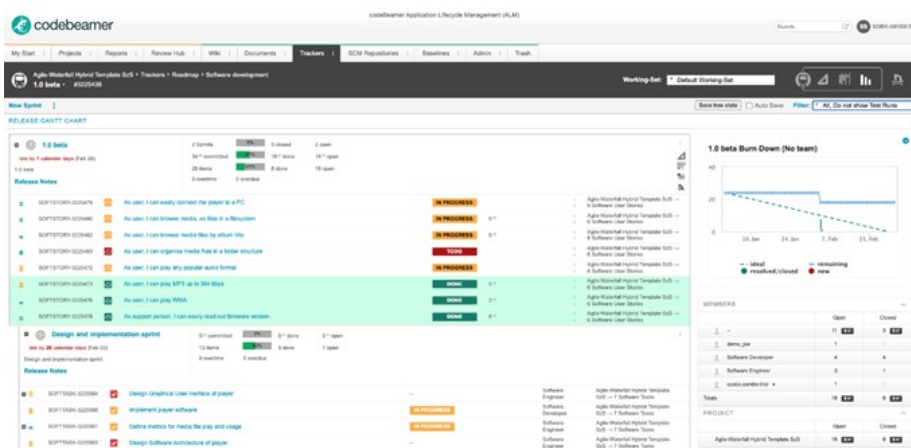


Figura 1. Vista de CodeBeamer. Fuente: Intland Software, s. f.

IBM

Ofrece varios productos para su solución ALM. Algunos de estos son: CLM para la gestión colaborativa del ciclo de vida, la herramienta Engineering Workflow Management para gestionar flujos de trabajo, servicio en la nube de entrega continua para automatizar compilaciones y pruebas de software, base de administración del ciclo de vida de ingeniería para administrar requisitos, calidad, cambios, planes y seguimiento de proyectos, entre otros.

Los productos de IBM son compatibles con las metodologías cascada y ágiles y se integran con IBM Tivoli para admitir la **integración continua**. Estas características hacen que los productos de IBM sean una buena opción para las organizaciones que

necesitan integrar diversos procesos. El producto es más antiguo con mucho soporte heredado, pero IBM lo ha mantenido actualizado, alineándose con las tendencias emergentes de ALM. IBM Rational CLM es un sistema ALM que incluye una gran selección de aplicaciones ALM que están integradas entre sí.

- ▶ Rational CLM es una combinación de IBM Rational Team Concert, IBM Rational DOORS Next Generation e IBM Rational Quality Manager, lo que la convierte en una **solución ALM completa**.
- ▶ La integración de los productos anteriores con RCLM ofrece gestión de requisitos, planificación de proyectos en tiempo real, gestión de calidad, gestión de cambios, trazabilidad del ciclo de vida, etc.
- ▶ IBM Rational Team Concert se utiliza para planificar, gestionar y rastrear el trabajo del proyecto.
- ▶ IBM Rational DOORS Next Generation se utiliza para gestionar requisitos como definirlos, analizarlos y gestionarlos.
- ▶ IBM Rational Quality Manager se utiliza para desarrollar, ejecutar e informar el plan de prueba.

MicroFocus ALM

Es un software *lean*, ágil y DevOps que permite a los equipos de todos los tamaños ofrecer aplicaciones de alta calidad con mayor velocidad. Permite que las partes interesadas interactúen y se coordinen para **lograr los objetivos del proyecto**. Proporciona seguimiento e informes sólidos y una integración perfecta de varias tareas relacionadas con el proyecto. Puede conectarse a sistemas de correo electrónico y enviar correos electrónicos a los miembros del equipo.

Microsoft ALM

La *suite* ALM de Microsoft utiliza **Visual Studio**, que es uno de los entornos de desarrollo integrados más utilizados en la comunidad de desarrolladores. Team Foundation Server (TFS) de Microsoft proporciona una buena funcionalidad de configuración y control de versiones. La *suite* ALM de Microsoft es una opción particularmente buena para las organizaciones que usan un marco .NET porque muchos desarrolladores de .NET usan Visual Studio y también tienden a disfrutar usando TFS.

TFS, renombrado en 2022 como **Azure DevOps Server**, es un **conjunto integrado** de herramientas de desarrollo, sistema de compilación, métricas y control de versiones que utilizan los equipos especializados o calificados para organizar y ejecutar los proyectos.

- ▶ TFS ofrece una gama de herramientas colaborativas para el desarrollo de software que se pueden integrar con su entorno de desarrollo actual.
- ▶ Al usar TFS, se puede reducir la repetición del trabajo en el software en desarrollo al aumentar su transparencia, lo que a su vez da como resultado el envío de productos de software de alta calidad.
- ▶ TFS opera como un punto de contacto esencial tanto para la gestión de procesos como para la gestión de proyectos.

Polarion ALM de Siemens

Es un software ALM de nivel empresarial completo creado para conectar equipos y personas para **mejorar el proceso de desarrollo de aplicaciones**. Facilita los nuevos proyectos al brindar la capacidad de reutilizar o bifurcar sus datos (o código) para proyectos o líneas de productos similares. Comparta datos de forma segura y establezca permisos de usuario.

Polarion es una plataforma integrada que facilita a las organizaciones automatizar y optimizar los procesos de desarrollo en varios proyectos:

- ▶ Se utiliza para lograr la brillantez de la fabricación al definir, construir, probar y administrar el software multifacético en una solución basada en navegador o en la plataforma en la nube.
- ▶ Sus características importantes son el desarrollo unificado, la gestión de cambios y configuraciones, la colaboración y el cumplimiento, la gestión de pruebas y calidad, etc.
- ▶ Admite procesos de desarrollo en varias operaciones de desarrollo como en cascada, ágiles e híbrido.
- ▶ Así como técnicas innovadoras de resolución de problemas para equipos distribuidos.

SpiraTeam

Es la galardonada solución ALM de Inflectra que ayuda a los equipos a administrar los requisitos, versiones, iteraciones, tareas, defectos y código de su programa en un entorno, con trazabilidad completa. Los requisitos y los módulos de gestión de casos de prueba vienen con flujos de trabajo totalmente personalizables. La solución es excelente para la gestión de carteras de proyectos, la planificación de programas y la gestión de líneas base y ofrece una configuración robusta y control de versiones para artefactos. El seguimiento de errores, problemas y tareas es fácil y facilita el trabajo de los equipos de control de calidad. Las funciones para firmas electrónicas y colaboración de documentos son especialmente valiosas para las empresas que trabajan en las industrias reguladas.

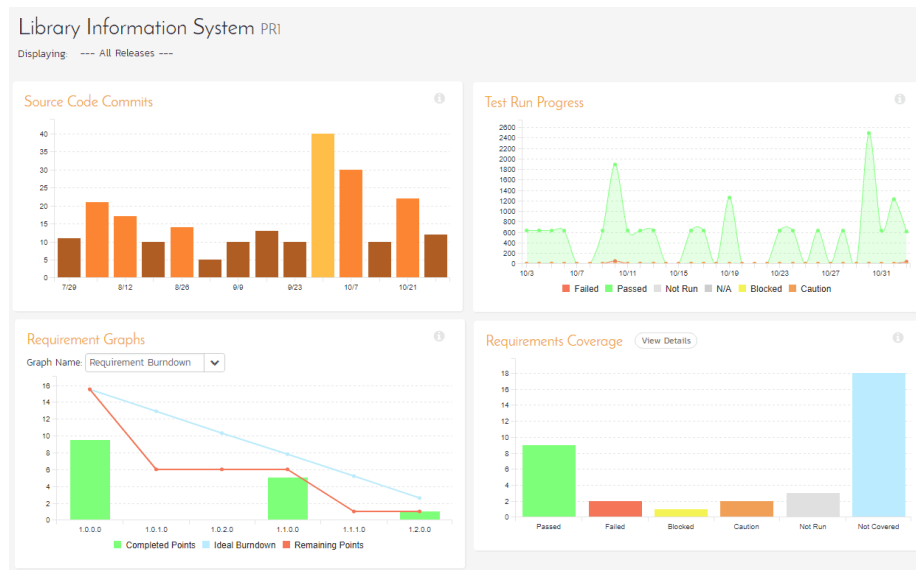


Figura 2. Cuadros de mando con SpiraTeam. Fuente: Inflectra, s. f.

SpiraTeam presenta una **solución absoluta** de gestión del ciclo de vida de las aplicaciones que se utiliza para gestionar requisitos, planes, pruebas, errores, tareas y problemas en un único entorno.

- ▶ Se utiliza para la recopilación de requisitos, control de calidad, pruebas, informes personalizables, toma de decisiones, etc. por parte de equipos de todos los tamaños.
- ▶ Admite metodologías ágiles de gestión de proyectos como Scrum y Kanban.
- ▶ Usando SpiraTeam podemos crear los *scripts* de prueba manual, automatizarlos y también podemos administrarlos.
- ▶ SpiraTeam ofrece tableros incorporados con métricas importantes del proyecto.

SwiftALM

Es una plataforma **colaborativa** para la gobernanza de programas, proyectos y procesos de TI. Alinea los objetivos de TI con las estrategias corporativas, cuenta con funciones sofisticadas de modelado de organización y control de acceso con la capacidad de realizar múltiples niveles de desglose de información. Puede configurar

los recursos, procesos y proyectos y controlar el acceso a planes e informes. La herramienta tiene una arquitectura de aplicaciones altamente **configurable y flexible** y tecnologías de análisis avanzadas para aumentar la productividad, la calidad y la entrega.

SwiftALM es una propuesta colaborativa para la gestión de proyectos, gestión de programas, SDLC y gobernanza de procesos.

- ▶ Es una herramienta de proyecto simple basada en la web que se utiliza para metodologías ágiles, en cascada e híbridas.
- ▶ Tiene un diseño estructural de aplicación extremadamente configurable y flexible que puede admitir hasta miles de usuarios.
- ▶ Se utiliza para configurar los procesos, proyectos, recursos y controlar el acceso a las funciones mediante una mejor planificación.
- ▶ Cubre la gestión de requisitos, cambios, problemas, riesgos, pruebas, defectos y documentos.

Resume las mejores prácticas de una organización en una plantilla para que sean reutilizables para proyectos futuros.

3.5. Plataformas CASE

La mayoría de las plataformas CASE en el ámbito del software soportan el **lenguaje unificado** de modelado como representación básica. Las principales plataformas CASE de ingeniería del software son:

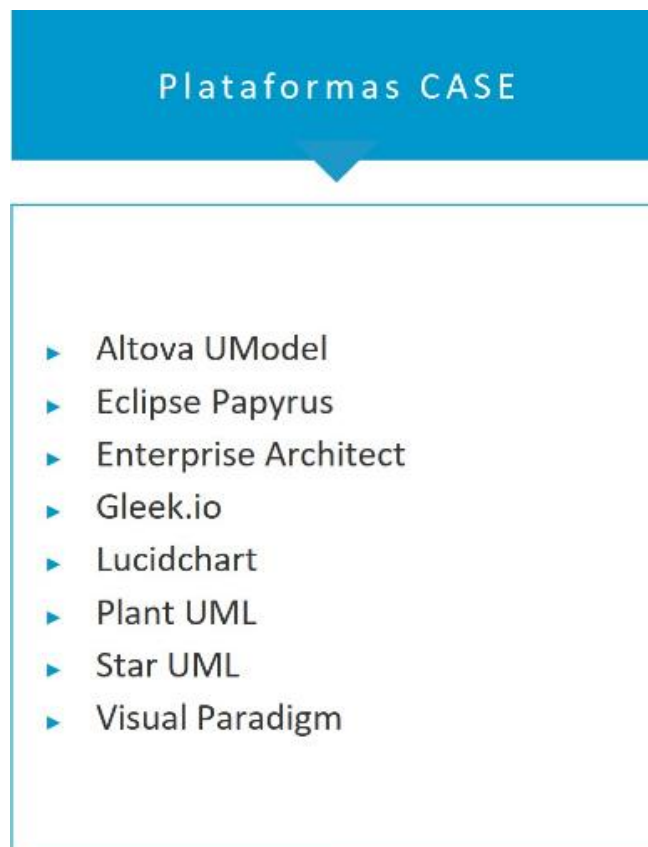


Tabla 2. Plataformas CASE. Fuente: elaboración propia.

Altova UModel

Es la herramienta lenguaje unificado de modelado (UML) que hace que el diseño de software visual sea **práctico** para cualquier proyecto. Es posible diseñar visualmente modelos de aplicaciones en UML y generar código Java, C++, C# o Visual Basic .NET y documentación de proyectos. O se puede aplicar ingeniería inversa a los programas existentes en diagramas UML, luego mejorar sus diseños y completar el viaje de ida y vuelta regenerando el código (Altova, 2022).

Mantenga sus vistas de diagramas y herramientas UML favoritas al alcance de su mano para mejorar la eficiencia y la productividad. Todo se puede **personalizar** en los diagramas UML que crea con UModel: tamaño, posición, color, características tipográficas y estilos de línea.

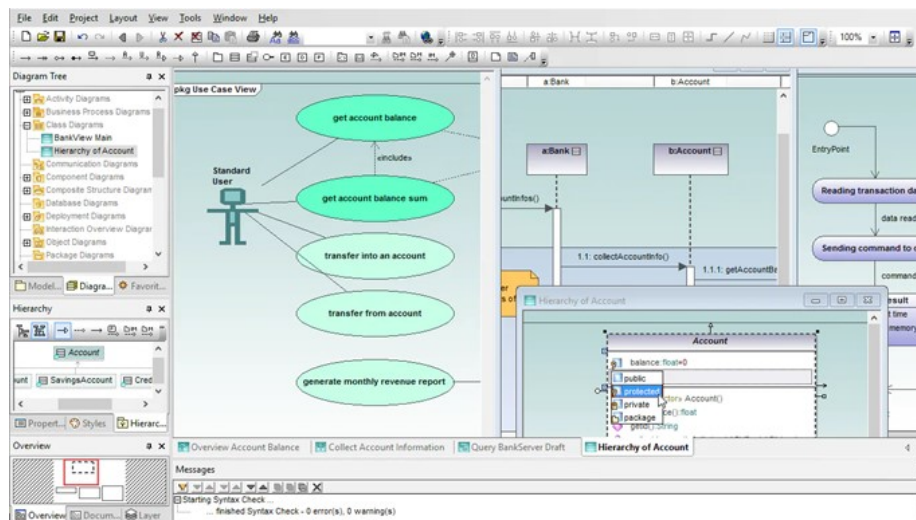


Figura 5. Vista de Altova UModel. Fuente: Altova, s. f.

- Es **compatible** con los catorce tipos de diagramas de arquitectura de software UML 2.5, además de diagramas de estilo UML para esquemas y tablas XML en bases de datos SQL populares.

- ▶ Permite agregar **hipervínculos** a cualquier elemento en cualquier diagrama UML. Puede crear hipervínculos a archivos externos, URL web u otros diagramas en su proyecto. Su descripción se convierte en texto emergente para el hipervínculo.
- ▶ Incluye un **generador de código integrado** que crea código Java, C++, C# o Visual Basic .NET basado en diagramas de clase, diagramas de secuencia y diagramas de máquina de estado en su modelo UML, liberándolo de la tediosa tarea de escribir a bajo nivel código de infraestructura para que pueda concentrarse en la lógica comercial y la arquitectura general de su proyecto.
- ▶ Incorpora una potente **capacidad de ingeniería inversa** para leer código fuente y archivos binarios de Java, C#, C++ y Visual Basic, y generar modelos UML para una representación visual mucho más clara y fácil de analizar que una copia impresa del código fuente como texto.
- ▶ Es posible modificar el código generado. La función de ingeniería de ida y vuelta lee el código modificado y actualiza automáticamente los diagramas UML en consecuencia.

Eclipse Papyrus

Es una herramienta de ingeniería basada en modelos de código abierto de grado industrial de la fundación Eclipse. Se ha utilizado notablemente con éxito en proyectos industriales y es la plataforma base para varias herramientas de modelado industrial (Eclipse, s. f.).

- ▶ **Basado en estándares:** UML 2.5, SysML 1.1 y 1.4, fUML 1.2.1, ALF 1.0.1, MARTE 1.1, BPMNProfile 1.0, BMM 1.3, SMM 1.1, PSCS 1.0, PSSM 1.0b, FMI 2.0 e ISO/IEC 42010.
- ▶ Posibilidad de **crear lenguajes específicos del dominio:** cada parte de Eclipse Papyrus se puede personalizar (perfil UML, explorador de modelos, notación y estilo de diagrama, vistas de propiedades, paleta y menús de creación y mucho más).

- **Model-Driven Engineering:** permite técnicas basadas en modelos (simulación basada en modelos, pruebas formales basadas en modelos, análisis de seguridad, análisis de rendimiento/compensaciones, exploración de arquitectura, etc.).

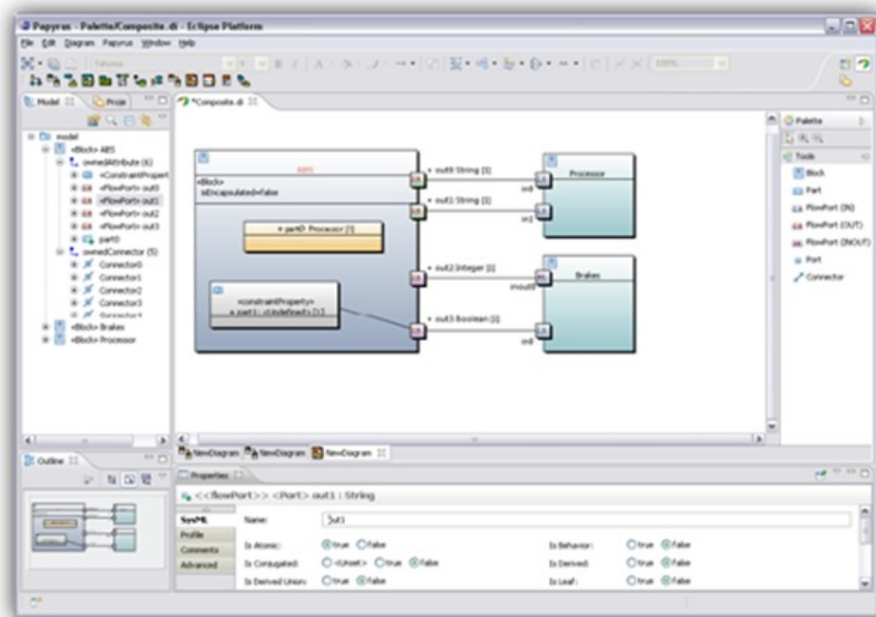
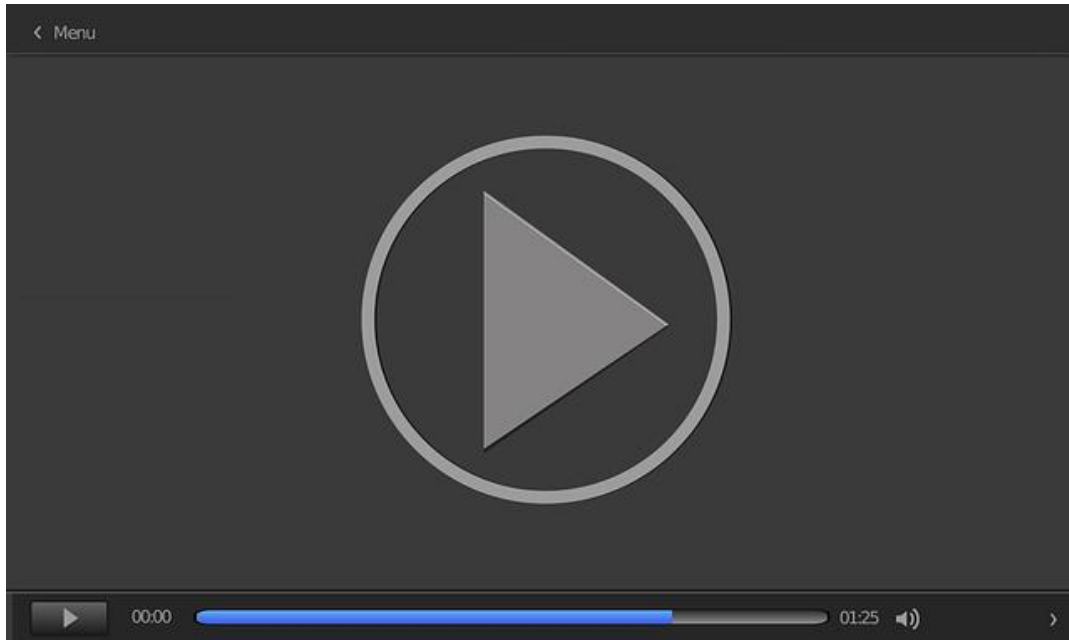


Figura 6. Vista de Papyrus. Fuente: Eclipse, s. f.

En este vídeo, *Modelo UML con Eclipse Papyrus*, se aprenderá a utilizar Papyrus, que está dentro de Eclipse, para modelar proyectos de software.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=fef059cb-eea3-4b4c-9afc-aec300cd975d>

Enterprise Architect

Es la solución comercial de Sparx Systems como herramienta gráfica diseñada para ayudar a sus equipos a construir sistemas **robustos y fáciles de mantener**. Con capacidades integradas de gestión de requisitos, ayuda a rastrear especificaciones de alto nivel para modelos de análisis, diseño, implementación, prueba y mantenimiento utilizando UML, SysML, BPMN y otros estándares abiertos.

- **Repositorio de modelos de alto rendimiento:** tiene un desempeño espectacularmente rápido, ya que carga modelos extremadamente grandes en segundos.

- ▶ **Simulación dinámica de modelos:** es posible verificar la corrección de los modelos de comportamiento y obtener una mejor comprensión de cómo funciona un sistema empresarial.
- ▶ **Trazabilidad completa:** proporciona una trazabilidad completa desde los requisitos, el análisis y los modelos de diseño hasta la implementación y el despliegue.
- ▶ **Del modelo al código fuente:** el editor de código fuente incorporado permite navegar rápidamente desde el modelo directamente al código fuente en el mismo entorno. Las plantillas de generación de código permiten personalizar el código fuente generado según las especificaciones de la empresa.

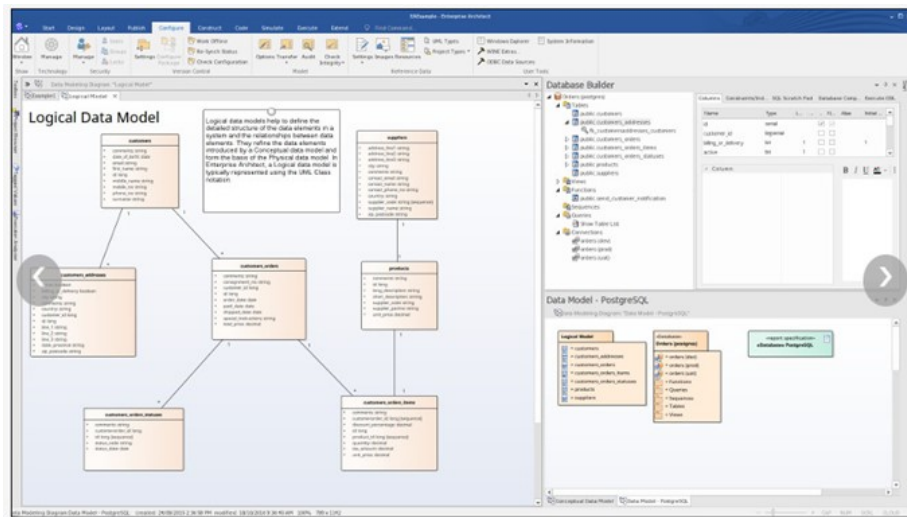


Figura 7. Vista de Enterprise Architect. Fuente: Sparx System, s. f.

Gleek.io

Crea varios tipos de **diagramas UML**: diagramas de secuencia, diagramas de clases y diagramas de objetos. Los equipos también la usan para crear organigramas, diagramas de flujo, mapas mentales, entre otros. Como saben los usuarios avanzados y los desarrolladores, usar el teclado es mucho más rápido que usar un mouse. Debido a que se basa en los comandos del teclado, los desarrolladores trabajan más rápido que con los programas de dibujo de arrastrar y soltar. Y tiene

ayuda de sintaxis disponible directamente en la ventana de diagramación en caso de que te quedes atascado.

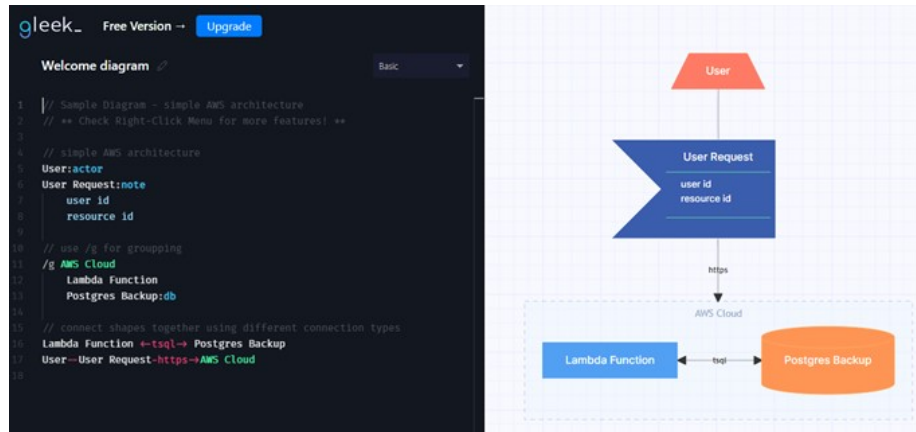


Figura 8. Vista de Geek.io. Fuente: elaboración propia basado en Gleek.io.

Lucidchart

Proporciona un conjunto de **herramientas simples** para facilitar el modelado en colaboración. Funciona en tiempo real en muchas plataformas gracias a que su código está basado en HTML 5. Se integra con Confluence y también con MS Teams, Slack y G Suite. Maneja múltiples formatos de información, desde diagramas de sistemas complejos hasta mapas mentales. Es una herramienta interesante entre los miembros del equipo de desarrollo menos técnicos debido a su sencilla interfaz de usuario de arrastrar y soltar.

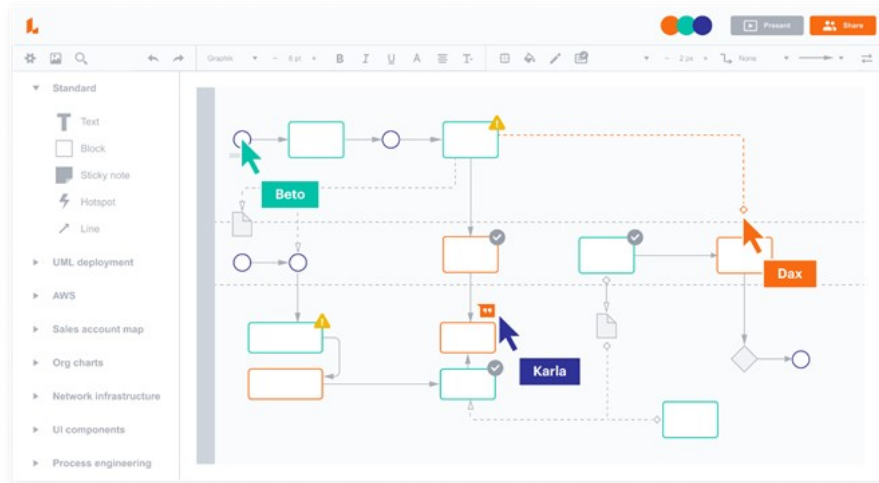


Figura 9. Vista de LucidChart. Fuente: Lucid, s. f.

Plant UML

Es un **proyecto de código abierto** que, con base en un lenguaje textual, genera multitud de diagramas como:

- ▶ Diagramas de secuencia.
- ▶ Diagramas de casos de uso.
- ▶ Diagramas de clases.
- ▶ Diagramas de objetos.
- ▶ Diagramas de actividades.
- ▶ Diagramas de componentes.
- ▶ Diagramas de despliegue.
- ▶ Diagramas de estados.
- ▶ Diagrama de tiempos.

- ▶ *Wireframe graphical interface.*
- ▶ Diagrama de Gantt.
- ▶ Mapa mental.
- ▶ Estructura de descomposición del trabajo.
- ▶ Diagrama entidad-relación.

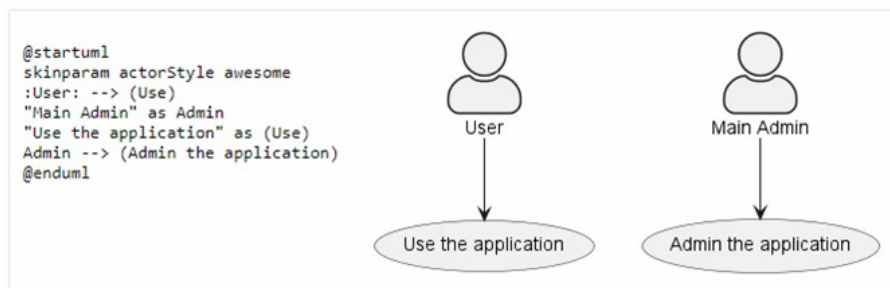
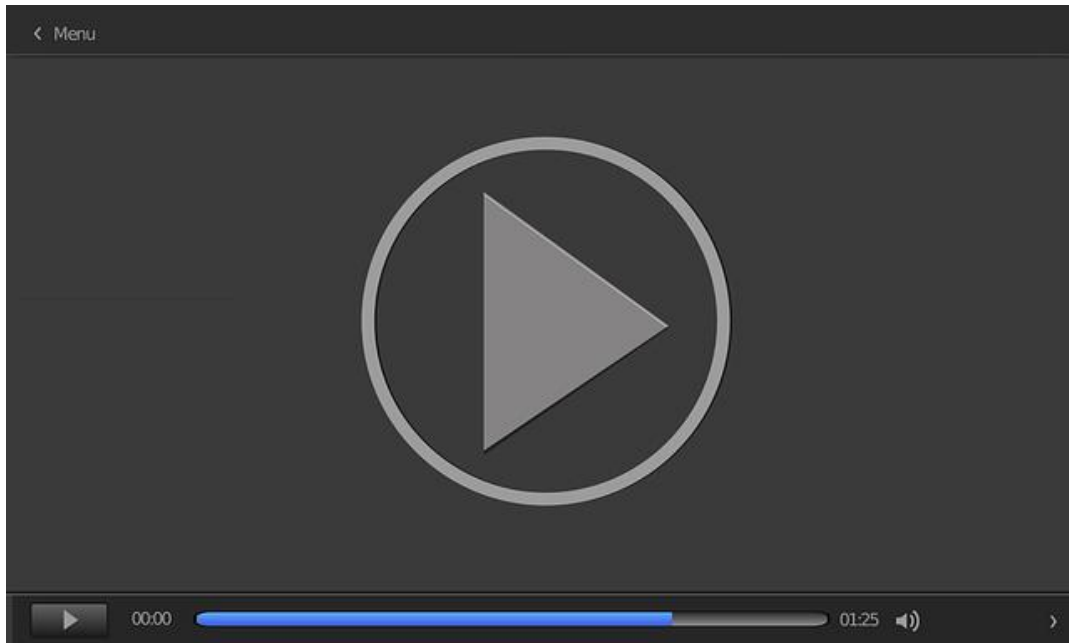


Figura 10. Vista del modelado de casos de uso con PlantUML. Fuente: PlantUML, s. f.

En este vídeo, *Documentando análisis y diseño técnico con PlantUML*, se aprenderá a utilizar la herramienta PlantUML, que sirve para facilitar la documentación técnica y de diseño.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=387c55b8-ca5a-4500-9f94-aec300cd989a>

Star UML

Es un modelador de software sofisticado para un modelado **ágil y conciso**.

- ▶ Compatible con diagramas y metamodelo estándar UML 2.x: clase, objeto, caso de uso, componente, implementación, estructura compuesta, secuencia, comunicación, diagrama de estado, actividad, sincronización, desbordamiento de interacción, flujo de información y diagrama de perfil.
- ▶ Admite el modelado con diagramas SysML: requisito, definición de bloque, bloque interno y diagrama paramétrico.

- Soporte para crear diagramas de entidad-relación (ERD), diagramas de flujo de datos (DFD) y diagramas de diagrama de flujo.

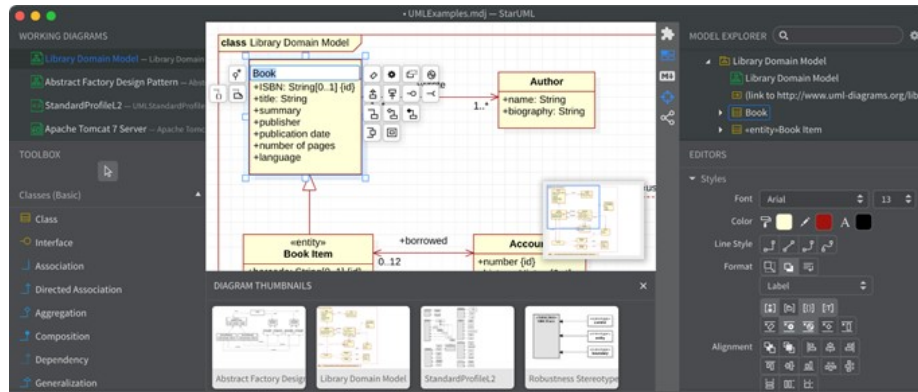


Figura 11. Vista de StarUML. Fuente: StarUML, s. f.

Visual Paradigm

Es una aplicación de software diseñada para equipos de desarrollo de software para **modelar** sistemas de información comercial y **administrar** procesos de desarrollo. Además del soporte de modelado, esta tecnología proporciona capacidades de ingeniería de código y generación de informes, incluida la generación de código. Esta tecnología puede realizar ingeniería inversa de diagramas a partir de código y proporcionar ingeniería de ida y vuelta para varios lenguajes de programación.

- **Editor de diagramas de arrastrar y soltar:** admite UML, BPMN, ArchiMate, DFD, ERD, SoaML, SysML y CMMN.
- **Arquitectura empresarial:** es una herramienta de ciclo de vida TOGAF.
- **Planificación estratégica:** herramienta de gestión de procesos que te ayuda a descubrir y definir los objetivos estratégicos de tu negocio.
- **Diseño de procesos:** es posible visualizar el flujo de trabajo empresarial con BPMN, documentar el procedimiento de trabajo para las actividades y generar informes de procesos de alta calidad.

- ▶ **Análisis de procesos:** es posible estudiar el flujo de trabajo con la herramienta de animación de procesos. A su vez, se puede determinar la mejor configuración de recursos con la herramienta de simulación de procesos.
- ▶ **Reingeniería de procesos:** herramienta de gestión de procesos de reingeniería que ayuda a documentar el plan, diseñar nuevos procesos, identificar brechas y monitorear mejoras con indicadores (*key performance indicators*, KPI).
- ▶ **Diseño de experiencia de usuario:** aclare las necesidades de las partes interesadas con nuestras potentes herramientas de experiencia del usuario, soporta *wireframes* y prototipado rápido.
- ▶ **Generación de código:** soporta generación de código hacia Java, ORM e interfaces REST.
- ▶ **Ingeniería inversa:** obtenga diagramas a partir de código fuente.

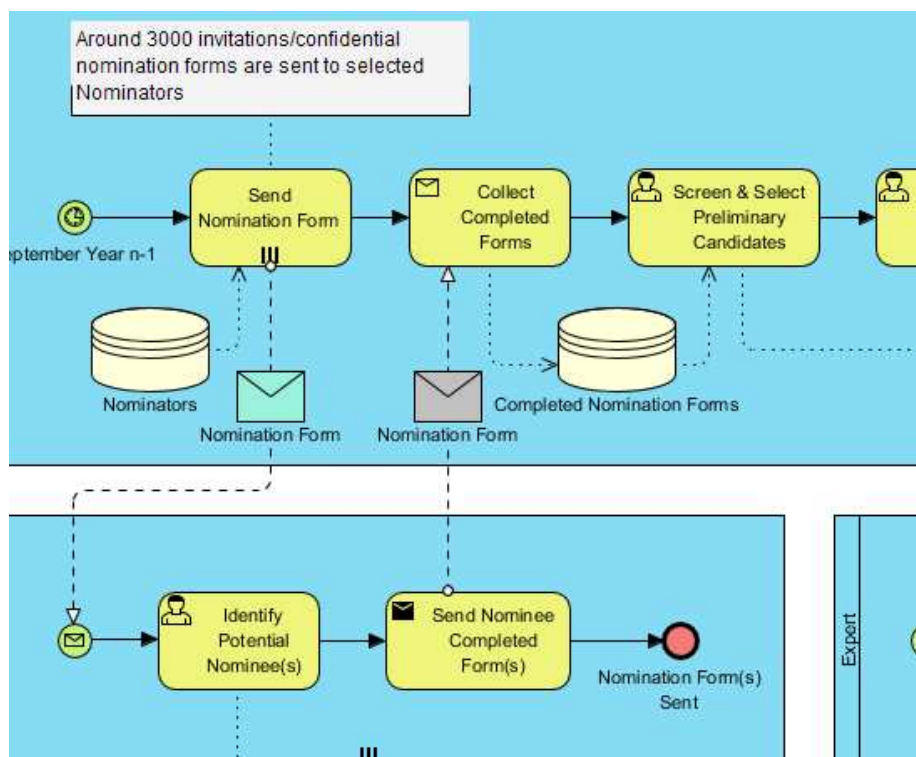
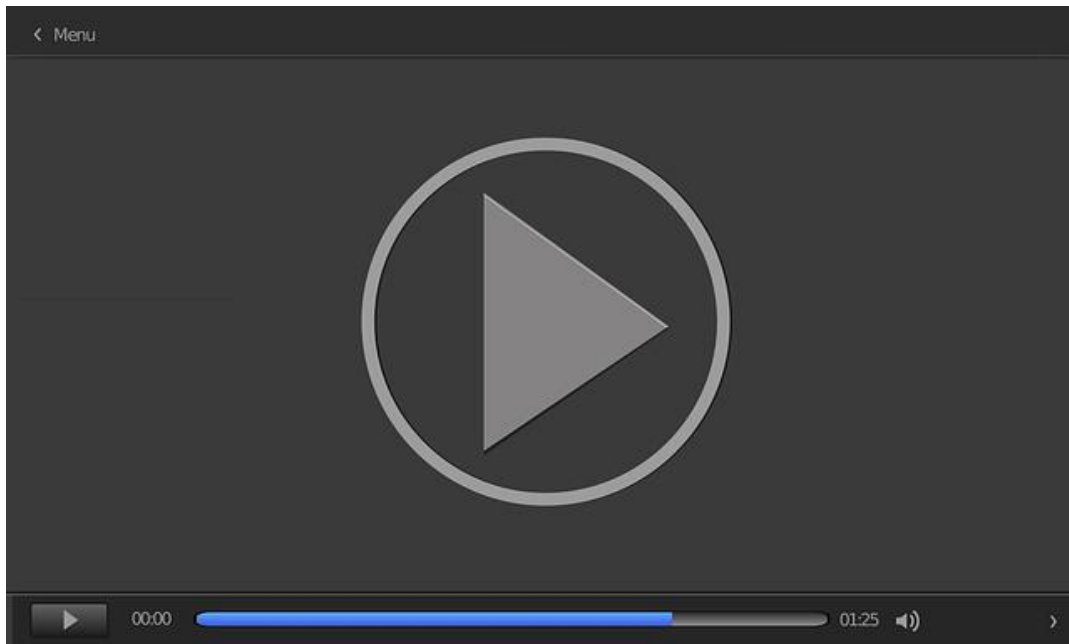


Figura 12. Vista de Visual Paradigm. Fuente: Visual Paradigm, s. f.

En este vídeo, *Creando nuestros propios metamodelos con Web Generic Modeling Environment (WebGME)*, se aprenderá a utilizar la herramienta WebGME, que no es la típica herramienta para modelar, pero puede ser muy útil para cuando es necesario crear con lenguajes propios.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=f6e77882-6fb4-49ad-ab4a-aec300cd973f>

3.6. Referencias bibliográficas

Altova. (s. f.). *UML Software Modeling Tool*. <https://www.altova.com/umodel>

Eclipse. (s. f.). *Eclipse Papyrus*. <https://www.eclipse.org/papyrus/>

Gleek.io. (s. f.). *Create diagrams without touching your mouse*. <https://www.gleek.io/>

Inflectra. (s. f.). *Perfecto for your team*. <https://www.inflectra.com/SpiraTeam/>

Intland software (s. f.). *Codebeamer*. <https://intland.com/codebeamer/>

Lucid. (s. f.). *La aplicación de diagramación inteligente para todo el equipo*.
<https://lucid.co/es/producto/lucidchart>

Página de [StarUML](#).

PlantUML. (s. f.). *Diagrama de Casos de Uso*. <https://plantuml.com/es/>

Sparx System. (s. f.) *Enterprise Architect*. <https://sparxsystems.com/products/ea/>

Visual Paradigm. (s. f.). *Process Design Tool*. <https://www.visual-paradigm.com/features/process-design-tool/>

Modelado ágil con UML

Rumpe, B. (2017). *Agile modeling with UML: Code generation, testing, refactoring*. Springer International Publishing.

Este libro se centra en el tratamiento metodológico de UML/P y aborda tres temas centrales del desarrollo de software basado en modelos: la generación de código, la prueba sistemática de programas utilizando una definición basada en modelos de casos de prueba y la refactorización y transformación evolutiva de modelos.

Aprendiendo UML 2.0

Miles, R. y Hamilton, K. (2006). *Learning UML 2.0*. O'Reilly Media.

El lenguaje de modelado unificado (UML) lo ayuda a administrar esta complejidad. Ya sea que esté buscando usar UML como un lenguaje de diseño, una herramienta de boceto o como un lenguaje de programación, este libro le brindará la información necesaria sobre cómo aplicar UML a su proyecto. Si bien hay muchos libros disponibles que describen UML, *Learning UML 2.0* le mostrará cómo usarlo.

1. El ámbito que se ocupa de la forma en la que se puede gestionar el ciclo completo de vida de la construcción de un software, desde la idea hasta su puesta en marcha, experimentación, implementación y soporte, se denomina:

- A. *Software development life cycle* (SDLC).
- B. *Application lifecycle management* (ALM).
- C. *Product lifecycle management* (PLM).
- D. *Computer-aided software engineering* (CASE).

2. Relaciona las características con las diversas plataformas de ingeniería del software:

Enterprise Architect	1	A	Modelado en colaboración web
Plant UML	2	B	Ingeniería basada en modelos de la fundación eclipse
Papyrus	3	C	Genera diagramas desde texto
Lucidchart	4	D	Solución comercial de Sparx Systems

3. Selecciona qué herramienta CASE soporta los siguientes diagramas: diagramas de secuencia, casos de uso, clases, mapa mental, *wireframes*, tiempo, Estructura de descomposición del trabajo:

- A. Plant UML.
- B. Papyrus.
- C. Star UML.
- D. Todas las anteriores.

4. ¿Qué solución de ALM funciona bajo Azure DevOps Server?

- A. MicroFocus ALM.
- B. Team Foundation Server (TFS).
- C. Microsoft ALM.
- D. Todas las anteriores.

5. Relaciona la plataforma con su característica:

Altova UModel	1	A	Trazabilidad completa desde los requisitos
Gleek.io	2	B	Permite agregar hipervínculos a cualquier elemento en cualquier diagrama UML
Lucidchart	3	C	Se modela con el teclado y no con el ratón
Enterprise Architect	4	D	Se modela online vía arrastrar y soltar

6. Marca aquella característica que no forma parte de las plataformas ALM:

- A. Gestión de requerimientos.
- B. Gestión de proyectos.
- C. Colaboración y comunicación.
- D. Análisis de modelos.

7. Selecciona qué herramienta CASE carga modelos extremadamente grandes en segundos:

- A. Enterprise Architect.
- B. Altova UModel.
- C. Papyrus.
- D. Plant UML.

8. Selecciona qué herramienta CASE es compatible con UML 2.5:
- A. Papyrus.
 - B. Visual Paradigm.
 - C. Star UML.
 - D. Todas las anteriores.
9. Marca aquella característica que no forma parte de las plataformas CASE:
- A. Gestión de proyectos y carteras.
 - B. Generación de documentación.
 - C. Elaboración de diagramas.
 - D. Generación de código.
10. Marca qué elemento no forma parte de los pilares *application lifecycle management* (ALM):
- A. Trazabilidad.
 - B. Gestión de equipos.
 - C. Automatización de procesos de alto nivel.
 - D. Visibilidad del progreso del desarrollo.

Plataformas de Desarrollo de Software

Tema 4. Plataformas low-code y no-code

Índice

Esquema

Ideas clave

- 4.1. Introducción y objetivos
- 4.2. Contexto de las plataformas low-code y no-code
- 4.3. Valor y utilidad de las plataformas low-code y no-code
- 4.4. Principales plataformas low-code y no-code comerciales
- 4.5. Principales plataformas low-code y no-code abiertas
- 4.6. Plataformas aceleradoras
- 4.7. Referencias bibliográficas

A fondo

- Low-Code for Dummies
- Curso online de desarrollo de aplicaciones web con OutSystems
- OutSystems Education Program

Test



4.1. Introducción y objetivos

En este tema se estudian algunas de las plataformas que nos ayudan a **automatizar** el desarrollo del software.

A la hora de desarrollar un programa, podemos escribir su código fuente o, para agilizar el proceso, utilizar diversos marcos o plataformas que nos faciliten el proceso mediante una mayor abstracción o el encapsulamiento de funcionalidades. Para ir reduciendo la curva de aprendizaje tecnológico, se han ido **creando herramientas** como generadores de código, marcos *full-stack* o las denominadas plataformas de bajo código (*low-code* y *no-code*).

En este tema posicionaremos estas plataformas en el desarrollo de software, detallaremos su relevancia actual y su clara tendencia creciente en los próximos años, resumiremos su utilidad y los posibles criterios para adoptarlas y veremos con mayor detalle las principales comerciales y abiertas.

Con el estudio de este tema pretendemos alcanzar los siguientes objetivos:

- ▶ Comprender **el valor y la utilidad** que tienen las plataformas *low-code* y *no-code* para la automatización de la construcción del software.
- ▶ Identificar los criterios principales que nos ayuden a seleccionar las plataformas en cada caso.
- ▶ Conocer las principales plataformas *low-code* y *no-code* comerciales y abiertas.
- ▶ Conocer algunas herramientas de automatización vía generación de código.
- ▶ Saber utilizar algunas de las principales plataformas *low-code* y *no-code*.

4.2. Contexto de las plataformas low-code y no-code

Toda la historia de la ingeniería de software se resume en acciones para elevar el nivel de abstracción (Booch, 2018). De esta forma, el desarrollo de bajo código (*low-code*) ha **evolucionado** para aprovechar las herramientas de diseño visual, como los modeladores de arrastrar y soltar y la creación de interfaces de apuntar y hacer clic, para permitir la rápida creación, lanzamiento, uso y cambio de potentes aplicaciones comerciales.

Los nuevos términos **low-code** y **no-code** representan en su esencia muchas de las características perseguidas con las plataformas basadas en modelos (*model-driven* o *model-based*), si bien en los últimos años han adquirido una gran popularidad en el mercado (Cabot, 2020).

Forrester (s. f.) define una plataforma *low-code* como aquella que permite **el desarrollo y la entrega rápida** de aplicaciones con una codificación manual mínima y una configuración e implementación.

Gartner (s. f. b) define una plataforma de aplicaciones empresariales de bajo código (*low-code application platform*, LCAP) como aquella que proporciona un **rápido desarrollo de aplicaciones** utilizando técnicas *low-code* y *no-code*, como el diseño y desarrollo de aplicaciones declarativas basadas en modelos, junto con la implementación simplificada de aplicaciones con un solo botón. Una LCAP normalmente crea metadatos e interpreta esos metadatos en tiempo de ejecución y abstrae la infraestructura del servidor subyacente para facilitar su uso y muchas también permiten extensiones de programación de procedimientos opcionales. Aquellas plataformas sin código (*no-code*) se refieren a una LCAP que solo requiere la entrada de **información visual o textual** para generar la aplicación, sin necesidad.

La Figura 1 intenta representar el posicionamiento de esta nueva categoría de plataformas que ayudan al desarrollo de software:



Figura 1. Contexto de las plataformas *low-code* y *no-code*. Fuente: elaboración propia.

- ▶ En su capa inferior, tendríamos el código fuente, es decir, el enfoque sin usar ninguna plataforma. En este nivel, el equipo desarrolla la solución **desde cero y lo personaliza** a cualquier tipo de necesidad en base a su profundo conocimiento sobre las tecnologías utilizadas.
- ▶ El siguiente nivel son marcos que **facilitan el desarrollo** de una de las partes del sistema. Existen diversos proveedores y la integración de las capas es laboriosa y requiere de nuevo de un elevado conocimiento técnico.
- ▶ Los generadores de código **simplifican el inicio del desarrollo** al crear un esqueleto rápidamente. Una vez que se ha construido esa base, el resto se completa manualmente haciendo uso de las capas inferiores.

- ▶ Los marcos *full-stack* cubren el **desarrollo completo** de un sistema incluyendo la perspectiva de los datos, la lógica y la interfaz de usuario. Su curva de aprendizaje es menor que las capas inferiores aunque la personalización o flexibilidad es menor, por lo que son especialmente útiles para prototipar con rapidez soluciones.
- ▶ Por último, las nuevas *low-code* o *no-code* nos permiten **agilizar el desarrollo** con un alto nivel de abstracción, lo que simplifica la curva de entrada y favorece la ampliación de los equipos de desarrollo con perfiles no necesariamente técnicos. Como desventaja, la flexibilidad o personalización se limita a aquellos aspectos considerados, lo que hace, en ocasiones, imposible la adaptación al no disponer incluso del código fuente.

La tendencia de estas soluciones es tan relevante que las principales consultoras de referencia en el ámbito TI se encargan de analizar y revisar la situación del mercado. La consultora Gartner, en *Magic Quadrant for Enterprise Low-Code Application Platforms* (Wong y Iijima, 2021), posiciona a los principales fabricantes de esta forma:

Figure 1: Magic Quadrant for Enterprise Low-Code Application Platforms

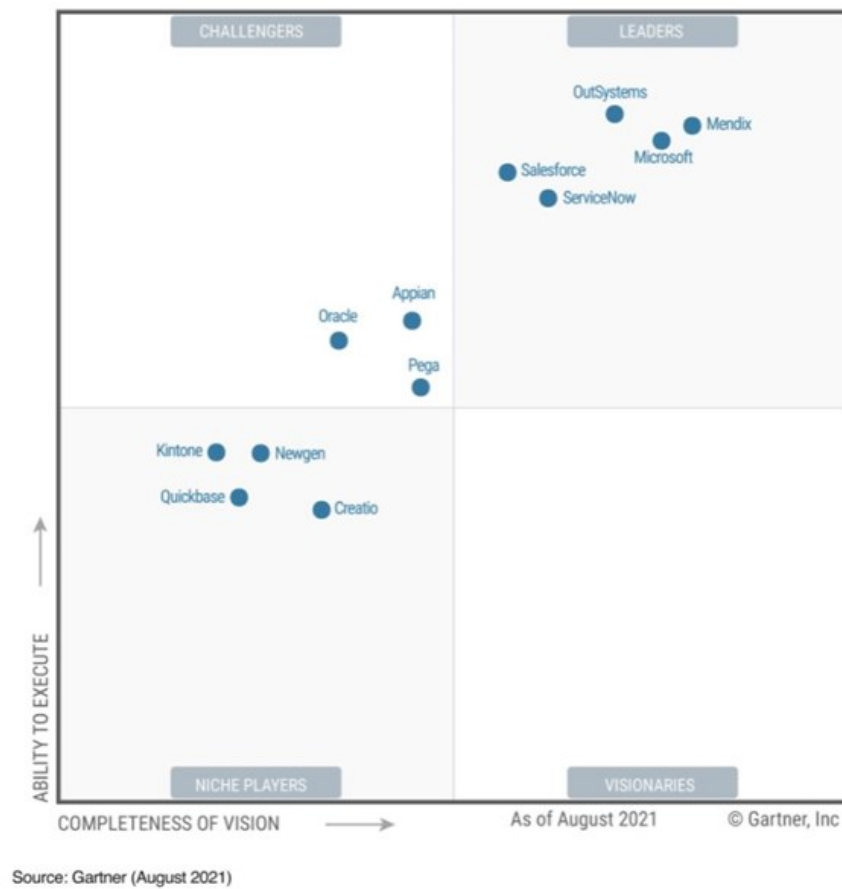


Figura 2. Gartner *Magic Quadrant for Enterprise Low-Code Application Platforms*. Fuente: Wong y Iijima, 2021.

- ▶ **Líderes:** OutSystems, Mendix, Microsoft PowerApps, Salesforce y ServiceNow.
- ▶ **Challengers:** Appian, Oracle y Pega.
- ▶ **Niche players:** Newgen, Kintone, Quickbase y Creatio.

Forrester, en su revisión de plataformas *low-code* (Bratincevic et al., 2021), destaca:



Figura 3. Plataformas de desarrollo *low-code* para desarrolladores profesionales, segundo trimestre de 2021 según Forrester. Fuente: Bratincevic et al., 2021.

- ▶ **Líderes:** OutSystems, Mendix, Microsoft PowerApps y ServiceNow.
- ▶ **Strong performers:** Appian, Salesforce, HCL Software y Pegasystems.
- ▶ **Contenders:** AgilePoint, Oracle, Unqork y Thinkwise.
- ▶ **Challengers:** GeneXus y Wavemaker.

OutSystems se ha posicionado como líder de soluciones *low-code* y *no-code* de propósito general y tiene una valoración de 9500 millones de euros (junio 2021).

El auge de estas plataformas ha llevado a la creación de **portales específicos** en los que se analizan y comparan las diversas soluciones, como el portalNoCodeJournal (s. f.):

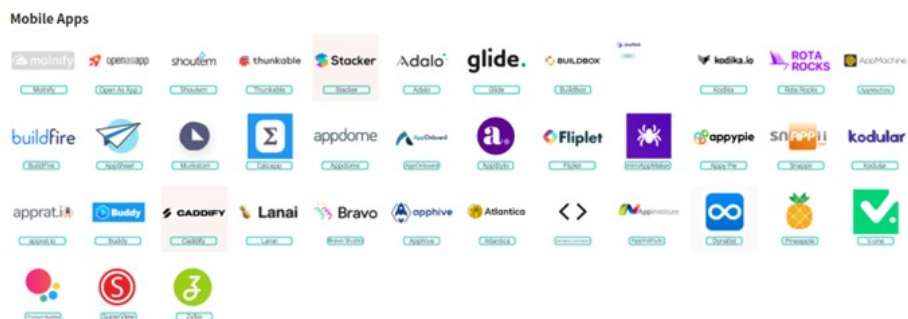


Figura 4. Vista de plataformas *low-code* y *no-code* para construir aplicaciones móviles.

Fuente: NoCodeJournal, s. f.

4.3. Valor y utilidad de las plataformas low-code y no-code

La demanda de creación de software está **aumentando exponencialmente**. En el período 2018-2023 se estima que se crearán unos 500 millones de nuevas aplicaciones a nivel mundial, lo que supondría una superación en el número de aplicaciones generadas en los últimos cuarenta años (Barcelona Digital Talent, 2022). Por ello, se prevé que, para el año 2025, el 70 % de las nuevas aplicaciones desarrolladas por las empresas se llevarán a cabo mediante tecnologías *no-code* y *low-code* (Goasduff, 2021). Este auge en el uso de estas plataformas derivará en un **nuevo perfil profesional** con capacidad de desarrollo, pero sin conocimientos profundos en lenguajes y tecnologías de programación.

Un desarrollador ciudadano (*citizen developer*) es un empleado que crea **capacidades de aplicación** para su propio consumo o el de otros, utilizando herramientas que no están prohibidas por los Departamentos de Tecnologías de la Información (TI) corporativas. Un desarrollador ciudadano es una persona, no un título o rol específico. Reportan a una unidad de negocio o función que no sea TI (Gartner, s. f. a). Se estima que, para el año 2023, el número de *citizen developers* en las organizaciones será **cuatro veces superior** al número de desarrolladores con conocimientos en programación (Carr, 2021).

El valor y la utilidad de estas plataformas, con carácter general, es **reducir el esfuerzo y coste** asociado al desarrollo de software, así como acelerar al máximo el proceso de puesta en producción con perfiles profesionales sin elevados conocimientos tecnológicos. En particular, algunas de las propuestas de valor de estas soluciones son:

- **Rentabilidad:** las herramientas de desarrollo *low-code* a menudo son más rentables que la alternativa del desarrollo de aplicaciones tradicional.

- No es necesario contratar un equipo de desarrollo interno, que es un gran punto de venta para las organizaciones que no se especializan en el desarrollo de aplicaciones.
- Cuando los nuevos empleados se unen a la empresa, pueden aprender herramientas de desarrollo *low-code* mucho más rápido que un lenguaje de programación desconocido.
- Las aplicaciones a menudo se pueden desarrollar mucho más rápido con herramientas *low-code* que con el desarrollo tradicional. Los usuarios de la aplicación disfrutarán antes de los beneficios, mientras que los desarrolladores pueden dedicar más tiempo a ser productivos en otras tareas.
- ▶ **Facilidad de uso:** las herramientas de desarrollo *low-code* son más fáciles de usar, con una curva de aprendizaje más suave y una interfaz de usuario más simple que el desarrollo de aplicaciones tradicional. Esta ventaja conlleva a que los perfiles requeridos para el desarrollo de software no requieren un profundo conocimiento tecnológico, lo que facilita la creación de equipos de desarrollo en las organizaciones.
- ▶ **Agilidad empresarial:** gracias a su menor coste y facilidad de uso, las plataformas *low-code* también tienen un efecto dominó al mejorar la agilidad de la organización en su conjunto. Los desarrolladores pueden acortar considerablemente el proceso de desarrollo, implementando nuevas aplicaciones más rápidamente en respuesta a los cambios en el panorama empresarial en rápida evolución.
- ▶ **Gobernanza de TI más sólida:** cuando una organización no puede afrontar el número de desarrollos internos requeridos, los empleados suelen intentar solventar sus necesidades por su cuenta. A este concepto se le denomina **TI en la sombra** (*shadow IT*), haciendo mención del uso de software que los empleados usan sin el permiso de su departamento de TI. Se trata de un problema relevante, puesto que, al escapar del control y conocimiento del equipo de TI, se dificulta su mantenimiento, evolución y estrategia empresarial, a la vez que puede suponer un riesgo a la

seguridad. Sin embargo, los orígenes de la TI en la sombra a menudo se encuentran en la insatisfacción de los empleados con las aplicaciones empresariales disponibles para ellos. Las herramientas *low-code* permiten a los usuarios tomar el desarrollo de aplicaciones en sus propias manos y crear el software que necesitan para hacer su trabajo, al mismo tiempo que le brindan más visibilidad y supervisión de TI.

- ▶ **Validación de ideas de negocio:** la construcción de productos mínimos viables (*minimal viable product*, MVP) que permitan validar hipótesis de negocio se convierte en una realidad con las soluciones *no-code*, en las que en cuestión de minutos en lugar de semanas se pueden desplegar en producción soluciones globales.
- ▶ **Simplificar el mantenimiento:** con el uso de plataformas *low-code* el mantenimiento evolutivo de la aplicación se simplifica, ya que se sigue usando un entorno de mayor nivel de abstracción que el código fuente que vuelve a ser regenerado de forma automática por la herramienta.
- ▶ **Menor coste de pruebas:** las plataformas *low-code* realizan la generación automática del código, por lo que toda la verificación de que el sistema está bien creado no es necesario hacerla y los desarrolladores pueden enfocarse únicamente en las pruebas de validación que permitan asegurar la aceptación de los requisitos.
- ▶ **Simplificación del proceso de operación:** la mayoría de las soluciones *low-code* incluyen la generación automática del código y su despliegue en infraestructuras en la nube, por lo que el proceso de paso a producción en entornos operativos se simplifica. Sin embargo, en el caso de necesitar despliegues específicos, es necesario analizar *a priori* si la herramienta en particular va a satisfacer o no esas necesidades de la empresa.

A la hora de seleccionar las plataformas *low-code* y *no-code*, cada organización debe analizar múltiples criterios de cara a valorar la idoneidad del uso de una concreta bien para un proyecto o con carácter general. Entre los principales criterios, destacan:

- ▶ La **finalidad** de la plataforma en la que existen, con carácter general, estas categorías:
 - **UI-driven**: enfocadas a soluciones con principal foco en la interfaz de usuario.
 - **Data-driven**: para aplicaciones que gestionan principalmente datos.
 - **Business process driven**: en soluciones de automatización de procesos.
 - **General purpose**: aquellas que permiten combinar la perspectiva del proceso, lógica, datos e interacción con el usuario indistintamente.
- ▶ El **coste** de licenciamiento y uso de la plataforma, diferenciando los costes fijos de los equipos de desarrollos y los posibles derivados del uso de las aplicaciones creadas.
- ▶ El riesgo de **lock-in**, entendido como la dependencia que tendríamos de una plataforma en el futuro. Si el código generado no es accesible, toda evolución futura del sistema obliga a mantener el acuerdo con el proveedor de la plataforma a medio y largo plazo.
- ▶ La **curva de aprendizaje**, que puede mitigarse con disponibilidad de contenidos formativos y soporte por parte del proveedor.
- ▶ La capacidad que tiene la solución de facilitar la **integración** con software o sistemas existentes ya sea a nivel de conectores o API abiertos.
- ▶ La capacidad de **generar código** en un entorno tecnológico concreto que sea compatible con los estándares y políticas corporativas.
- ▶ La capacidad de **desplegar** la solución en entornos *on-premise* o en la nube concretos.
- ▶ La posibilidad de **adaptar** o **extender** el comportamiento o la visualización de forma concreta a las necesidades particulares que tengamos.

Debido al elevado número de posibles criterios y a la gran cantidad de soluciones disponibles, la propia elección de una solución *low-code* que se convierta en una base corporativa para el desarrollo en una organización se ha convertido en una tarea compleja. Por ello, existen algunas organizaciones que han creado sistemas de ayuda a la toma de esta decisión (Pal, 2020) e incluso calculadoras que permiten estimar el retorno de la inversión en la adquisición de una plataforma *low-code* empresarial (DronaHQ, s. f.).

[illegible]

Figura 5. Herramienta de apoyo a la elección de plataformas low-code. Fuente: Pal, 2022.

4.4. Principales plataformas low-code y no-code comerciales

Algunas de las herramientas comerciales mejor valoradas por los analistas para automatizar el desarrollo de aplicaciones *low-code* son:

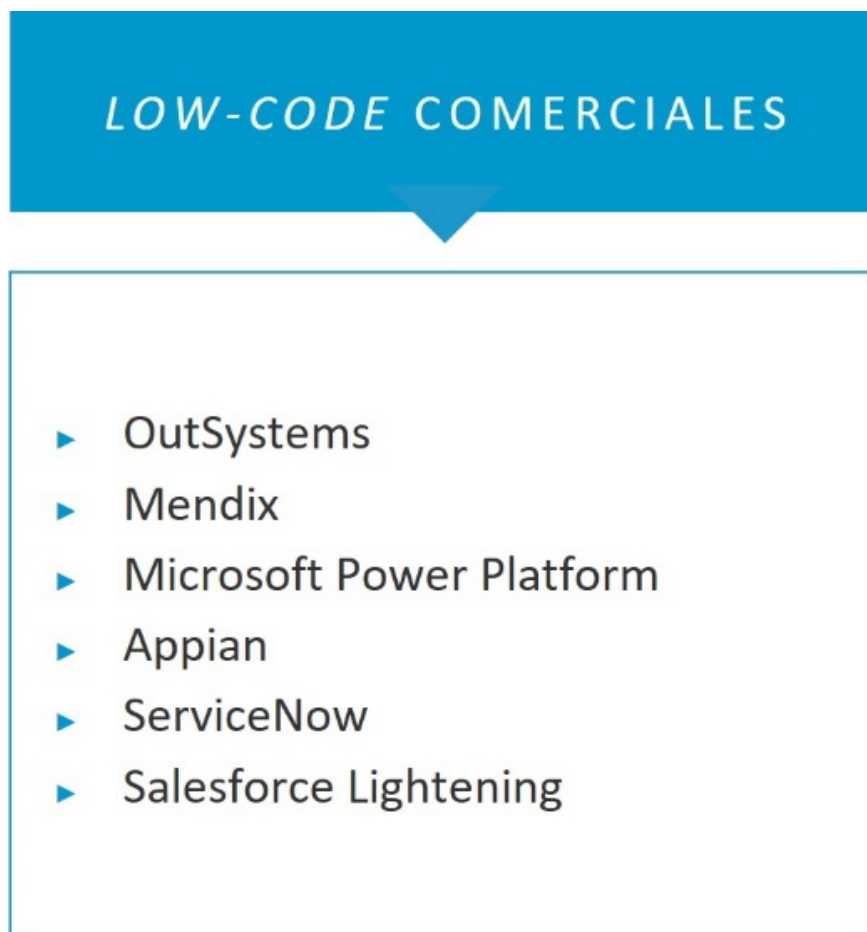


Tabla 1. Principales plataformas *low-code* y *no-code* comerciales. Fuente: elaboración propia.

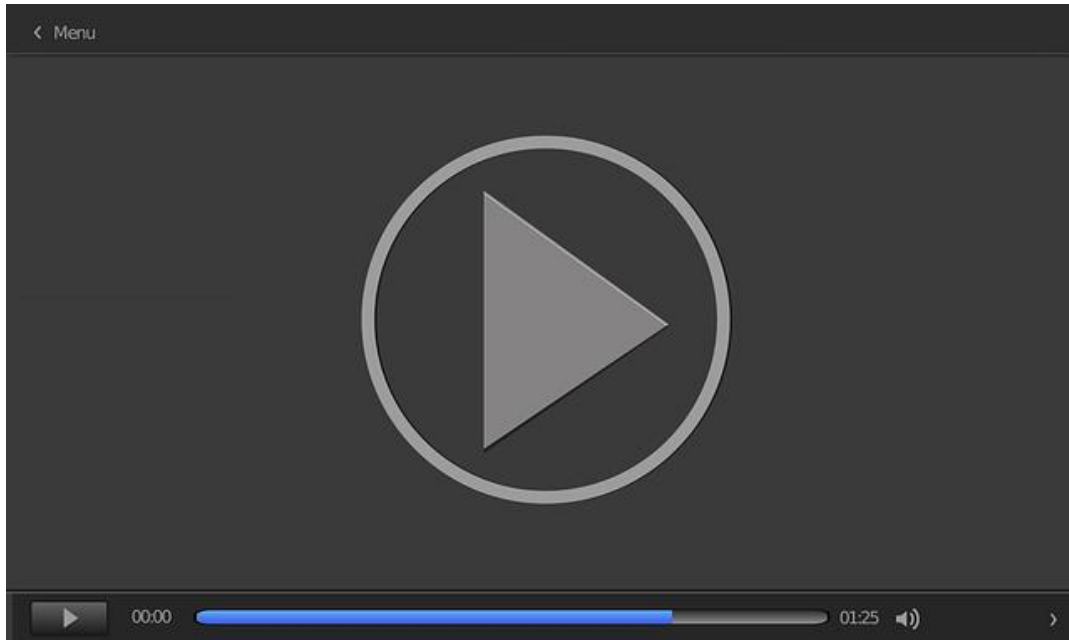
OutSystems

Es la plataforma *low-code* número uno bajo el criterio de diversos analistas como Gartner o Forrester. Permite que la aplicación de nivel empresarial se desarrolle mediante el **uso de su IDE** (entorno de desarrollo integrado) y se despliegue en su

nube. El IDE le permite crear aplicaciones para computadoras de escritorio, tabletas y dispositivos móviles. La aplicación también se puede generar como aplicaciones nativas. Admite la **integración** con cualquier sistema empresarial externo, base de datos o aplicación personalizada a través de conectores de código abierto prediseñados, servicios en la nube populares y API. Incluye impresionantes plantillas de interfaz de usuario modernas preconstruidas para escritorio, tabletas y aplicaciones móviles. Los contenidos formativos y su capa de abstracción hacen que la herramienta no sea solo para desarrolladores profesionales, sino que también esté diseñado para líderes de TI, dueños de negocios, arquitectos empresariales y *citizen developers*.

En el apartado A fondo tendrás más recursos para poder aprender a utilizar *OutSystems*, así como acceder al programa educativo para utilizar la plataforma y su entorno en la nube de forma gratuita, lo que permite que se pueda llegar a alcanzar una certificación.

En este vídeo, *OutSystems: desarrollando una aplicación en 5 minutos*, aprenderemos a desarrollar una aplicación con la plataforma *no-code* OutSystems.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=3367bd5d-8e7a-4010-a3c4-aec300cd9ac5>

Algunas de sus características son:

- ▶ **Velocidad inigualable:** desde el desarrollo visual de alta productividad hasta potentes herramientas para implementar y administrar las aplicaciones.
- ▶ **Desarrollo visual *full-stack*:** interfaz de usuario de arrastrar y soltar, procesos de negocio, lógica y modelos de datos para crear aplicaciones multiplataforma *full-stack*. Permite añadir nuestro propio código cuando sea necesario.

- **Implementación y despliegue con un solo clic:** entregar aplicaciones y actualizarlas con un solo clic. La herramienta verifica automáticamente las dependencias y maneja todos los procesos de generación, compilación y despliegue de forma autónoma.
- **Comentarios en la aplicación:** los usuarios pueden compartir comentarios de voz y escritos directamente dentro de la aplicación, lo que simplifica todo el proceso de gestión de cambios.
- **Refactorización automática:** analiza todos los modelos e inmediatamente refactoriza las dependencias. Ante una modificación de una tabla de base de datos todas sus consultas se actualizarán automáticamente.
- **Aplicaciones móviles:** facilita la creación de experiencias móviles de gran apariencia con sincronización de datos fuera de línea, acceso a dispositivos nativos y lógica comercial en el dispositivo.
- **Arquitectura escalable:** permite combinar microservicios con un análisis profundo de dependencias. Permite la creación y actualización de servicios y aplicaciones reutilizables de forma rápida y a gran escala.

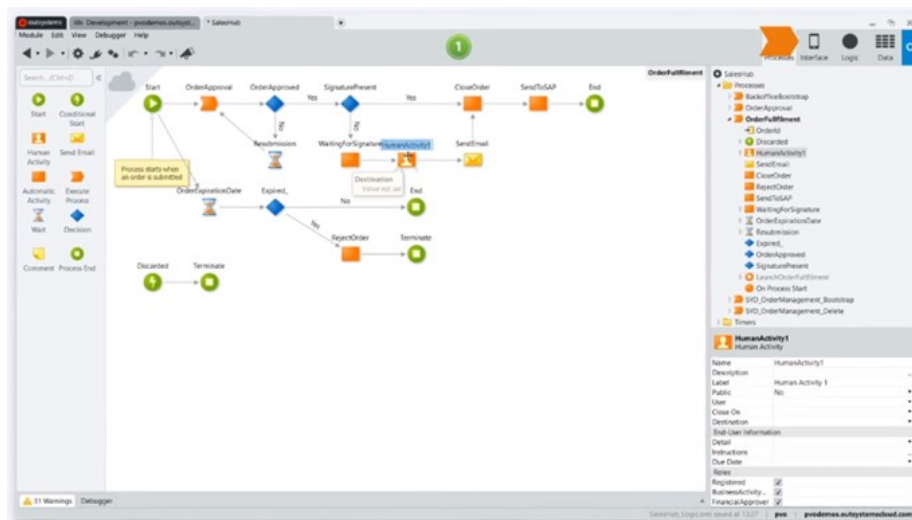


Figura 6. Vista del IDE de OutSystems. Fuente: OutSystems, s. f.

Mendix

Es una plataforma de aplicaciones de **alta productividad** que le permite crear y mejorar continuamente aplicaciones móviles y web a escala. Mendix Studio es un IDE que le permite diseñar aplicaciones web y móviles mediante una función de arrastrar y soltar. La plataforma está diseñada para **acelerar la entrega** de aplicaciones empresariales en todo el ciclo de vida de desarrollo de aplicaciones, desde la ideación hasta la implementación y las operaciones.

Algunas de sus características son:

- ▶ Como solución *no-code*, proporciona un entorno de modelado de aplicaciones visuales **basado en la web**, que está diseñado para expertos en dominios comerciales.
- ▶ Como solución *low-code*, también proporciona un amplio y potente entorno de modelado de aplicaciones **visuales basado en escritorio**, que está diseñado para desarrolladores profesionales y se puede integrar con IDE de codificación para ampliar las capacidades.
- ▶ Se dirige a líderes de TI, desarrolladores de negocios y desarrolladores profesionales. Para crear e implementar aplicaciones, no necesita tener **ningún conocimiento de codificación**.
- ▶ Permite **modelar el dominio**: creando objetos, atributos, validaciones y modelos de datos con un modelador de datos basado en UML (lenguaje unificado de modelado).
- ▶ **Facilita el modelo de interfaz de usuario**: a través de un diseñador de páginas gráficas *what you see is what you get* (WYSIWYG) se pueden crear interfaces de usuario web y móviles totalmente adaptables.

- ▶ **Microflujos y nanoflujos:** se pueden utilizar modelos visuales para crear una lógica empresarial del lado del cliente, aplicaciones móviles enriquecidas fuera de línea, así como una interacción de usuario dinámica de alta capacidad de respuesta.
- ▶ **Depuración en vivo:** es posible aislar los posibles problemas mediante sistemas de depuración y avance visual a través de la lógica e interroga los datos para resolver rápidamente los errores de la aplicación.

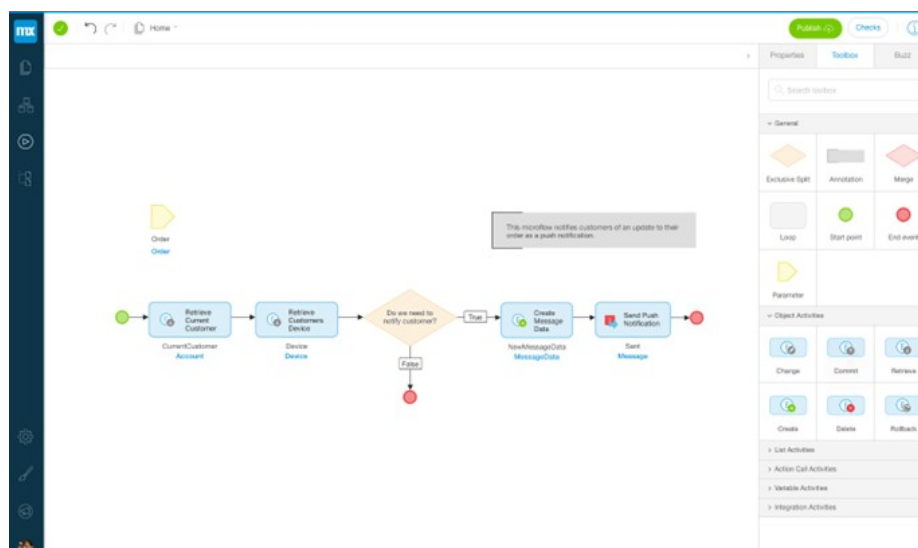


Figura 8. Vista del IDE Mendix Studio. Fuente: Mendix, s. f.

Microsoft Power Platform

Es la *suite* de herramientas de **automatización del desarrollo de código** de Microsoft. Permite a los usuarios comerciales crear interfaces de usuario, flujos de trabajo comerciales y modelos de datos e implementarlos en la nube Azure de Microsoft. Las cuatro ofertas de Microsoft Power Platform son Power BI, Power Apps, Power Automate y Power Virtual Agents. Cada uno de estos productos tiene su propio modelo de precios, por usuario, a partir de 9,99 \$ por mes. Una vez que las aplicaciones se construyen con las herramientas de Power Platform, las aplicaciones de producción se pueden implementar en la nube de Azure.

Algunas de sus características son:

- ▶ **Power BI:** enfocada en facilitar la toma de decisiones empresariales fundamentadas y fiables proporcionando a todas las personas implicadas conocimientos basados en los datos.
- ▶ **Power Apps:** centrada en facilitar la generación de aplicaciones personalizadas que resuelvan los desafíos corporativos.
- ▶ **Power Automate:** enfocada en la automatización de los procesos de la organización.
- ▶ **Power Virtual Agents:** centrada en facilitar la creación de *chatbots* para conversar con los clientes y empleados sin necesidad de escribir código.

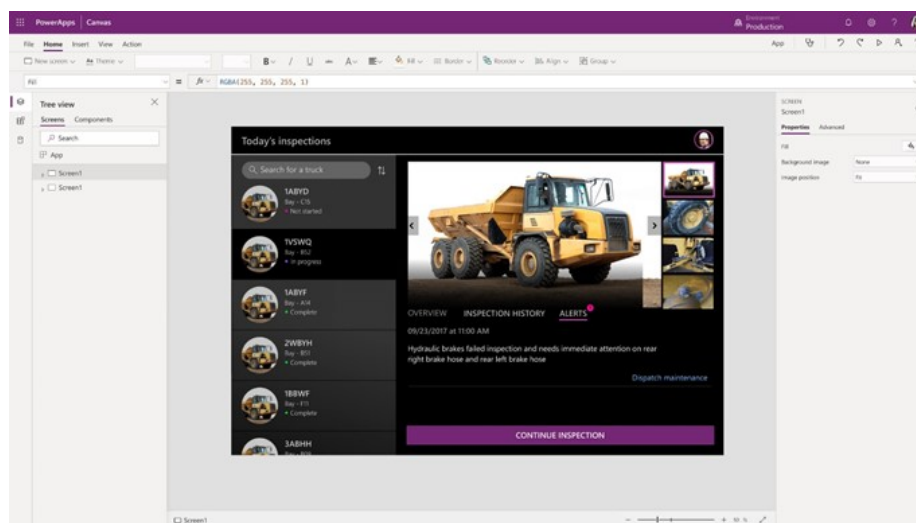


Figura 8. Vista de Microsoft Power Apps. Fuente: Microsoft, s. f.

Appian

Es una plataforma *low-code* basada en la nube. Su funcionalidad clave gira en torno a la **gestión de procesos empresariales** (*business process management*, BPM), la automatización de procesos (*robotic process automation*, RPA), la gestión de casos, la gestión de contenidos y la automatización inteligente. Es compatible con

implementaciones en la nube pública y en la nube de la herramienta basada en AWS, Google Cloud y Azure. El producto Appian Low-Code Automation Platform admite la automatización de desarrollo de aplicaciones de software *full-stack*.

Algunas de sus características son:

- ▶ **Automatización** de procesos y tareas rutinarias en un flujo de trabajo e integración de sistemas sin API.
- ▶ **Gestión de procesos empresariales:** diseño, ejecución, gestión y optimización de procesos complejos con iBPM líder en la industria.
- ▶ **Gestión de casos:** automatización del trabajo colaborativo y el manejo de excepciones.
- ▶ **Inteligencia artificial:** facilita que las aplicaciones funcionen mejor y de forma más inteligente con la IA (inteligencia artificial) integrada de Google, AWS y Azure.
- ▶ **Reglas de decisión:** permite definir rápidamente la lógica empresarial.
- ▶ **Diseñador visual *no-code*:** potencia la colaboración empresarial y de TI mediante el desarrollo visual para crear aplicaciones rápidamente.
- ▶ Potentes herramientas de **generación de código:** manejo de aplicaciones comerciales simples a complejas.
- ▶ **Desarrollo colaborativo:** permite superar la división entre profesionales del negocio y técnicos de desarrollo dibujando aplicaciones empresariales en lugar de codificarlas.



Figura 9. Vista de Appian. Fuente: Appian, s. f.

ServiceNow

Es una plataforma que permite el desarrollo de aplicaciones empresariales con muy poco o ningún código y soporta dispositivos móviles. Su principal factor diferencial es que es compatible con una amplia variedad de aplicaciones *plug-and-play* para *IT service management* (ITSM), operaciones de seguridad, servicio de atención al cliente y recursos humanos.

El mercado de plataformas *low-code* y *no-code* está en continuo movimiento por lo que los principales analistas del mercado (Forrester y Gartner) actualizan sus informes con carácter trimestral.

Salesforce Lightning

Esta plataforma es exclusiva para los **usuarios de Salesforce CRM** que desean crear sus propias experiencias de usuario y brindar soporte para computadoras de escritorio, tabletas y dispositivos móviles sin escribir ningún código. Es un marco para que los usuarios de Salesforce creen una interfaz de usuario empresarial y un flujo de trabajo en Salesforce sin involucrar a los desarrolladores.

4.5. Principales plataformas low-code y no-code abiertas

Algunas de las herramientas *open source* más populares, ordenadas alfabéticamente, para automatizar el desarrollo de aplicaciones *low-code* son:

LOW-CODE ABIERTAS	
▶ Abp.io ▶ BaseRow ▶ Budibase ▶ Convertigo ▶ Digdag ▶ Joget DX ▶ JUDO.codes ▶ La plataforma CUBA ▶ Metabase ▶ nuBuilder	▶ OpenCatapult ▶ OpenXava ▶ OS.bee ▶ Rentagi ▶ Saltcorn ▶ Skyve ▶ Stackstorm ▶ Teleport ▶ Tymly ▶ WebRatio

Tabla 2. Principales plataformas *low-code* y *no-code* abiertas. Fuente: elaboración propia.

Abp.io

ABP Framework es una **infraestructura completa** para crear aplicaciones web modernas siguiendo las mejores prácticas y convenciones de desarrollo de software. Ofrece una arquitectura para crear **soluciones de software** sobre las plataformas .NET y ASP.NET Core. Proporciona la infraestructura fundamental, las plantillas de inicio listas para la producción, los módulos, los temas, las herramientas, las guías y la documentación para implementar esa arquitectura correctamente y automatizar los detalles y los trabajos repetitivos tanto como sea posible.

BaseRow

Es una herramienta gratuita de **creación y administración de bases de datos** basada en la web de código abierto que no requiere habilidades previas de codificación o habilidades técnicas avanzadas como otras herramientas. Su principal propuesta de valor es ser una base de datos *no-code* de código abierto y una alternativa a Airtable. Permite la creación de bases de datos en línea sin la necesidad de salir del navegador.

Budibase

Es una plataforma *low-code* de código abierto para crear **aplicaciones internas**, como paneles de administración, herramientas internas o portales de clientes, entre otros. Cuenta con licencia GPLv3. Permite la integración con fuentes de datos externas como MongoDB, CouchDB, PostgreSQL, MySQL, Airtable, S3, DynamoDB o mediante conectores vía REST API. Al facilitar el control de acceso, tiene múltiples plantillas y soporta el despliegue en la nube del cliente.

Convertigo

Es una mezcla entre la plataforma *no-code* y *low-code*. Su objetivo es que exista una **colaboración** entre desarrolladores profesionales y los *citizen developers* para crear aplicaciones y herramientas listas para la empresa **en poco tiempo**. Ofrece una instalación local, una versión en la nube y una edición *mobile backend as a service* (mBaaS) para desarrolladores. Cuenta con una herramienta de creación móvil, interfaz de usuario visual de arrastrar y soltar, *back-end low-code*, convertidor REST/XML, convertidor REST/JSON, consola de administración y más. Ofrece soporte de desarrollo móvil completo para *progressive web applications* (PWA), iOS y Android.

Digdag

Es una solución empresarial que tiene como objetivo una implementación fácil, una configuración de múltiples nubes y una estructura modular para crear y escalar aplicaciones comerciales. Incluye un panel de administración enriquecido, soporte multilingüe, manejo de errores, herramienta de configuración y herramientas de control de versiones. La solución se basa en Java y Node.js. Es compatible con AWS, nube privada, nube de IBM y Digital Ocean.

Joget DX

Es una plataforma *low-code* de creación de aplicaciones para **facilitar la transformación digital** de las empresas. Combina la gestión de automatización de procesos comerciales, la personalización del flujo de trabajo con herramientas de desarrollo de aplicaciones *low-code*. Puede ejecutarse en la nube y en la infraestructura propia del cliente. Tiene una rica documentación, un panel de control fácil de usar con un generador visual que admite arrastrar y soltar e independiente del sistema operativo y la base de datos.

JUDO.codes

Es otra plataforma *low-code* destinada al uso empresarial en términos de modelado de datos, diseño y desarrollo de la interfaz de usuario. Ofrece más **flexibilidad** para los desarrolladores que otras plataformas *low-code* en esta lista. Está disponible para Windows, macOS y Linux. El portal contiene abundante **documentación** y un conjunto de **tutoriales** para los recién llegados. También proporciona una extensa **guía de usuario**, así como instrucciones de instalación para plataformas de aplicaciones.

Plataforma CUBA

Es un sistema de **desarrollo rápido de aplicaciones** con licencia Apache 2.0 para empresas. Viene con docenas de herramientas, como un entorno de desarrollo integrado, un estudio de creación de aplicaciones, una interfaz de línea de comandos y una infraestructura extensible sólida. Tiene un rico sistema de complementos, en los que se incluye uno para la gestión de procesos de negocio (*business process management*, BPM) que toma unos minutos para construir e instalar.

Metabase

Es una herramienta *low-code* abierta de generación de aplicaciones basadas en datos que admite una **amplia gama de back-ends** de bases de datos como MongoDB, MySQL, PostgreSQL, SQL Server, Oracle y más. Proporciona una capa para administrar registros de bases de datos, manipular datos, enfoque visual para registros manipulados, admite uniones, agregaciones múltiples, filtrado avanzado y búsqueda de texto completo. Es la solución definitiva para crear un *dashboard* productivo listo para la base de datos en minutos con una alta productividad y disponibilidad para la empresa. Incluye un conjunto de *widgets* visuales que incluyen: gráficos, mapas, mapas vectoriales SVG, análisis y una amplia documentación detallada para usuarios, desarrolladores y equipos de desarrollo y operaciones (DevOps).

nuBuilder

Es una herramienta abierta para crear aplicaciones de bases de datos empresariales **en minutos**. Es **autohospedado**, *low-code* digo con un sistema de *back-end* altamente personalizable. Utiliza PHP, HTML y JavaScript con base de datos MySQL. Permite crear aplicaciones de bases de datos basadas en web en minutos con herramientas para construir formularios rápidos e informes. Facilita la personalización de las aplicaciones con eventos de JavaScript y PHP que permiten administrar la experiencia del usuario y actualizar los registros de la base de datos.

OpenCatapult

Es una plataforma *low-code* de automatización DevOps de código abierto. Ayuda a los equipos DevOps a tener un mejor control de sus servidores mediante la gestión de tareas rutinarias automatizadas sobre la marcha. Desafortunadamente, solo está disponible para la plataforma Windows x64, pero los desarrolladores están planeando versiones de Linux y macOS en un futuro cercano.

OpenXava

Es una plataforma de creación de aplicaciones *low-code* con un enfoque principal en **la productividad, la simplicidad y la usabilidad**. Como sistema multiplataforma construido con tecnologías Java, se ejecuta en servidores Linux y Windows. Garantiza una alta productividad, una curva de aprendizaje funcional corta, un gran conjunto de funcionalidades para empresas y un diseño receptivo completo para dispositivos móviles y tabletas. Dispone de una **edición comunitaria gratuita** de código abierto, pero la empresa puede comprar diferentes ediciones con funciones adicionales y soporte.



Figura 10. Vista de DSL y resultado con OpenXava. Fuente: OpenXava, s. f.

S.bee

Es una herramienta de automatización del desarrollo de software de código abierto que **combina** el desarrollo de aplicaciones automatizadas y *no-code*, con *low-code* y tradicional. Basado en lenguajes específicos de dominio, está diseñado para usuarios que desean modelar aplicaciones comerciales personalizadas de **la manera más rápida y sencilla posible**. La plataforma fue creada por una empresa que se especializó en la creación de un sistema de planificación de recursos empresariales (ERP) durante más de treinta años. Ofrece un generador de diagramas y modelos visuales para crear modelos de datos de aplicaciones. El sistema está creado con Java y ofrece una instalación e integración con Eclipse.

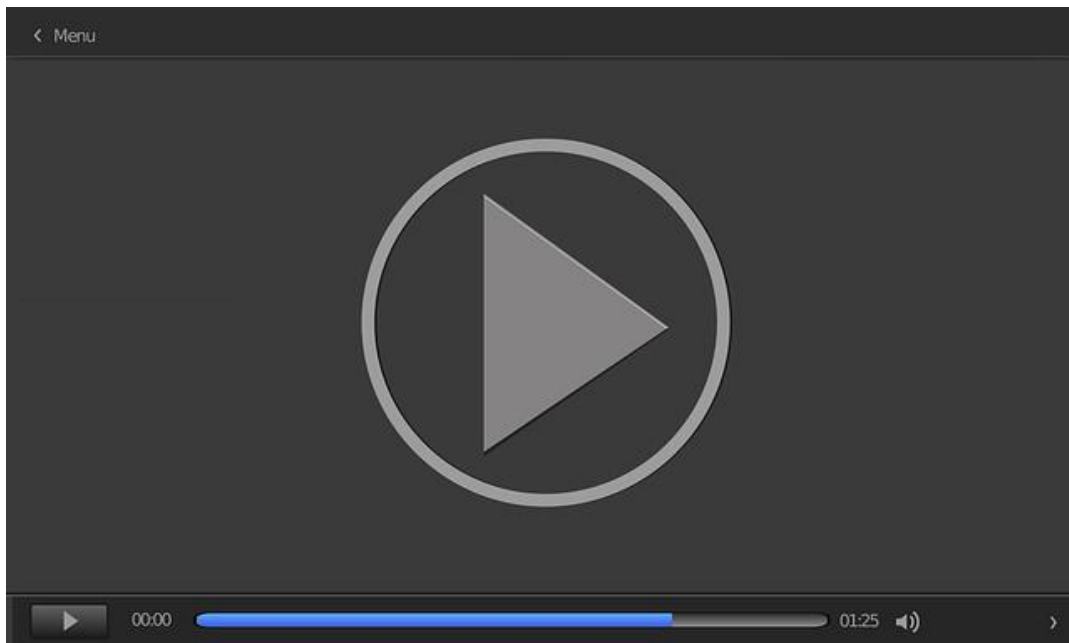
Rentagi

Es una plataforma de creación de aplicaciones de nivel empresarial *low-code* y enfocada al desarrollo de aplicaciones móviles. También es una solución completamente gratuita y de código abierto, lo que la convierte en una solución perfecta para **pequeñas y medianas empresas**. Viene con herramientas ricas y sofisticadas para crear aplicaciones rápidamente para una alta productividad. También ofrece una API amplia y fácil de usar para desarrolladores móviles.

Saltcorn

Es una plataforma *no-code* para crear aplicaciones web de bases de datos. A través de su panel de interacción web, se pueden definir bases de datos estructuradas, vistas y páginas. Los **usuarios sin experiencia** en programación pueden crear aplicaciones de base de datos ricas e interactivas en cuestión de minutos. Las empresas también pueden utilizarlo para crear sus herramientas de uso diario y remodelarlas sobre la marcha. Tiene una **lista impresionante de aplicaciones** de ejemplo que incluyen: blog, libreta de direcciones, sistema de gestión de proyectos, seguimiento de problemas, wiki, gestión de equipos y más. Cuenta con licencia MIT y se pueden desplegar aplicaciones en su nube o bien en su propio entorno local.

En el siguiente vídeo *Saltcorn: una herramienta no-code open source y extensible para el desarrollo de aplicaciones basadas en datos* aprenderemos a utilizar la plataforma *no-code* de código abierto Saltcorn para desarrollar una aplicación web de gestión de datos.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=45368e02-ac48-4171-a697-aec300cd9c98>

Skyve

Es una plataforma de creación de software de código abierto para empresas. **Puede funcionar como *low-code* o *no-code*** y se orienta al desarrollo rápido de aplicaciones empresariales. Es compatible con diferentes motores de base de datos. Viene con un rico conjunto de API para el desarrollo de bajo nivel y un asistente de creación de aplicaciones para el desarrollo *no-code*. Se compone de una serie de módulos como:

- ▶ Aplicación Builder: para crear aplicaciones móviles con React Native.
- ▶ Módulo Skyve Bus: se integra con otros servicios de terceros.
- ▶ Skyve Confidence: proporciona una funcionalidad de prueba para el desarrollo basado en pruebas.
- ▶ Skyve Portal: que amplía las aplicaciones empresariales con funcionalidades de portal web.
- ▶ Skyve CRM: aplicación Skyve CRM personalizada.
- ▶ Skyve Replica: ofrece una sincronización perfecta entre las instancias distribuidas de Skyve.

Stackstorm

Es una plataforma especializada para simplificar la integración de aplicaciones, ya que está diseñada como un paraguas para **conectar, administrar y monitorear** aplicaciones empresariales dentro de una plataforma. Se centra en arquitecturas dirigidas por eventos, el diseño de flujo de trabajo personalizado y las funciones y permisos de los usuarios. La automatización basada en eventos ofrece sensores, disparadores, acciones definidas, reglas, flujo de trabajo y auditorías con un rico sistema de registro.

Teleport

Es una plataforma de desarrollo *low-code* **gratuita**. Permite a los desarrolladores, diseñadores y creadores de contenido crear fácilmente sitios web estáticos receptivos de manera visual, sin escribir ningún código, todo mientras colaboran en tiempo real. Su creador visual de sitios web viene con una **interfaz de arrastrar y soltar** fácil de usar para agregar y personalizar rápidamente bloques de contenido. Los usuarios también pueden definir componentes reutilizables con propiedades personalizadas, plantillas de tienda y secciones. El editor puede generar

automáticamente código y diseños CSS receptivos para los marcos JS más populares como React, Vue, Angular, Next.js o CMS (sistema de gestión de contenidos) como Gatsby. El kit de desarrollo de software (SDK) para los generadores de código es de código abierto, por lo que los desarrolladores pueden ampliar fácilmente la plataforma para admitir nuevos lenguajes o marcos de programación.

Tymly

Es una plataforma empresarial *low-code* para crear aplicaciones de servidor escalables. Se lanza como un proyecto de código abierto bajo licencia MIT. Introdujo un concepto de **modelo que encapsula** los procesos comerciales, las funciones y los flujos de trabajo en un modelo. Tiene un ecosistema y una tienda de planos que protegen una gran cantidad de recursos de desarrollo. Los modelos se guardan en esquemas JSON, mientras que los datos se almacenan en la base de datos PostgreSQL. Los desarrolladores pueden escribir sus modelos definiendo sus requisitos, funciones comerciales y flujo de trabajo en esquemas JSON.

WebRatio

Es una herramienta de desarrollo *low-code* para crear aplicaciones web, móviles y de automatización de procesos de negocio. Basada en el estándar *interaction flow modeling language* (IFML) y el paradigma visual *model-driven*, permite la creación de aplicaciones web y móviles.

En el apartado A fondo tendrás más recursos para poder aprender a utilizar Webratio Platform, así como acceder al programa universitario para utilizar la plataforma de forma gratuita.



Figura 13. Vista de un diagrama IFML con WebRatio. Fuente: WebRatio, s. f.

4.6. Plataformas aceleradoras

Aunque las herramientas de generación de código no se consideran propiamente herramientas *low-code*, vamos a ver alguna adicional, ya que pueden ser de gran utilidad como aceleradoras del proceso de desarrollo.

Las herramientas de generación de código aceleran el proceso de arranque (Bootstrap) de un proyecto, generando rápidamente esqueletos y arquetipos. Su principal problema es que, una vez creada la estructura, el resto debe ser completado a mano sobre el código fuente y no es posible sincronizar la información entre el modelo y el código.

JHipster

Es una plataforma de desarrollo para **generar, desarrollar e implementar** rápidamente aplicaciones web modernas y arquitecturas de microservicios. El objetivo es generar una arquitectura de aplicaciones web o microservicio completa y moderna, unificando:

- ▶ Una pila del lado del servidor robusta y de alto rendimiento con una excelente cobertura de prueba.
- ▶ Una interfaz de usuario elegante, moderna y móvil con Angular, React o Vue + Bootstrap para CSS.
- ▶ Un poderoso flujo de trabajo para construir su aplicación con Webpack y Maven o Gradle.

- ▶ Una arquitectura de microservicio resistente con principios nativos en la nube en mente.
- ▶ Una infraestructura como código para que pueda implementar rápidamente en la nube.

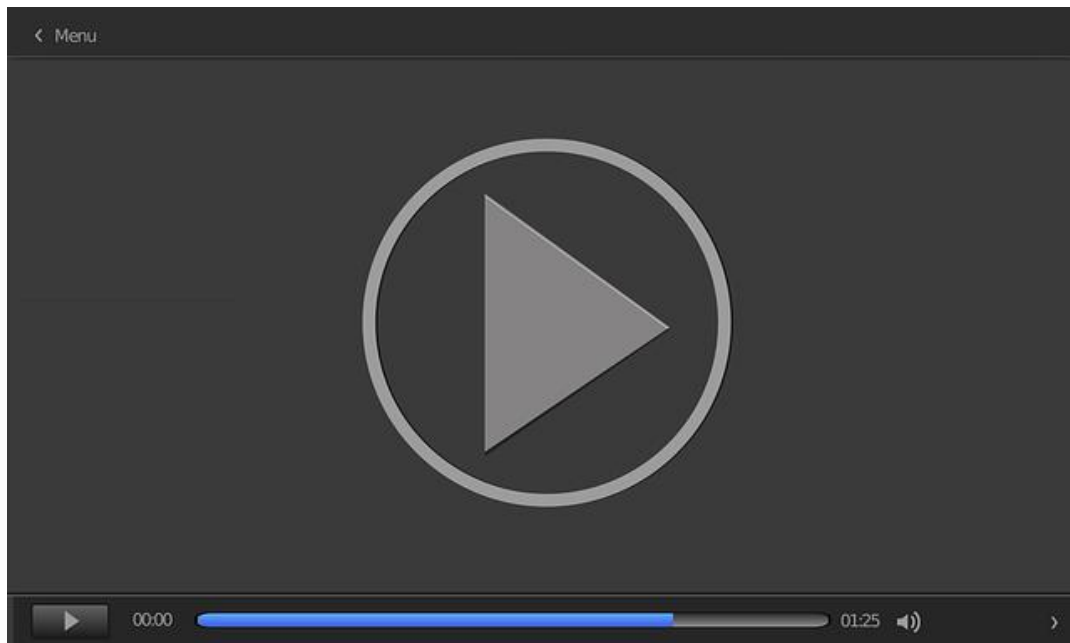
Admite una gran variedad de tecnologías:



Tabla 3. Vista de las tecnologías que incorpora JHipster. Fuente: elaboración propia adaptado de JHipster, s.f.

- ▶ Para el lado del cliente soporte la generación para Angular, React y Vue.
- ▶ Incluye también el soporte para aplicaciones móviles para Ionic y React Native.
- ▶ Para el *back-end* admire Spring Boot (con Java o Kotlin), Micronaut, Quarkus, Node.js y .NET.
- ▶ Para la implementación, adopta los principios nativos de la nube con Docker y Kubernetes.
- ▶ Existe soporte de implementación para AWS, Azure, Cloud Foundry, Google Cloud Platform, Heroku y OpenShift.

En el vídeo, *JHipster: plataforma para acelerar el desarrollo de aplicaciones web modernas y arquitecturas de microservicios*, aprenderemos a acelerar el desarrollo de una aplicación monolítica cliente servidor con Angular y SpringBoot utilizando JHipster.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=9a59c426-3bb9-48f5-8f99-aec300cd9c7b>

4.7. Referencias bibliográficas

Appian. (s. f.). *Build Apps Fast*. <https://appian.com/platform/low-code-development/low-code-application-development.html>

Barcelona Digital Talent (2022). *Análisis del Low-code. Nuevo paradigma en el desarrollo de software*. Mobile World Capital Barcelona; NTT Data. https://es.nttdata.com/documents/low_code_es.pdf

Bratincevic J, Koplowitz R, Powers S, Sjoblom S, Hartig K. (2021). The Forrester Wave™: Low-Code Development Platforms For Professional Developers, Q2 2021. *Forrester*. <https://www.forrester.com/report/The-Forrester-Wave-LowCode-Development-Platforms-For-Professional-Developers-Q2-2021/RES161668>

Cabot, J. (2020, octubre 18). Low-code vs model-driven: are they the same? *Modeling Languages*. <https://modeling-languages.com/low-code-vs-model-driven/>

Carr, D. F. (2021, octubre 22). Gartner: Citizen developers will soon outnumber professional coders 4 to 1. *VentureBeat*. <https://venturebeat.com/2021/10/22/gartner-citizen-developers-will-soon-outnumber-professional-coders-4-to-1/>

DronaHQ. (s. f.), *Quantifying the ROI of Low-Code*. <https://www.dronahq.com/quantifying-roi-of-low-code-development/#calculator>

Forrester. (s. f.). *Low-Code Plataforms*. <https://www.forrester.com/blogs/category/low-code-platforms/>

Gartner. (s. f. a) *Citizen Developer*. <https://www.gartner.com/en/information-technology/glossary/citizen-developer>

Gartner. (s. f. b). *What is an Enterprise Low-Code Application Platforms (LCAP)?* <https://www.gartner.com/reviews/market/enterprise-low-code-application-platform>

Goasduff, L. (2021). Gartner Says Cloud Will Be the Centerpiece of New Digital Experiences. *Gartner*. <https://www.gartner.com/en/newsroom/press-releases/2021-11-10-gartner-says-cloud-will-be-the-centerpiece-of-new-digital-experiences>

Booch, G. (2018, abril 25). History of Software Engineering with Grady Booch. *ACM Learning Center Site*. <https://learning.acm.org/techtalks/histofsofteng>

Mendix. (s. f.). *Visual Development*. <https://www.mendix.com/collaborative-visual-development/>

Microsoft. (s. f.). *El mundo necesita grandes soluciones. Cree las suyas en menos tiempo*. <https://powerapps.microsoft.com/es-es/>

NoCodeJournal (s. f.). *State Of NoCode*. <https://www.nocodejournal.com/state-of-nocode>

OpenXava. (s. f.). *El mejor time-to-market para Java*. <https://www.openxava.org/es/>

OutSystems. (s. f.). *The Low-Code Platform for Serious Applications*. <https://www.outsystems.com/platform/>

Página de [JHipster](#).

Pal K. (2020, septiembre 2). *Comparing low code platforms for internal tool development*. DronaHQ. <https://www.dronahq.com/evaluating-low-code-internal-tool-builders/>

WebRatio. (s. f.) *Community*. <https://my.webratio.com/community>

Wong J., Iijima K (2020, septiembre 20). *Magic Quadrant for Enterprise Low-Code Application Platforms*. Gartner. <https://www.gartner.com/doc/reprints?id=1-275QSBDL&ct=210813&st=sb>

Low-Code for Dummies

Bloomberg, J. (2020). *Low-Code for Dummies*. John Wiley & Sons.

Libro que proporciona una explicación sobre los conceptos y beneficios del desarrollo de software utilizando plataformas low-code.

Curso online de desarrollo de aplicaciones web con OutSystems

OutSystems. (s. f.). *Developing Web Apps (OutSystems 11)*.
<https://www.outsystems.com/training/courses/114/developing-web-apps-outsystems-11/?LearningPathId=2>

El curso se enfoca en el desarrollo web y la integración del lado del servidor, cubriendo todo, desde la arquitectura de la aplicación y el modelado de datos hasta la creación de interfaces de usuario, así como aspectos de seguridad e integración con servicios web.

OutSystems Education Program

Outsystems. (s. f.). *Education Program for Students*.
<https://www.outsystems.com/education-program/students/>

Con el programa educativo de OutSystems podrás acceder a todos los recursos formativos abiertos en línea, hacer uso de la plataforma de forma gratuita y poder obtener una certificación de desarrollador con OutSystems.

1. Un *citizen developer* es:

- A. Un nuevo término para representar el rol de programador de una empresa sin titulación técnica.
- B. Un empleado que crea aplicaciones utilizando herramientas con un mayor nivel de abstracción que los lenguajes de programación.
- C. Un nuevo cargo funcional en metodologías de desarrollo ágiles.
- D. Una persona que ha aprendido a programar realizando cursos online.

2. Relaciona las características con las diversas plataformas *low-code*:

OutSystems	1	A	Creación de <i>chatbots</i>
Microsoft Power Virtual Agents	2	B	Soporte para <i>IT service management</i> (ITSM)
Appian	3	C	Implementación y despliegue con un solo clic
ServiceNow	4	D	Soporte para la automatización de procesos (<i>robotic process automation</i> , RPA)

3. ¿Cuáles son las principales plataformas *low-code* que atienden los informes de las consultoras Gartner y Forrester?

- A. OutSystems, Saltcorn, JHipster y WebRatio.
- B. OutSystems, Quickbase y Creatio.
- C. OutSystems, Mendix, Microsoft PowerApps y ServiceNow.
- D. OutSystems, Appian, Salesforce, Oracle.

4. Un criterio para seccionar una plataforma *low-code* es su finalidad, que con carácter general puede agruparse en:

- A. *UI-driven* y *data-driven*.
- B. Escritorio, móvil y Cloud.
- C. Lógica, datos y procesos.
- D. *UI-driven*, *data-driven*, *business process driven* y *general purpose*.

5. Ordena cómo se agiliza el desarrollo de software con las siguientes tecnologías:

Vue.js	1	A	Velocidad de desarrollo muy alta
Java	2	B	Velocidad de desarrollo alta
OutSystems	3	C	Velocidad de desarrollo media
Spring Boot	4	D	Velocidad de desarrollo baja

6. El principal valor y utilidad de las plataformas *low-code* y *no-code* es:

- A. Generar código en cualquier lenguaje de programación.
- B. Agilizar el desarrollo incluyendo perfiles profesionales sin elevados conocimientos tecnológicos.
- C. Generar código con múltiples arquitecturas.
- D. Formar equipos de desarrollo exclusivos con *citizen developers*.

7. Marca aquellas tecnologías sobre las que se puede generar un proyecto utilizando jhipster:

- A. HTML5, Angular, React, Vue.
- B. Spring Boot, Consul, Liquibase, Swagger.
- C. Docker, Kubernetes, OpenShift.
- D. Todas las anteriores.

8. El uso de plataformas *low-code* tiene el riesgo de *lock-in*, que significa:
- A. No poder elegir el lenguaje de programación.
 - B. No poder elegir la nube para su despliegue y operación.
 - C. La dependencia futura de nuestra aplicación respecto a la herramienta usada para el desarrollo.
 - D. No poder elegir la arquitectura de la aplicación.
9. El concepto de TI en la sombra (*shadow IT*) hace referencia a:
- A. Utilizar herramientas de generación de código multiplataforma.
 - B. Que el equipo de desarrollo está ubicado en un sótano.
 - C. Utilizar herramientas *low-code*.
 - D. Desarrollar aplicaciones corporativas sin el permiso de su departamento de TI.
10. Saltcorn utiliza los siguientes elementos de modelado:
- A. Tablas y relaciones.
 - B. Tablas, vistas y páginas.
 - C. *Plugins* y *packs*.
 - D. Tablas y páginas.

Plataformas de Desarrollo de Software

Tema 5. Plataformas para el desarrollo en Java

Índice

Esquema

Ideas clave

- 5.1. Introducción y objetivos
- 5.2. Contexto de las plataformas para el desarrollo en Java
- 5.3. Valor y utilidad de las plataformas para el desarrollo en Java
- 5.4. Entornos de desarrollo integrados
- 5.5. Plataformas para la gestión de las dependencias y procesos de compilación
- 5.6. Plataformas aceleradoras del desarrollo en Java
- 5.7. Referencias bibliográficas

A fondo

- Spring in Action
- Java design patterns: A hands-on experience with real-world examples

Test



5.1. Introducción y objetivos

En este tema se estudian algunas de las plataformas que nos ayudan a desarrollar sistemas basados en la tecnología Java.

En primer lugar, estableceremos el **contexto** de una tecnología independiente de la plataforma que requiere de una máquina virtual que interpreta el código en la plataforma destino, característica fundamental que marca las ventajas e inconvenientes de su uso.

Posteriormente, veremos algunos de los principales entornos de desarrollo integrado que facilitan **la escritura, compilación, depuración y despliegue** de soluciones basadas en Java, así como algunas plataformas que facilitan la automatización de la construcción y el despliegue de este tipo de aplicaciones. Por último, expondremos algunas de las plataformas que aceleran el desarrollo de soluciones basadas en tecnologías Java.

Con el estudio de este tema pretendemos alcanzar los siguientes objetivos:

- ▶ Comprender el contexto, el valor y la utilidad que tiene el desarrollo con una tecnología independiente de la plataforma como Java.
- ▶ Conocer los principales entornos de desarrollo integrado para Java .
- ▶ Introducir algunas herramientas de automatización para la gestión de dependencias y procesos de compilación y despliegue sobre Java.
- ▶ Conocer algunas plataformas aceleradoras del desarrollo a nivel de interfaz de usuario y servidores empresariales.

5.2. Contexto de las plataformas para el desarrollo en Java

Java es, por un lado, un **lenguaje de programación** y por otro una **plataforma**. Como lenguaje de programación, es un lenguaje de alto nivel que se puede caracterizar por las siguientes características: orientado a objetos, multiproceso, de arquitectura neutra, portable, seguro y robusto (Bloch, 2017).

Aunque originalmente era un lenguaje orientado a objetos, Java ha comenzado a adoptar conceptos de **otros paradigmas** de programación como la programación funcional. Algunas de sus características principales como lenguaje son (Sierra, Bates, y Gee, 2022):

- ▶ Orientado a objetos.
- ▶ Fuertemente tipado estáticamente.
- ▶ Independiente de la plataforma.
- ▶ Con un recogedor de elementos no usados para liberar la memoria (*garbage collector*).
- ▶ Multiproceso.

Java es un lenguaje de programación de propósito general que adopta el paradigma orientado a objetos. Una característica central del lenguaje es la **portabilidad multiplataforma**, lo que significa que los programas escritos en una plataforma se pueden ejecutar en cualquier combinación de software y hardware mediante una máquina virtual.

En el lenguaje de programación Java, todo el código fuente se escribe primero en archivos de texto sin formato que terminan con la extensión `.java`. Esos archivos

fuentes luego se compilan en archivos **.class** por el compilador `javac`. Un archivo **.class** **no contiene código nativo** de su procesador, en su lugar, contiene *bytecodes*, el lenguaje de máquina de máquina virtual de Java (*Java virtual machine*, JVM). Las instrucciones de código de *bytes* de Java son análogas al código de máquina, pero son interpretadas por una máquina virtual de Java (JVM) específica para la combinación de sistema operativo *host* y hardware. De esta forma, se consigue esta característica de portabilidad mencionada. Los *bytecodes* pueden ser decompilados y se puede obtener una visión del código fuente escrito, salvo los comentarios.

Como plataforma, se compone de dos elementos: la **máquina virtual de Java** y la **interfaz de programación de aplicaciones Java** (*Java application programming interface*, API).

Debido a que JVM está disponible en **muchos sistemas operativos diferentes**, los mismos archivos **.class** pueden ejecutarse en Microsoft Windows, el sistema operativo Solaris (Solaris OS), Linux o macOS. Algunas máquinas virtuales, como Java SE HotSpot, realizan pasos adicionales en tiempo de ejecución para mejorar el rendimiento de su aplicación. Esto incluye varias tareas, como encontrar cuellos de botella en el rendimiento y volver a compilar (en código nativo) las secciones de código que se usan con frecuencia.

La API es una gran **colección de componentes de software** listos para usar que brindan muchas capacidades útiles. Se agrupa en bibliotecas de clases e interfaces relacionadas. Estas bibliotecas se conocen como paquetes.

A nivel de plataforma, podemos encontrar un diverso grupo de tecnologías:



Figura 1. Tecnologías en la plataforma Java. Fuente: Oracle, s. f.

- ▶ **Java Platform, Standard Edition (Java SE):** permite desarrollar e implementar aplicaciones Java en equipos de escritorio y servidores.
- ▶ **Oracle Java Embedded:** se orienta a soluciones de Internet de las cosas y dispositivos embebidos con menos recursos computacionales.
- ▶ **Java Platform, Enterprise Edition (Java EE):** es el estándar en software empresarial impulsado por la comunidad. Java EE se desarrolla utilizando el proceso de la comunidad de Java, con contribuciones de expertos de la industria, organizaciones comerciales y de código abierto, grupos de usuarios de Java e innumerables personas.
- ▶ **Java Platform, Micro Edition (Java ME):** proporciona un entorno sólido y flexible para aplicaciones que se ejecutan en dispositivos integrados y móviles en el Internet de las cosas, como microcontroladores, sensores, puertas de enlace, teléfonos móviles, televisores o impresoras. Las aplicaciones basadas en Java ME son portátiles en muchos dispositivos, pero aprovechan las capacidades nativas de cada dispositivo.
- ▶ **Java Card:** permite que elementos seguros, como tarjetas inteligentes y otros chips de seguridad resistentes a la manipulación, alojen aplicaciones que emplean tecnología Java. Ofrece una plataforma de ejecución segura e interoperable que

puede almacenar y actualizar múltiples aplicaciones en un solo dispositivo con recursos limitados, al mismo tiempo que conserva los más altos niveles de certificación y compatibilidad con los estándares.

- ▶ **Java TV:** es una tecnología basada en Java ME que proporciona una solución eficaz, segura y fácil de implementar para desarrollar aplicaciones Java que se ejecutan en televisores y decodificadores. Al usar el tiempo de ejecución de Java TV, los desarrolladores pueden crear fácilmente aplicaciones, como guías electrónicas de programas (EPG), clientes de vídeo a pedido (VOD), juegos y aplicaciones educativas, aplicaciones para acceder a datos de Internet (por ejemplo, clima, teletipos de noticias, redes sociales o redes) y en la mayoría de los títulos de disco Blu-ray, la interfaz de usuario y el contenido adicional.
- ▶ **Java DB** es una distribución de la base de datos de código abierto Apache Derby. Admite SQL ANSI/ISO estándar a través de las API JDBC y Java EE.

A partir de la versión 11, Oracle modificó el modelo de licenciamiento de la JVM, por lo que es posible utilizar las JDK para desarrollar una aplicación, pero es necesario pagar una **suscripción**. Oracle Java SE Subscription y Oracle Java SE Desktop Subscription son soluciones de Oracle para desarrolladores empresariales que ejecutan aplicaciones e infraestructura de misión crítica en la plataforma Java SE.

La suscripción de Java SE es una suscripción mensual simple (en abril de 2022, el precio de escritorio es de 2,5 dólares por mes por usuario) que incluye licencias y soporte de Java SE para usar en escritorios, servidores o implementaciones en la nube. Sigue un **modelo de uso común**, popular con las distribuciones de Linux. La suscripción proporciona acceso a actualizaciones de rendimiento, estabilidad y seguridad probadas y certificadas para Java SE, directamente desde Oracle.

Existen diversas implementaciones de la máquina virtual de Java:

- ▶ **Azul Platform Core:** es una compilación OpenJDK compatible con Azul Systems que cumple con los estándares Java SE 17, 16, 15, 13, 11, 8, 7 y 6.

- ▶ **Codename One:** utiliza el ParparVM de código abierto.
- ▶ **Eclipse OpenJ9:** código abierto de IBM J9 para Windows, AIX, Linux (x86, Power y Z), macOS, MVS, OS/400, Pocket PC, z/OS.
- ▶ **GraalVM:** se basa en HotSpot/OpenJDK y tiene una función políglota para mezclar y combinar de forma transparente los idiomas admitidos.
- ▶ **HotSpot:** la implementación de Java VM de código abierto de Oracle.
- ▶ **Jikes RVM** (Jikes Research Virtual Machine): es un proyecto de investigación. Admite las bibliotecas Apache Harmony y GNU Classpath con una licencia Eclipse.
- ▶ **leJOS:** la *suite* de robótica, un reemplazo de *firmware* para los bloques funcionales programables Lego Mindstorms, proporciona un entorno de programación Java para los robots Lego RCX y NXT.
- ▶ **Maxine:** máquina virtual de investigación de código abierto de Oracle Labs y la Universidad de Manchester.
- ▶ **JamaicaVM** (aicas): una máquina virtual Java en tiempo real para sistemas integrados.

Las más habituales son la oficial de Oracle (OracleJDK), la versión abierta oficial (OpenJDK) o la distribución binaria basada en la anterior ([Adoptium](#)).

Además de Java, **otros lenguajes** pueden ejecutarse en la máquina virtual de Java:

- ▶ **Scala:** significa 'lenguaje escalable'. Creado en 2004, es un lenguaje de tipo estático que combina dos importantes paradigmas de programación: la programación funcional y la orientada a objetos. Scala es un lenguaje orientado a objetos puro, ya que no admite primitivas. Brinda la capacidad de definir clases, objetos, métodos

junto con características de programación funcional como rasgos, tipos de datos algebraicos o clases de tipos.

- ▶ **Kotlin:** es un lenguaje de código abierto, de propósito general y tipificado estáticamente desarrollado por el equipo de JetBrains, que reúne los paradigmas funcional y orientado a objetos. El enfoque principal durante el desarrollo de Kotlin fue la interoperabilidad de Java, la seguridad (manejo de excepciones), la concisión y un mejor soporte de herramientas. Desde el lanzamiento de Android Studio 3.0, Kotlin es un lenguaje de programación totalmente compatible con Google en la plataforma Android. También se incluye en el paquete IDE de Android Studio como una alternativa al compilador estándar de Java (Jemerov, 2016).
- ▶ **Groovy:** es un lenguaje específico de dominio dinámico (*domain specific language*, DSL) orientado a objetos, tipado opcionalmente, con soporte para capacidades de compilación y tipado estáticos. Su objetivo es mejorar la productividad de los desarrolladores con una sintaxis fácil de aprender. Se integra fácilmente con cualquier programa de Java e inmediatamente agrega funciones potentes, como capacidades de secuencias de comandos, metaprogramación en tiempo de ejecución y tiempo de compilación y funciones de programación funcional.
- ▶ **Clojure:** es un lenguaje de programación funcional de propósito general. El lenguaje se ejecuta en JVM, así como en la *common language runtime* (CLR) de Microsoft. Sigue siendo un lenguaje compilado y dinámico, ya que sus funciones son compatibles en tiempo de ejecución. Los diseñadores de Clojure querían diseñar un Lisp moderno que pudiera ejecutarse en JVM. Es por eso por lo que también se le conoce como un dialecto del lenguaje de programación Lisp y, como sucede en este último lenguaje mencionado, trata el código como datos y también tiene un sistema de macros.

- ▶ **Jython:** es la implementación de la plataforma Java de Python que se ejecuta en la JVM. Este lenguaje se diseñó inicialmente para permitir la escritura de aplicaciones de alto rendimiento sin sacrificar la interactividad. Jython está orientado a objetos, tiene múltiples subprocesos y utiliza el recolector de basura de Java para limpiar la memoria de manera eficiente. Incluye la mayoría de los módulos que forman parte del lenguaje Python pero también puede importar y usar cualquier clase de las bibliotecas de Java.
- ▶ **JRuby:** es una implementación del lenguaje de programación Ruby para ejecutarse en la máquina virtual Java. Es un lenguaje de alto rendimiento y multiproceso, con una gran variedad de bibliotecas disponibles de Java y Ruby. Además, combina características de ambos lenguajes como la programación orientada a objetos y el tipado pato (*duck typing*).

En el índice de la comunidad de programación TIOBE, que mide la popularidad de los lenguajes de programación, los principales (en marzo 2022) que funcionan bajo la JVM son: Java (núm. 3), Groovy (núm. 25), Kotlin (núm. 31), Scala (núm. 36) y Clojure (núm. 47).

5.3. Valor y utilidad de las plataformas para el desarrollo en Java

El valor y la utilidad del desarrollo en Java se centra en las **características asociadas** a la ejecución sobre la máquina virtual, en particular:

- ▶ **Independencia de la plataforma:** el *bytecode* de Java se puede escribir una vez y luego ejecutar en múltiples plataformas. No necesita ser portado a un entorno de hardware específico porque se ejecuta en la máquina virtual de Java. Cualquier aplicación compilada en el código de *bytes* de Java se puede ejecutar en una plataforma informática que tenga una máquina virtual de Java.
- ▶ **API común:** el otro gran beneficio de usar JVM es el de una API común para trabajar con recursos específicos de la plataforma. Por ejemplo, no hay necesidad de pensar en cómo manejar la entrada/salida de archivos (E/S) en todas las plataformas posibles, ya que JVM lo hará por nosotros. JVM manejará no solo la E/S de archivos específica de la plataforma, sino también otros recursos como la memoria, las redes, los dispositivos de entrada y salida, etc.
- ▶ **Control de la corrección:** cada vez que se carga un *bytecode*, la JVM realiza una verificación de acuerdo con la especificación de la máquina virtual de Java. Garantiza que el código generado esté correctamente escrito y compilado por un compilador válido. Se realizan muchas comprobaciones de verificación, como la verificación de acuerdo con las restricciones de JVM y la asignación de memoria.

- ▶ **Seguridad:** Java fue escrito con la seguridad en mente. La máquina virtual de Java tiene características de seguridad integradas que permiten a los programadores escribir programas Java altamente seguros. También evita que el software malicioso comprometa el sistema operativo (SO) porque evita que las aplicaciones Java interactúen con los recursos del sistema operativo.
- ▶ **Gestión de la memoria:** no hay necesidad de reservar memoria para las variables o gestionar punteros a espacios de memoria. La JVM se encarga de la gestión de la memoria al asignar la memoria requerida para variables y objetos. Luego, se realiza un seguimiento del acceso a la memoria y el recolector de basura (*garbage collector*, GC) del subsistema JVM puede recuperar la memoria no utilizada cuando sea el momento. Esta funcionalidad libera al desarrollador de gestionar cuándo asignar memoria y cuándo liberarla, lo que reduce los problemas de fuga de memoria.

Adicionalmente, el uso de un lenguaje interpretado conlleva una **serie de desventajas:**

- ▶ **Menor eficiencia de ejecución:** los programas Java que se ejecutan en una máquina virtual Java tienden a funcionar más lentamente que los programas equivalentes escritos en C++. La neutralidad del sistema del código de *bytes* actúa como una desventaja en lo que respecta al rendimiento. Esto se debe a que la optimización del código depende, en gran medida, de las funciones específicas del sistema. Dado que el código de *bytes* de Java es neutral para el sistema, no se puede optimizar para un conjunto de hardware específico (Oaks, 2014).

- ▶ **Mayor consumo de memoria:** la JVM, necesaria para ejecutar una aplicación en Java, consume memoria, por lo que el consumo es superior. Esto supone un inconveniente en dispositivos con menos recursos, como los teléfonos inteligentes, que justifica la implementación de JVM específicas para los mismos que optimizan el consumo de memoria, pero no ofrecen todas las funcionalidades. Sin embargo, hay proveedores de la nube que tarifican según los recursos utilizados por los programas, por lo que en este caso sí que estaríamos pagando un sobre coste adicional necesario derivado de la JVM.
- ▶ **Posibles errores en la JVM:** dado que un programa Java se basa en la máquina virtual de Java para ejecutarlo, la JVM debe estar libre de errores para que el programa funcione correctamente. Esta dependencia de la máquina virtual de Java introduce un posible punto de falla para el programa. Afortunadamente, el software Java virtual machine se produce con estándares muy altos y, por lo tanto, no es probable que se envíe con algún error. De todos modos, una falla en la máquina virtual de Java es una posibilidad que debe considerarse.

5.4. Entornos de desarrollo integrados

Un **entorno de desarrollo integrado** (*integrated development environment*, IDE) agrupa todas las herramientas que un programador necesita para escribir, depurar y probar su código. Un IDE permite a los desarrolladores trabajar en un solo entorno y mejorar su flujo de trabajo de programación. Cada IDE tiene diferentes recursos, pero todos incluyen un editor de texto, herramientas de compilación y un depurador.

Las principales funcionalidades que nos ofrecen los IDE para el desarrollo con Java son:

- ▶ **Edición de código:** esta herramienta identifica e inserta componentes de código para ahorrarle tiempo y reducir el riesgo de errores tipográficos y errores. El resaltado de la sintaxis incorporado es crucial para la legibilidad y la eficiencia. Esta herramienta puede ayudar a encontrar errores en su código y permite filtrar su código a una velocidad más rápida.
- ▶ **Automatización de compilación:** un buen IDE permitirá escribir y ejecutar el código Java en la misma ventana, lo que aumenta la productividad y la eficiencia de su programa.
- ▶ **Entorno de ejecución:** para lanzar la aplicación desarrollada.
- ▶ **Depuración:** si bien los depuradores varían entre los diferentes IDE, es importante tener una herramienta para resaltar posibles errores y ayudarlo a revisar su código, con ejecuciones paso a paso y visualización del valor de variables.

Algunos de los IDE más populares para el desarrollo en Java son:



Tabla 1. IDE para Java. Fuente: elaboración propia.

Eclipse

Es uno de los IDE de Java **más populares** del mercado. Es gratuito, de código abierto y tiene un extenso ecosistema de complementos que permite a los usuarios personalizar las funcionalidades para el desarrollo de aplicaciones. El IDE de Eclipse viene en ediciones de escritorio y en la nube. Esta última permite a los desarrolladores codificar en un navegador web. Eclipse es una de las mejores

opciones debido a sus **herramientas de desarrollo**. Ofrece un *marketplace* que contiene una variedad de complementos disponibles para descargar, junto con un entorno de desarrollo de complementos que permite a los desarrolladores crear sus propias funciones (Vogel, 2013).

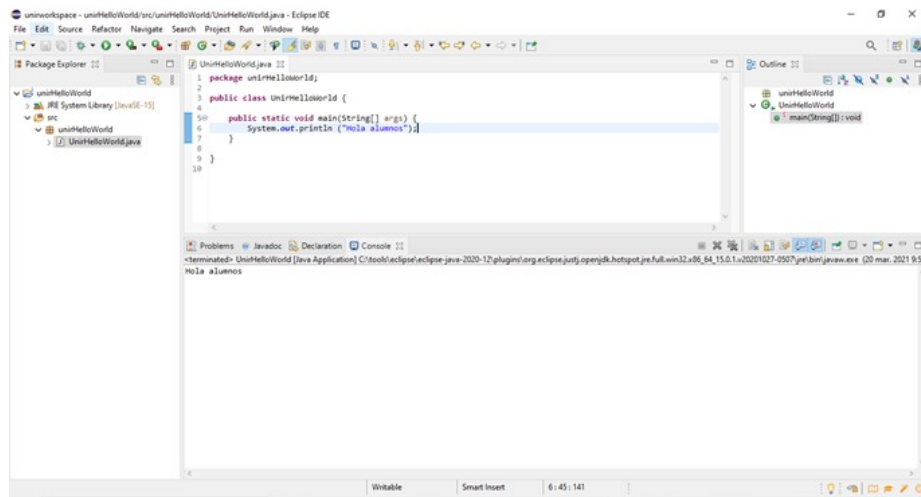


Figura 2. Captura del IDE eclipse. Fuente: elaboración propia.

Apache NetBeans

Es el IDE oficial para Java 8, lo que lo convierte en uno de los **principales** IDE de Java para crear aplicaciones de escritorio, móviles y web. A su vez, es de código abierto. Es fácil **personalizar** las aplicaciones de software porque resalta el código Java sintácticamente y semánticamente. Con potentes herramientas de refactorización y depuración, NetBeans ayuda a aumentar la precisión y la eficiencia.

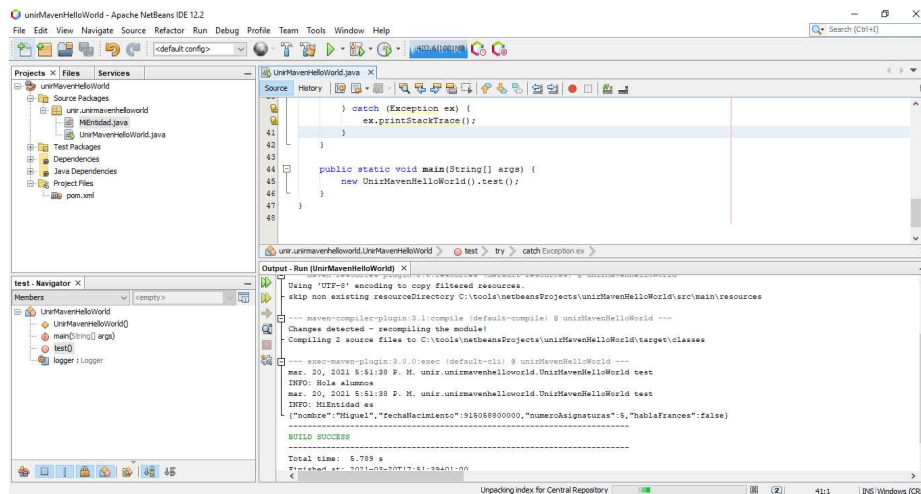


Figura 3. Captura del IDE Apache NetBeans. Fuente: elaboración propia.

IntelliJ

Es uno de los mejores IDE para el desarrollo de Java. Es un IDE **capaz y ergonómico**, con características para aumentar la productividad sin saturar la interfaz de usuario. Incluye un conjunto de herramientas que facilitan la programación, como finalización inteligente, refactorización entre idiomas, análisis de flujo de datos e inyección de lenguaje. IntelliJ está disponible en dos ediciones: la edición comunitaria con licencia de Apache 2 y la edición Ultimate propietaria que ofrece funcionalidades avanzadas para el desarrollo web y empresarial.

Oracle JDeveloper

Es un IDE gratuito ofrecido por Oracle que cubre **todo el ciclo de vida del desarrollo**: codificación, diseño, depuración, optimización, creación de perfiles e implementación. Se integra con Oracle Application Development Framework (Oracle ADF) para simplificar el desarrollo y ofrece editores visuales y declarativos y permite a los desarrolladores editar directamente sus aplicaciones desde el entorno de codificación. Tiene una función de arrastrar y soltar que simplifica el desarrollo de aplicaciones y es compatible con Java EE y Java SE.

Greenfoot

Fue diseñado con la **intención educativa** de hacer que la programación Java sea más simple y agradable para los jóvenes desarrolladores. Utiliza gráficos bidimensionales para crear programas, juegos y simulaciones interactivos. Este IDE atrae la atención de profesores y estudiantes y proporciona una plataforma para que interactúen en línea. Ofrece acceso gratuito a sonidos y animaciones e ilustra conceptos tales como programación orientada a objetos, interacciones de objetos y parámetros.

DrJava

Es un IDE liviano que fue desarrollado para **principiantes y estudiantes**. Aunque no es tan poderoso como algunos de los otros IDE en esta lista, tiene una interfaz ordenada y una apariencia consistente en diferentes plataformas. Permite a los desarrolladores evaluar de forma interactiva su código desde una consola y presentar sus resultados en la misma consola. Da soporte con características como buscar y reemplazar, finalización automática, sangría automática, comentarios y coloreado de sintaxis.

MyEclipse

Es un IDE de Java comercial que se basa en la plataforma Eclipse. Utiliza **tanto código propietario como código abierto** en el entorno de desarrollo y sus herramientas se pueden utilizar para el desarrollo de aplicaciones Java y el desarrollo web. Es compatible con conectores de servidores de aplicaciones y bases de datos y tiene soporte mejorado para Maven y Spring, lo que ayuda a construir *front-ends* y *back-ends* potentes.

Codenvy

Es un IDE comercial de Java construido sobre Eclipse Che. Hay un total de tres variantes del IDE entre las que los programadores de Java pueden elegir: desarrollador, empresa y equipo. Es un espacio de trabajo para desarrolladores **bajo demanda y basado en la nube**. Además de escribir código Java que utiliza el editor basado en navegador que ofrece, los programadores de Java pueden beneficiarse de potentes funciones como finalización de código, refactorización y resaltado de sintaxis. El editor también facilita la vida de los programadores al ofrecer una forma eficaz de compilar, depurar, editar y ejecutar varios proyectos.

Si se atiende a la última encuesta para profesionales de Stack Overflow (2021), IntelliJ es usado por el 28 % de los desarrolladores, Eclipse por el 15 % y NetBeans por el 7 %, aunque en estos resultados no se acotan al uso de Java.

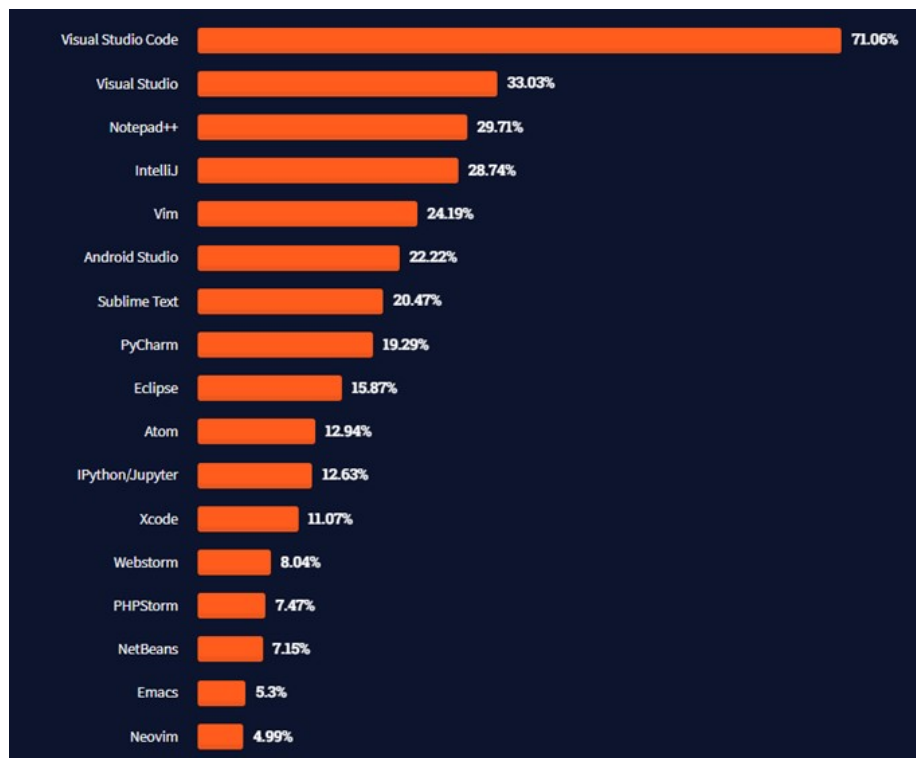


Figura 4. IDE más populares. Fuente: Stack Overflow, 2021.

5.5. Plataformas para la gestión de las dependencias y procesos de compilación

Dentro del ciclo de vida del desarrollo, una vez que hemos escrito el código fuente, es necesario compilarlo, empaquetarlo, testearlo y desplegarlo. Existen una serie de **herramientas de automatización de la construcción** (*building automation tools*) que se utilizan para automatizar todo el proceso de creación de compilación de software y otros procesos relacionados, como el empaquetado de código binario y la ejecución de pruebas automatizadas.

En este tema vamos a exponer alguna de las herramientas principales para el desarrollo con Java como Ant, Maven y Gradle.

Apache Ant

Es una biblioteca de Java y una herramienta de línea de comandos cuya misión es **impulsar** los procesos descritos en los archivos de compilación como objetivos y puntos de extensión que dependen unos de otros. Proporciona una serie de tareas integradas que permiten compilar, ensamblar, probar y ejecutar aplicaciones Java. Es extremadamente flexible y no impone convenciones de codificación o diseños de directorios a los proyectos Java que lo adoptan como herramienta de construcción. Para usarlo, basta con crear un fichero build.xml en el que definimos los objetivos para poder hacer una compilación (*compile*), un empaquetado (*jar*) o una ejecución (*run*) que se podrá hacer desde un IDE o desde la propia línea de comandos (por ejemplo: *ant build*).

```

<project>
  <target name="clean">
    <delete dir="classes" />
  </target>
  <target name="compile" depends="clean">
    <mkdir dir="classes" />
    <javac srcdir="src" destdir="classes" />
  </target>
  <target name="jar" depends="compile">
    <mkdir dir="jar" />
    <jar destfile="jar/HelloWorld.jar" basedir="classes">
      <manifest>
        <attribute name="Main-Class"
value="net.unir.missi.HelloWorld" />
      </manifest>
    </jar>
  </target>
  <target name="run" depends="jar">
    <java jar="jar/HelloWorld.jar" fork="true" />
  </target>
</project>

```

Apache Maven

Es una herramienta de gestión de **dependencias** y **automatización** de compilaciones, utilizada principalmente para aplicaciones Java. Facilita el proceso de construcción, proporciona un sistema de construcción uniforme y una información de proyectos de calidad y fomenta mejores prácticas de desarrollo. Utiliza archivos XML como Ant, pero de una manera mucho más manejable.

Para usarlo deberemos **crear el fichero** configuración de Maven, que contiene instrucciones de gestión de dependencias y compilación y que se llama por convención pom.xml:


```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>net.unir.missi</groupId>
  <artifactId>demoMaven</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <description>Maven example</description>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.12</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-dependency-plugin</artifactId>
        <executions>
          <execution>
            <id>copy-dependencies</id>
            <phase>package</phase>
            <goals>
              <goal>copy-dependencies</goal>
            </goals>
            <configuration>

            <outputDirectory>target/dependencies
            </outputDirectory>
          </configuration>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
</project>
```

Con Maven, el proyecto debe tener una estructura de proyecto fija, diferenciando el código fuente (*src*) de las pruebas (*test*) e incorporando ficheros de configuración o

recursos externos que no sea código (*resources*).

```
+---src
|   +---main
|   |   +---java
|   |   |   \---net
|   |   |       \---unir
|   |   |           \---missi
|   |   |               HelloWorld.java
|   |   |
|   |   \---resources
|   \---test
|       +---java
|       \---resources
```

Podemos compilar el proyecto simplemente con un `mvn compile`, o empaquetarlo con un `mvn package`, o bien ejecutarlo con un `mvn compile exec:java -Dexec.mainClass="net.unir.missi.HelloWorld"`.

Gradle

Es una herramienta de **automatización de compilación** de código abierto centrada en la flexibilidad y el rendimiento. Los *scripts* de compilación de Gradle se escriben con Groovy o Kotlin DSL. Es **altamente personalizable** ya que está modelado de una manera que es extensible en las formas más fundamentales. Completa las tareas rápidamente al reutilizar los resultados de ejecuciones anteriores y procesa solo aquellos elementos que cambiaron y puede ejecutar tareas en paralelo. Es la herramienta de compilación oficial para Android y viene con soporte para muchos lenguajes y tecnologías populares (Muschko, 2014).

Para usarlo crearemos un fichero `build.gradle` en el que definiremos las dependencias y tareas:

```
apply plugin: 'java'

repositories {
    mavenCentral()
}

jar {
    baseName = 'gradleExample'
    version = '0.0.1-SNAPSHOT'
}

dependencies {
    testImplementation 'junit:junit:4.12'
}
```

A partir de aquí podremos compilar (*gradle classes*) o empaquetar (*gradle jar*).

Accede al código fuente de ejemplos de proyectos con Ant, Maven y Gradle a través del aula virtual

5.6. Plataformas aceleradoras del desarrollo en Java

Existen una serie de plataformas a modo de librerías o *frameworks* que nos permiten acelerar el desarrollo de sistemas con Java. De forma general, veremos con más detalle Spring Framework, las librerías para el desarrollo de interfaces gráficas de usuario y alguna utilidad práctica como Lombok.

Las principales plataformas aceleradoras del desarrollo en Java que veremos son:

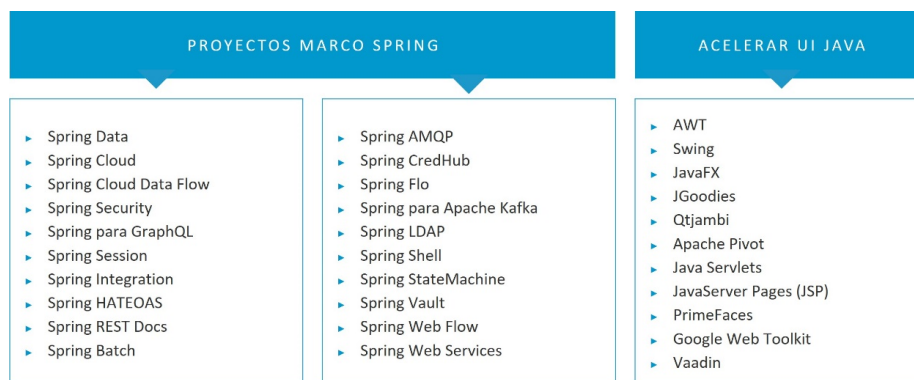


Tabla 2. Plataformas aceleradoras del desarrollo en Java. Fuente: elaboración propia.

Spring Framework

Es una plataforma Java que proporciona **soporte de infraestructura integral** para el desarrollo de aplicaciones Java. Spring maneja la infraestructura para que podamos concentrarnos en la aplicación. Nos permite la creación de aplicaciones a partir de **objetos Java simples y antiguos** (*plain old Java object*, POJO) y aplicar servicios empresariales de forma no invasiva a los POJO. Esta capacidad se aplica al modelo de programación Java SE y a Java EE completo y parcial.

El elemento principal del marco es el **control de las dependencias** y la **inversión del control**. Una aplicación Java —un término impreciso que abarca toda la gama desde aplicaciones integradas restringidas hasta aplicaciones empresariales del lado

del servidor de n niveles—, por lo general, consta de objetos que colaboran para formar la aplicación propiamente dicha. Por lo tanto, los objetos de una aplicación tienen dependencias entre sí. Aunque la plataforma Java proporciona una gran cantidad de funciones de desarrollo de aplicaciones, carece de los medios para organizar los componentes básicos en un **todo coherente**, lo que deja esa tarea a los arquitectos y desarrolladores. El componente Spring Framework Inversion of Control (IoC) aborda esta preocupación al proporcionar un **medio formalizado** para componer componentes dispares en una aplicación completamente funcional lista para usar.

Algunos casos de uso de aplicación del marco son:

- ▶ Permite que un método Java se ejecute en una transacción de base de datos sin tener que lidiar con las API de transacciones.
- ▶ Facilita la conversión de un método Java local en un punto final HTTP sin tener que lidiar con la API de Servlet.
- ▶ Permite convertir un método Java local en un controlador de mensajes sin tener que lidiar con la API de JMS.
- ▶ Facilita que un método Java local en una operación de administración sin tener que lidiar con la API JMX.

Spring se basa en la inversión de control como principio de diseño de software, de forma que el flujo de ejecución de un programa se invierte respecto a los métodos de programación tradicionales y es gestionado por la plataforma.

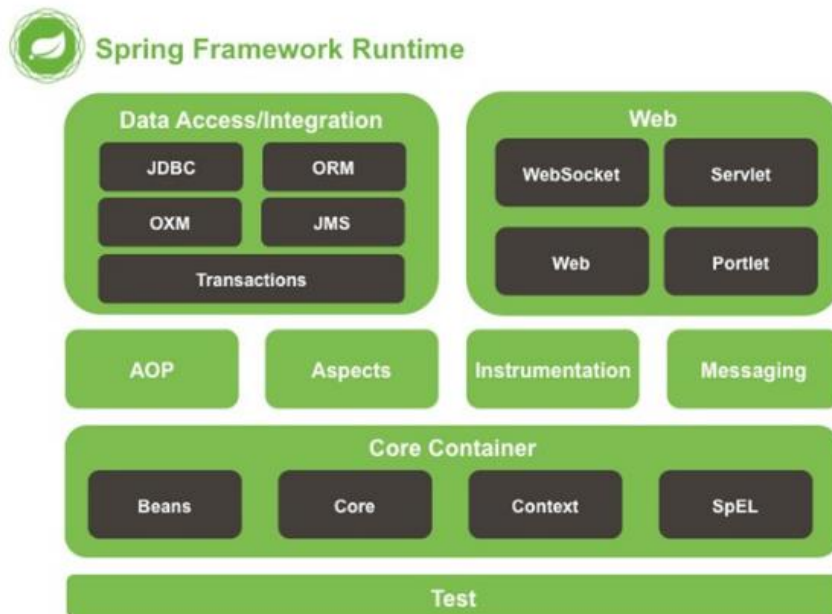


Figura 5. Vista de los componentes de Spring Framework Runtime. Fuente: Spring Framework, s. f.

El marco Spring Framework proporciona un **modelo integral** de programación y configuración para aplicaciones empresariales modernas basadas en Java, en cualquier tipo de plataforma de implementación.

Un elemento clave de Spring es el soporte de infraestructura a nivel de aplicación: Spring se enfoca en la integración de las aplicaciones empresariales para que los equipos puedan enfocarse en la lógica comercial a nivel de aplicación, sin vínculos innecesarios con entornos de implementación específicos.

Adicionalmente a este marco general, existen un conjunto de proyectos dentro del ecosistema Spring que facilitan el desarrollo de *software* y que se resumen a continuación:

Spring Boot

Facilita la creación de aplicaciones basadas en Spring independientes y de grado de producción que podemos simplemente ejecutar. Es la forma más sencilla de iniciar una solución basada en Spring.

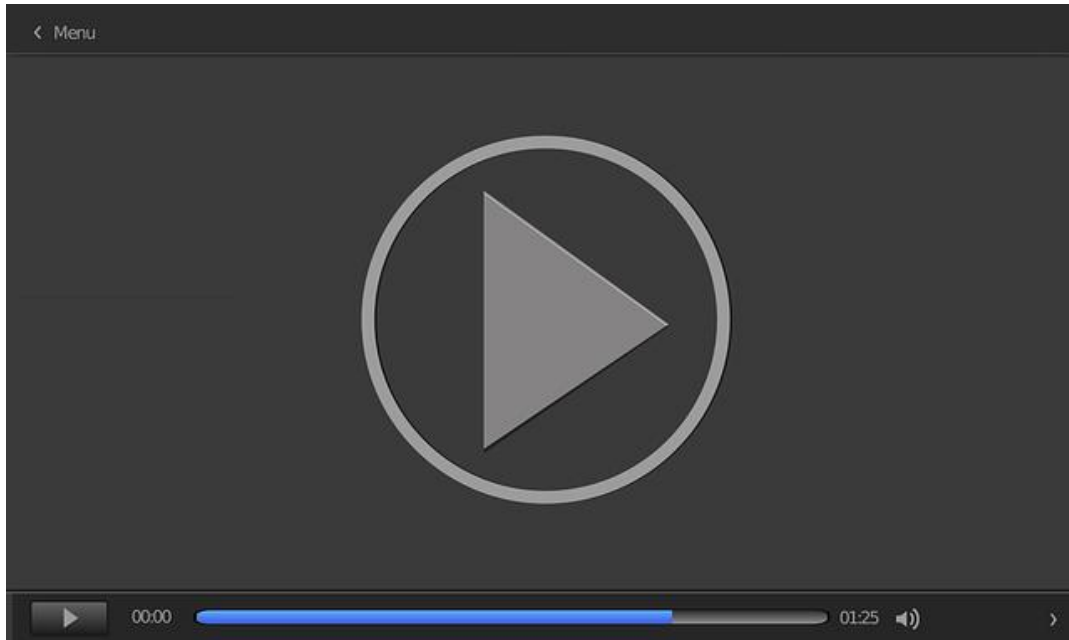
Spring Data

Proporciona un modelo de programación familiar y coherente basado en Spring para el acceso a los datos y, al mismo tiempo, conservar las características especiales del almacén de datos subyacente. Facilita el uso de tecnologías de acceso a datos, bases de datos relacionales y no relacionales, marcos de reducción de mapas y servicios de datos basados en la nube.

Spring Cloud

Proporciona herramientas para que los desarrolladores construyan rápidamente algunos de los patrones comunes en sistemas distribuidos (por ejemplo, gestión de configuración, descubrimiento de servicios, disyuntores, enrutamiento inteligente, micro-proxy, bus de control, tokens únicos, bloqueos globales, elección de liderazgo, distribución sesiones, estado del clúster). La coordinación de los sistemas distribuidos conduce a patrones repetitivos, y al usar Spring Cloud, los desarrolladores pueden implementar rápidamente servicios y aplicaciones que implementan esos patrones.

En este vídeo, *Framework Spring*, aprenderemos a crear un proyecto con el framework Spring.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=1add5b23-e837-4b3e-b610-aec300cd9a13>

Spring Cloud Data Flow

Proporciona herramientas para crear topologías complejas para canalizaciones de transmisión de datos por lotes. Las canalizaciones de datos consisten en aplicaciones Spring Boot, creadas con los marcos de microservicios Spring Cloud Stream o Spring Cloud Task. Facilita el procesamiento de datos por lotes y streaming basado en microservicios para Cloud Foundry y Kubernetes. Admite una variedad de casos de uso de procesamiento de datos, desde ETL hasta importación/exportación, transmisión de eventos y análisis predictivo.

Spring Security

Es un marco de autenticación y control de acceso potente y altamente personalizable. Es el estándar de facto para proteger las aplicaciones basadas en Spring. Se enfoca en proporcionar autenticación y autorización a las aplicaciones Java.

Spring para GraphQL

Brinda soporte para aplicaciones Spring creadas en GraphQL Java.

Spring Session

Proporciona una API e implementaciones para administrar la información de la sesión de un usuario.

Spring Integration

Amplía el modelo de programación de Spring para admitir los conocidos patrones de integración empresarial (*enterprise integration patterns*, EIP). Permite la mensajería ligera dentro de las aplicaciones basadas en Spring y admite la integración con sistemas externos a través de adaptadores declarativos. Su objetivo principal es proporcionar un modelo simple para crear soluciones de integración empresarial manteniendo la separación de preocupaciones que es esencial para producir código comprobable y mantenible.

Spring HATEOAS

Proporciona algunas API para facilitar la creación de representaciones REST que siguen el principio hipermedia como motor del estado de la aplicación (*Hypermedia As The Engine Of Application State*, HATEOAS) cuando se trabaja con Spring y especialmente con Spring MVC. El problema central que intenta abordar es la creación de enlaces y el ensamblaje de representaciones.

Spring REST Docs

Ayuda a documentar los servicios RESTful.

Spring Batch

Es un marco de trabajo por lotes ligero y completo diseñado para permitir el desarrollo de aplicaciones por lotes sólidas vitales para las operaciones diarias de los sistemas empresariales. Proporciona funciones reutilizables que son esenciales en el procesamiento de grandes volúmenes de registros, incluido el registro/rastreo, la gestión de transacciones, las estadísticas de procesamiento de trabajos, el reinicio de trabajos, la omisión y la gestión de recursos.

Spring AMQP

Aplica conceptos básicos de Spring al desarrollo de soluciones de mensajería basadas en el protocolo *Advanced Message Queuing Protocol* (AMQP). Proporciona una plantilla como una abstracción de alto nivel para enviar y recibir mensajes. También proporciona soporte para POJO controlados por mensajes con un contenedor de escucha.

Spring CredHub

Brinda soporte del lado del cliente para almacenar, recuperar y eliminar credenciales de un servidor CredHub que se ejecuta en una plataforma Cloud Foundry.

Proporciona una API para almacenar, generar, recuperar y eliminar de forma segura credenciales de varios tipos.

Spring Flo

Es una biblioteca de JavaScript que ofrece un generador visual HTML5 integrado básico para canalizaciones y gráficos simples. Esta biblioteca se utiliza como base del generador de secuencias en Spring Cloud Data Flow.

Spring para Apache Kafka

Aplica conceptos básicos de Spring al desarrollo de soluciones de mensajería basadas en Kafka. Proporciona una plantilla como abstracción de alto nivel para enviar mensajes. También brinda soporte para POJO controlados por mensajes con anotaciones `@KafkaListener` y un contenedor de escucha.

Spring LDAP

Es una biblioteca para simplificar la programación LDAP en Java, construida sobre los mismos principios que Spring Jdbc.

Spring Shell

Permite crear fácilmente una aplicación de Shell con todas las funciones (también conocida como línea de comandos) dependiendo de los jars de Spring Shell y agregando sus propios comandos (que vienen como métodos en Spring Beans).

Spring StateMachine

Es un marco para que los desarrolladores de aplicaciones utilicen conceptos de máquinas de estado con aplicaciones Spring.

Spring Vault

Proporciona abstracciones familiares de Spring y soporte del lado del cliente para acceder, almacenar y revocar secretos. Ofrece abstracciones de alto y bajo nivel para interactuar con Vault, lo que libera al usuario de preocupaciones de infraestructura.

Spring Web Flow

Se basa en Spring MVC y permite implementar los flujos de una aplicación web. Un flujo encapsula una secuencia de pasos que guían a un usuario a través de la ejecución de alguna tarea empresarial. Abarca múltiples solicitudes HTTP, tiene estado, trata con datos transaccionales, es reutilizable y puede ser dinámico y de larga duración por naturaleza.

Spring Web Services

Es un producto centrado en la creación de servicios web basados en documentos. Tiene como objetivo facilitar el desarrollo de servicios SOAP con contrato primero, lo que permite la creación de servicios web flexibles utilizando una de las muchas formas de manipular las cargas útiles XML.

Spring implementa el patrón de diseño de **inyección de dependencias**, de forma que se suministran objetos a una clase en lugar de ser la propia clase la que cree dichos objetos. Estos objetos cumplen un contrato (de ahí el concepto de dependencia) que nuestra clase necesita para poderlos usar.

Librerías para el desarrollo de interfaces gráficas de usuario

Las pilas de Java tradicionales se diseñaron para **aplicaciones monolíticas** con largos tiempos de inicio y grandes requisitos de memoria en un mundo donde la

nube, mientras que los contenedores y Kubernetes no existían. Los marcos de Java necesitaban evolucionar para satisfacer las necesidades de este nuevo mundo. Para ello, se ha creado **Quarkus** como plataforma para permitir a los desarrolladores de Java crear aplicaciones para un mundo moderno nativo de la nube. Quarkus es un *framework* nativo de Java para Kubernetes diseñado para las máquinas virtuales GraalVM y HotSpot, creado a partir de las mejores bibliotecas y estándares de Java. El objetivo es hacer de Java la **plataforma líder** en Kubernetes y entornos sin servidor al mismo tiempo que ofrece a los desarrolladores un marco para abordar una gama más amplia de arquitecturas de aplicaciones distribuidas.

Para el desarrollo de interfaces gráficas de usuario con Java, existen una serie de librerías que nos permiten acelerar el desarrollo y que enumeramos a continuación:

AWT

Abstract Window Toolkit (AWT) fue la primera versión de la interfaz gráfica de usuario (GUI) para Java creada en el año 1995. Se trata de un conjunto de clases básicas que en su visualización son diferentes en función del sistema operativo. Con Java Swing esta librería quedó obsoleta.

Swing

Es un **conjunto de clases** que se extienden de AWT, incorporan un número elevado de componentes visuales que son independientes de la plataforma y que siguen el patrón modelo-vista-controlador. Java Swing forma parte de las clases básicas de Java (*Java foundation classes*, JFC). Las JFC son un conjunto completo de componentes y servicios de interacción gráfica con el usuario, creados para Java8, que simplifican drásticamente **el desarrollo y la implementación** de aplicaciones de escritorio e Internet/Intranet de calidad comercial (Zukowski, 2000).

JavaFX

Es una plataforma de aplicaciones cliente de próxima generación de código abierto

para sistemas integrados, móviles y de escritorio contruidos en Java. Se compone de un conjunto de herramientas **moderno, eficiente y con todas las funciones** para desarrollar aplicaciones de clientes enriquecidos. Permite a los desarrolladores integrar gráficos vectoriales, animación, sonido y activos web de vídeo en una aplicación interactiva, completa y atractiva (Chin, Vos y Weaver, 2021).

JGoodies

Ofrece **software, productos y servicios gratuitos** para desarrolladores de escritorio Java. Nos enfocamos en el diseño de la interfaz de usuario de Java y su implementación en Swing. Como organización, ofrecen capacitación, consultoría, bibliotecas, aplicaciones de ejemplo, patrones de escritorio, plantillas de diseño, una arquitectura de aplicaciones y un proceso de producción para acelerar el proceso de desarrollo de aplicaciones de escritorio con Java.

Qtjambi

Es la biblioteca Qt disponible para Java. Qt es el marco C ++ estándar de facto para el desarrollo de software multiplataforma de alto rendimiento. Es una tecnología con **soporte oficial** dirigida a todos los programadores de escritorio que desean escribir clientes GUI enriquecidos utilizando el lenguaje Java, mientras que al mismo tiempo aprovechan la potencia y el rendimiento de Qt.

Apache Pivot

Permite a los desarrolladores crear fácilmente **aplicaciones conectadas**, multiplataforma y visualmente atractivas en Java. Es también el único marco para *installable internet applications* (IIA) abierto y está impulsado en su totalidad por la comunidad de desarrollo de software. Permite a los desarrolladores crear **soluciones** utilizando las herramientas que ya conocen, lo que reduce el tiempo de entrega y la expansión tecnológica.

Java Servlets

Se utilizan para crear una **aplicación web**, reside en el lado del servidor y genera una página web dinámica. Podemos verlo como una forma de construir una página HTML dinámica mediante una programación Java. En un servidor web que sea capaz de contener *servlets*: al recibir la petición, se ejecuta el código Java correspondiente y, como resultado, se genera la página web.

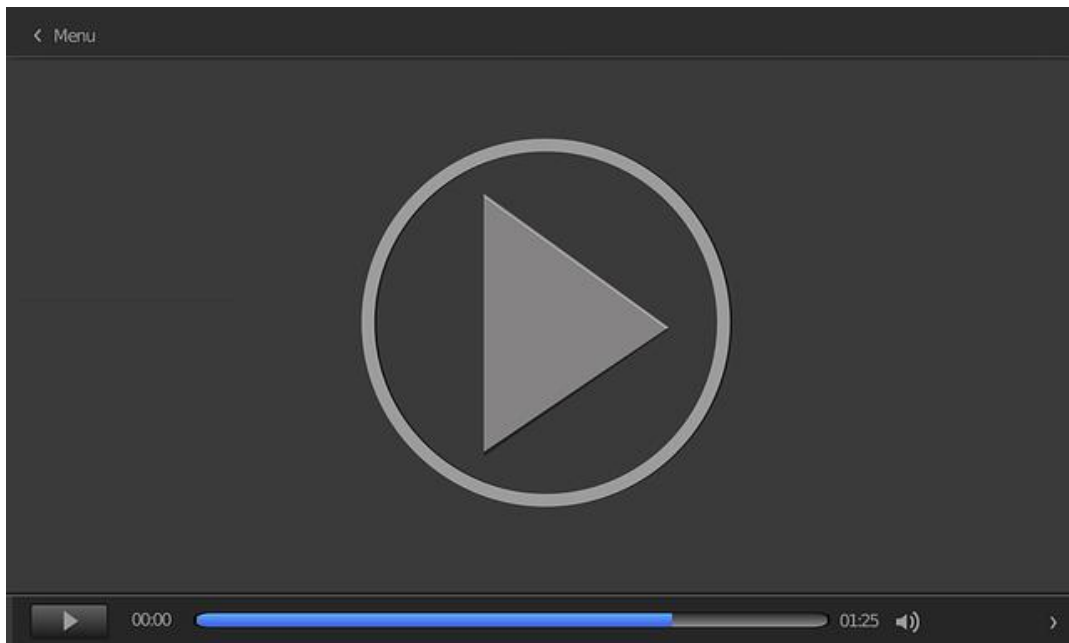
JavaServer Pages (JSP)

Conceptualmente similar a Java Servlets, sirve para construir una página HTML dinámica mediante una programación Java. La diferencia es que **se separa** la vista de la página de la lógica de control, por lo que la visualización queda en el archivo JSP mientras desarrollamos en Java el resto de la lógica que es **invocada** desde el JSP siempre en el lado del servidor (Murach, 2014).

PrimeFaces

Es un marco de código abierto popular para JavaServer Faces que presenta más de cien componentes, kit móvil, validación del lado del cliente y motor de temas. Dispone de un amplio conjunto de componentes (HtmlEditor, Dialog, autocompletar, gráficos y muchos más), integra AJAX basado en el API JSF AJAX estándar. Simplemente requiere la **integración de un fichero .jar**, sin más dependencias. Tiene un marco para crear temas, amplia documentación y una gran comunidad de usuarios (Caliskan y Varaksin, 2015).

En este vídeo, *Primefaces: desarrollando interfaces de usuario web en Java*, aprenderemos a ejecutar el proyecto primefaces-showcase para ver todos sus componentes y crear una solución Java cuya interfaz gráfica web está basada en el *framework* PrimeFaces.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=45dbd105-9402-4000-bf6a-aec300cd98bc>

Google Web Toolkit (GWT)

GWT es un conjunto de herramientas de desarrollo para **crear y optimizar aplicaciones complejas** basadas en navegador web. Su objetivo es permitir el desarrollo productivo de aplicaciones web de alto rendimiento sin que el desarrollador tenga que ser un experto en peculiaridades del navegador, XMLHttpRequest y JavaScript. Es de código abierto, completamente gratuito y lo utilizan miles de desarrolladores de todo el mundo. Fue creado por Google para ocultar la complejidad de varios aspectos de la tecnología AJAX, al ser compatible

con los principales navegadores.

Vaadin

Es una plataforma comercial para desarrollar rápidamente aplicaciones web en *back-ends* Java. Permite crear **interfaces de usuario escalables** en Java o TypeScript y provee herramientas, componentes y un sistema de diseño integrados para iterar más rápido, diseñar mejor y simplificar el proceso de desarrollo (Duarte, 2021).

Lombok

A nivel de utilidades, el proyecto **Lombok** es una biblioteca de Java que se conecta automáticamente a su editor y herramientas de compilación. Permite no tener que volver a escribir métodos *get* o *set*, sino que en su lugar con una anotación la clase tiene un constructor con todas las funciones. De esta manera, podemos **simplificar** el código necesario y en tiempo de compilación será añadido. La integración con los principales IDE hace que no se generen errores aunque esos métodos no los veamos escritos como tal.

De esta forma una clase quedará escrita como:

```
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@ToString
@Builder
public class Usuario implements Serializable {
    private String nombre;
    private String apellido;
    private int edad;
}
```

Y podrá ser utilizada como:

```
Usuario u =  
Usuario.builder().nombre("Nombre1").apellido("Apellido1").edad(20).build()  
;  
System.out.println ("Mi usuario es: " + u.toString());
```

Para instalarlo descargaremos el **fichero lombok.jar** de la web del proyecto y ejecutaremos **java -jar lombok.jar**, y empezará a buscar posibles IDE instalados, o también podremos indicarle la ruta en la que están desplegados.



Figura 6. Captura de la instalación de Lombok con los IDE del equipo. Fuente: elaboración propia

Accede al código fuente del ejemplo con Lombok a través del aula virtual.

5.7. Referencias bibliográficas

Bloch, J. (2017). *Effective Java* (3ª ed.). Addison-Wesley Educational.

Caliskan, M., y Varaksin, O. (2015). *PrimeFaces Cookbook* (2ª ed.). Packt Publishing.

Chin, S., Vos, J. y Weaver, J. (2021). *The definitive guide to modern java clients with JavaFX 17 the definitive guide to modern java clients with JavaFX 17: Cross-platform mobile and cloud development* (2ª ed.). Apress.

Duarte, A. (2021). *Practical Vaadin: Developing web applications in java* (1ª ed.). Apress.

Jemerov, D. y Isakova, S. (2016). *Kotlin in Action*. Manning Publications.

Murach, J. (2014). *Murachs Java Servlets & JSP*. Mike Murach & Associates.

Muschko, B. (2014). *Gradle in Action*. Manning Publications.

Oaks, S. (2014). *Java performance: The Definitive Guide*. O'Reilly Media.

Oracle. (s. f.). *Java is the Language of Possibilities*.
<https://www.oracle.com/java/technologies/>

Stack Overflow. (2021, mayo). *2021 Developer Survey*.
<https://insights.stackoverflow.com/survey/2021>

Sierra, K., Bates, B. y Gee, T. (2022). *Head First Java* (3ª ed.). O'Reilly Media.

Spring Framework (s. f.). *Overview of Spring Framework*.
<https://docs.spring.io/spring-framework/docs/5.0.0.RC2/spring-framework-reference/overview.html?msclkid=8b4c6a89d12a11ec803e435f3fb600aa>

Vogel, L. (2013). *Eclipse IDE* (3ª ed.). Vogella.

Zukowski, J. (2000). *The definitive guide to java swing* (3ª ed.). Apress.

Spring in Action

Walls, C. (2019). *Spring in Action* (5ª ed.). Manning Publications.

Guía básica y principal para aprender los conceptos de inyección de dependencias e inversión del control de ejecución, para desarrollar aplicaciones web basadas en el modelo-vista-controlador con el *framework* Spring.

Java design patterns: A hands-on experience with real-world examples

Sarcar, V. (2018). *Java design patterns: A hands-on experience with real-world examples* (2ª ed.). Apress.

Guía práctica para implementar 26 de los patrones de diseño más comunes que usa Java y Eclipse. Para cada patrón, se incluye al menos su descripción teórica, una implementación práctica y una aplicación a un caso real.

1. Una característica central del lenguaje Java es la portabilidad multiplataforma, que significa:

- A. Que al compilar con javac podemos seleccionar en qué dispositivo se ejecutará.
- B. Que los programas escritos en una plataforma se pueden ejecutar en cualquier combinación de software y hardware mediante una máquina virtual.
- C. Que existe una máquina virtual que traduce el código Java escrito a un código nativo para cada hardware específico y sistema operativo.
- D. Que al compilar con javac se generan unos *bytecodes* que son entendidos por cualquier sistema operativo.

2. Relaciona las características con las diversas plataformas *low-code*:

Entornos de desarrollo integrados	1	A	Maven y Gradle
Java virtual machines	2	B	Azul Platform Core, Maxine y Eclipse OpenJ9
Plataformas para la gestión de las dependencias y procesos de compilación	3	C	Scala, Kotlin y Groovy
Lenguajes de programación	4	D	Apache NetBeans, MyEclipse y DrJava

3. Java es:

- A. Solo un lenguaje de programación.
- B. Solo una plataforma.
- C. Un lenguaje de programación y una plataforma.
- D. Solo una máquina virtual.

4. El *framework* Quarkus sirve para:

- A. Desarrollar rápidamente aplicaciones de escritorio en Java.
- B. Migrar aplicaciones Java de JavaSE a JavaEE.
- C. Integrar Maven con AWS.
- D. Facilitar el desarrollo de aplicaciones Java en la nube sobre Kubernetes.

5. Marca algunas de las ventajas e inconvenientes del desarrollo con Java:

Ventaja en ejecución multiplataforma	1	A	Eficiencia
Inconveniente en ejecución	2	B	Independencia de la plataforma
Ventaja en desarrollo	3	C	Consumo de memoria
Inconveniente en equipos con pocos recursos	4	D	Gestión de la memoria

6. Spring se basa en la inversión de control como principio de diseño de software, lo que implica:

- A. Que nuestras clases no hacen un *new* de un objeto sino que es suministrado por otro contenedor.
- B. Que el flujo de ejecución de un programa se invierte respecto a los métodos de programación tradicionales y es gestionado por la plataforma.
- C. Que mediante anotaciones se genera código intermedio invisible.
- D. Que Spring genera un método *main* con instrucciones de forma automática.

7. El código fuente compilado como *bytecode*:
- A. Está cifrado y no es posible su lectura.
 - B. Es código binario y no es posible su lectura.
 - C. Viene ofuscado por lo que no es posible su lectura.
 - D. Existen herramientas de decompilación que permiten volver a reconstruir el código fuente, salvo los comentarios.
8. El framework PrimeFaces es:
- A. Una alternativa al framework Spring para el desarrollo de aplicaciones empresariales.
 - B. Una plataforma para facilitar el despliegue de aplicaciones Java en la nube.
 - C. Un marco para JavaServer Faces para facilitar el desarrollo de interfaces de usuario.
 - D. Un entorno de desarrollo integrado que incorpora una máquina virtual de Java.
9. El proyecto Lombok facilita el desarrollo de aplicaciones Java porque:
- A. Implementa el patrón de inyección de las dependencias.
 - B. Implementa el patrón de inversión de control.
 - C. Invoca automáticamente al generador de código de eclipse y nos añade métodos sin tener que escribirlos y podemos retocarlos.
 - D. Sin alterar el código que vemos y escribimos, generará código antes de compilar lo que completará nuestras clases con constructores *get*, *set* y *toString*, entre otros.

10. Spring implementa el patrón de diseño de inyección de dependencias lo que implica:

- A. Que todas las clases son estáticas.
- B. Que nuestras clases no hacen un *new* de un objeto, sino que es suministrado por otro contenedor.
- C. Que en un fichero pom.xml ponemos todas las librerías de las que depende nuestra aplicación.
- D. Que el flujo de ejecución de un programa se invierte respecto a los métodos de programación tradicionales y es gestionado por la plataforma.

Plataformas de Desarrollo de Software

Tema 6. Plataformas para el desarrollo en .NET

Índice

Esquema

Ideas clave

- 6.1. Introducción y objetivos
- 6.2. Contexto de las plataformas para el desarrollo en .NET
- 6.3. Valor y utilidad de las implementaciones de la plataforma .NET
- 6.4. Entornos de desarrollo integrados
- 6.5. Plataformas para la gestión de paquetes
- 6.6. Plataformas aceleradoras del desarrollo en .NET
- 6.7. Referencias bibliográficas

A fondo

C# 10 and .NET 6 - Modern Cross-Platform Development: Build apps, websites, and services with ASP.NET Core 6, Blazor, and EF Core 6 using Visual Studio 2022 and Visual Studio Code

Pro microservices in .NET 6: With examples using ASP.NET core 6, MassTransit, and Kubernetes

Test

PLATAFORMAS PARA EL DESARROLLO EN .NET			
CONCEPTOS	VALOR Y UTILIDAD	HERRAMIENTAS	
.NET como implementación .NET Framework .NET 5+ (incluido .NET Core) Mono Universal Windows Platform	Ventajas .NET: - Independencia de la plataforma - Interoperabilidad de los lenguajes - Desarrollo de varios tipos de aplicaciones - Biblioteca Común - Gestión de la memoria - Mayor eficiencia en ejecución	IDE: - Visual Studio - Visual Studio Code - OmniSharp - JetBrains Rider - .NET Cli	Gestores paquetes: - NuGet - ProGet
Bibliotecas de clases Específicas de la plataforma Portables .NET Standard	Desventajas .NET: - Limitaciones <i>Entity Framework</i> - Dependencia de <i>Microsoft</i> - Fugas de memoria - Transición a <i>.NET Core</i>	Aceleradores: - ReSharper - NDepend - LINQPad - .NET Reflector - ILSpy - dotPeek - dotTrace - dotMemory - PerfCollect - PerfView - BenchmarkDotNet	
.NET es independiente del lenguaje C#, F#, Visual Basic	Lenguajes .NET: - C# - Visual Basic - TypeScript - F# - XAML - ASP.net		

6.1. Introducción y objetivos

En este tema se estudian las plataformas relacionadas con la tecnología .NET. En primer lugar, estableceremos el contexto de la tecnología para conocer cómo ha ido evolucionando a lo largo de los años y cuál es su situación actual, así como las **ventajas e inconvenientes** de su uso.

Posteriormente veremos algunos de los **principales entornos de desarrollo integrado** que facilitan la escritura, compilación, depuración y despliegue de soluciones basadas en .NET, así como algunas plataformas que facilitan la automatización de la construcción y el despliegue de este tipo de aplicaciones. Por último, expondremos algunas de las plataformas que aceleran el desarrollo de soluciones basadas en tecnologías .NET.

Con el estudio de este tema pretendemos alcanzar los siguientes objetivos:

- ▶ Comprender el contexto, el valor y la utilidad que tiene el desarrollo con una tecnología con una importante presencia en el mercado como es .NET.
- ▶ Conocer los principales entornos de desarrollo integrado para .NET .
- ▶ Introducir algunas herramientas de automatización para la gestión de dependencias y paquetes sobre .NET.
- ▶ Conocer algunas plataformas aceleradoras del desarrollo a nivel de interfaz de usuario y servidores empresariales.

6.2. Contexto de las plataformas para el desarrollo en .NET

Bajo el nombre de .NET incluimos el conjunto de plataformas de desarrollo de software que Microsoft ha ido lanzando a lo largo de los años, fruto de la evolución propia de la tecnología, así como de las necesidades que como proveedor de plataformas ha querido implementar.

Para poder llegar a entender la situación actual es necesario conocer la **historia** desde su aparición. La historia de .NET se remonta a 1996, cuando el ingeniero danés Anders Hejlsberg abandona Borland, donde era el arquitecto del equipo de desarrollo de los productos Turbo Pascal y Delphi y se incorpora a Microsoft. Bill Gates pretendía conseguir una **plataforma similar a Java** —que por aquel entonces se estaba convirtiendo en la plataforma de desarrollo de referencia para muchos programadores—, pero mejorada, aprendiendo de sus fallos y limitaciones.

Hejlsberg comienza su andadura en Microsoft asumiendo el rol de arquitecto principal de la construcción de Visual J++, una implementación mezcla de C++ y Java 1.1 que tenía como principal característica la creación de interfaces gráficas. Dicha implementación no llegó a ver la luz puesto que Microsoft fue demandado por Sun y tuvo que paralizar su desarrollo. Al abandonarse dicho proyecto, surge uno nuevo: el proyecto .NET. El objetivo de este proyecto consistía en ofrecer las **herramientas necesarias** para crear software de propósito general con la más alta calidad.

A finales del 2000 se lanza la primera versión beta y es en febrero de 2002 cuando se libera oficialmente la versión 1.0 de lo que se conoció como .NET Framework. Desde esa primera versión, se ha ido liberando **diferentes versiones mejoradas** estructuralmente y de acceso a los recursos hardware disponibles.

Actualmente, .NET es una plataforma de desarrollo gratuita, de código abierto y multiplataforma con el que se pueden implementar diferentes tipos de aplicaciones, entre las que destacan, tal y como se muestran en la Figura 1, los siguientes:

- ▶ Aplicaciones de consola.
- ▶ Aplicaciones de escritorio, incluyendo Window Presentation Foundation (**WPF**), Windows Forms y la plataforma universal de Windows (**UWP**).
- ▶ Aplicaciones móviles.
- ▶ Aplicaciones web y microservicios.
- ▶ Aplicaciones nativas de la nube.
- ▶ Funciones sin servidor en la nube.
- ▶ Otras, como pueden ser juegos, IoT (Internet de las cosas) o servicios de Windows.



Figura 1. Tipos de aplicaciones desarrolladas con .NET. Fuente: Díaz Alcolea, 2020.

Con .NET, el código y los archivos del proyecto tienen el **mismo aspecto** independientemente del tipo de aplicación con que se compile. Y cada aplicación tiene acceso a las mismas funcionalidades del lenguaje, API y tiempo de ejecución. Por otro lado, al ser multiplataforma es posible generar aplicaciones para diferentes

sistemas operativos (Windows, macOS, Linux, iOS, Android, etc.) y permite, además, usar funcionalidades específicas de la plataforma como son las API propias del sistema operativo.

.NET ofrece lo que se conoce como **implementaciones**, que ofrecen características diferentes en función del tipo de aplicación que se desea desarrollar y el sistema operativo al que va dirigido. Cada implementación incluye un entorno de ejecución y una biblioteca de clases, lo que permite incorporar también marcos de aplicaciones y herramientas de desarrollo.

Dentro de las implementaciones, actualmente conviven:

- ▶ **.NET Framework:** es la implementación original de .NET que existe desde el 2002, solo se ejecuta en Windows y está optimizado para crear aplicaciones de escritorio de Windows. A partir de la versión 4.5 implementa el .NET Standard.
- ▶ **.NET 5+ (incluido .NET Core):** es la implementación más reciente y se ejecuta en cualquier plataforma. Es la implementación más reciente y se ejecuta en cualquier plataforma. Se ejecuta en Windows, macOS y Linux, e implementa .NET Standard.
- ▶ **Mono:** utilizada cuando se requiere un tiempo de ejecución pequeño. Se suele emplear como un compilador Just-In-Time, aunque también incluye un compilador estático completo. Se activa desde las aplicaciones Xamarin para dispositivos móviles y también para juegos creados con el motor Unity. Admite todas las versiones de .NET Standard publicadas actualmente.
- ▶ **Universal Windows Platform:** se utiliza para compilar aplicaciones de IoT. Se ha diseñado para unificar los diferentes tipos de dispositivos de destino, como tabletas, teléfonos e incluso la consola Xbox. Proporciona un entorno de ejecución (AppContainer) y un conjunto de API de Windows para usar en lugar del Win32 (WinRT). Pueden escribirse aplicaciones C++, C#, Visual Basic y JavaScript.

La implementación más reciente de .NET es .NET 5+, que ya dispone de un calendario de versiones programado y que permite el desarrollo de aplicaciones que se ejecutan en cualquier sistema operativo.

Por otro lado, Microsoft desarrolló .NET Standard no como una implementación, sino como una especificación de API que establece los **criterios comunes** que permiten desarrollar bibliotecas de clases para varias implementaciones de .NET. Hasta su aparición de la especificación de .NET Standard, las distintas implementaciones tenían el esquema que aparece en la Figura 2.

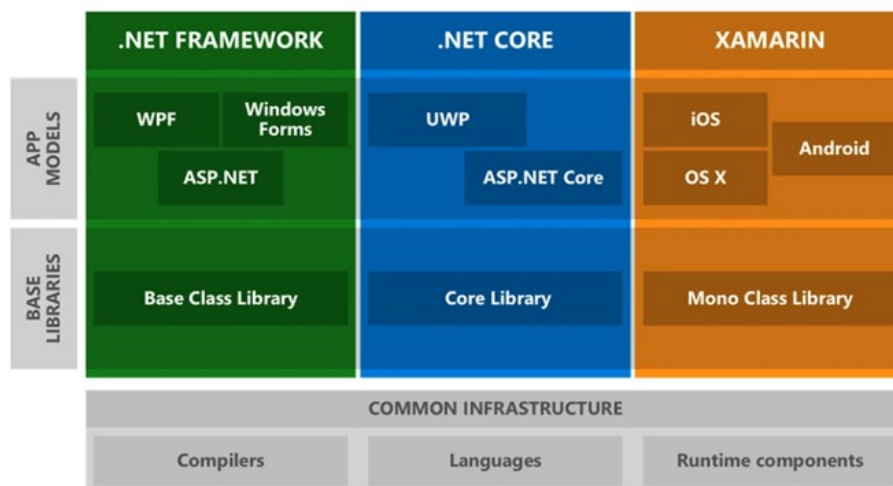


Figura 2. Implementaciones previas de .NET. Fuente: Díaz Alcolea, 2020.

El grado de acoplamiento de la librería de clases respecto de la implementación hacía más complicada su evolución, por lo que, una vez definido y lanzado el .NET Standard, el esquema que de dichas implementaciones sería el que aparece en la Figura 3.

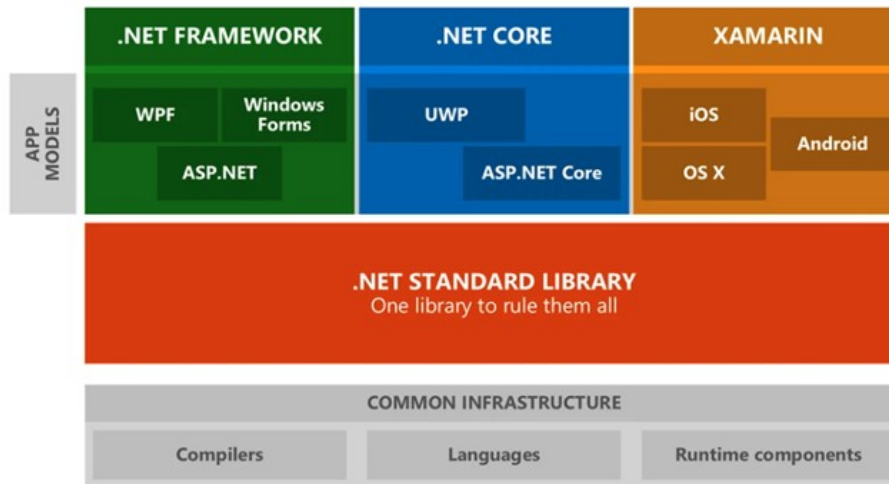


Figura 3. Incorporación de .NET Standard a las implementaciones. Fuente: Díaz Alcolea, 2020.

Las bibliotecas de clases son el concepto que introduce .NET para dividir las funcionalidades útiles **en módulos** y que puedan ser utilizados por varias aplicaciones.

Hay tres tipos de bibliotecas de clases:

- ▶ Las **específicas de la plataforma**, que permiten el acceso a las API de una plataforma determinada, pero que solo pueden ser usadas por las aplicaciones que van a ejecutarse en dicha plataforma.
- ▶ Las **portables**, que tienen acceso a subconjunto de API y las pueden usar aplicaciones que tienen como destino distintas plataformas.
- ▶ Las de **.NET Standard**, que ofrece lo mejor de las anteriores. Son específicas de la plataforma, ya que exponen toda la funcionalidad de la plataforma subyacente, y portables, puesto que funcionan en todas las plataformas compatibles. .NET Standard expone un conjunto de **contratos** y las implementaciones de .NET deben cumplir el contrato por completo o no admitirlo.

.NET es independiente del lenguaje. Esto significa que, como desarrollador es posible trabajar en uno de los muchos lenguajes que tienen como destino las implementaciones de .NET.

La independencia del lenguaje implica una **interoperabilidad** entre estos, es decir, es posible tener acceso a los tipos y miembros de bibliotecas de clases desarrolladas para implementaciones de .NET sin tener que conocer el lenguaje en el que se escribieron originalmente y sin tener que seguir las convenciones del lenguaje original. Para que esto sea posible, solo se deben exponer a aquellos que tengan las características comunes a todos los lenguajes y que se encuentran descritos en el **common language specification (CLS)**, que es el conjunto de reglas que se aplican a los ensamblados generados y se encuentra definido en el estándar ECMA-335 (ECMA International, 2019).

Los **tres principales lenguajes de programación** que puedes utilizar para programar una aplicación .NET son C#, F# y Visual Basic. Los tres lenguajes de programación son compatibles con Windows, así como con macOS y Linux. Los usuarios de Windows tienen otra opción adicional que es C++/CLI, una variante del C++ clásico desarrollada por Microsoft.

- ▶ **C#** es un lenguaje de programación moderno, basado en objetos y con seguridad de tipos. C# tiene sus raíces en la familia de lenguajes C y a los programadores de C, C++, Java y JavaScript les resultará familiar inmediatamente.
- ▶ **F#** es un lenguaje de programación interoperable, multiplataforma y de código abierto para escribir código conciso, sólido y de gran rendimiento. El foco permanece en el dominio del problema, en lugar de en los detalles de la programación. La programación de F# está orientada a datos, donde el código implica la transformación de datos con funciones.
- ▶ **Visual Basic** es un lenguaje clásico que, entre los lenguajes de .NET, es el que presenta una sintaxis lo más parecida al lenguaje humano normal, lo que facilita su

aprendizaje. A diferencia de C# y F#, para los que Microsoft desarrolla nuevas características de forma activa, el lenguaje Visual Basic es estable. Visual Basic no es compatible con las aplicaciones web, pero sí con las API web.

6.3. Valor y utilidad de las implementaciones de la plataforma .NET

El valor y utilidad del desarrollo en .NET se centran en las características propias de su desarrollo y evolución, en particular:

- ▶ **Independencia de la plataforma:** cuando Microsoft lanzó .NET en 2002 estaba pensado solo para usuarios de productos de Microsoft, lo que significaba que no se podía codificar ninguna aplicación para otro sistema operativo. Sin embargo, con la aparición de .NET Core, y actualmente con la versión .NET 6, los principales sistemas operativos son compatibles con ellas.
- ▶ **Interoperabilidad de los lenguajes:** uno de los principales beneficios y razones por las que los desarrolladores se decantan por el uso de .NET como plataforma es la posibilidad de programar en varios lenguajes, de forma que, de cara a desarrollar herramientas que cumplan requisitos de negocio concretos, se pueda hacer uso de lo mejor de cada lenguaje.
- ▶ **Desarrollo de varios tipos de aplicaciones:** esto es algo especialmente fluido en .NET, ya que no es necesario conocer el sistema operativo en el que se ejecutará una aplicación a la hora de desarrollarla, puesto que será posible programar de la misma forma independientemente de la plataforma destino y el tipo de aplicación: web, escritorio, aplicaciones para dispositivos móviles, aplicaciones de consola, etc.
- ▶ **Biblioteca común:** al hilo de la interoperabilidad de los lenguajes, se dispone de una biblioteca de clases muy amplia y en diferentes lenguajes de programación. Además, existe una amplia comunidad de usuarios y documentación sólida por parte de Microsoft.

- ▶ **Gestión de la memoria:** no hay necesidad de reservar memoria para las variables o gestionar punteros a espacios de memoria. El *runtime* se encarga de la gestión de la memoria, al asignar la necesaria requerida para variables y objetos. Cuenta con un recolector de basura (*garbage collector*, GC) que detecta si el programa no está siendo utilizado y procede a liberar el espacio de memoria que se encontraba dispuesto para su uso.
- ▶ **Seguridad:** permite manejar niveles de seguridad en el código, lo que posibilita la lectura pero no la escritura, por ejemplo, de aquellos accesos que procedan de la web.
- ▶ **Mayor eficiencia en ejecución:** las aplicaciones en .NET se compilan inicialmente en un lenguaje intermedio y, en tiempo de ejecución, el compilador Just-In-Time (JIT) convierte el lenguaje intermedio en código nativo. Además, para aplicaciones UWP, Microsoft ha desarrollado la tecnología .NET Native que permite la compilación directa en código nativo, con lo que aumenta todavía más su rendimiento.

Algunos de los inconvenientes del uso de .NET pueden ser:

- ▶ Soporte para **objetos relacionales limitado:** .NET funciona con el enfoque de programación orientada a objetos, que se basa en objetos en lugar de acciones y datos en lugar de lógica. Su Entity Framework proporciona el soporte para el desarrollo de aplicaciones de software orientado a datos en .NET Framework. La entidad actúa como puente entre el .NET Framework orientado a objetos y las bases de datos SQL. Sin embargo, algunos desarrolladores creen que Entity Framework no ofrece la flexibilidad requerida y no es compatible con todos los diseños de bases de datos disponibles.

- ▶ **Vendor lock-in:** .NET es un marco de Microsoft. Si bien Xamarin y .NET Core son fuentes abiertas, todo el ecosistema está lejos de ser impulsado por la comunidad. Esto significa que sus productos dependerán de las decisiones que tome Microsoft.
- ▶ **Fugas de memoria:** el problema de las fugas de memoria es común con todas las tecnologías. .NET también es una de esas plataformas que con frecuencia ha sido criticada por problemas relacionados con la memoria y la fuga de memoria. Aunque .NET posee un recolector de basura para este problema, los ingenieros aún deben invertir esfuerzos adicionales en la gestión de recursos.
- ▶ **Dificultad en la transición a .NET Core:** la transición a .NET Core podría ser larga y difícil. Las aplicaciones creadas con versiones anteriores de .NET son mucho más que las creadas con .NET Core. La transición de estas aplicaciones a las últimas versiones de .NET puede ser un poco complicada.

6.4. Entornos de desarrollo integrados

Como se ha introducido anteriormente, los entornos de desarrollo integrado (IDE) permiten a los desarrolladores trabajar en un solo entorno y mejorar su flujo de trabajo de programación. Cada IDE tiene diferentes recursos, pero todos incluyen un editor de texto, herramientas de compilación y un depurador.

Los IDE para el desarrollo con .NET más **populares** son:



Figura 4. IDE para .NET. Fuente: elaboración propia.

Visual Studio

Es un entorno de desarrollo integrado (IDE) desarrollado por Microsoft. Se trata de un conjunto de herramientas que permiten desarrollar aplicaciones de escritorio, aplicaciones móviles, aplicaciones web ASP.NET, y servicios web XML. Tienen las siguientes características:

- ▶ Es **multiplataforma**: aunque es un producto desarrollado por Microsoft, es posible utilizarlo en Mac, Android, iOS y Linux. Así como también tanto en la web como en la nube.
- ▶ Promete **eficiencia pura**: no sólo se puede escribir códigos más rápidamente sino que permite depurar y emitir diagnósticos de forma mucho más sencilla.
- ▶ Es **compatible** con los siguientes lenguajes de programación: C#, C++, Visual Basic .NET, Python, Java, JavaScript, TypeScript, Node.js, Ruby, PHP y F#. También es compatible en las siguientes plataformas y flujos de trabajo: .NET, Web, Nube, escritorio, juegos, móvil (Android/iOS), ciencia de datos y aprendizaje automático/inteligencia artificial.
- ▶ Puedes crear aplicaciones **nativas como híbridas**: tanto para Windows como para Android e iOS.
- ▶ Utiliza gráficos **vanguardistas**.
- ▶ Incluye **soporte técnico** de primerísima categoría para el desarrollo web

Visual Studio es una herramienta de desarrollo muy completa, ya que le permite tanto a desarrolladores con años de experiencia como a desarrolladores novatos escribir código de **forma sencilla**. Visual Studio ofrece las siguientes características y funcionalidades:

- ▶ Ofrece opciones para depurar el código, generar perfiles y emitir diagnósticos de forma fácil.
- ▶ Permite integración de pruebas.

- ▶ Permite acceder a un *marketplace* para instalar extensiones.
- ▶ Permite utilizar el control de versiones de Git para rastrear y guardar los cambios en los ficheros.
- ▶ Permite desarrollar e implementar bases de datos de SQL Server y Azure SQL fácilmente.
- ▶ Permite desarrollar extensiones propias.
- ▶ Permite crear, administrar e implementar aplicaciones de escala de nube en Azure.
- ▶ Permite desarrollar tanto aplicaciones nativas como híbridas para Android, iOS y Windows.

Esta herramienta es muy útil para programadores, principalmente gracias a su **versatilidad**, ya que les permite:

- ▶ Crear softwares y programas relativos al universo web, como es el caso de aplicaciones (videojuegos incluso) o móvil, incluso instrumentos de *big data* y analítica.
- ▶ Editar, depurar y compilar código para posteriormente publicar una aplicación.
- ▶ Reducir no solo el esfuerzo en actividades rutinarias, sino además mejorar la eficiencia de las tareas esenciales de los desarrolladores.
- ▶ Instalar integraciones y extensiones relacionadas con *machine learning* (aprendizaje automático) o GitHub.
- ▶ Desarrollar extensiones propias en Visual Studio.

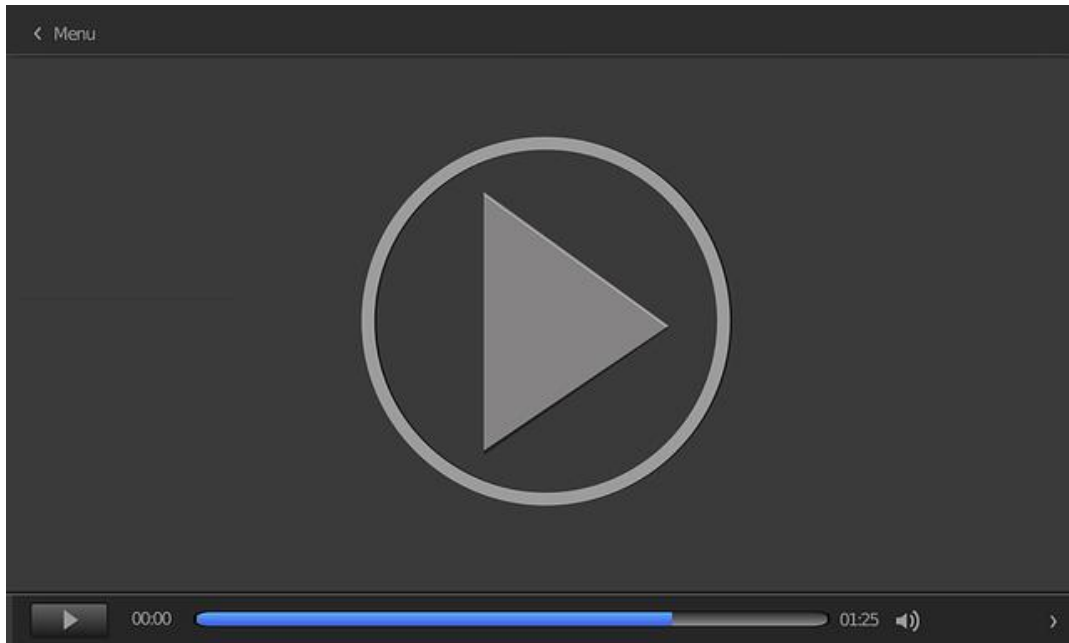


Figura 4. Captura de los componentes para instalar en Visual Studio. Fuente: elaboración propia

Con más de una docena de versiones públicas, su formato actual (que es Visual Studio 2022, en su versión 17.1.1) ofrece una serie de **mejoras** y se posiciona como el mejor IDE completo para desarrolladores .NET y C++ en Windows. Las alternativas que ofrece Visual Studio son las siguientes:

- ▶ **Community:** IDE eficiente y gratis para estudiantes, colaboradores de código abierto y usuarios particulares. Permite crear aplicaciones para Android, iOS, Windows y la Web.
- ▶ **Professional:** IDE profesional adecuado para equipos pequeños.
- ▶ **Enterprise:** solución completa y escalable para equipos de cualquier tamaño.
- ▶ **Team Explorer:** una solución gratuita para que los usuarios que no son desarrolladores interactúen con Azure DevOps Server y Azure DevOps.

En este vídeo, *Introducción al desarrollo de aplicaciones con .NET y Visual Studio*, aprenderemos a crear varios proyectos con .NET y a editarlos, ejecutarlos y depurarlos sobre Visual Studio.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=793c9a5f-58ff-46cc-8ec6-aec300cd969c>

Las últimas características incorporadas son:

- ▶ **Herramientas remotas:** permite la implementación de aplicaciones, la depuración y la comprobación remotas, la generación de perfiles de rendimiento y la ejecución de pruebas unitarias en equipos que no tengan instalado Visual Studio.
- ▶ **IntelliTrace *stand-alone collector*:** permite recopilar datos de diagnóstico de sus aplicaciones en servidores de producción sin instalar Visual Studio ni volver a implementar la aplicación.
- ▶ **Agentes:** se puede usar para realizar pruebas de carga, funcionales y

automatizadas.

- **Build tools:** permite compilar proyectos de Visual Studio desde una interfaz de la línea de comandos. Algunos proyectos admitidos son: ASP.NET, Azure, escritorio de C++, ClickOnce, contenedores, .NET Core, .NET Desktop, Node.js, Office y SharePoint, Python, TypeScript, pruebas unitarias, UWP, WCF (Windows Communication Foundation) y Xamarin.

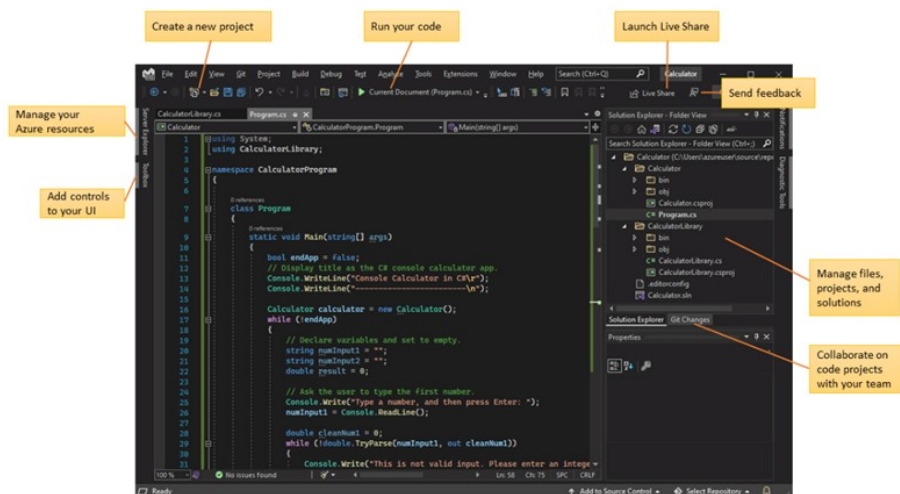


Figura 5. Captura de pantalla de Visual Studio 2022. Fuente: Lee y Olprod, 2022.

OmniSharp

Permite el desarrollo de aplicaciones .NET multiplataforma en editores como Atom, Brackets, Sublime Text, Emacs y Vim.

OmniSharp es un conjunto de herramientas, integraciones de editor y bibliotecas que juntas crean un **ecosistema** que le permite tener una gran experiencia de programación sin importar cuál sea su editor y sistema operativo de elección.

El proyecto OmniSharp ha sido creado **por la comunidad para la comunidad**. Si bien algunos de nosotros trabajamos para Microsoft, este proyecto es un esfuerzo de la comunidad y no está patrocinado ni respaldado por Microsoft.

.NET CLI

La interfaz de línea de comandos (CLI) es una nueva herramienta de **plataforma cruzada** para crear, restaurar paquetes, construir, ejecutar y publicar aplicaciones multiplataforma en Linux, macOS y Windows. Se instala con el SDK de .NET, por lo que no necesitamos instalarlo por separado en la máquina de desarrollo.

Visual Studio Code

Es un editor proporcionado por Microsoft de código fuente ligero pero potente que se ejecuta en su escritorio y multiplataforma, ya que está disponible para Windows, macOS y Linux. Dispone de soporte incorporado para JavaScript, TypeScript y Node.js y tiene un rico ecosistema de extensiones para otros lenguajes (como C++, C#, Java, Python, PHP, Go) y *runtimes* (como .NET y Unity). El entorno no dispone de ningún alarde, centrándose exclusivamente en ofrecer **flexibilidad y simpleza** para precisamente promover esa compatibilidad en distintas plataformas que ofrece, más allá de soporte para repositorios Git o posibilidad de abrir varias instancias de archivos en una misma ventana.

Este editor tiene, según la última encuesta para profesionales de Stack Overflow (Stack Overflow, 2021), un liderazgo significativo en cuanto a los IDE de desarrollo.

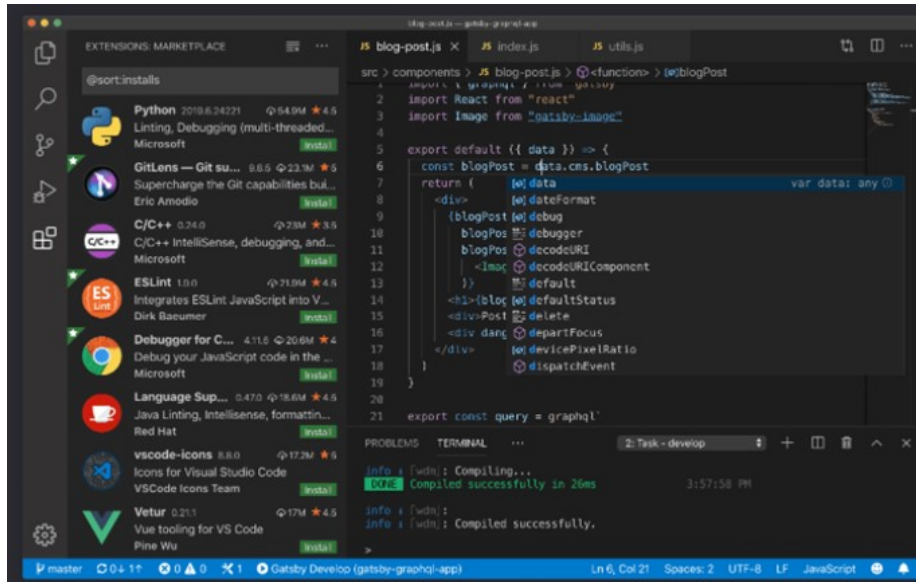


Figura 6. Captura de pantalla de Visual Sudio Code. Fuente: Visual Studio Code, s. f.

JetBrains Rider

Rider de JetBrains es un IDE .NET multiplataforma basado en la plataforma IntelliJ y ReSharper. Ofrece soporte para aplicaciones .NET y .NET Core en todas las plataformas.

Rider es compatible con proyectos basados en .NET Framework, el nuevo .NET Core multiplataforma y en Mono. Esto le permite desarrollar una amplia variedad de aplicaciones, incluidos servicios, bibliotecas y aplicaciones para escritorio .NET, juegos Unity, aplicaciones Xamarin y aplicaciones web ASP.NET y ASP.NET Core.

Cuenta con más de 2200 inspecciones de código en directo, cientos de acciones de contexto y refactorizaciones aportadas por ReSharper y combina todo esto con las sólidas funcionalidades de IDE de la plataforma IntelliJ. A pesar de un conjunto de funcionalidades pesado, Rider se ha diseñado para ser **rápido y con capacidad de respuesta**.

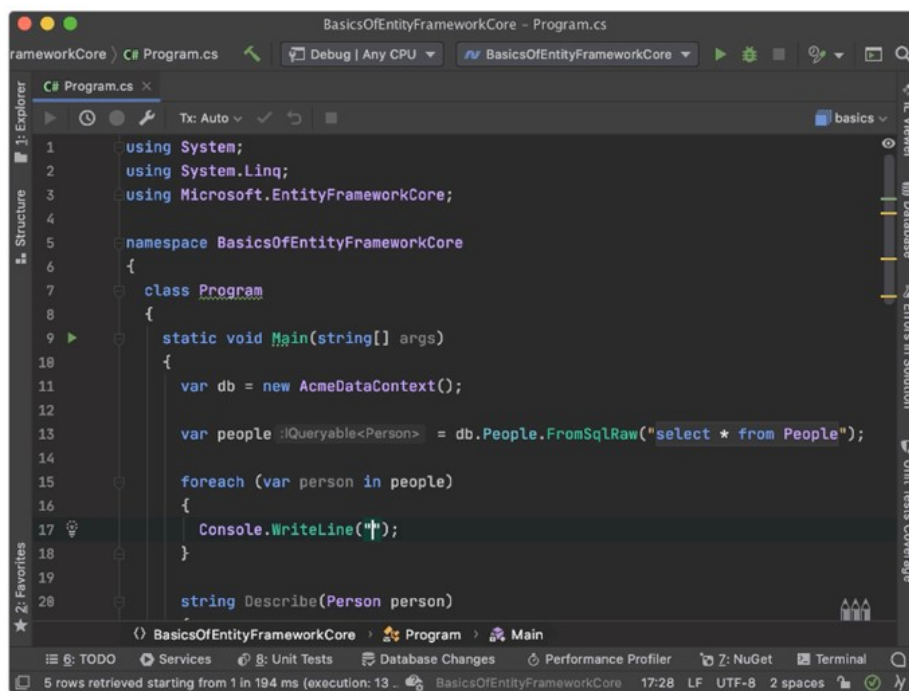


Figura 7. Captura de pantalla de JetBrains Rider. Fuente: JetBrains, s. f. a.

6.5. Plataformas para la gestión de paquetes

El soporte de administración de paquetes permite flujos de trabajo de **entrega continua** al alojar nuestros paquetes, módulos y bibliotecas, para ponerlos a disposición del equipo de desarrollo o en los procesos de construcción y compilación o en los de despliegue. Entre las principales soluciones de gestión de paquetes para .NET se encuentran: NuGet, su NuGet Package Explorer y ProGet.

NuGet es el **administrador de paquetes de referencia para .NET**. Los paquetes NuGet son una forma para que los desarrolladores creen, compartan y consuman código reutilizable. La galería NuGet es el repositorio central de paquetes que utilizan todos los autores y consumidores de paquetes. Tiene alrededor de 150 mil paquetes únicos y más de un millón quinientos mil de versiones de paquetes. Es posible utilizarlo de forma integrada desde el IDE Visual Studio, pero también viene con una función de línea de comandos. Es equivalente a *npm* o *yarn* para Node, *brew* para macOS, *apt-get* para Linux o *Maven* para Java (NuGet, 2022).

En pocas palabras, un paquete NuGet es un único archivo ZIP con la extensión *.nupkg* que contiene código compilado (DLL), otros archivos relacionados con ese código y un manifiesto descriptivo que incluye información como el número de versión del paquete. Los desarrolladores con código para compartir crean paquetes y los publican en un *host* público o privado. Los consumidores de paquetes obtienen esos paquetes de *hosts* adecuados, los agregan a sus proyectos y luego llaman a la funcionalidad de un paquete en su código de proyecto. Luego, NuGet se encarga de todos los **detalles intermedios**.

Dado que NuGet admite *hosts* privados junto con el *host* público de *nuget.org*, puede usar paquetes de NuGet para **compartir código exclusivo** de una organización o un grupo de trabajo. También puede usar los paquetes de NuGet como una manera conveniente de factorizar su propio código para usarlo únicamente en sus propios

proyectos. En resumen, un paquete NuGet es una unidad de código que se puede compartir, pero no requiere ni implica ningún medio particular de compartir.

Las principales funcionalidades que aporta NuGet son:

- ▶ Proporciona el **repositorio central** nuget.org con soporte para hospedaje privado.
- ▶ Proporciona las **herramientas** que los desarrolladores necesitan para crear, publicar y consumir paquetes.
- ▶ Lo que es más importante, mantiene una **lista de referencia** de los paquetes usados en un proyecto y la capacidad de **restaurar y actualizar** esos paquetes de esa lista.

Para que estos procesos funcionen de manera eficiente, administra una caché de paquetes y una carpeta de paquetes globales para atajar la instalación y la reinstalación. La caché evita descargar un paquete que ya se ha instalado en la máquina. La carpeta de paquetes globales permite que varios proyectos compartan el mismo paquete instalado, lo que **reduce el espacio total** de NuGet en la computadora. La caché y la carpeta de paquetes globales también son muy útiles cuando se restaura con frecuencia una gran cantidad de paquetes, como en un servidor de compilación.

Dentro de un proyecto individual, administra el **gráfico de dependencia general**, que nuevamente incluye la resolución de múltiples referencias a diferentes versiones del mismo paquete. Es bastante común que un proyecto dependa de uno o más paquetes que tienen las mismas dependencias. Muchos otros paquetes emplean algunos de los paquetes de utilidades más útiles de nuget.org. Para evitar traer varias versiones de ese paquete a la propia aplicación, NuGet determina qué versión única pueden usar todos los consumidores.

NuGet Package Explorer (NPE)

Es una aplicación que **facilita la creación y exploración** de paquetes NuGet. Puede cargar un archivo .nupkg o .snupkg desde el disco o directamente desde una fuente como nuget.org (GitHub, 2022a).

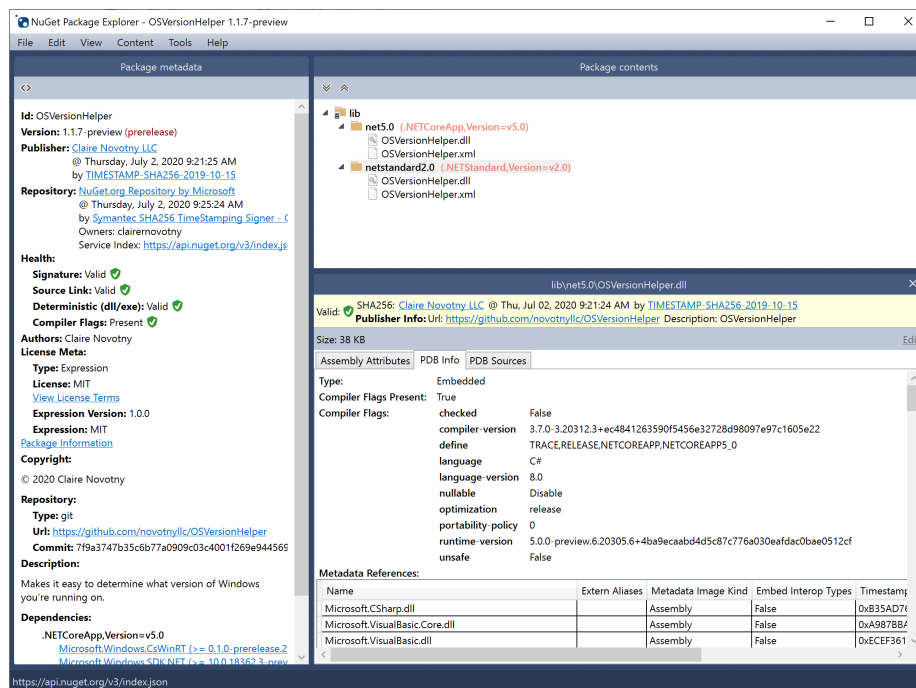


Figura 8. Vista de NuGet Package Explorer. Fuente: GitHub, 2022a.

ProGet

Es un sistema de gestión de paquetes, diseñado por la empresa de software Inedo. Permite a los usuarios alojar y administrar paquetes, aplicaciones y componentes personales o de toda la empresa. Originalmente, fue diseñado como un administrador privado de NuGet y un servidor fuente y de símbolos. A partir de 2015, ProGet amplió el soporte, agregó funciones de nivel empresarial y está **diseñado para adaptarse a una metodología de desarrollo y operaciones (DevOps)**. Las empresas utilizan ProGet para «empaquetar aplicaciones y componentes» (Inedo, s. f.) con el objetivo de garantizar que el software se cree solo

una vez y se implemente de manera uniforme en todos los entornos.

Las principales funcionalidades que aporta ProGet son:

- ▶ Permite utilizar **paquetes universales** para distribuir uniformemente sus aplicaciones y componentes o imágenes de **Docker** y gráficos de Helm (Kubernetes) para su software en contenedores.
- ▶ Facilita **compartir código creado y probado** previamente entre equipos de desarrollo, ya sea que use C#/.NET (NuGet), Python (PyPI o Conda), JavaScript/Node.js (npm), Ruby (Gems) o Java (Maven).
- ▶ Permite **administrar** software de Windows de terceros (Chocolatey), paquetes de PowerShell/recursos DSC y paquetes de Linux (RPM/YUM y Debian/apt).
- ▶ Soporta el **almacenamiento de Amazon S3 o Azure Blob** para escalar de forma económica y con pago por uso el almacenamiento de archivos de paquetes y una recuperación ante desastres fiable.
- ▶ Permite agregar **tantos nodos** de servidor web como necesite para equilibrar la carga del tráfico de varios equipos y crear servidores en todo el mundo.
- ▶ Da soporte para la detección y el **bloqueo de licencias de código** abierto que no puedan ser usadas a nivel comercial.
- ▶ Escanea los paquetes en **busca de vulnerabilidades** y bloquee el acceso a aquellos que se consideren demasiado riesgosos o inseguros según las reglas que configure.
- ▶ **Registra los despliegues de otras herramientas**, incluidas Otter, Octopus Deploy o incluso sus propios *scripts* personalizados, para que pueda ver qué paquetes se desplegaron en qué servidores.

6.6. Plataformas aceleradoras del desarrollo en .NET

Adicionalmente a las plataformas anteriormente presentadas, existen otras herramientas que extienden las funcionalidades del IDE, apoyan el análisis de rendimiento o ayudan a obtener el código cuando solo disponemos de ficheros compilados entre otras. Algunas de las principales plataformas aceleradoras del desarrollo en .NET son:

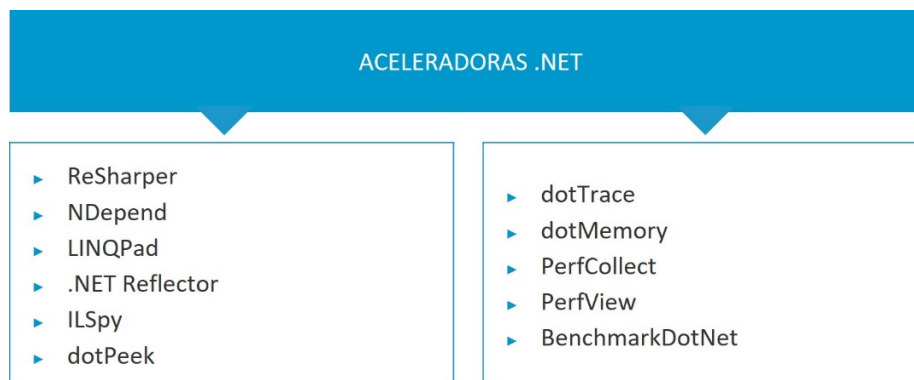


Tabla 2. Plataformas aceleradoras del desarrollo en .NET. Fuente: elaboración propia.

ReSharper

Es una extensión de Visual Studio creada por JetBrains para ayudar a los desarrolladores en la codificación de .NET. Es una de las herramientas de .NET para desarrolladores de .NET que ofrece la capacidad de evaluar la calidad del código de .NET y descubrir y reparar problemas rápidamente. También tiene muchos **atajos** para refactorizar y navegar rápidamente. Permite a los usuarios brindar soporte sin igual a todos los lenguajes tecnológicos desde C#, XAML, JavaScript, C++, TypeScript, JSON, ASP.NET MVC, XML, HTML, CSS, Protobuf, NAnt y secuencias de comandos MSBuild, incluida la funcionalidad integral entre idiomas (JetBrains, s. f. b).

Es una herramienta de productividad de código cuyo trabajo principal es ofrecer optimización del rendimiento de los códigos a través del rendimiento. Ayuda a los desarrolladores a monitorear perfiles para eliminar pérdidas de memoria. Es un corredor de unidades, un descompilador que se libera por sí solo. La suscripción a ReSharper Ultimate también proporciona dotCover (un ejecutor de pruebas unitarias de .NET y una herramienta de cobertura de código), así como otros como dotTrace, dotMemory y dotPeek.

Algunas de sus funcionalidades son:

- ▶ **Análisis de código:** amplía Visual Studio con más de 2200 inspecciones de código sobre la marcha para C#, VB.NET, ASP.NET, JavaScript, TypeScript y otras tecnologías. Para la mayoría de las inspecciones, ReSharper proporciona arreglos rápidos (bombillas) para mejorar el código.
- ▶ **Refactorizaciones:** dispone de más de sesenta refactorizaciones y más de 450 acciones de contexto ayudan a organizar el código de forma segura y moverlo por la solución, distribuir la responsabilidad, desacoplar, disminuir la complejidad o simplemente usar una sintaxis de lenguaje alternativo.
- ▶ **Navegación y búsqueda:** ayuda a llegar instantáneamente a cualquier código en una solución, sin importar cuán grande sea la solución. También puede navegar desde cualquier símbolo a su código relacionado, como implementaciones de una interfaz dada, métodos de extensión de una clase o usos de un campo.
- ▶ **Formateo y limpieza de código:** permite configurar y aplicar el estilo de código según sus preferencias personales o el estándar de su equipo. La configuración de formato y estilo de código para C#, VB.NET, ASP.NET, JavaScript, TypeScript y otros lenguajes se puede aplicar en cualquier ámbito, desde una selección hasta una solución.

- **Generación de código:** puede crear una gran cantidad de código para usted, desde nuevos archivos y clases hasta bloques condicionales y GUID y desde métodos y propiedades requeridas por una interfaz que se implementa hasta comprobaciones de igualdad y formateo de miembros.

NDepend

Es una extensión de Visual Studio que ayuda en la herramienta de análisis de código administrado de .NET estática utilizada por un desarrollador de .NET. La aplicación permite la **medición de la calidad del código** al utilizar múltiples métricas de código, así como la visualización de su diseño. También proporciona una perspectiva de vista de pájaro de la organización de la aplicación y el código. Proporciona un lenguaje de consulta personalizado especializado para examinar el acoplamiento de la aplicación, así como la capacidad de evaluar la eficacia de la adhesión al código (NDepend, s. f.).

El programa también compara instantáneas de código base, gráficos y criterios de calidad. Las consultas LINQ se pueden usar para crear reglas definidas por el usuario. CQLinq es el nombre de esta función. Además, la herramienta incluye un número significativo de reglas de código CQLinq preestablecidas. En Visual Studio, las reglas de código se pueden verificar automáticamente o durante la integración continua.

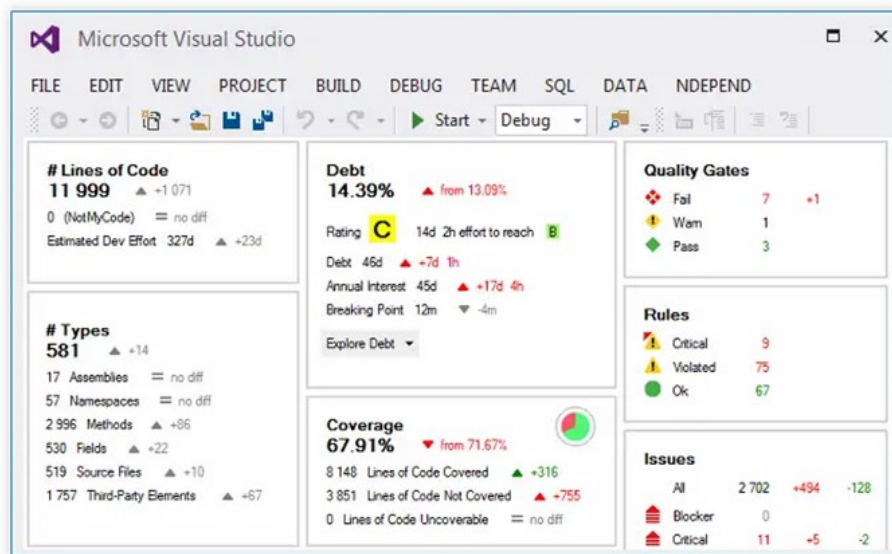


Figura 9. Vista del analizador NDepend. Fuente: NDepend, s. f.

LINQPad

Es un **playground** para programadores .NET, entendido como una aplicación liviana de .NET que se usa para escribir código C# de forma interactiva **sin un IDE**. La herramienta puede ejecutar no solo consultas simples, sino también una serie de consultas. Tiene un depurador integrado y capacidades de autocompletar (LINQPad, 2022).

La herramienta facilita que los desarrolladores puedan usarlo inmediatamente para probar fragmentos de código y consultar bases de datos usando LINQ o SQL Server Express. Es **popular** debido a su extenso formato de salida, depuración integrada, autocompletado opcional y ayuda a la creación de prototipos con comentarios instantáneos, soportando C#, F# o Visual Basic .NET.

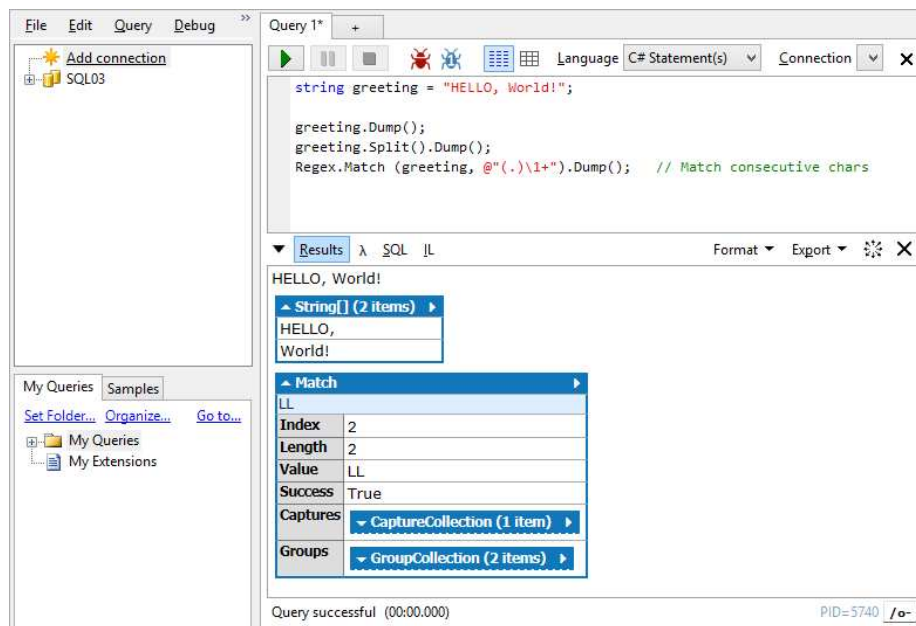


Figura 12. Vista del *playground* LINQPad. Fuente: LINQpad, s. f.

.NET Reflector

Es un conocido descompilador y analizador estático de .NET. También está disponible como extensión de Visual Studio y es una de las herramientas más útiles para los desarrolladores de .NET. El beneficio de usar la herramienta reflectora .NET es depurar su programa y descubrir problemas. Incluso puede entrar en código de terceros escrito en Visual Studio.

ILSpy

Es el navegador y descompilador de ensamblados .NET de código abierto. Es una buena alternativa libre a .NET Reflector, que ya no tiene una versión gratuita. Las herramientas pueden descompilar cualquier archivo DLL de .NET a Visual Basic o C#. En caso de que haya perdido su código fuente o no lo haya tenido en primer lugar, ILSpy le permite ver cómo funciona el código detrás de las DLL del marco.

DotPeek

es un descompilador confiable basado en ReSharper, útil para depurar ensamblajes y códigos de terceros. Con dotPeek, puede examinar problemas que ocurren en componentes de terceros, incluso sin tener acceso directo a su código fuente. El descompilador admite múltiples formatos, incluidas bibliotecas (.dll), ejecutables (.exe) y archivos de metadatos de Windows (.winmd).

PerfCollect y PerfView

Son dos soluciones enfocadas para que las herramientas de desarrollo de Dotnet sean más eficientes. PerfCollect es un *script bash* que recopila datos y los automatiza. Los *scripts bash* son una serie de **comandos contenidos en un archivo** simple para su ejecución. Mientras que, por otro lado, PerfView es una herramienta de rendimiento que se centra principalmente en Windows. PerfView es una herramienta de análisis de rendimiento y PerfCollect automatiza la recopilación de datos.

PerfView también se puede utilizar para ajustar la CPU y la asignación de memoria. También es como una caja de herramientas de ajuste de rendimiento de .NET. No requiere ninguna instalación para comenzar a usarlo. Su funcionamiento simplemente copiando la aplicación a la PC con Windows donde se requieren datos de rendimiento. Después de eso, debe configurarse sobre lo que necesita ser perfilado, como una aplicación, y comenzará a capturar sin problemas.

DotTrace y dotMemory

Son herramientas de creación de perfiles para realizar un seguimiento de los problemas de rendimiento y memoria en las aplicaciones .NET. Capaces de procesar grandes volúmenes de datos, los muestran al usuario de forma integral.

Las funciones de dotTrace incluyen detección de cuellos de botella en el rendimiento, análisis de pilas de llamadas, informes de líneas de tiempo de rendimiento y muchas otras acciones de creación de perfiles (JetBrains, s. f. c).

DotMemory tiene perfiles remotos, inspecciones automáticas para cualquier problema de memoria, análisis de tráfico de memoria, comparación de instantáneas de memoria, etc. (JetBrains, s. f. d).



Figura 13. Vista del analizador de memoria dotMemory. Fuente: JetBrains, s. f. d

BenchmarkDotNet

Es un proyecto de código abierto que elimina las suposiciones de rendimiento y le brinda métricas al aislar automáticamente cada punto de referencia y ejecutar varias iteraciones, incluido el calentamiento. Permite escribir pruebas de rendimiento, para que puedas medir y comparar el rendimiento de tus algoritmos. Ayuda a transformar métodos en puntos de referencia, realizar un seguimiento de su rendimiento y compartir experimentos de medición reproducibles (GitHub, 2022b).

Los resultados se presentan en un formato fácil de usar que destaca todos los hechos importantes sobre su experimento. La biblioteca es adoptada por más de 6800 proyectos que incluyen .NET Runtime y es compatible con .NET Foundation.

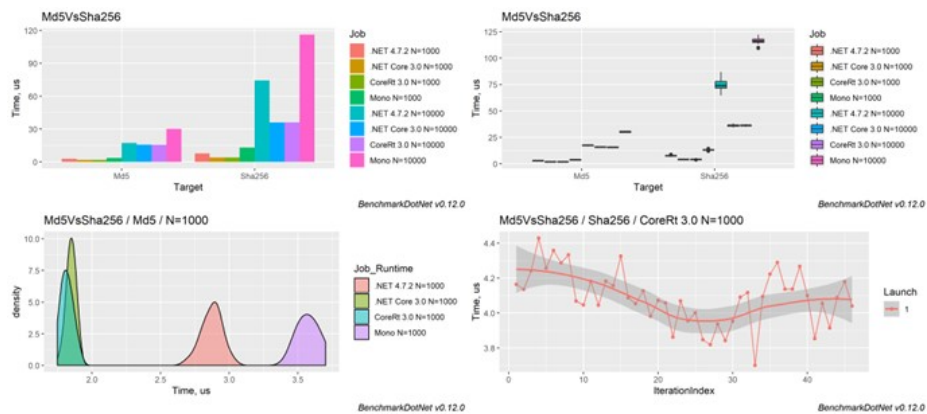


Figura 14. Vista del analizador de rendimiento de BenchmarkDotNet. Fuente: GitHub, 2022b.

6.7. Referencias bibliográficas

Díaz Alcolea, C. (2020, noviembre 9). *Qué es .NET Core*. Open Webinars. <https://openwebinars.net/blog/que-es-net-core/>

ECMA International. (2019, agosto 16). *ECMA-335. Common Language Infrastructure (CLI)*. <https://www.ecma-international.org/publications-and-standards/standards/ecma-335/>

GitHub. (2022a, abril 25) *NuGetPackageExplorer*. <https://github.com/NuGetPackageExplorer/NuGetPackageExplorer>

GitHub. (2022b, julio 1). *BenchmarkDotNet*. <https://github.com/dotnet/BenchmarkDotNet>

Inedo. (s. f.). *ProGet*. <https://inedo.com/proget>

JetBrains. (s. f. a). *Rider*. <https://www.jetbrains.com/es-es/rider/>

JetBrains. (s. f. b). *ReSharper: la extensión de Visual Studio para desarrolladores .NET*. <https://www.jetbrains.com/es-es/resharper/>

JetBrains. (s. f. c) *dotTrace. .NET Performance profiling*. <https://www.jetbrains.com/dottrace>

JetBrains. (s. f. d) *dotMemory. The .NET Memory Profiler*. <https://www.jetbrains.com/dotmemory/>

Lee, T. G. y Olprod. (2022, marzo 14). *Le damos la bienvenida al IDE de Visual Studio*. Microsoft. <https://docs.microsoft.com/es-es/visualstudio/get-started/visual-studio-ide?view=vs-2022>

LINQPad. (2022). *The .NET Programmer's Playground*. <https://www.linqpad.net/>

NDepend. (s. f.) *The "Swiss Army Knife" for .NET Developers, Architects and Teams.*

<https://www.ndepend.com/>

NuGet (s. f.). *Create .NET apps faster with NuGet.* <https://www.nuget.org/>

Stack Overflow. (2021, mayo). *2021 Developer Survey.*

<https://insights.stackoverflow.com/survey/2021>

Visual Studio Code. (s. f.). *Code editing. Redefined.* <https://code.visualstudio.com/>

C# 10 and .NET 6 - Modern Cross-Platform Development: Build apps, websites, and services with ASP.NET Core 6, Blazor, and EF Core 6 using Visual Studio 2022 and Visual Studio Code

Price, M. J. (2021). *C# 10 and .NET 6 - Modern Cross-Platform Development: Build apps, websites, and services with ASP.NET Core 6, Blazor, and EF Core 6 using Visual Studio 2022 and Visual Studio Code* (6ª ed.). Packt Publishing.

Una guía completa para que los principiantes aprendan los conceptos clave, las aplicaciones del mundo real y las funciones más recientes de C# 10 y .NET 6 con ejercicios prácticos con Visual Studio 2022 y Visual Studio Code.

Pro microservices in .NET 6: With examples using ASP.NET core 6, MassTransit, and Kubernetes

Whitesell, S., Richardson, R., & Groves, M. D. (2022). *Pro microservices in .NET 6: With examples using ASP.NET Core 6, MassTransit, and Kubernetes* (1ª ed.). Apress.

Conoce los fundamentos de la creación e implementación de microservicios con .NET 6 y obtén información sobre cuándo y por qué incorporarlos.

1. Selecciona qué tipo de aplicaciones podemos desarrollar en .NET:

- A. Aplicaciones de escritorio.
- B. Aplicaciones móviles.
- C. Aplicaciones web y microservicios.
- D. Todas las anteriores.

2. Relaciona las siguientes plataformas con su tipo:

JetBrains Rider	1	A	Descompilador
NuGet	2	B	IDE
ILSpy	3	C	<i>Playground</i>
LINQPad	4	D	Administrador de paquetes

3. Di cómo se llama la plataforma de código abierto que permite escribir pruebas de rendimiento en .NET y que ha sido usada en más de 6800 proyectos:

- A. dotTrace.
- B. BenchmarkDotNet. Foundation.
- C. PerfView.
- D. dotPeek.

4. NuGet es el administrador de paquetes de referencia para .NET, marca la característica que no está incluida en el mismo:

- A. Proporciona el repositorio central nuget.org con soporte para hospedaje privado.
- B. Proporciona las herramientas que los desarrolladores necesitan para crear, publicar y consumir paquetes.
- C. Realiza la compilación y despliegue automático de la solución.
- D. Mantiene una lista de referencia de los paquetes usados en un proyecto.

5. Marca el nombre del IDE con su característica:

Visual Studio	1	A	IDE oficial .NET
Visual Studio Code	2	B	La interfaz de línea de comandos
.NET CLI	3	C	IDE <i>open source</i> de Microsoft
JetBrains Rider	4	D	IDE con tecnología IntelliJ y ReSharper

6. ReSharper es una extensión de Visual Studio creada por JetBrains para ayudar a los desarrolladores en la codificación de .NET, que proporciona múltiples funcionalidades, excepto (marca la que no es):

- A. Análisis de código.
- B. Generación de test.
- C. Generación de código.
- D. Refactorizaciones.

7. Marca aquellas funcionalidades incluidas en el gestor de paquetes ProGet:

- A. Bloqueo de licencias de código abierto.
- B. Almacenamiento de Amazon S3 o Azure Blob.
- C. Registro de despliegue en Otter u Octopus Deploy.
- D. Todas las anteriores.

8. .NET es independiente del lenguaje, lo que significa:
- A. Que podemos construir una aplicación con diversos lenguajes como C#, F# o Visual Basic indistintamente, combinando módulos escritos en distintos lenguajes.
 - B. Que podemos construir una aplicación con diversos lenguajes como C#, F# o Visual Basic, pero no combinarlos dentro del mismo sistema.
 - C. Que puedes escribir en cualquier lenguaje de programación.
 - D. Que puedes ejecutar en cualquier sistema operativo.
9. Selecciona la versión de Visual Studio adecuada para equipos de desarrollo empresariales pequeños:
- A. Community.
 - B. Professional.
 - C. Enterprise.
 - D. Team Explorer.
10. Selecciona la implementación que se utiliza para compilar aplicaciones de IoT:
- A.NET Framework.
 - B. .NET 5+ (incluido .NET Core).
 - C. Mono.
 - D. Universal Windows Platform.

Plataformas de Desarrollo de Software

Tema 7. Plataformas para el desarrollo de servicios

Índice

Esquema

Ideas clave

- 7.1. Introducción y objetivos
- 7.2. Contexto de las plataformas para el desarrollo de servicios
- 7.3. Valor y utilidad de las plataformas para el desarrollo de servicios
- 7.4. Modelado y descripción de servicios
- 7.5. Documentación e interacción con servicios
- 7.6. Plataformas aceleradoras del desarrollo de servicios
- 7.7. Referencias bibliográficas

A fondo

- Java web services: Up and running
- Spring REST

Test

PLATAFORMAS PARA EL DESARROLLO DE SERVICIOS			
CONCEPTOS	VALOR Y UTILIDAD	HERRAMIENTAS	
Ingeniería de Servicios Servicio Arquitectura orientada a servicios Servicio web Microservicio	Valor al crear un sistema distribuido: - Separación de preocupaciones - Bajo acoplamiento - Desarrollo simultáneo - Diferentes lenguajes y tecnologías - Independencia de la plataforma - Escalabilidad - Resiliencia - Reutilización de código	Documentar: - Lucybot - Redoc - RapidDoc - Swagger UI	
Representational state transfer (REST) - Interfaz uniforme - Cliente-servidor - Sin estado - Caché - Sistema en capas - Código bajo demanda	Utilidad plataformas para servicios: - Descripción formal - Auto documentación - Auto generación esqueletos servidor - Auto generación clientes - Auto validación	Mock: - MockServer - WireMock	
Estándares: - API Blueprint - OpenAPI Specification - RESTful API modeling language - Web application description language - Web services definition language	Emular (mock) un servicio: - Agilizar desarrollo en paralelo - Integración con sistemas de terceros - Escenarios para pruebas automáticas	Interacción: - Postman - SoapUI	
		Aceleradores: - OpenAPI Generator - Swagger Codegen - RAML Tools para .NET - Spring WebFlux	

7.1. Introducción y objetivos

En este tema se estudian algunas de las plataformas que nos ayudan a desarrollar sistemas distribuidos basados en servicios.

En primer lugar, estableceremos el **contexto** de la ingeniería de servicios, aclarando conceptos clave como servicio, arquitectura orientada a servicio, servicio web o microservicio entre otros. Detallaremos las principales características de arquitecturas basadas en *representational state transfer* (REST) y los estándares para modelar interfaces o contratos de servicios.

Posteriormente, veremos algunas de las plataformas y herramientas que **facilitan** el desarrollo de servicios, ya que aportan automatización o simplifican el proceso de definición de API, generación de documentación, emulación o interacción. Finalmente, enumeraremos algunas plataformas adicionales que simplifican la creación de esqueletos o de librerías de cliente para implementar o consumir servicios.

Con el estudio de este tema pretendemos alcanzar los siguientes objetivos:

- ▶ Comprender el contexto, el valor y la utilidad que tiene el desarrollo de sistemas distribuidos basados en servicios.
- ▶ Conocer los principales estándares de definición de servicios.
- ▶ Introducir algunas herramientas de automatización para la documentación, emulación, interacción de servicios.
- ▶ Conocer algunas plataformas aceleradoras del desarrollo de sistemas distribuidos basadas en servicios.

7.2. Contexto de las plataformas para el desarrollo de servicios

En el ámbito de la ingeniería del software, existen una serie de conceptos relacionados con la creación de sistemas distribuidos basados en servicios.

Un **servicio** software es una descripción de una o más operaciones que utilizan (múltiples) mensajes para intercambiar datos entre un proveedor y un consumidor. El efecto típico de una llamada de servicio es que el consumidor obtiene alguna información y/o modifica el estado del sistema o componente proveedor. Los servicios pueden tener **diferentes atributos** y pueden ser de **diferentes categorías**, se describen mediante una interfaz y su visión desde el punto de vista del consumidor (firma y semántica) se denomina «interfaz bien definida» o contrato (Josuttis, 2007).

La **ingeniería de servicios**, también llamada ingeniería de software orientada a servicios, es un proceso de ingeniería de software que intenta descomponer el sistema en unidades autoejecutables que realizan servicios o exponen servicios (servicios reutilizables). Las aplicaciones orientadas a servicios están diseñadas en torno a servicios **débilmente acoplados**, lo que significa que existen estándares y protocolos simples que todos los interesados siguen, mientras que detrás de ellos hay una amplia variedad de servicios tecnológicos que pueden ser mucho más complejos. Los servicios reutilizables, a menudo, son proporcionados por muchos proveedores de servicios diferentes, en los que todos colaboran dinámicamente con los usuarios de servicios y los registros de servicios (Fox, 2013).

Una **arquitectura orientada a servicios** (*service oriented architecture*, SOA) es aquella diseñada para construir aplicaciones comerciales como un conjunto de componentes de caja negra débilmente acoplados y orquestados para ofrecer un determinado nivel de servicio que de soporte a una serie de procesos de negocio (Bloor et al., 2009). En SOA hay tres tipos de actores:

- ▶ **Proveedores de servicios:** son servicios de software que publican sus capacidades (interfaces o contratos) y su disponibilidad en los registros de servicios.
- ▶ **Usuarios de servicios:** son sistemas de software (que pueden ser servicios en sí mismos) que utilizan los servicios proporcionados por los proveedores de servicios. Los usuarios de servicios pueden utilizar los registros de servicios para descubrir y localizar los proveedores de servicios que necesitan.
- ▶ **Registros de servicios:** son catálogos de información en constante evolución que se pueden consultar para ver qué tipo de servicios están disponibles.

Un **servicio web** (*web service*, WS) es un sistema de software diseñado para admitir la **interacción interoperable** de máquina a máquina a través de una red. Tiene una interfaz descrita en un formato procesable por máquina, como Web Service Description Language (Alhajj y Rokne, 2014). Otros sistemas interactúan con el servicio web de la manera prescrita por su descripción utilizando mensajes y siguiendo el protocolo Simple Object Access Protocol (W3, s. f.), normalmente transmitidos mediante HTTP (protocolo de transferencia de hipertexto) con una serialización XML junto con otros estándares relacionados con la web (W3, 2004).

Los **microservicios**, también conocidos como arquitectura de microservicios, son un estilo arquitectónico que estructura una aplicación como una colección de servicios que son:

- ▶ Fácilmente mantenibles.
- ▶ Débilmente acoplados.

- ▶ Desplegables de forma independiente.
- ▶ Organizados en torno a las capacidades empresariales.
- ▶ Bajo la responsabilidad de un equipo de desarrollo pequeño.

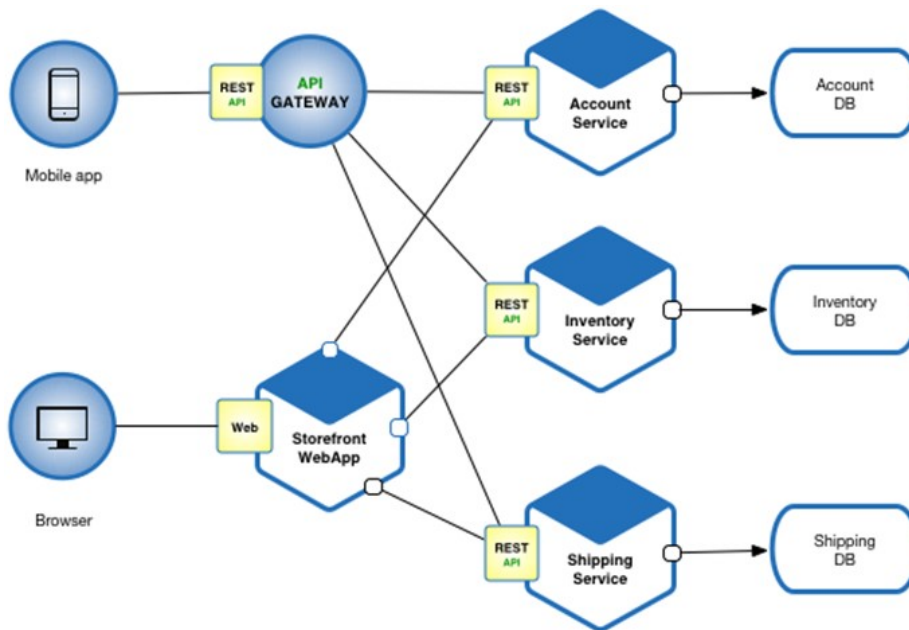


Figura 1. Ejemplo de arquitectura de microservicios. Fuente: Microservice Architecture, s. f.

La arquitectura de microservicios permite la entrega **rápida, frecuente y confiable** de aplicaciones grandes y complejas. También permite que una organización evolucione su pila de tecnología. En ocasiones, se confunde el concepto de microservicio con el de servicios desplegados en contenedores, pero que forma parte de un sistema cliente-servidor monolítico global.

Para que hablemos de microservicios, en realidad, cada uno debe verse como un proyecto independiente, que tiene su propio ciclo de vida y modelo de despliegue y que, en ocasiones, interactúa con otra colección de servicios para satisfacer un objetivo de negocio común.

Representational state transfer (REST) es un estilo de arquitectura de software que se creó para guiar el diseño y desarrollo de la arquitectura para la *World Wide Web*. REST define un **conjunto de restricciones** sobre cómo debe comportarse la arquitectura de un sistema hipermedia distribuido a escala de Internet, como la web. El estilo arquitectónico REST enfatiza la escalabilidad de las interacciones entre los componentes, las interfaces uniformes, la implementación independiente de los componentes y la creación de una arquitectura en capas para facilitar el almacenamiento en caché de los componentes para reducir la latencia percibida por el usuario, reforzar la seguridad y encapsular los sistemas heredados (Fielding, 2000).

REST se basa en recursos, lo que significa que en un servicio REST trabajaremos con **cosas** para definir el servicio. Los clientes pueden utilizar este recurso para acceder a un identificador de recursos uniformes (URI) único y luego el recurso se devuelve a la persona que llama. REST usa sustantivos (es decir: usuario, país, estado, etc.) para definir qué tipo de recurso debe proporcionar el servicio. La representación de los recursos no está ligada al servicio en sí, lo que significa que la representación del recurso tiene un **formato libre** y se puede representar mediante varios tipos de formatos de datos (es decir, JSON, XML, CVS, etc.).

A continuación se muestra un ejemplo de definición de servicio REST:

```
Resource: alumno
Service: contact information (GET)
Representation:
  - name, address, e-mail
  - JSON or XML format
URI: http://unir.net/missi/alumnos
```

El estilo de arquitectura REST tiene **seis restricciones**, que son reglas de diseño aplicadas a la arquitectura y que establecen la característica distintiva de los servicios REST. Para definir un servicio de arquitectura REST, debemos tener las

siguientes **características arquitectónicas** que hacen de cualquier servicio web una verdadera API RESTful:

- ▶ **Interfaz uniforme:** el servicio debe tener una interfaz definida entre el cliente y el servidor. Simplifica y desacopla la arquitectura, lo que permite que cada parte evolucione de forma independiente.
- ▶ **Cliente-Servidor:** la separación de preocupaciones entre cliente y servidor. A los clientes no les preocupa el almacenamiento de datos porque es un rol de servidor. Del mismo modo, los servidores no se preocupan por la interfaz de usuario porque es un rol de cliente.
- ▶ **Sin estado:** los servidores no almacenan ningún contexto de cliente. Cada solicitud de cliente debe contener toda la información requerida al servidor para comprender lo que se solicita. Por lo tanto, el estado de la sesión se mantiene completamente en el lado del cliente.
- ▶ **Caché:** para mejorar la escalabilidad y el rendimiento, se debe aplicar el almacenamiento en caché en los recursos cuando corresponda y así reducir la interacción cliente-servidor. El almacenamiento en caché se puede implementar en el lado del cliente o del servidor
- ▶ **Sistema en capas:** REST permite utilizar una arquitectura de sistema en capas, lo que significa que el servicio se puede implementar en un servidor y el almacenamiento de datos se puede ubicar en un servidor diferente. El cliente no puede saber si está o no conectado al servidor final.
- ▶ **Código bajo demanda:** los clientes pueden personalizar su aplicación para devolver un código ejecutable. Un cliente puede llamar a una API REST para obtener un código de representación del *widget* de la interfaz de usuario (UI).

7.3. Valor y utilidad de las plataformas para el desarrollo de servicios

El principal valor del uso de una arquitectura basada en servicios es la capacidad de crear **sistemas distribuidos** (ISO, 2017), cuyos componentes están ubicados en diferentes computadoras en red, las cuales se comunican y coordinan sus acciones pasándose mensajes entre sí desde cualquier sistema para lograr un objetivo común.

El desarrollo de sistemas distribuidos basados en servicios proporciona una serie de beneficios como:

- ▶ **Separación de preocupaciones** entre los diferentes servicios: cada servicio dentro de la aplicación ejecuta un grupo reducido de tareas y aprovecha la infraestructura de hardware que garantiza su mejor rendimiento.
- ▶ **Bajo acoplamiento** de los servicios: inspirado en el paradigma de diseño orientado a objetos, que reduce el acoplamiento entre clases para apreciar un entorno en el que las clases se pueden cambiar sin romper la relación existente, SOA fomenta mucho el desarrollo de servicios independientes que interactúan a través de interfaces o contratos.
- ▶ **Desarrollo simultáneo** y autónomo: cada servicio puede ser desarrollado, implementado y mantenido por un equipo independiente con mínima influencia en el resto del sistema.
- ▶ Combinación de **diferentes lenguajes y tecnologías**: cada servicio puede escribirse en el lenguaje que mejor se adapte a su finalidad y se comunica con el resto a través de un protocolo de comunicación estandarizado.

- ▶ **Independencia de la plataforma:** a través de protocolos estándares es posible desarrollar una aplicación formada por múltiples servicios que interoperan a pesar de estar creados con diversas tecnologías y desplegados en diversas plataformas.
- ▶ **Escalabilidad:** podemos disponer de múltiples instancias de los servicios desplegados en diversas infraestructuras, por lo que ante peticiones masivas de servicios, el sistema es capaz de escalar más fácilmente repartiendo la carga entre los entornos operativos.
- ▶ **Confiabilidad y resiliencia:** la posibilidad de disponer de múltiples instancias del mismo servicio replicadas en diversas infraestructuras computacionales, se aumenta la robustez del sistema ante caídas parciales del mismo.
- ▶ **Reutilización** de código: dado que los servicios representan paquetes de código autónomos, permiten la reutilización del código a nivel macro, lo que puede reducir significativamente el tiempo de desarrollo de nuevas funciones.

Las diversas plataformas y herramientas existentes facilitan el desarrollo de sistemas distribuidos basados en servicios, ofreciendo beneficios en base a funcionalidades como:

- ▶ Soporte para el modelado de la interfaz o contrato de un servicio atendiendo a diversos estándares.
- ▶ Autogeneración de parte del código de la implementación de un servicio.
- ▶ Autogeneración del código para consumir un servicio a partir de su interfaz.
- ▶ Autogeneración de la documentación legible asociada a un servicio.
- ▶ Autogeneración de un *front* web sobre el que podemos lanzar ejecuciones de servicios.
- ▶ Facilitación de interacción con servicios al atender a diversos protocolos.

- ▶ Emulación del comportamiento de servicios para disponer de entornos de pruebas o permitir el desarrollo de integraciones cuando los servicios de terceros no están disponibles en tiempo de desarrollo.
- ▶ Facilitación del testeo de servicios distribuidos al producir llamadas a servicios, analizar las respuestas o generar tráfico de alta demanda, entre otros.
- ▶ Simplificación la monitorización de una infraestructura basada en servicios.
- ▶ Facilitación el despliegue de servicios y microservicios.

7.4. Modelado y descripción de servicios

Un paso fundamental a la hora de afrontar el diseño de una arquitectura basada en servicios es la descripción de la interfaz, contrato o API que un servicio ofrece. Esta tarea puede hacerse desde un inicio, siguiendo una filosofía API-First o bien a medida que vamos avanzando en el desarrollo. En todo caso, si queremos que otros sistemas interactúen con nuestros servicios o debemos consumir servicios de terceros, será preciso disponer de un modelo que contenga la descripción de estos. Las principales tecnologías para ello son:

- ▶ API Blueprint (API Blueprint, s. f. a).
- ▶ OpenAPI Specification, OAS (OAS, 2021).
- ▶ RESTful API Modeling Language, RAML (GitHub, 2021a).
- ▶ Web Application Description Language, WADL (Hadley, 2009).
- ▶ Web Services Definition Language, WSDL (Christensen et al., 2001).

API Blueprint

Es un lenguaje de alto nivel para describir API web. La sintaxis es una combinación de la sintaxis Markdown —una sintaxis de formato de texto ligera— con Markdown Syntax for Object Notation (MSON), que es una extensión para describir objetos de datos. API Blueprint es simple y accesible para todos los involucrados en el ciclo de vida de la API. Su sintaxis es **concisa pero expresiva**. Algunas de sus características son:

- ▶ Está diseñado para fomentar el **diálogo y la colaboración** entre las partes interesadas del proyecto, los desarrolladores y los clientes en cualquier punto del ciclo de vida de la API.

- ▶ Es completamente de **código abierto** bajo la licencia MIT.
- ▶ Está diseñado para fomentar mejores diseños de API a través de la **abstracción**. El objetivo de API Blueprint es desacoplar elementos de API para habilitar la modularidad mientras encapsula el comportamiento de la implementación del *back-end*.
- ▶ Se basa en la filosofía de **diseño primero**. Conceptualmente parejo al desarrollo dirigido por las pruebas, API Blueprint representa un contrato para una API. Discutir su API y acordar el contrato antes de que se desarrolle tiende a conducir a mejores diseños de API.

API Blueprint cuenta con un conjunto de herramientas que facilitan el desarrollo de servicios especificados con esta tecnología (API Blueprint, s. f. b), como editores, *parsers*, *testers*, convertidores o analizadores léxicos entre otros.

Seguidamente, se muestra un ejemplo real de una descripción de un servicio utilizando API Blueprint (API Blueprint, s. f. c):

```
FORMAT: 1A
HOST: https://alpha-api.app.net

# Real World API
This API Blueprint demonstrates a real world example documenting a portion
of
[App.net API](http://developers.app.net).

NOTE: This document is a work in progress.

# Group Posts
This section groups App.net post resources.

## Post [/stream/0/posts/{post_id}]
A Post is the other central object utilized by the App.net Stream API. It
has
rich text and annotations which comprise all of the content a users sees in
their feed. Posts are closely tied to the follow graph...

+ Parameters
```

```
+ post_id: `1` (string) - The id of the Post.

+ Model (application/json)

```js
{
 "data": {
 "id": "1", // note this is a string
 "user": {
 ...
 },
 "created_at": "2012-07-16T17:25:47Z",
 "text": "@berg FIRST post on this new site #newsocialnetwork",
 "html": "<span itemprop=\"mention\" data-mention-name=\"berg\"
data-mention-id=\"2\">@berg FIRST post on this new site <span
itemprop=\"hashtag\"
data-hashtag-
name=\"newsocialnetwork\">#newsocialnetwork.",
 "source": {
 "client_id": "udxGzAVBdXwGtKHmvswR5MbMEeVnq6n4",
 "name": "Clientastic| for iOS",
 "link": "http://app.net"
 },
 "machine_only": false,
 "reply_to": null,
 "thread_id": "1",
 "num_replies": 3,
 "num_reposts": 0,
 "num_stars": 0,
 "entities": {
 "mentions": [{
 "name": "berg",
 "id": "2",
 "pos": 0,
 "len": 5
 }],
 "hashtags": [{
 "name": "newsocialnetwork",
 "pos": 34,
 "len": 17
 }]
 }
 }
}
```

```

 },
 "links": [{
 "text": "this new site",
 "url": "https://join.app.net"
 "pos": 20,
 "len": 13
 }]
 },
 "you_reposted": false,
 "you_starred": false
},
"meta": {
 "code": 200,
}
}
...

Retrieve a Post [GET]
Returns a specific Post.

Retrieve a Post [GET]
Returns a specific Post.

+ Response 200

[Post][]

Delete a Post [DELETE]
Delete a Post. The current user must be the same user who created the Post.
It
returns the deleted Post on success.

+ Response 204

Posts Collection [/stream/0/posts]
A Collection of posts.

+ Model (application/json)

```js
{
  "data": [
    {

```

```
### Retrieve a Post [GET]
```

Returns a specific Post.

```
+ Response 200
```

```
[Post][[]]
```

```
### Delete a Post [DELETE]
```

Delete a Post. The current user must be the same user who created the Post.

It

returns the deleted Post on success.

```
+ Response 204
```

```
## Posts Collection [/stream/0/posts]
```

A Collection of posts.

```
+ Model (application/json)
```

```
```js
{
 "data": [
 {
 "id": "1", // note this is a string
 ...
 },
 {
 "id": "2",
 ...
 },
 {
 "id": "3",
 ...
 },
],
 "meta": {
 "code": 200,
 }
}
```
```

A User's stars are visible to others, but they are not automatically added to your followers' streams.

+ Parameters

+ post_id: `1` (string) - The id of the Post.

Star a Post [POST]

Save a given Post to the current User's stars. This is just a "save" action, not a sharing action.

Note: A repost cannot be starred. Please star the parent Post.

+ Response 200

[Post][]

Unstar a Post [DELETE]

Remove a Star from a Post.

+ Response 200

[Post][]

```
schema:
  type: array
  items:
    type: string
```

OpenAPI (OAS)

La especificación **OpenAPI** (OAS) define una descripción de interfaz estándar e independiente del lenguaje de programación para las API HTTP, que permite, tanto a humanos como a computadoras, descubrir y comprender **las capacidades de un servicio** sin necesidad de acceder al código fuente, documentación adicional o inspección del tráfico de red. Cuando se define correctamente a través de OpenAPI, un consumidor puede comprender e interactuar con el servicio remoto con una cantidad mínima de lógica de implementación. Similar a lo que han hecho las descripciones de interfaz para la programación de nivel inferior, la especificación OpenAPI elimina las conjeturas al llamar a un servicio.

La especificación OpenAPI también proporciona:

- ▶ Un **formato no propietario**.
- ▶ El ecosistema de **herramientas** más desarrollado: OpenAPI ofrece una gran cantidad de herramientas para trabajar con él (OpenAPI Tools, s. f.).
- ▶ Un formato **legible tanto por máquinas como por humanos**: aunque escribir documentos OpenAPI a mano no es la forma más conveniente de hacerlo, son archivos de texto sin formato que se pueden navegar fácilmente en caso de que sea necesario depurar algo.

Seguidamente se muestra un ejemplo real de una descripción de un servicio utilizando OpenAPI (Swagger, s. f. a.):

```
openapi: 3.0.0
info:
  title: Sample API
  description: Optional multiline or single-line description in
[CommonMark](http://commonmark.org/help/) or HTML.
  version: 0.1.9
servers:
  - url: http://api.example.com/v1
    description: Optional server description, e.g. Main (production) server
  - url: http://staging-api.example.com
    description: Optional server description, e.g. Internal staging server
for testing
paths:
  /users:
    get:
      summary: Returns a list of users.
      description: Optional extended description in CommonMark or HTML.
      responses:
        '200': # status code
          description: A JSON array of user names
          content:
            application/json:
```

Creada en 2010 y conocida hasta del 2016 como Swagger, existen herramientas que a partir de esta especificación **facilitan** la generación de documentación, la generación de código para generar servidores y clientes en varios lenguajes de

programación, así como la automatización de las pruebas entre otros casos de uso. Describir un servicio formalmente en un formato legible por máquina permite que las herramientas automatizadas la procesen, lo que facilita:

- ▶ **La validación de la interfaz:** verificar que el archivo de descripción sea sintácticamente correcto y se adhiera a una versión específica de la especificación.
- ▶ **La validación de datos:** comprobar que los datos que fluyen a través de la API (en ambas direcciones) son correctos durante el desarrollo y una vez implementados.
- ▶ **Generación de documentación:** crear documentación tradicional legible por humanos basada en la descripción legible por máquina, que siempre se mantiene actualizada.
- ▶ **Generación de código:** crear código de servidor y de cliente en cualquier lenguaje de programación, lo que libera a los desarrolladores de tener que realizar la validación de datos o escribir código SDK kit de desarrollo de software), por ejemplo.
- ▶ **Editores gráficos:** permiten la creación sencilla de archivos de descripción utilizando una interfaz de usuario en lugar de escribirlos a mano.
- ▶ **Servidores simulados:** crear servidores falsos que brinden respuestas de ejemplo que usted y sus clientes pueden comenzar a probar antes de escribir una sola línea de código.
- ▶ **Análisis de seguridad:** descubrir posibles vulnerabilidades en la etapa de diseño de la API en lugar de mucho, mucho más tarde.

RESTful API Modeling Language (RAML)

Significa lenguaje de modelado de API RESTful. Es una forma de describir las API prácticamente RESTful de una manera que es altamente legible, tanto para humanos como para computadoras. Decimos **prácticamente RESTful** porque, hoy en día, en el mundo real, muy pocas API obedecen realmente a todas las restricciones de REST. RAML no es estricto: por ahora, se enfoca en describir claramente los recursos, métodos, parámetros, respuestas, tipos de medios y otras construcciones HTTP que forman la base de las API modernas que obedecen muchas, aunque quizás no todas, las restricciones RESTful.

Con RAML podemos **diseñar un API visualmente**, probarla y obtener comentarios de los usuarios sin tener que escribir una sola línea de código. Algunas de sus características y beneficios son:

- ▶ **Diseño visual:** permite describir un API en un formato legible por humanos, en otras palabras, texto sin formato. Con herramientas como API Workbench o API Designer, podemos obtener una representación visual de la API a medida que lo diseñamos.
- ▶ **Reutilización de código:** permite incorporar dos mejores prácticas para desarrolladores, es decir, patrones de diseño y reutilización de código. En lugar de tener que confiar en la verificación humana para garantizar que la API sea consistente, RAML permite aprovechar las bibliotecas, las superposiciones, las características y los tipos de recursos para garantizar que al API permanece consistente en todo momento.
- ▶ **Prototipado** de la API REST: podemos crear fácilmente un prototipo de la API y enviarla a posibles usuarios.
- ▶ **Pruebas tempranas:** facilita que estos desarrolladores prueben la API, con integración incorporada en herramientas populares como Postman. Incluso podemos guiar a los usuarios a través de la API simulada utilizando API Notebook.

Seguidamente, se muestra un ejemplo real de una descripción de un servicio utilizando RAML (RAML, s. f.):

```
##RAML 1.0
title: Mobile Order API
baseUri: http://localhost:8081/api
version: 1.0

uses:
  assets: assets.lib.raml

annotationTypes:
  monitoringInterval:
    type: integer

/orders:
  displayName: Orders
  get:
    is: [ assets.paging ]
    (monitoringInterval): 30
    description: Lists all orders of a specific user

    queryParameters:
      userId:
        type: string
        description: use to query all orders of a user
  post:
    /{orderId}:
      get:
        responses:
          200:
            body:
              application/json:
                type: assets.Order
              application/xml:
                type: !include schemas/order.xsd
```

Web Application Description Language(WADL)

Es el lenguaje de descripción de aplicaciones web y una descripción XML legible por

máquina de los servicios web basados en HTTP. WADL **modela los recursos** proporcionados por un servicio y las relaciones entre ellos y pretende simplificar la reutilización de los servicios web que se basan en la arquitectura HTTP existente de la web. Es una especificación independiente de la plataforma y el idioma y tiene como objetivo promover la reutilización de aplicaciones más allá del uso básico en un navegador web.

Seguidamente, se muestra un ejemplo real de una descripción de un servicio utilizando WADL (Oracle, 2010):

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<application xmlns="http://research.sun.com/wadl/2006/10">
  <doc xmlns:jersey="http://jersey.dev.java.net/"
    jersey:generatedBy="Jersey: 1.0-ea-SNAPSHOT 10/02/2008 12:17
PM"/>
  <resources base="http://localhost:9998/storage/">
    <resource path="/containers">
      <method name="GET" id="getContainers">
        <response>
          <representation mediaType="application/xml"/>
        </response>
      </method>
      <resource path="{container}">
        <param xmlns:xs="http://www.w3.org/2001/XMLSchema"
          type="xs:string"
style="template" name="container"/>
        <method name="PUT" id="putContainer">
          <response>
            <representation mediaType="application/xml"/>
          </response>
        </method>
        <method name="DELETE" id="deleteContainer"/>
        <method name="GET" id="getContainer">
          <request>
            <param xmlns:xs="http://www.w3.org/2001/XMLSchema"
type="xs:string" style="query" name="search"/>
          </request>
          <response>
            <representation mediaType="application/xml"/>
          </response>
        </method>
        <resource path="{item: .+}">
          <param xmlns:xs="http://www.w3.org/2001/XMLSchema"
type="xs:string" style="template" name="item"/>
```

```
<method name="PUT" id="putItem">
  <request>
    <representation mediaType="*/*/">
  </request>
  <response>
    <representation mediaType="*/*/">
  </response>
</method>
<method name="DELETE" id="deleteItem"/>
<method name="GET" id="getItem">
  <response>
    <representation mediaType="*/*/">
  </response>
</method>
</resource>
</resource>
</resource>
</resources>
</application>
```

Web Services Definition Language (WSDL)

Es un lenguaje de **descripción de interfaces** basado en XML que se utiliza para describir la funcionalidad que ofrece un servicio web. Proporciona una descripción legible por máquina de cómo se puede llamar al servicio, qué parámetros espera y qué estructuras de datos devuelve. La última versión de WSDL, que se convirtió en recomendación del W3C en 2007, es WSDL 2.0.

Los elementos que se definen dentro de un WSDL son:

- ▶ **Service:** conjunto de funciones del sistema que se exponen como interfaz.
- ▶ **Endpoint:** dirección o el punto de conexión a un servicio web. Por lo general, se representa mediante una cadena de URL HTTP simple.
- ▶ **Binding:** especifica la interfaz y define el estilo de enlace SOAP (RPC/documento) y el transporte (protocolo SOAP). La sección de enlace también define las operaciones.

- ▶ **Interface:** define un servicio web, las operaciones que se pueden realizar y los mensajes que se utilizan para realizar la operación.
- ▶ **Operation:** define las acciones SOAP y la forma en que se codifica el mensaje, por ejemplo, **literal**. Una operación es como una llamada de método o función en un lenguaje de programación tradicional.
- ▶ **Types:** describe los datos. El lenguaje XML Schema (también conocido como XSD) se usa (en línea o referenciado) para este propósito.

WADL y WSDL sirven ambos para describir servicios web. WSDL especifica aquellos que interaccionan usando SOAP. WADL se aplica para los que están basados en REST.

Seguidamente se muestra un ejemplo real de una descripción de un servicio utilizando WSDL (W3, 2007):

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:description
  targetNamespace="http://example.org/TicketAgent.wsdl20"
  xmlns:xsTicketAgent="http://example.org/TicketAgent.xsd"
  xmlns:wsdl="http://www.w3.org/ns/wsdl"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.w3.org/ns/wsdl
http://www.w3.org/2007/06/wsdl/wsdl20.xsd">

  <wsdl:types>
    <xs:import schemaLocation="TicketAgent.xsd"
      namespace="http://example.org/TicketAgent.xsd" />
  </wsdl:types>

  <wsdl:interface name="TicketAgent">
    <wsdl:operation name="listFlights">
```

```
        pattern="http://www.w3.org/ns/wsdl/in-out">
        <wsdl:input element="xsTicketAgent:listFlightsRequest"/>
        <wsdl:output element="xsTicketAgent:listFlightsResponse"/>
    </wsdl:operation>

    <wsdl:operation name="reserveFlight"
        pattern="http://www.w3.org/ns/wsdl/in-out">
        <wsdl:input element="xsTicketAgent:reserveFlightRequest"/>
        <wsdl:output element="xsTicketAgent:reserveFlightResponse"/>
    </wsdl:operation>
</wsdl:interface>
</wsdl:description>
```

Adicionalmente, existe un conjunto de especificaciones en el ámbito de los servicios web que han ido evolucionando y que cubren aspectos de seguridad, interoperabilidad, mensajería, orquestación o soporte para procesos de negocio. Un resumen preparado por innoQ se expone a continuación en la Tabla 1.

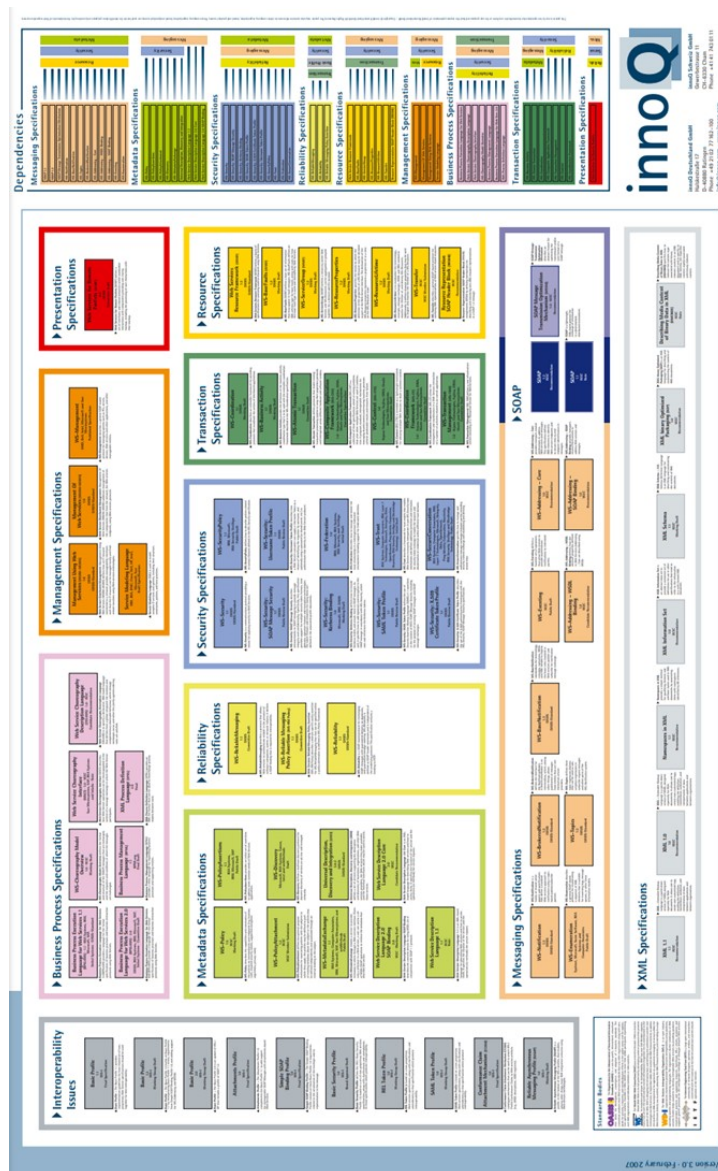


Tabla 1. Web Services Standards. Fuente: InnoQ, 2007.

Accede al documento a través del aula virtual o desde el siguiente enlace : <https://www.innoq.com/soa/ws-standards/poster/innoQ%20WS-Standards%20Poster%202007-02.pdf>

7.5. Documentación e interacción con servicios

Uno de los objetivos principales de realizar una descripción de servicios en base a algunos de los estándares presentados anteriormente, es la capacidad de que otras herramientas puedan procesar esa información de manera automática para facilitar la:

- ▶ La documentación.
- ▶ La emulación.
- ▶ La interacción con los servicios.

Las principales plataformas para documentación, emulación o interacción con servicios son:

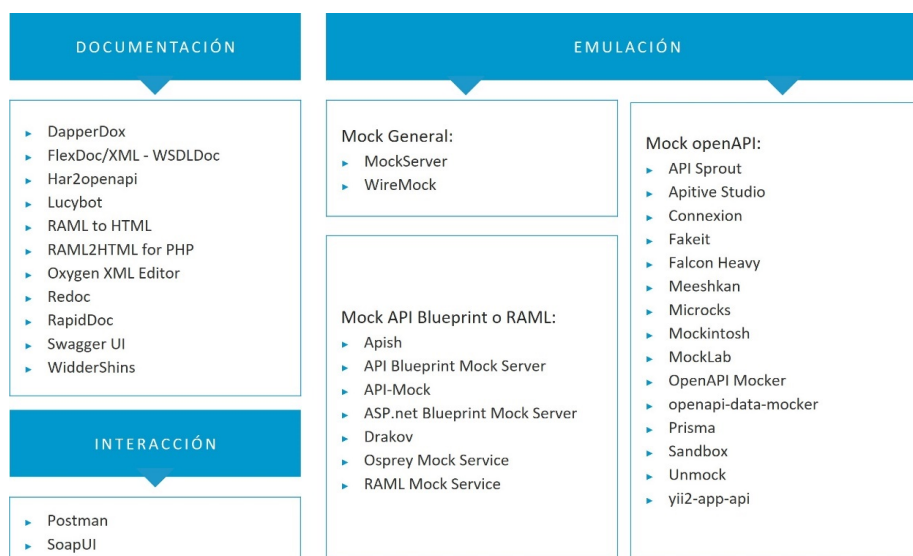


Tabla 2. Plataformas para documentación, emulación o interacción con servicios. Fuente: elaboración propia.

Plataformas para la documentación

En primer lugar, a nivel de documentación, algunas de las plataformas más representativas son:

DapperDox

Es una solución de código abierto para generar documentación a partir de una especificación de API. La herramienta es muy **flexible** y permite diseñar las páginas de documentación de la forma que deseemos. Proporciona una interfaz de usuario clara y sencilla con dos columnas: en la primera columna, tiene su explorador y en la segunda columna, tiene los detalles del método API. Una desventaja es que no se actualiza con frecuencia y aún no es compatible con OpenAPI v3.

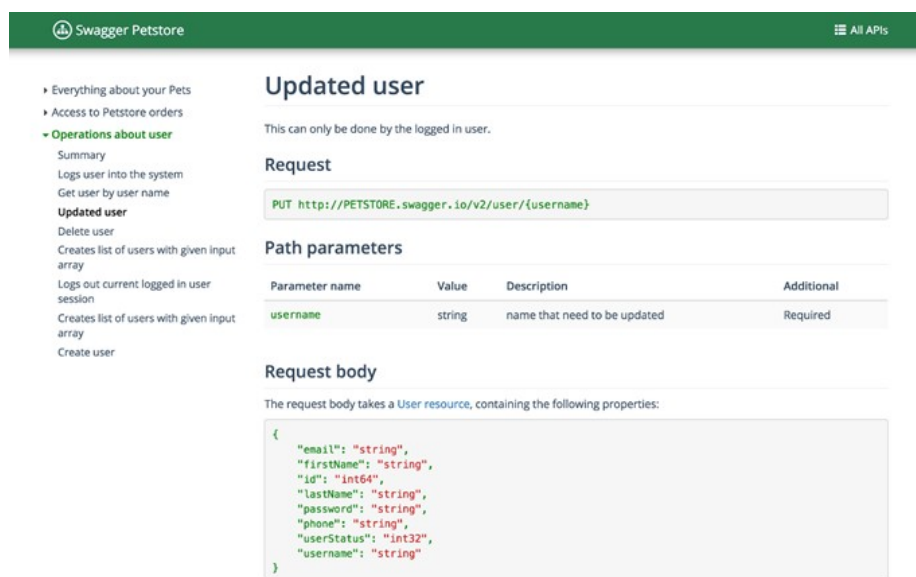


Figura 2. Documentación con DapperDox. Fuente: DapperDox, s. f.

FlexDoc/XML - WSDLDoc

Es un poderoso generador de documentación WSDL/XSD que crea una documentación **HTML o RTF** (con diagramas XSD) para cualquier número de archivos WSDL y XSD.

Har2openapi

Genera automáticamente documentación de una interfaz a partir del uso de solicitudes de red capturadas en uno o más archivos HAR.

Lucybot

Permite generar documentación a partir de los **principales estándares abiertos** como Markdown, OpenAPI, RAML o WADL para crear un sitio web totalmente personalizable. Genera automáticamente documentación para cada operación, parámetro y respuesta. También permite usar Markdown o HTML para crear páginas de documentación personalizadas. También soporta la ejecución manual de los servicios desde el entorno web generado.

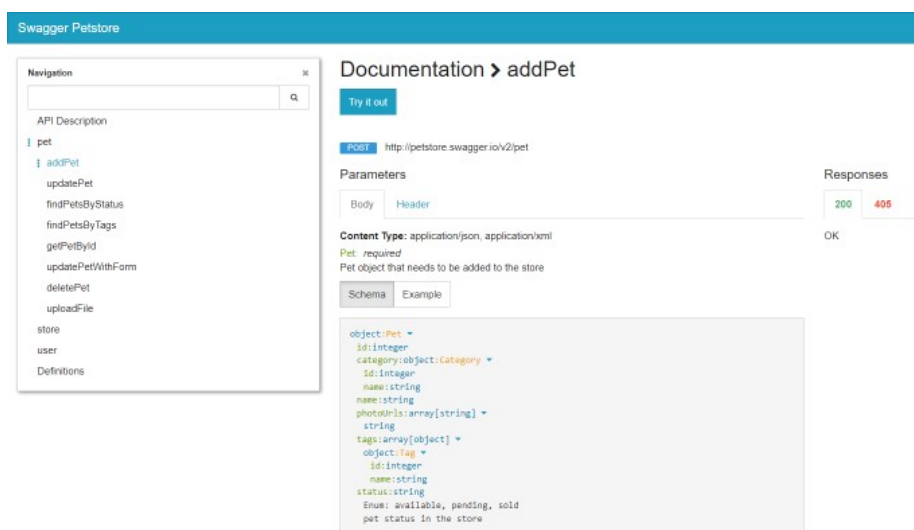


Figura 3. Documentación con Lucybot. Fuente: Lucybot, s. f.

RAML to HTML:

Es una herramienta de documentación que genera una **única consola** de página HTML basada en una definición RAML. Está escrito en Node.js y se puede ejecutar como una línea de comando.

RAML2HTML for PHP

Es una aplicación simple que utiliza **múltiples plantillas** para permitir crear y personalizar documentos API usando RAML.

Oxygen XML Editor

Genera **documentación detallada** para los componentes de un documento WSDL en formato HTML. Podemos seleccionar los componentes WSDL para incluir en la salida y el nivel de detalles para presentar para cada uno de ellos.

ReDoc

Es una herramienta de código abierto para generar documentación a partir de definiciones OpenAPI. Muestra la información en tres paneles:

- ▶ El panel izquierdo contiene una barra de búsqueda y un menú de navegación.
- ▶ El panel central contiene la documentación.
- ▶ El panel derecho contiene ejemplos de solicitudes y respuestas (GitHub, 2022).

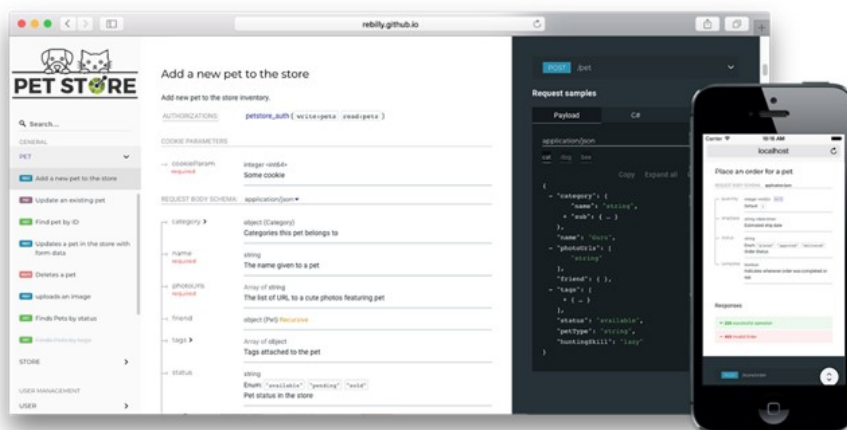


Figura 4. Documentación con Redoc. Fuente: GitHub, 2022.

RapidDoc

Tiene la mejor interfaz de usuario para la documentación de API. Permite crear documentación desde OpenAPI, lo que genera un contenido **interactivo y muy visual**. Esta herramienta es muy flexible y permite personalizar el tema, la fuente y los colores. También podemos incrustar código HTML externo en la documentación o incrustar los documentos en otro código HTML. Viene con una consola lista para usar, que podemos usar fácilmente para hacer llamadas API y probar su API. Es posible cambiar sus atributos usando JavaScript. Tiene una vista de una sola columna con barras contraíbles y estas contienen la consola con las cargas útiles de JSON.

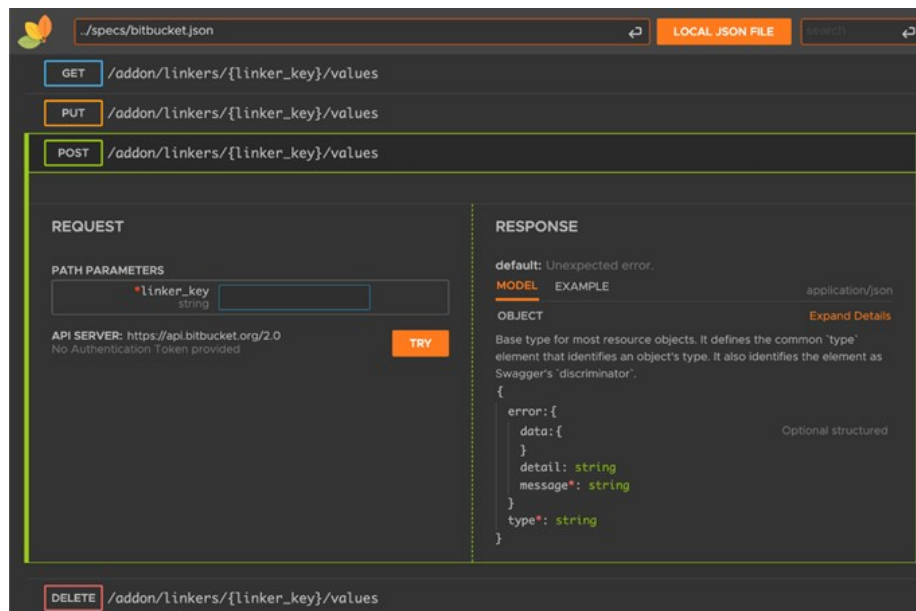


Figura 5. Documentación con Rapidoc. Fuente: Rapidoc, s. f.

Swagger UI

La interfaz de usuario de Swagger permite que cualquier persona, ya sea su equipo de desarrollo o sus consumidores finales, **visualice e interactúe** con los recursos de la API sin tener implementada ninguna lógica de implementación. Se genera automáticamente a partir de su especificación OpenAPI, con la documentación visual

que facilita la implementación de *back-end* y el consumo del lado del cliente (Swagger, s. f. b.). Soporta no solo una documentación HTML, sino que es, a su vez, un entorno interactivo en el que podemos ejecutar de forma manual los servicios al introducir los valores de entrada y comprobando los resultados.

Swagger es una de las herramientas más utilizadas por los desarrolladores, ya que a partir del código fuente la plataforma se encarga de generar la especificación OpenAPI, la documentación y permite su ejecución manual

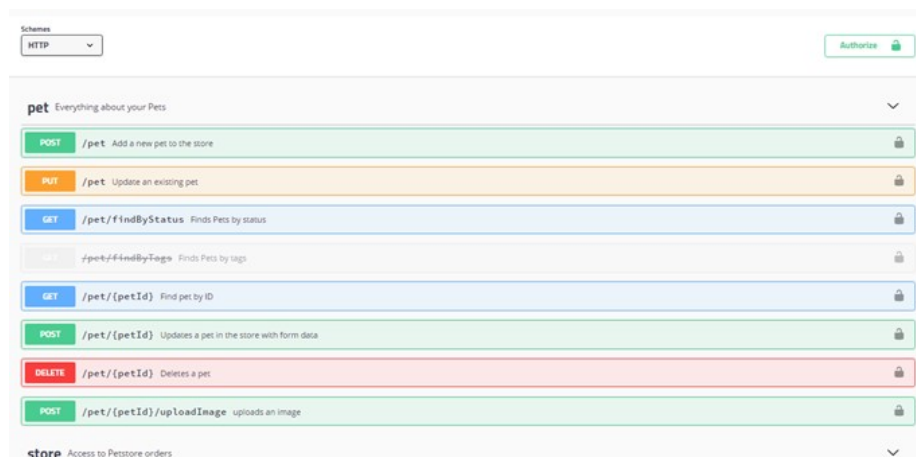
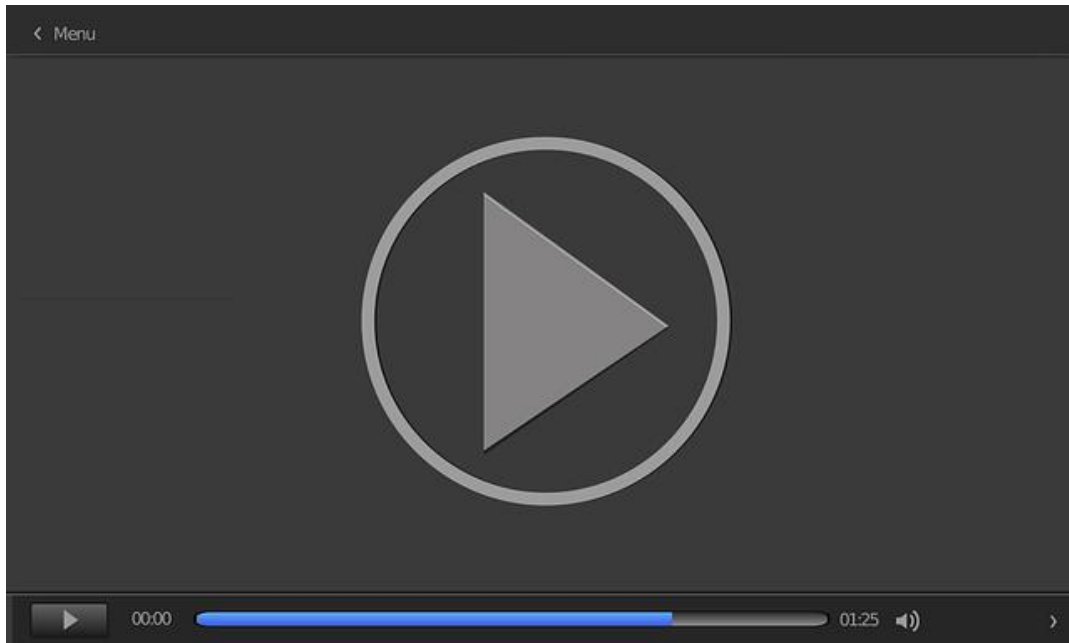


Figura 6. Documentación con Swagger UI. Fuente: Swagger, s. f. b.

En este vídeo, *OpenAPI* y *Swagger*, se hará una introducción a la especificación de servicios con OpenAPI y, en particular, al uso de Swagger como marco de documentación e interacción de servicio REST.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=b76d6261-b518-42ba-a1c1-aec300cd99f9>

WidderShins

Permite crear documentación de API a partir de definiciones de OpenAPI 3.0. La interfaz de usuario de esta herramienta es muy similar a ReDoc, ya que tiene una página de tres columnas: en la primera columna hay un explorador, en la segunda columna tiene la descripción y en la tercera columna tiene su consola para hacer llamadas a la API. Usa plantillas para generar los resultados y son personalizables.

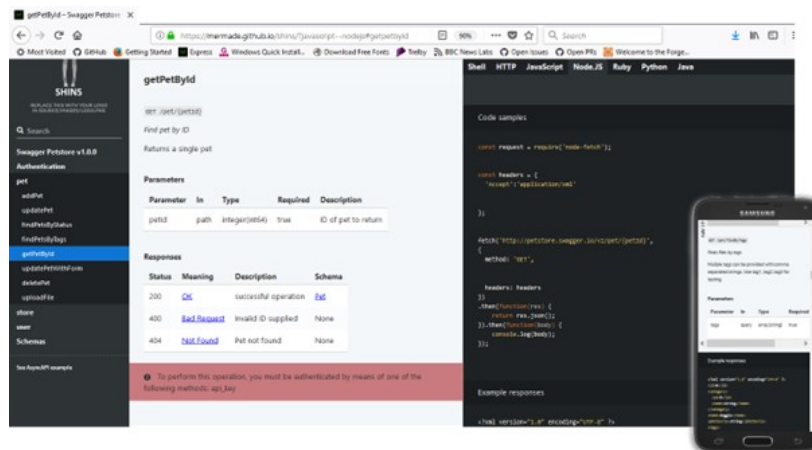


Figura 7. Documentación con WidderShins. Fuente: GitHub, 2021b.

En segundo lugar, una de las utilidades más comunes es autogenerar una implementación emulada de un servicio (en inglés **mock a service**). Esto es necesario en una variedad de situaciones, como:

- **Agilizar el desarrollo en paralelo:** En un desarrollo *API-first design-first*, debemos diseñar y especificar la interfaz o contrato de nuestros servicios. Antes de implementarlos, podemos generar una versión emulada de los mismos que sea capaz de responder a una serie de peticiones predefinidas. De esa forma, el equipo encargado de desarrollar piezas de código que consumirán estos servicios (típicamente el frontend) puede evolucionar en paralelo sin necesidad de esperar a que la implementación esté disponible.
- **Facilitar la integración con sistemas de terceros:** en el caso de que el sistema deba interaccionar con servicios de terceros, es posible que no tengamos un entorno de desarrollo o pruebas y que no podamos hacer uso del servicio real en producción. En estos escenarios es preciso emular el comportamiento de estos servicios para disponer de un entorno lo más parecido al real y que podamos verificar el correcto funcionamiento antes de su puesta en producción.

- ▶ **Disponer de escenarios para pruebas automáticas:** en el caso de querer hacer pruebas, generalmente automáticas, sobre un conjunto de escenarios, en ocasiones debemos disponer de un contexto en el que uno de los elementos bajo testeo (*System Under Test, SUT*) requiera de un comportamiento anómalo o erróneo de un subsistema o servicio. En esos casos, la solución pasa por disponer de un entorno en el que alguno de los servicios en lugar de tener una implementación real está emulado para devolver errores o resultados no esperados, con el objetivo de poder comprobar que incluso en esas situaciones no deseadas nuestro sistema es robusto y es capaz de reconducir la situación.

Plataformas para la emulación

A nivel de emulación, algunas de las plataformas más representativas son:

MockServer

Es una herramienta para simular/aplicar API HTTP externas. Para cualquier sistema con el que se integre a través de HTTP o HTTPS, MockServer se puede utilizar como:

- ▶ Un simulacro configurado para devolver respuestas específicas para diferentes solicitudes.
- ▶ Una grabación de proxy y, opcionalmente, modificar solicitudes y respuestas.
- ▶ Tanto un proxy para algunas solicitudes como un simulacro para otras solicitudes al mismo tiempo.

Cuando MockServer recibe una solicitud, compara la solicitud con las expectativas activas que se han configurado. Luego, si no se encuentran coincidencias, transmite la solicitud si corresponde, de lo contrario, se devuelve un 404. De forma general, la herramienta puede:

- ▶ Generar y devolver respuestas fijas.
- ▶ Reenviar una solicitud a otro servidor.
- ▶ Ejecutar devoluciones de llamada.
- ▶ Devolver respuestas erróneas.
- ▶ Verificar una solicitud.
- ▶ Registrar logs para facilitar la monitorización y depuración.

WireMock

Es una herramienta para simular API basadas en HTTP que se ejecuta en sus pruebas unitarias, en su escritorio o en su entorno de prueba. En esencia, es un servidor web que se puede preparar para brindar respuestas a solicitudes particulares (*stubbing*) y que captura las solicitudes entrantes para que puedan verificarse más tarde. También tiene una variedad de otras características útiles que incluyen grabación/reproducción de interacciones con otras API, inyección de fallas y retrasos, simulación de comportamiento con estado. Cualquier aplicación JVM puede utilizarlo como una biblioteca o ejecutarlo como un proceso independiente en el mismo *host* que el sistema bajo prueba o en un servidor remoto. Se puede acceder a todas las funciones de WireMock a través de su interfaz REST (JSON) y su API Java.

Algunas plataformas de emulación para servicios descritos con OpenAPI son:

- ▶ **API Sprout:** un servidor simulado ligero, ultrarrápido y multiplataforma con cliente de validación sobre Docker.
- ▶ **Apitive Studio:** una plataforma para gerentes de productos digitales y consultores de API para diseñar API REST con simulacros y documentación incorporados.
- ▶ **Connexion:** OpenAPI First framework para Python sobre Flask con validación automática de punto final y compatibilidad con OAuth2 Python.
- ▶ **Fakeit:** es posible crear un servidor simulado a partir de la especificación OpenAPI con generación de respuestas aleatorias y validación de solicitudes.
- ▶ **Falcon Heavy:** el marco para crear *back-ends* de aplicaciones y microservicios a través del flujo de trabajo de diseño de API primero.
- ▶ **Meeshkan:** simula las API de HTTP a través de una combinación de definiciones de API, tráfico registrado y código. Se utiliza para *sandboxes*, así como para pruebas exploratorias y automatizadas.
- ▶ **Microcks:** herramienta nativa de Kubernetes para simulación y pruebas de API. Se pueden convertir los ejemplos de contratos OpenAPI en simulacros.
- ▶ **Mockintosh:** simulacros para entornos nativos de la nube: convierte archivos de OpenAPI en emulaciones y los usa para desarrollar en entornos aislados y probar casos extremos, llamadas asincrónicas a colas como Kafka o RabbitMQ o simulación de pruebas de rendimiento y caos.
- ▶ **MockLab:** plataforma SaaS para cargar su especificación para crear un servidor simulado SaaS
- ▶ **OpenAPI Mocker:** servidor simulado OpenAPI basado en Node.js independiente, compatible con Docker con validación de solicitudes y recarga automática.

- ▶ **openapi-data-mocker:** pequeña biblioteca para generar tipos de datos básicos de OpenAPI. Es como un paquete Faker extendido. Puede ser útil para escribir pruebas unitarias personalizadas en PHP.
- ▶ **Prisma:** permite convertir cualquier archivo OpenAPI en un servidor API con simulación, transformaciones y validaciones.
- ▶ **Sandbox:** herramienta SaaS, autohospedada o de consola para convertir las descripciones de OpenAPI en un servidor simulado, donde podemos modificar el comportamiento y simular el tiempo de inactividad gracias a un editor de código integrado. SaaS/Java
- ▶ **Unmock:** biblioteca de prueba de integración de API que intercepta solicitudes salientes y devuelve datos simulados en función de las descripciones de OpenAPI sobre Node.js.
- ▶ **yii2-app-api:** genera código API del lado del servidor con enrutamiento, modelos, validación de datos y esquema de base de datos a partir de una descripción de OpenAPI basado en el marco yii.

Plataformas de emulación con API Blueprint o RAML

Algunas plataformas de emulación para servicios descritos con API Blueprint o RAML son:

- ▶ **Apish:** facilita la creación de servicios y pruebas con JavaScript desde API Blueprint.
- ▶ **API Blueprint Mock Server:** facilita la ejecución de un servidor simulado simple y rápido en Java desde un API Blueprint.
- ▶ **API-Mock:** permite generar un servidor simulado simple y rápido a partir de un API Blueprint.
- ▶ **ASP.NET Blueprint Mock Server:** permite crear un servidor simulado con tecnologías ASP.NET desde un API Blueprint.

- ▶ **Drakov:** servidor simulado que implementa la especificación API Blueprint, con algunos extras incorporados. Se puede utilizar como servidor independiente y como módulo Node.js.
- ▶ **Osprey Mock Service:** genera un servicio simulado de API a partir de una definición RAML utilizando Osprey de MuleSoft, un generador de *middleware* API a partir de una definición de RAML, que se puede usar local o globalmente para validar solicitudes y respuestas de API.
- ▶ **RAML Mock Service:** otra implementación para generar un servicio simulado de API a partir de una definición RAML usando Osprey.

En tercer lugar, tenemos herramientas que facilitan la interacción con los servicios al generar llamadas con REST o SOAP y comprobando los resultados. A diferencia de las anteriores, en las que el objetivo era documentar y permitían una cierta interacción manual, estas plataformas permiten automatizar no solo una llamada, sino un **flujo de llamadas**, lo que sistematiza la inserción de datos en el *header* y es posible añadir lógica para en función de la respuesta realizar los siguientes pasos, por lo que son válidas para automatizar flujos de testeo con un mayor nivel de complejidad.

Plataformas para la invocación

Algunas de las principales herramientas de invocación de servicios son:

Postman

Es una plataforma API para construir y usar API. Puede ejecutarse en múltiples sistemas operativos desde Windows y macOS hasta Linux. Les ofrece a los desarrolladores más espacio para desarrollar API. Para los probadores funcionales, permite crear pruebas para llamadas a la API sin gastar mucho esfuerzo en codificar *scripts*. Sus principales beneficios son:

- ▶ Facilidad de uso.
- ▶ Accesibilidad.
- ▶ Funcionalidades avanzadas, como ser admitir todos los métodos HTTP, guardar el progreso, convertir de API a código, etc.
- ▶ Soporta el análisis de las respuestas para condicionar los siguientes pasos del flujo.

Sus principales desventajas son:

- ▶ Aunque es ideal para pruebas de API RESTful, no está bien diseñado para API SOAP y otras API.
- ▶ Los usuarios no pueden reutilizar sus guiones prescritos ni agregar más solicitudes, por lo que los equipos de QA (análisis de calidad) tienen que crear nuevos scripts de prueba una y otra vez para cada proyecto.
- ▶ La herramienta en sí no admite muchas capacidades de integración por lo que se convierte en un obstáculo para conectar Postman con los sistemas existentes y colaborar dentro del equipo.

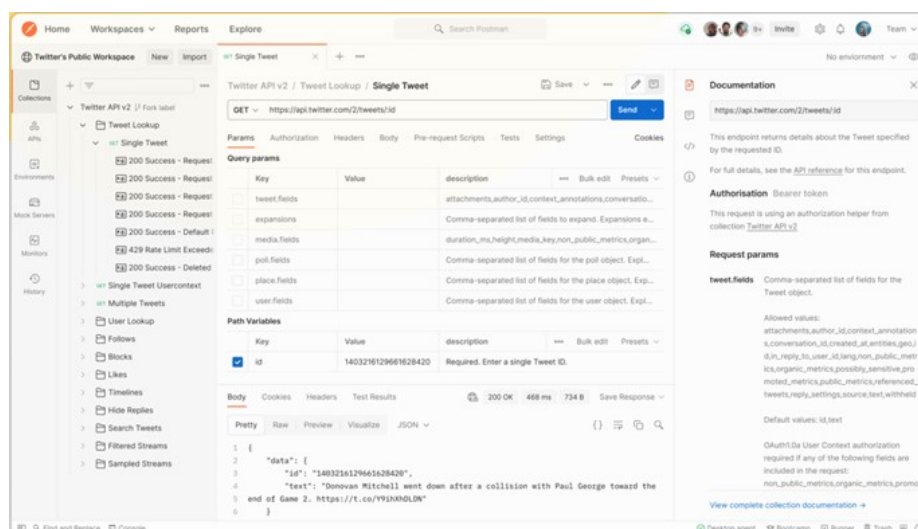


Figura 8. Pantalla de uso de PostMan. Fuente: Postman, s. f.

SoapUI

Es una herramienta para probar servicios web, tanto usando SOAP como HTTP o los servicios web RESTful. Es una solución de código abierto y completamente gratuita, cuenta con una versión comercial (ReadyAPI) que aporta funcionalidad adicional para empresas con servicios web de misión crítica. Esta herramienta se considera el **estándar de facto** para las pruebas de servicio de API por el número de descargas y uso en la comunidad, por lo que hay mucho conocimiento sobre la herramienta, lea los blogs en la red para obtener más información sobre el uso de SoapUI en la vida real. Con la herramienta podemos realizar pruebas funcionales, de rendimiento, de interoperabilidad y de regresión. Podemos emular servicios web, grabar pruebas y usarlas más tarde, crear apéndices de código desde el *WSDL* e incluso crear especificaciones REST (*WADL*) a partir de comunicaciones grabadas.

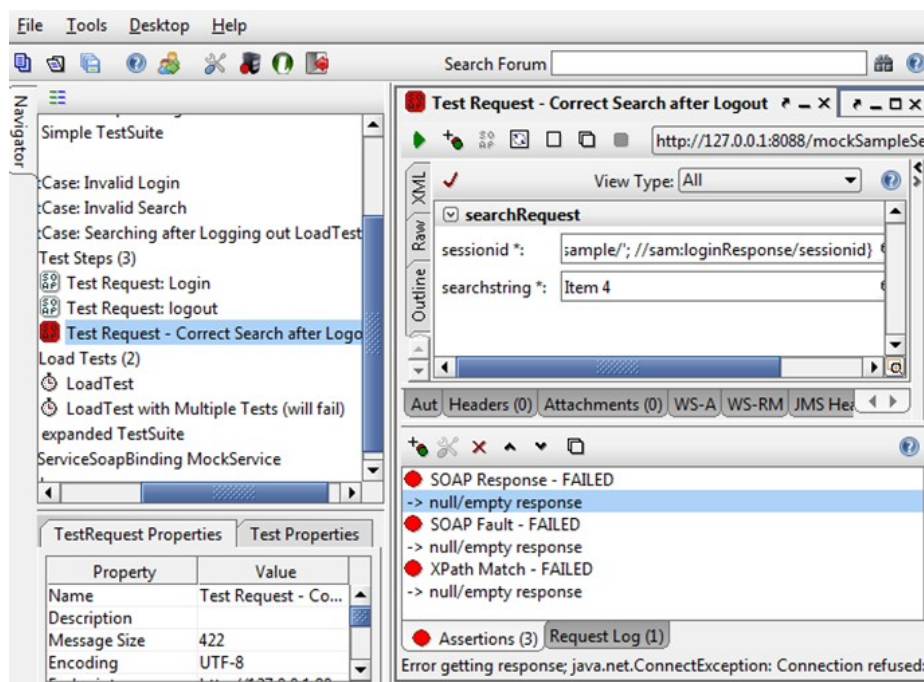


Figura 9. Pantalla de uso de SoapUI. Fuente: SoapUI, s. f.

7.6. Plataformas aceleradoras del desarrollo de servicios

A la hora de desarrollar sistemas distribuidos basados en servicios, existen una serie de plataformas que nos facilitan y aceleran el desarrollo. Además de las que hemos visto que ayudan a la definición, emulación o interacción, o las específicas de testeo, en esta sección expondremos algunas de las principales a la hora de **simplificar la construcción**, tanto de implementaciones de servicios como de consumidores de servicios.

Las principales plataformas que aceleran la creación o consumo de servicios son:



Tabla 3. Plataformas aceleradoras del desarrollo de servicios. Fuente: elaboración propia.

Plataformas aceleradoras para la implementación de servicios

A nivel de implementación de servicios, algunas de las plataformas aceleradoras más representativas son:

Apache CXF

Su funcionalidad `wsdl2java` toma un documento WSDL y genera un código Java completamente anotado a partir del cual implementar un servicio.

Eclipse Jersey

Es un marco REST que proporciona una implementación JAX-RS (JSR-370, la API de Java para servicios web RESTful). Permite realizar un **seguimiento** de la API de JAX-RS y proporciona versiones periódicas de implementaciones de referencia de calidad de producción que se envían con GlassFish. Facilita la creación de API para extender Jersey así como el desarrollo de servicios web RESTful utilizando Java.

OpenAPI Generator

A partir de una especificación OpenAPI, es posible generar un esqueleto con la implementación en **múltiples lenguajes** de programación como Ada, ASP.NET, C++, F#.NET, Go, Haskell, Java, PHP, Python, Ruby y Scala.

RAML Tools para .NET

Es una **extensión de Visual Studio** que permite a los desarrolladores generar una nueva implementación de API web ASP.NET desde cero utilizando un enfoque de contrato primero a partir de una RAML u OAS creada previamente.

Spring WebFlux

Proporciona soporte de programación **reactivo, asíncrono y sin bloqueo** para aplicaciones web en un formato de controlador anotado similar a SpringMVC. Este enfoque es similar a cómo Node.js usa un modelo asíncrono sin bloqueo que ayuda a que sea más escalable. Spring WebFlux usa un modelo similar pero con múltiples bucles de eventos. Se aleja del modelo de bloqueo de subprocesos por solicitud en SpringMVC tradicional (con Tomcat de forma predeterminada) y avanza hacia un paradigma multi-EventLoop, asíncrono, sin bloqueo (con Netty de forma predeterminada) que es más escalable y eficiente que el código de bloqueo tradicional.

Swagger Codegen

Puede simplificar el proceso de compilación al generar *stubs* de servidor para cualquier API definida con la especificación OpenAPI.

Plataformas aceleradoras para la generación de clientes de servicios

A nivel de generación de clientes de servicios, algunas de las plataformas aceleradoras más representativas son:

EasyWSDL

Genera el código para consumir cualquier servicio web SOAP especificado con WSDL, en varios lenguajes como Java para Android, Dart/Flutter Java o clases de Objective-C para iOS.

OpenAPI Generator

A partir de una especificación OpenAPI, es posible generar una **librería cliente** con la implementación del consumo de los servicios en múltiples lenguajes de programación como Ada, Android, C, Clojure, C++, C#.NET, Go, Java, JavaScript, Kotlin, Perl, PHP, Python, R, Ruby, Scala y TypeScript.

RAML Client Generator

Es un generador de clientes de API descritos con RAML sobre Node.js.

RAML Tools para .NET

Es una **extensión de Visual Studio** que permite a los desarrolladores integrar y consumir fácilmente API que exponen una definición RAML u OAS.

Swagger Codegen

Genera *software development kits* (SDK) de cliente de una API definida con la especificación OpenAPI.

7.7. Referencias bibliográficas

Alhajj, R. y Rokne, J. (Eds.) (2014). *In Encyclopedia of Social Network Analysis and Mining*. Springer New York.

API Blueprint. (s. f. a) *API blueprint specification*.
<https://apiblueprint.org/documentation/specification.html>

API Blueprint (s. f. b.). *API Blueprint Tools*. <https://apiblueprint.org/tools.html>

API Blueprint (s. f. c.). *Real World API*.
<https://apiblueprint.org/documentation/examples/real-world-api.html>

Bloor, R., Hurwitz, J., Kaufman, M. y Halper, F. (2009). *Service Oriented Architecture (SOA) For Dummies* (2a ed.). John Wiley & Sons.

Christensen, E., Curbera, F., Meredith, G. y Weerawarana, S. (2001, marzo 15). *Web Services Description Language (WSDL) 1.1*. W3C.

DapperDox. (s. f.). *DapperDox use case*. <http://dapperdox.io/>

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Tesis doctoral, Universidad de California, Irvine]. ICS-UCI.
<https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

Fox, A. (2013). *Engineering Software as a Service* (2a ed.). Strawberry Canyon.

GitHub. (2021a, abril 9). *RAML Version 1.0: RESTful API Modeling Language*.
<https://github.com/raml-org/raml-spec/blob/master/versions/raml-10/raml-10.md/>

GitHub. (2021b, julio 9). *Widdershins*. <https://github.com/Mermade/widdershins>

GitHub. (2022, junio 2). *Redoc*. <https://github.com/Redocly/redoc>

Innoq. (2007). *Web Services Standards as of Q1 2007*.

<https://www.innoq.com/resources/ws-standards-poster/>

Hadley, M. (2009, agosto 31). *Web Application Description Language*. W3C.

<https://www.w3.org/Submission/wadl/>

ISO. (2017). *ISO/IEC/IEEE 24765:2017 Systems and software engineering — Vocabulary*. <https://www.iso.org/standard/71952.html>

Josuttis, N. M. (2007). *SOA in Practice: The Art of Distributed System Design*. Soa in Practice. <http://www.soa-in-practice.com/soa-glossary.html>

Lucybot. (s. f.). *Documentation*. <http://demo.lucybot.com/>

Microservice Architecture. (s. f.). *Pattern: Microservice Architecture*.

<https://microservices.io/patterns/microservices.html>

OpenAPI (2021, febrero 15). *OpenAPI Specification OAS v3.1.0*.

<https://spec.openapis.org/oas/v3.1.0>

OpenAPI. (s. f.). *OpenAPI Tools*. <https://openapi.tools/>

Oracle. (2010). *Example: The Storage-Service WADL*.

<https://docs.oracle.com/cd/E19776-01/820-4867/ghmfe/>

Página de [Postman](#).

Página de [RAML](#).

RapiDoc. (s. f.). *OpenAPI Spec Documentations*. <https://mrin9.github.io/RapiDoc/>

SoapUI. (s. f.). *Accelerating API Quality Through Testing*. <https://www.soapui.org/>

Swagger. (s. f. a) *Basic Structure*. <https://swagger.io/docs/specification/basic-structure/>

Swagger. (s. f. b). *Swagger UI*. <https://swagger.io/tools/swagger-ui/>

W3. (2004, febrero 11). *Web Services Glossary*. <https://www.w3.org/TR/ws-gloss/>

W3. (2007, junio 2007). *Web Services Description Language (WSDL) Version 2.0 Part 1: Core Language*. <https://www.w3.org/TR/wsdl/#Iri-ref-ex>

W3. (s. f.). Latest SOAP versions. <https://www.w3.org/TR/soap/>

Java web services: Up and running

Kalin, M. (2013). *Java Web Services: Up and Running* (2a ed.). O'Reilly Media.

Guía para desarrollar clientes y servicios web basados en SOAP y REST. Este libro práctico ofrece un enfoque claro y pragmático de los servicios web al proporcionar una descripción general de la arquitectura, ejemplos completos de códigos de trabajo e instrucciones breves pero precisas para compilarlos, implementarlos y ejecutarlos.

Spring REST

Varanasi, B. y Belida, S. (2014). *Spring REST* (1a ed.). Apress.

Es una guía práctica para diseñar y desarrollar API RESTful utilizando Spring Framework. Detalla sobre varios proyectos de Spring, como Spring Boot, Spring MVC, Spring Data JPA y Spring Security, y el papel que desempeñan en la simplificación del desarrollo de aplicaciones REST.

1. ¿Cuál es la diferencia entre Web Application Description Language (WADL) y Web Services Definition Language (WSDL)?

- A. Ninguna, ya que ambos se orientan a la descripción de servicios web.
- B. WADL es para arquitecturas cliente-servidor y WSDL para microservicios en la nube.
- C. Aunque ambos se orientan a describir servicios web. WSDL especifica aquellos que interaccionan usando SOAP y WADL se aplica para los que están basados en REST.
- D. WADL es para cualquier tecnología y WSDL solo para entornos Microsoft .NET.

2. Relaciona la utilidad con la plataforma:

Interacción	1	A	Postman
Documentación	2	B	Lucybot
Emulación	3	C	Spring WebFlux
Aceleración del desarrollo	4	D	WireMock

3. En sistemas distribuidos basados en servicios se pueden combinar diferentes lenguajes y tecnologías, lo que significa:

- A. Que podemos elegir el lenguaje de programación porque todos tienen librerías para servicios.
- B. Que podemos implementar cada servicio en un lenguaje diferente siempre que no exista interacción entre ellos.
- C. Que cada servicio puede escribirse en el lenguaje que mejor se adapte a su finalidad y se comunica con el resto a través de un protocolo de comunicación estandarizado.
- D. Que el cliente puede estar desarrollado en cualquier tecnología y se comunicará con nuestros servicios usando siempre SOAP.

4. Entre las situaciones en las que emular el comportamiento de un servicio es necesario no se encuentra:

- A. Agilizar el desarrollo en paralelo.
- B. Automatizar el paso de REST a SOAP.
- C. Facilitar la integración con sistemas de terceros.
- D. Disponer de escenarios para pruebas automáticas.

5. Selecciona la plataforma con su estándar:

OpenAPI	1	A	Apache CXF
WSDL	2	B	Osprey Mock Service
RAML	3	C	Apish
API Blueprint	4	D	WidderShins

6. Destaca un beneficio del desarrollo de sistemas distribuidos basados en servicios:

- A. Desarrollo nativo en la nube.
- B. Cliente multiplataforma.
- C. Bajo acoplamiento de los servicios.
- D. Poca escalabilidad.

7. ¿Qué característica arquitectónica no es necesaria para una verdadera API RESTful?

- A. Interfaz uniforme.
- B. Guardar el estado.
- C. Código bajo demanda.
- D. Caché.

8. Un microservicio es:

- A. Un servicio que no puede superar las cien líneas de código.
- B. Un servicio que siempre está desplegado en un contenedor Docker.
- C. Un servicio independiente del resto, con su propio ciclo de vida y potencialmente modelo de datos.
- D. Un servicio que no puede consumir otro servicio.

9. Marca qué lenguaje no es apropiado para describir servicios:

- A. RESTful API Modeling Language.
- B. OpenAPI Specification.
- C. Representational State Transfer (REST).
- D. Web Services Definition Language.

10. En las arquitecturas orientadas a servicio hay tres tipos de actores:
- A. Proveedores de servicios, registros de servicios y usuarios de servicios.
 - B. Clientes y servidores.
 - C. Stubs, Skeletons y registros.
 - D. Proveedores de servicios y usuarios de servicios.

Plataformas de Desarrollo de Software

Tema 8. Plataformas para el desarrollo móvil

Índice

Esquema

Ideas clave

- 8.1. Introducción y objetivos
- 8.2. Contexto de las plataformas para el desarrollo móvil
- 8.3. Valor y utilidad de las plataformas para el desarrollo móvil
- 8.4. Entornos de desarrollo integrados para desarrollo de apps nativas
- 8.5. Plataformas aceleradoras del desarrollo móvil
- 8.6. Referencias bibliográficas

A fondo

- Native mobile development: A cross-reference for iOS and android native programming
- Learn simply: Android mobile application development using kotlin

Test

PLATAFORMAS PARA EL DESARROLLO MÓVIL			
CONCEPTOS	VALOR Y UTILIDAD	HERRAMIENTAS	
App: aplicación móvil Aplicación nativa Aplicación híbrida Aplicación web progresiva	Aplicaciones nativas: - Mayor rendimiento - Acceso a todo el dispositivo - Mayor interacción Aplicaciones híbridas: - Múltiples plataformas con menos código - Desarrollo y mantenimiento más corto - Interacción uniforme multiplataforma Aplicaciones progresivas: - No requiere instalación desde <i>marketplaces</i> - Menor tiempo de carga - Actualización inmediata	IDE: - Android Studio (Android) - XCode (IOS) Aceleradores: - React Native - Flutter - Xamarin - Ionic - Apache Cordova - Sencha Touch - Mobile Angular UI - JQuery mobile - NativeScript - Framework7	
Android: - Multitáctil y multitarea - Se adaptan a diferentes dispositivos - Varios puntos de entrada - Ejecución en Linux Multiusuario - Principio de mínimo privilegio - Componentes: actividades; servicios; proveedores de contenido; receptores de mensajes iOS: - Multitarea - iCloud - Compras - Centros de notificaciones - Acelerómetro, GPS, Giroscopio			

8.1. Introducción y objetivos

En este tema se estudian algunas de las plataformas que nos ayudan a desarrollar **aplicaciones móviles**.

En primer lugar, estableceremos el **contexto** del desarrollo móvil y sus aplicaciones (apps) y se hará una diferenciación entre las nativas, las híbridas y las webs progresivas, junto con sus características que nos ayudarán a identificar cuál es más idónea en cada caso. Seguidamente, veremos con mayor detalle los dos sistemas operativos móviles principales, Android e iOS, junto con sus características.

Posteriormente, veremos algunos de los principales **entornos de desarrollo** integrado que facilitan la creación de aplicaciones nativas para Android, Android Studio y para iOS, Xcode. Por último, analizaremos algunas plataformas o *frameworks* que permiten acelerar el proceso de desarrollo de aplicaciones móviles.

Con el estudio de este tema pretendemos alcanzar los siguientes objetivos:

- ▶ Comprender el contexto, el valor y la utilidad que tiene el desarrollo de aplicaciones móviles.
- ▶ Conocer los principales entornos de desarrollo integrado para crear aplicaciones móviles nativas para Android e iOS.
- ▶ Conocer algunas plataformas y *frameworks* aceleradores del desarrollo de aplicaciones móviles nativas e híbridas.

8.2. Contexto de las plataformas para el desarrollo móvil

Una aplicación móvil, comúnmente conocida como **app**, es un tipo de software de aplicación diseñado para ejecutarse en un dispositivo móvil, como un teléfono inteligente, una tableta o un reloj inteligente. Las aplicaciones móviles frecuentemente sirven para proporcionar a los usuarios servicios similares a los que se acceden en los ordenadores de escritorio o portátiles. Las apps son generalmente **pequeñas unidades de software** individuales con funciones limitadas.

En ocasiones, para que un sistema sea accesible desde un terminal móvil no es necesario desarrollar una app, sino que basta con hacer que la interfaz web se adapte al dispositivo. Un sitio **web responsive** es aquel que ha sido diseñado para ser receptivo en cualquier dispositivo, por lo que su visualización e interacción se adapta al tamaño de la pantalla, ya sea un teléfono o una tableta. El objetivo principal de un sitio web *responsive* es brindar a los usuarios una **experiencia de usuario perfecta**, independientemente del dispositivo que utilicen, al hacer uso de un diseño flexible de forma que su contenido se escale y adapte al tamaño de la pantalla. Estas soluciones quedan fuera del concepto de desarrollo móvil que cubrimos en este tema, pero, en ocasiones, puede ser suficiente para no perder usuarios o facilitar su interacción sin necesidad de un desarrollo adicional al de una aplicación web tradicional.

En el caso de que necesitemos **mayores funcionalidades**, existen diversos tipos de apps:

- Una **aplicación nativa** es un software creado para un sistema operativo móvil específico y escrito en un lenguaje de programación determinado para esa plataforma. En el 2022, iOS y Android son líderes en el mercado de dispositivos móviles. El principal beneficio de las aplicaciones nativas es una experiencia de

usuario perfecta y características sólidas diseñadas para un sistema operativo.

Dichas aplicaciones se pueden instalar en un teléfono inteligente o tableta y, según su modelo, pueden funcionar en modo fuera de línea, accediendo a los diversos elementos físicos del dispositivo. Con una aplicación nativa, debe enviarla a Google Play o App Store para su revisión, aprobar todos los requisitos para que su aplicación esté disponible para los usuarios y cumplir con las normas todo el tiempo mientras su aplicación esté en una tienda de aplicaciones.

- ▶ Una **aplicación híbrida** es aquella que funciona en todos los dispositivos móviles. Comparte la misma base de código y permite acelerar el tiempo de desarrollo y ahorrar dinero. En lugar de desarrollar una aplicación separada para Android e iOS, las empresas pueden crear una sola aplicación. Independientemente de las ventajas, las aplicaciones híbridas **no brindan la mejor experiencia de usuario** y no pueden presumir de un alto rendimiento. En la mayoría de los casos, no puede utilizar todas las funciones peculiares de un sistema operativo y crear aplicaciones que dependan en gran medida del hardware del dispositivo. Al igual que las aplicaciones nativas, las aplicaciones híbridas se distribuyen a través de las tiendas de aplicaciones. Por lo tanto, debe cumplir con sus requisitos para colocar la solicitud allí. Además, siempre existe la amenaza de que una tienda de aplicaciones elimine la aplicación sin previo aviso.
- ▶ Una **aplicación web progresiva** (*progressive web application*, PWA) es un sitio web que se ve y se comporta como si fuera una aplicación móvil. Las PWA están diseñados para aprovechar las funciones nativas de los dispositivos móviles sin que el usuario final tenga que visitar una tienda de aplicaciones, realizar una compra y descargar el software localmente. Eliminan la necesidad de que los comerciantes de comercio electrónico desarrollen aplicaciones nativas para múltiples sistemas operativos móviles. Al igual que los vídeos de YouTube, el contenido de PWA se descarga progresivamente, lo que brinda al usuario final una mejor experiencia de usuario que un sitio web tradicional que utiliza un diseño *responsive*. Usan **tecnologías basadas en estándares** y se ejecutan en un contenedor que es seguro y accesible para cualquier persona en la web. Pueden enviar notificaciones *push*

web, trabajar sin conexión y ser accesibles desde la pantalla de inicio, al igual que una aplicación móvil de una tienda de aplicaciones. También pueden aprovechar las API y los complementos del navegador para garantizar que la implementación y el mantenimiento de un sitio web sigan siendo lo más fáciles posible. Otro beneficio que las PWA han heredado de los sitios web tradicionales es el uso de URI (identificador de recursos uniforme) para indicar el estado actual, lo que permite que la aplicación web retenga o vuelva a cargar su estado cuando el usuario marca o comparte la URL de la aplicación. La interfaz de usuario de una aplicación web progresiva debe tener el **tamaño correcto** para el factor de forma y el tamaño de la pantalla del dispositivo. Además, debe tener el **aspecto de una aplicación nativa** y estar construido sobre un modelo de *shell* de aplicación. La PWA debería tener pocas actualizaciones de página y la aplicación debería funcionar en áreas de baja conectividad o incluso sin conexión. Además, el contenido nuevo debe estar disponible en la aplicación. Debido a que las solicitudes de red se pueden interceptar, la aplicación debe alojarse en HTTPS para evitar ataques de intermediarios.

Los dos principales sistemas operativos de dispositivos móviles son Android e iOS. A continuación vamos a ver alguna de las características principales de cada uno.

Android

Android es un sistema operativo de código abierto basado en Linux para dispositivos móviles como teléfonos inteligentes y tabletas. Android fue desarrollado por Open Handset Alliance, dirigida por Google y otras empresas. Ofrece un enfoque **unificado** para el desarrollo de aplicaciones para dispositivos móviles, lo que significa que los desarrolladores solo necesitan desarrollar para Android y sus aplicaciones deberían poder ejecutarse en diferentes dispositivos con Android. Google lanzó la primera versión beta del kit de desarrollo de software (SDK) de Android en 2007, mientras que la primera versión comercial, Android 1.0, se lanzó en septiembre de 2008. El código fuente de Android está disponible bajo licencias de software libre y de código abierto. Google publica la mayor parte del código bajo la

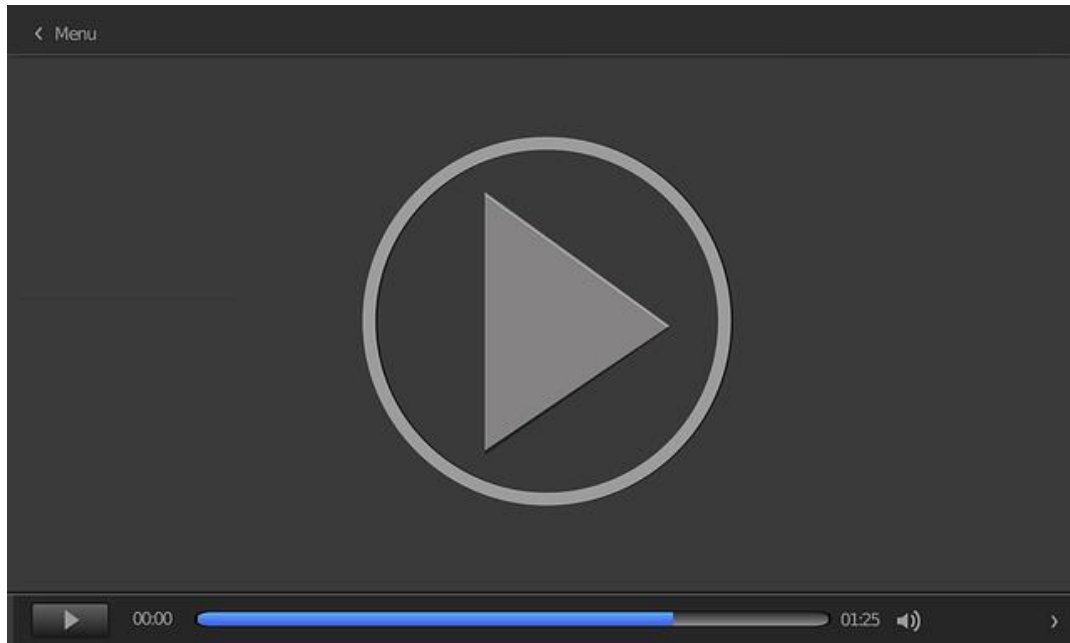
licencia Apache versión 2.0 y el resto, los cambios del kernel de Linux, bajo la Licencia Pública General (GNU) versión 2 (Android, s. f.).

Alguna de las principales características de Android son:

- ▶ **Interfaz de usuario intuitiva:** la pantalla básica del sistema operativo Android proporciona una interfaz de usuario intuitiva.
- ▶ **Conectividad:** soporta múltiples protocolos como GSM/EDGE, IDEN, CDMA, EV-DO, UMTS, Bluetooth, wifi, LTE, NFC y WiMAX.
- ▶ **Almacenamiento:** incorpora SQLite, una base de datos relacional ligera para el almacenamiento de datos.
- ▶ **Soporte de medios:** incluye el soporte para múltiples formatos de archivos multimedia como H.263, H.264, MPEG-4 SP, AMR, AMR-WB, AAC, HE-AAC, AAC 5.1, MP3, MIDI, Ogg Vorbis, WAV, JPEG, PNG, GIF y BMP.
- ▶ **Mensajería:** soporta SMS y MMS.
- ▶ **Navegador web:** basado en el motor de diseño WebKit de código abierto, junto con el motor JavaScript V8 de Chrome que admite HTML5 y CSS3.
- ▶ **Multitáctil:** tiene soporte nativo para multitáctil.

- ▶ **Multitarea:** el usuario puede saltar de una tarea a otra y, al mismo tiempo, varias aplicaciones pueden ejecutarse simultáneamente.
- ▶ **Widgets redimensionables:** los usuarios pueden expandirlos para mostrar más contenido o reducirlos para ahorrar espacio.
- ▶ **Multi lenguaje:** admite texto de dirección única y bidireccional.
- ▶ **Google Cloud Messaging (GCM):** es un servicio que permite a los desarrolladores enviar datos de mensajes cortos a sus usuarios en dispositivos Android, sin necesidad de una solución de sincronización patentada.
- ▶ **Wifi directo:** integra una tecnología que permite que las aplicaciones descubran y se emparejen directamente, a través de una conexión punto a punto de gran ancho de banda.
- ▶ **NFC (comunicación de campo cercano):** incorpora una tecnología NFC que permite a los usuarios compartir instantáneamente al simplemente tocar dos teléfonos habilitados para NFC.
- ▶ **Tiendas de apps:** una vez desarrolladas, las aplicaciones de Android pueden empaquetarse fácilmente y venderse a través de una tienda como Google Play, SlideME, Opera Mobile Store, Mobango, F-Droid y Amazon Appstore.

En este vídeo, *Introducción al desarrollo de aplicaciones móviles con Android Studio*, aprenderemos las principales características de una aplicación Android y cómo crearla y ejecutarla a nivel práctico con el IDE Android Studio.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=8e0ea169-99c1-459b-b4bf-aec300cd980f>

Las aplicaciones de Android generalmente se desarrollan en el lenguaje Java utilizando el kit de desarrollo de software de Android. Son las herramientas del Android SDK las encargadas de la **compilación y empaquetado** en un APK (un paquete de Android). Este archivo .apk es el archivo que usan los dispositivos Android para instalar la aplicación. Las aplicaciones Android:

- ▶ **Se adaptan a diferentes dispositivos:** es posible crear diferentes archivos de diseño XML para diferentes tipos de pantallas y el sistema decide en tiempo de ejecución el más adecuado.
- ▶ **Proporcionan varios puntos de entrada:** esto es debido a que estas aplicaciones están compuestas por varios componentes que pueden ser invocados de manera individual.
- ▶ **Se ejecutan sobre un Linux multiusuario:** donde cada aplicación es considerada como un usuario diferente, con su respectivo ID de usuario. Además, cada proceso tiene su propio equipo virtual (EV), por lo que las aplicaciones se ejecutan de forma independiente unas de las otras, o sea cada aplicación ejecuta su proceso de Linux propio.
- ▶ **Se ejecutan bajo el principio de mínimo privilegio:** donde cada aplicación tiene acceso a los componentes que necesita y nada más, para crear un entorno extremadamente seguro.

Las aplicaciones sobre Android están basadas en cuatro tipos de componentes:

COMPONENTES DE APLICACIONES ANDROID

ACTIVIDADES	Una actividad representa una pantalla con interfaz de usuario. Por ejemplo, una aplicación de correo electrónico tiene una actividad que muestra una lista de los correos electrónicos nuevos, otra actividad para redactar el correo electrónico y otra actividad para leer correos electrónicos. Hay que saber que las actividades trabajan juntas, pero cada una es independiente de las demás y puede ser invocada independientemente.
SERVICIOS	Un servicio es un componente que se ejecuta en segundo plano. Normalmente este tipo de componentes se usa para ejecutar tareas para procesos remotos u operaciones prolongadas y no tiene interfaz gráfica de usuario. Por ejemplo, un servicio podría reproducir música en segundo plano mientras el usuario se encuentra en otra aplicación.
PROVEEDORES DE CONTENIDOS	Un proveedor de contenido administra y maneja un conjunto de datos de la aplicación que pueden compartirse. Así, otras aplicaciones pueden consultarlos o modificarlos. Por ejemplo, Android proporciona un proveedor de contenido que administra la información de contacto del usuario. De esta manera, cualquier aplicación con los permisos correspondientes puede consultar parte del proveedor de contenido para la lectura y escritura de información sobre una persona específica.
RECEPTORES DE MENSAJES	Un receptor de mensaje es un componente que responde a los anuncios de mensajes en todo el sistema. Muchos mensajes son originados por el sistema. Por ejemplo, un mensaje que anuncie que se apagó la pantalla, que la batería tiene poca carga o que se tomó una foto.

Tabla 1. Componentes de aplicaciones Android. Fuente: elaboración propia.

Los componentes actividades, servicios y receptores de mensajes se activan mediante un mensaje asincrónico llamado *intent*, que sirven para enlazar componentes individuales en tiempo de ejecución.

iOS

iOS significa sistema operativo iPhone. Es un sistema operativo móvil propietario de Apple para su computadora de mano. Admite los lenguajes de programación Objective-C, C, C++ y Swift. Muchos de los dispositivos móviles de Apple, incluidos iPhone, iPad y iPod, funcionan con este sistema operativo. Para controlar el dispositivo, iOS emplea una **interfaz multitáctil**, como deslizar el dedo por la

pantalla para avanzar a la página siguiente o pellizcar los dedos para acercar o alejar la pantalla. Las principales características de iOS son:

- ▶ **Multitarea:** el sistema operativo soporta la ejecución concurrente de múltiples procesos.
- ▶ **Redes sociales:** permite compartir contenido y mostrar un flujo de actividad como formas de simplificar la integración de las interacciones de las redes sociales en una app.
- ▶ **iCloud:** es un servicio de Apple que ofrece almacenamiento de datos basado en Internet. Funciona en todos los dispositivos Apple, tiene cierta compatibilidad con Windows y maneja la mayoría de las operaciones en segundo plano. Está altamente encriptado. Ofrece la opción de copia de seguridad para ayudar al usuario a no perder ninguno de sus datos.
- ▶ **Compras en la aplicación:** disponibles en todas las plataformas de Apple, brindan a los usuarios material y servicios adicionales, como elementos digitales (iOS, iPadOS, macOS, watchOS), suscripciones y contenido premium directamente dentro de la aplicación. Incluso puede usar la App Store para promocionar y vender compras dentro de la aplicación.
- ▶ **Centro de notificaciones:** es una característica de iOS que le muestra todas las alertas de su aplicación en un solo lugar. En lugar de necesitar una resolución inmediata, muestra notificaciones hasta que el usuario completa una acción asociada.
- ▶ **Acelerómetro:** un acelerómetro es un dispositivo que detecta cambios en la velocidad a lo largo de un solo eje. Cada dispositivo iOS incorpora un acelerómetro de tres ejes que proporciona lecturas de aceleración en cada uno de los tres ejes.

- ▶ **Giroscopio:** la velocidad a la que un dispositivo gira alrededor de un eje espacial se mide con un giroscopio. Un giroscopio de tres ejes se encuentra en muchos dispositivos iOS y proporciona datos de rotación en cada uno de los tres ejes.
- ▶ **GPS:** para detectar su ubicación, el iPhone utiliza un chip de GPS asistido (AGPS) incorporado. Ni siquiera es necesario instalar esta función porque ya está integrada en su iPhone. Como proporciona una aproximación de su ubicación basada en información satelital, este sistema es más rápido que el GPS estándar.
- ▶ **Accesibilidad:** todos los productos y servicios de Apple están contruidos con capacidades de accesibilidad de un solo toque.
- ▶ **Bluetooth:** Apple proporciona el marco Core Bluetooth, que incluye clases para conectarse a tecnología inalámbrica de bajo consumo habilitada para Bluetooth.
- ▶ **Orientaciones:** las aplicaciones de iOS se pueden usar tanto en modo vertical como horizontal. Apple, por otro lado, proporciona clases de tamaño en Xcode para crear interfaces en orientación horizontal y vertical.
- ▶ **Integración de la cámara:** en iOS, Apple proporciona el subsistema de captura AVFoundation, que es una arquitectura estándar de alto nivel para la captura de audio, imagen y vídeo.
- ▶ **Servicios de ubicación:** permiten que las aplicaciones y los sitios web accedan a la ubicación del dispositivo del usuario con el permiso del usuario. Cuando los servicios de ubicación están operativos, la barra de estado muestra un icono de flecha blanca o negra.
- ▶ **Mapas:** Apple ofrece un servicio de mapas en línea que se puede utilizar como el sistema de mapas predeterminado de iOS. Tiene una variedad de funciones, como un modo de sobrevuelo.

8.3. Valor y utilidad de las plataformas para el desarrollo móvil

Los principales **valores** de desarrollar aplicaciones móviles, frente a un desarrollo *web responsive*, es la capacidad de personalizar los contenidos, mejorar la experiencia del usuario, dar soporte a la funcionalidad *offline* y poder hacer uso de los elementos físicos del terminal como la videocámara, la navegación GPS, la función de reconocimiento de voz, etc.).

El desarrollo de desarrollo de aplicaciones móviles **nativas** presenta una serie de ventajas:

- ▶ **Mayor rendimiento:** la app se crea y optimiza para una plataforma específica, por lo que presenta un nivel de rendimiento extremadamente alto. Al estar diseñadas y compiladas para el lenguaje de programación central de las plataformas y sus API de forma específica, su ejecución es más eficiente. El dispositivo almacena la aplicación, lo que permite que el software aproveche la velocidad de procesamiento del dispositivo. A medida que los usuarios navegan a través de una aplicación móvil nativa, los contenidos y elementos visuales ya están almacenados en su teléfono y eso significa que los tiempos de carga son rápidos.

- ▶ Permiten a los desarrolladores **acceder al conjunto completo de funciones de los dispositivos**: se desarrollan para su plataforma particular, lo que hace posible aprovechar al máximo el software y las características de los sistemas operativos. Estas aplicaciones pueden acceder directamente al hardware del dispositivo como el GPS, la cámara, el micrófono o el detector de huellas. Las notificaciones automáticas son otra gran ventaja al elegir el desarrollo de aplicaciones nativas usando el servidor iOS (APNS) o el Cloud Messaging de Google (GCM). Con una aplicación nativa podemos sacar el máximo provecho del terminal que lo ejecuta.
- ▶ **Son más interactivas e intuitivas**: heredan las interfaces del sistema operativo de sus dispositivos, lo que hace que se vean y se sientan como una parte integrada del dispositivo y presentan mejor experiencia en la interacción con el usuario. Las aplicaciones nativas se crean específicamente para un sistema operativo. Se adhieren a las pautas que, en última instancia, mejoran y alinean la experiencia del usuario con el sistema operativo específico. Como resultado, el flujo de la aplicación es más natural ya que tienen estándares de interfaz de usuario específicos para cada plataforma. Esto permite que el usuario aprenda la aplicación, como eliminar un elemento rápidamente. Cumplir con las pautas específicas elimina la curva de aprendizaje y permite a los usuarios interactuar con las aplicaciones mediante acciones y gestos con los que ya están familiarizados.

El desarrollo de desarrollo de aplicaciones móviles **híbridas** presenta una serie de ventajas:

Ventajas	Descripción
Múltiples plataformas con menos código	Este enfoque permite apuntar a múltiples plataformas sin mantener numerosas bases de código. A diferencia de las aplicaciones nativas, con una única base de código para varias plataformas, las aplicaciones híbridas permiten a los equipos de desarrollo acelerar el proceso de desarrollo. Reduce significativamente el tiempo de comercialización. La aplicación se puede publicar en varias tiendas de aplicaciones móviles con un código común.
Mayor facilidad de mantenimiento	En el caso de aplicaciones nativas, los equipos de desarrollo deben implementar nuevas versiones para cada plataforma con cada actualización. Una aplicación híbrida elimina el control de versiones y hace que la administración de la aplicación sea tan simple como actualizar una página web, también en tiempo real.
Plazo de desarrollo más corto	En general, son más fáciles y rápidas de desarrollar que una aplicación nativa. Permite a las organizaciones aprovechar su kit de herramientas de desarrollo web existente para eliminar la necesidad de crear una nueva base de código en cada plataforma.
Alcance de mercado más amplio	Son perfectos para apuntar a diferentes plataformas con una solución. En este mercado competitivo, se convierte en una opción lucrativa y rentable para llegar a una gran cantidad de usuarios que utilizan diferentes plataformas móviles manteniendo un único código.
Interacción uniforme multiplataforma	El desarrollo móvil híbrido combina las ventajas de las aplicaciones nativas y web. Ofrece una experiencia de usuario uniforme e impecable en las plataformas iOS y Android. Se trata de hacer que funcione para dispositivos móviles adaptando la web. Además, la interfaz de usuario de la aplicación híbrida ligera ayuda a cargar contenido y gráficos rápidamente. Las aplicaciones se adaptan a diferentes pantallas de dispositivos para una visualización rápida de datos y una transmisión continua de datos.

Tabla 2. Ventajas de las aplicaciones móviles híbridas. Fuente: elaboración propia.

El desarrollo de desarrollo de aplicaciones móviles **progresivas** presenta una serie de ventajas:

- ▶ **No** es necesario acceder a los **marketplaces** de aplicaciones para realizar instalaciones.
- ▶ **Menor tiempo de carga** en el dispositivo.
- ▶ Buen rendimiento y **eficiencia con mal ancho de banda**.
- ▶ **Actualización inmediata** y completamente transparente al usuario.
- ▶ Posibilidad de incorporar ciertas funcionalidades similares a una app nativa (por

ejemplo: botón pantalla *home*, notificaciones *push*, modo *offline*).

Si simplemente queremos soportar un acceso responsive desde el móvil con ciertos cortes de red, se debe elegir la PWA. Si queremos tener un único código multiplataforma, se debe elegir la híbrida. Si queremos obtener lo máximo del terminal, máxima eficiencia o interacción *pixel-perfect*, se debe elegir la nativa.

8.4. Entornos de desarrollo integrados para desarrollo de apps nativas

En esta sección vamos a exponer los entornos de desarrollo integrados (IDE) para los dos sistemas operativos móviles principales, Android e iOS.

Android Studio

Android Studio es el **IDE oficial de Android** y se creó con el único propósito de acelerar el proceso de desarrollo y crear aplicaciones de alta calidad para todos los dispositivos Android. Sus principales características son:

- ▶ **Codificación más rápida e iteración veloz:** basado en IntelliJ IDEA, permite inyectar código y cambiar recursos a su aplicación en ejecución sin tener que reiniciarla. Su editor de código inteligente sugiere código que puede seleccionar de una lista desplegable. Y por último, pero no menos importante, el emulador de Android instala e inicia sus aplicaciones más rápido. Esto le permite crear prototipos y probar aplicaciones en todos los dispositivos Android.
- ▶ **Programación con confianza:** proporciona plantillas de código y aplicaciones de muestra para un desarrollo mejor y más rápido. Además, también proporciona herramientas y **marcos de prueba** impresionantes como JUnit.
- ▶ **Generación de APK para todos los dispositivos:** proporciona automatización de compilación, administración de dependencias y configuraciones de compilación personalizables. También cuenta con la integración de herramientas de control de versiones para trabajo en equipo y cuenta con un ambiente unificado.

- ▶ **Integración de herramientas de interfaz gráfica de usuario (GUI):** integra un editor visual de arrastrar y soltar. Además, ofrece un analizador de APK, Vector Asset Studio y un editor de traducciones.
- ▶ **Editor de código inteligente:** facilita la escritura de código con su editor de código inteligente que proporciona finalización de código para los lenguajes Kotlin, Java y C/C++.

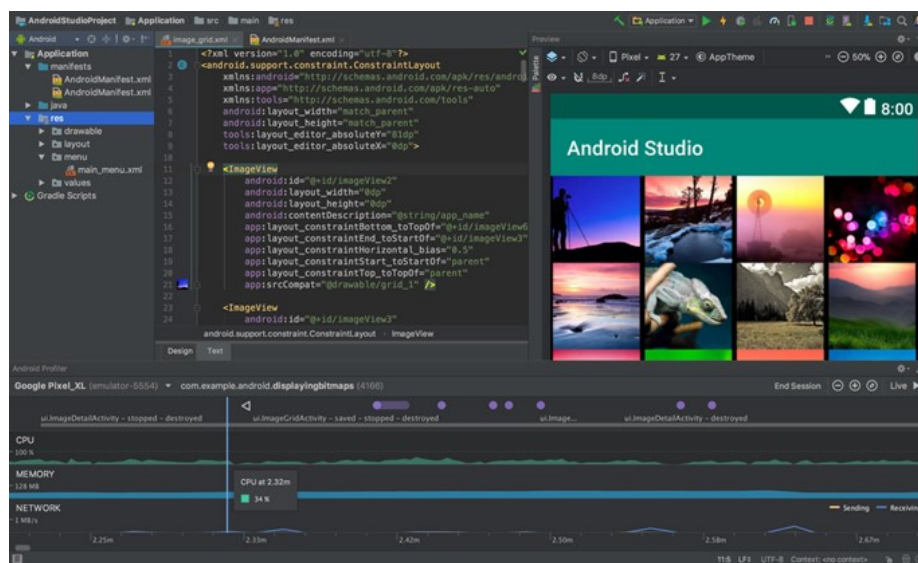


Figura 1. Imagen del IDE Android Studio. Fuente: Developer Android, s. f.

- ▶ **Editor de diseño visual:** facilita la creación de diseños complejos con ConstraintLayout al agregar restricciones de cada vista a otras vistas y pautas. Permite obtener una vista previa de su diseño en cualquier tamaño de pantalla seleccionando una de varias configuraciones de dispositivos o simplemente cambiando el tamaño de la ventana de vista previa.

- ▶ **Analizador de APK:** permite reducir el tamaño de su aplicación de Android inspeccionando el contenido del archivo APK de su aplicación, incluso si no se creó con Android Studio. Inspecciona el archivo de manifiesto, los recursos y los archivos DEX. Compara dos APK para ver cómo cambió el tamaño de una aplicación entre las versiones de la aplicación.
- ▶ **Emulador rápido:** permite instalar y ejecutar las aplicaciones más rápido que con un dispositivo físico y simula diferentes configuraciones y funciones, incluida ARCore, la plataforma de *Google* para crear experiencias de realidad aumentada.
- ▶ Sistema de **construcción flexible:** con la tecnología de **Gradle**, el sistema de compilación que permite personalizar una compilación para generar múltiples variantes de compilación para diferentes dispositivos a partir de un solo proyecto.
- ▶ **Perfiladores en tiempo real:** las herramientas de generación de perfiles integradas brindan estadísticas en tiempo real sobre la actividad de la CPU, la memoria y la red de su aplicación. Identifica cuellos de botella en el rendimiento registrando seguimientos de métodos, inspeccionando el montón y las asignaciones, y vea las cargas útiles de la red entrantes y salientes.

Xcode

Xcode es el IDE **propio de Apple** y se utiliza para crear aplicaciones para productos Apple, como iPad, iPhone, Apple Watch y Mac. Es una poderosa herramienta que le permite administrar todo su flujo de trabajo de desarrollo de principio a fin, eso es: crear su aplicación, probarla, optimizarla y enviarla a la App Store de Apple. Algunas de sus funcionalidades destacadas son:

- ▶ **Interfaz de ventana única:** obtiene todas las herramientas de desarrollo en una ventana; editor de codificación, diseño de interfaz de usuario, gestión de activos, pruebas y depuración, respectivamente.
- ▶ **Editor de código fuente inteligente:** cualquiera que sea el lenguaje de programación con el que esté trabajando, ya sea Swift, Objective-C y C++; el editor

verifica su código, sugiere completarlo, resalta los errores y ofrece realizar correcciones automáticas. Además, también proporciona fragmentos de código listos para usar y plantillas de archivos fuente para que los use y acelere el proceso de programación.

- ▶ **Diseño de interfaz de usuario gráfica:** tiene integrado Interface Builder, un editor de diseño visual, para crear interfaces para sus aplicaciones iOS, watchOS u OS X.
- ▶ **Depurador integrado:** un componente esencial en todo IDE, también proporciona un depurador y un simulador perfectos.
- ▶ **Pruebas e integraciones:** para un mejor rendimiento de la aplicación, Xcode viene con un marco de prueba para pruebas funcionales, de rendimiento y de interfaz de usuario.
- ▶ **Gestión de guardado automático y control de versiones del código fuente:** guarda automáticamente su progreso y cambios en el proyecto. También está integrado con Git y Subversion para la gestión del control de código fuente.

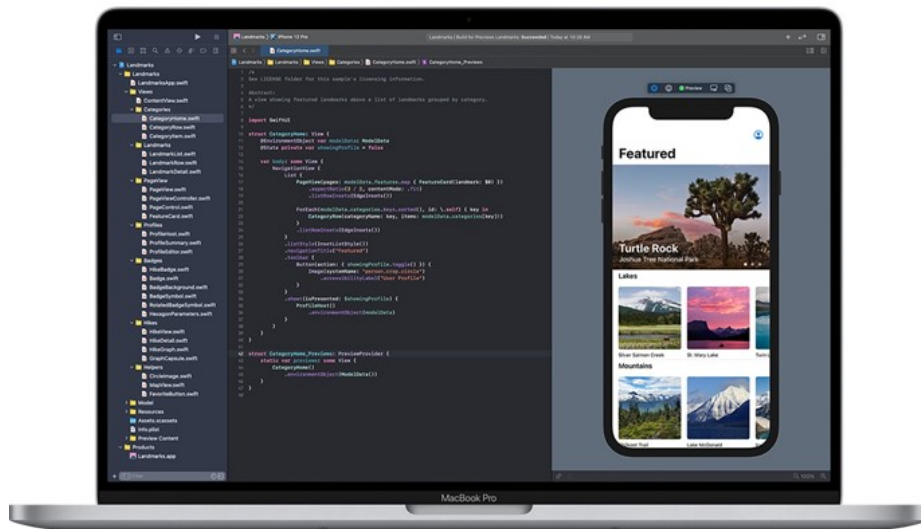


Figura 2. Imagen del IDE Xcode. Fuente: Developer Apple, s. f.

- ▶ **Documentación integrada:** proporciona una amplia documentación sobre el uso de

Xcode y ofrece mucho más, que incluye SDK doc., guías de programación, tutoriales, código de muestra, referencias detalladas de la API del marco y más.

- ▶ **Distribución de aplicaciones para pruebas y App Store:** brinda soporte para administrar cuentas y derechos de programas de desarrolladores y mucho más. Desde la prueba beta hasta su lanzamiento en la App Store después de la aprobación, Xcode brinda asistencia en todo momento.

8.5. Plataformas aceleradoras del desarrollo móvil

Las principales plataformas y *frameworks* para el desarrollo de apps móviles son:



Tabla 3. Plataformas aceleradoras del desarrollo móvil. Fuente: elaboración propia.

React Native

Es una de las mejores opciones de los equipos de desarrollo en todo el mundo. No solo es la opción preferida para crear aplicaciones **híbridas**, sino que también es popular para aplicaciones **nativas y multiplataforma**. Esto se debe a que viene equipado con un mundo de características y funcionalidades avanzadas, como reutilización de código, recarga en vivo, soporte comunitario, complementos nativos y más. Es una opción lógica para las empresas que buscan crear aplicaciones de alto rendimiento brindando a los usuarios una experiencia similar a la nativa (React Native, s. f.).

Flutter

Desarrollado por Google, es un marco de código abierto conocido por su **velocidad y confiabilidad** para construir interfaces nativas altamente interactivas. Ofrece una rica colección de *widgets* para personalización, SDK y muchas otras herramientas de desarrollo que facilitan y agilizan la creación de aplicaciones híbridas. Incorpora un

conjunto de herramientas de interfaz de usuario multiplataforma que está diseñado para permitir la reutilización de código en sistemas operativos como iOS y Android, al mismo tiempo que permite que las aplicaciones interactúen directamente con los servicios de la plataforma subyacente.

El objetivo es permitir que los desarrolladores entreguen aplicaciones de alto rendimiento que se sientan **naturales** en diferentes plataformas, aceptando las diferencias donde existan y compartiendo la mayor cantidad de código posible. Durante el desarrollo, las aplicaciones de Flutter se ejecutan en una máquina virtual (VM) que ofrece una recarga en caliente con estado de los cambios sin necesidad de una recopilación completa. Para el lanzamiento, las aplicaciones de Flutter se compilan directamente en código de máquina, ya sean instrucciones Intel x64 o ARM, o en JavaScript si se dirigen a la web. El marco es de código abierto, con una licencia BSD y tiene un ecosistema de paquetes de terceros que complementan la funcionalidad de la biblioteca principal. (Flutter, s. f. a).

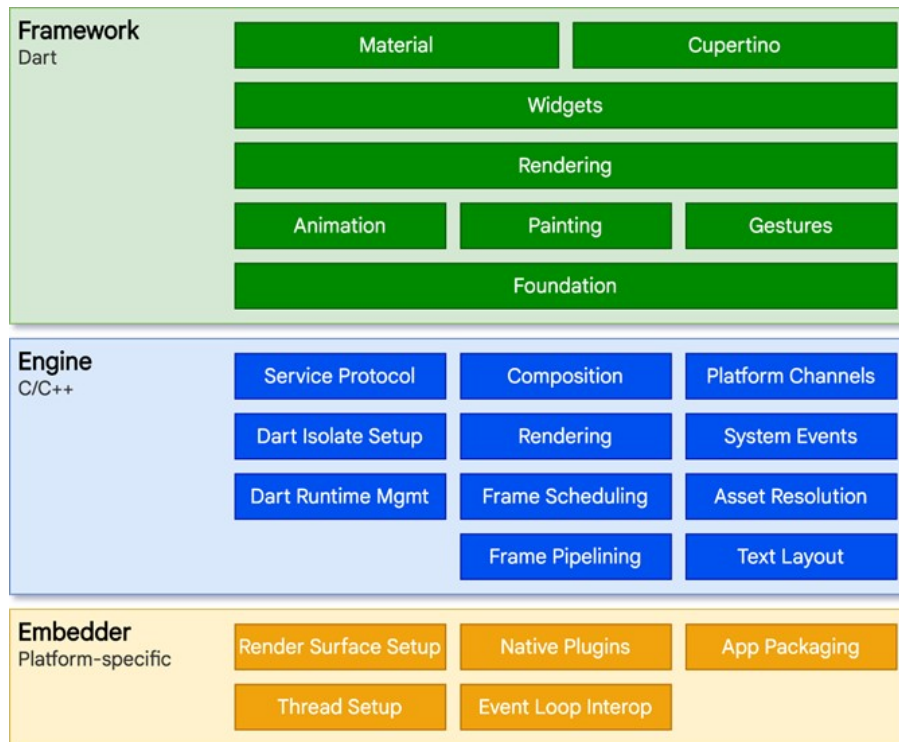


Figura 3. Modelo de arquitectura de Flutter. Fuente: Flutter, s. f. b.

Xamarin

Es una plataforma de código abierto para compilar aplicaciones modernas y con mejor rendimiento para iOS, Android y Windows con .NET. Aporta una capa de abstracción que administra la comunicación de código compartido con el código de plataforma subyacente. Se ejecuta en un entorno administrado que **proporciona ventajas** como la asignación de memoria y la recolección de elementos no utilizados. Permite a los desarrolladores compartir un promedio del 90 % de la aplicación entre plataformas. Este patrón permite a los desarrolladores escribir toda la lógica de negocios en un solo lenguaje (o reutilizar el código de aplicación existente), pero conseguir un rendimiento y una apariencia nativos en cada plataforma. Las aplicaciones de Xamarin se pueden escribir en PC o Mac y compilar en paquetes de aplicación nativos, como un archivo .apk en Android o .ipa en iOS (Microsoft, s. f.).

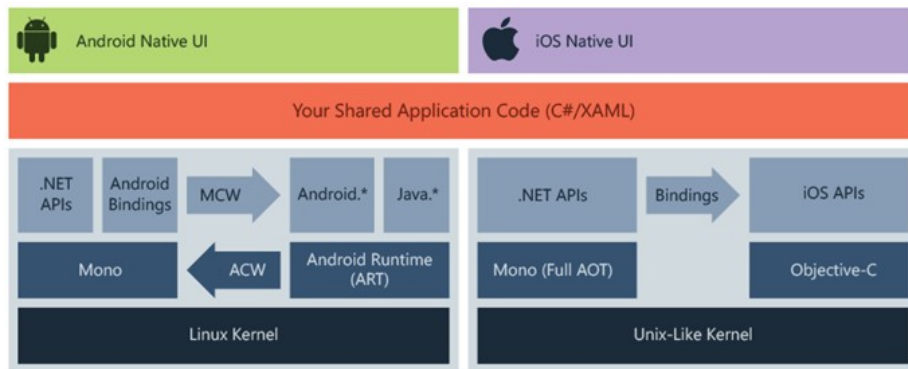


Figura 4. Modelo de arquitectura de Xamarin. Fuente: Microsoft, 2022.

Ionic

Es conjunto de herramientas móviles de código abierto para crear experiencias de aplicaciones web y nativas multiplataforma de alta calidad. Permite desarrollar más rápido con una **única base de código** al ejecutarse en todas partes con JavaScript y la web. Ofrece un amplio conjunto de funciones y herramientas. Está alimentado con integraciones que incluyen componentes y diseños de interfaces de usuario (IU) móviles nativos, así como herramientas de depuración y prueba para crear aplicaciones interactivas. Permite crear aplicaciones nativas para iOS, Android y web, contiene más de cien componentes de interfaz de usuario optimizados para dispositivos móviles, soporta acceso nativo completo y ejecuta las aplicaciones con buen rendimiento (Ionic, s. f.).

Sencha Touch

Es una biblioteca de JavaScript de UI, o marco web, creado específicamente para la web móvil. Los desarrolladores web pueden utilizarlo para desarrollar interfaces de usuario para aplicaciones web móviles que **se vean y se sientan** como aplicaciones nativas en dispositivos móviles compatibles. Se basa en estándares web como HTML5, CSS3 y JavaScript. Su objetivo es facilitar el desarrollo rápido de aplicaciones móviles basadas en HTML5 que se ejecutan en dispositivos Android e iOS (Sencha Touch, 2015).

Apache Cordova

Es un marco de desarrollo móvil de código abierto. Permite utilizar tecnologías web estándar: HTML5, CSS3 y JavaScript para el desarrollo multiplataforma. Las aplicaciones se ejecutan dentro de **contenedores destinados a cada plataforma** y se basan en enlaces API que cumplen con los estándares para acceder a las capacidades de cada dispositivo, como sensores, datos, estado de la red, etc. Sus usos habituales son: desarrollar una app móvil que queremos extenderla a más de una plataforma, sin tener que volver a implementarla con el lenguaje y el conjunto de herramientas de cada plataforma; desarrollar una app web que está empaquetada para su distribución en varios portales de tiendas de aplicaciones; desarrollar apps que mezclan componentes de aplicaciones nativas con una WebView (ventana especial del navegador) que puede acceder a API a nivel de dispositivo, o si desea desarrollar una interfaz de complemento entre componentes nativos y WebView. (Apache Cordova, s. f.). Es la versión gratuita de la solución comercial Adobe Phonegap, que fue descontinuada en 2020 (Phonegap d, s. f.).

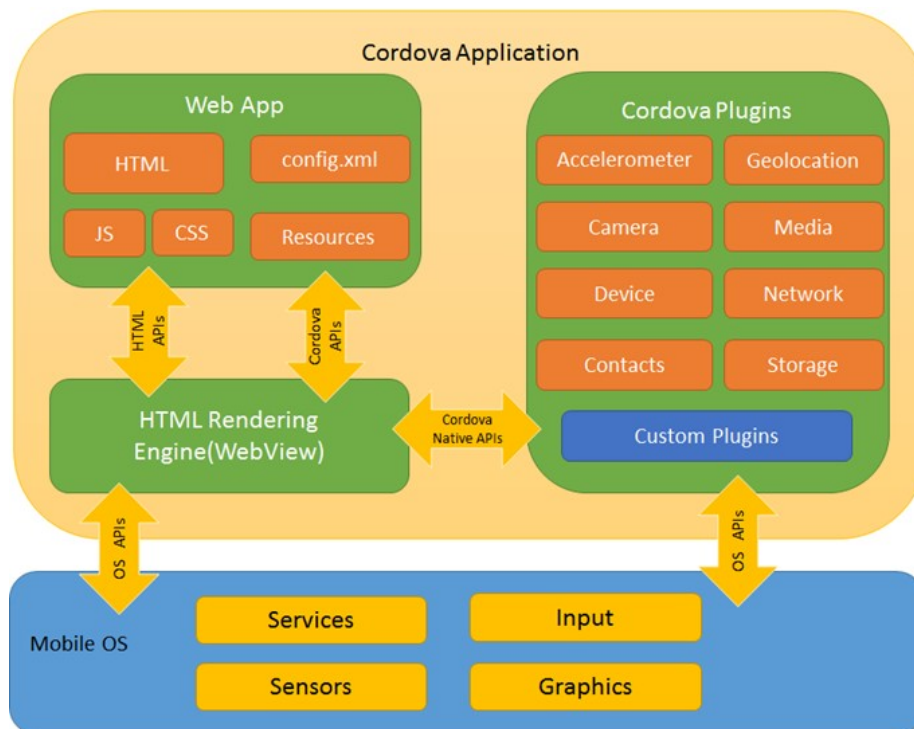


Figura 5. Modelo de arquitectura de Apache Cordova. Fuente: Apache Cordova, s. f.

Mobile Angular UI

Es un marco de desarrollo de aplicaciones móviles de código abierto que combina la implementación de Bootstrap y Angular *framework*. El marco ofrece varios **componentes de interfaz de usuario**, como superposiciones, interruptores, barras laterales, barras de navegación, áreas desplazables, etc. para proporcionar interfaces de usuario sólidas y atractivas para las aplicaciones. Además, admite fastclick.js y overthrow.js para crear experiencias ricas, así como la compatibilidad entre navegadores que permite el manejo automático del código JavaScript para cada navegador (Casimirri, 2016).

JQuery Mobile

Es un marco de desarrollo multiplataforma que se utiliza para crear aplicaciones móviles y web para varios dispositivos. El marco permite ejecutar una **sola versión**

de código para todos los dispositivos. Además, el marco de desarrollo basado en HTML5 viene con varios complementos como Image Slider, Content-Slider, Pop-Up Boxes, etc. que ayudan a crear un diseño interactivo rico en funciones (jQuery Mobile, 2022).

NativeScript

Es un marco de código abierto que se utiliza para crear aplicaciones móviles nativas mediante Angular, Vue.js, TypeScript o JavaScript. El marco le permite crear aplicaciones para Android e iOS a partir de **un único código base**. Las aplicaciones creadas usan las mismas API que en Xcode o Android Studio. Además, el marco se recomienda a aquellos desarrolladores que desean crear aplicaciones para múltiples plataformas en una duración menor y un costo efectivo (NativeScript, s. f.).

Framework7

Es un marco de desarrollo de código abierto que permite desarrollar aplicaciones móviles, de escritorio o web con una perspectiva nativa. El marco le permite usar las herramientas de sus preferencias con HTML, CSS y JavaScript y ofrece el soporte adicional de los componentes Vue.js y React para el desarrollo de aplicaciones. Además, proporciona una **amplia gama de elementos y widgets** de la interfaz de usuario, como hojas de acción, paneles laterales, vistas de lista, elementos de formulario, etc. para que el proceso de desarrollo sea más conveniente y eficiente (Framework7, s. f.).

8.6. Referencias bibliográficas

Android. (s. f.). *Experience even more on Android*. <https://www.android.com/>

Apache Cordova. (s. f.). *Overview*. <https://cordova.apache.org/docs/en/latest/guide/overview/index.html>

Casimirri, M. (2016, noviembre 6.). *Mobile Angular UI*. GitHub. <https://github.com/mcasimir/mobile-angular-ui>

Developer Android. (s. f.) *Android Studio*. <https://developer.android.com/studio>

Developer Apple. (s. f.) *Resources*. <https://developer.apple.com/xcode/resources/>

Framework7. (s. f.) *Build full featured iOS, Android & Desktop apps*. <https://framework7.io/>

Flutter. (s. f. a). *Build apps for any screen*. <https://flutter.dev/>

Flutter. (s. f. b). *Flutter architectural overview*. <https://docs.flutter.dev/resources/architectural-overview#:~:text=Flutter%20is%20a%20cross%2Dplatform,directly%20with%20underlying%20platform%20services>

Ionic Framework. (s. f.). *One codebase. Any platform*. <https://ionicframework.com/>

jQuery Mobile. (s. f.). *A Touch-Optimized Web Framework*. <https://jquerymobile.com/>

Microsoft. (2022, agosto 10). *¿Qué es Xamarin?* <https://docs.microsoft.com/es-es/xamarin/get-started/what-is-xamarin>

Microsoft. (s. f.) *Visual Studio Tools para Xamarin*. <https://visualstudio.microsoft.com/es/xamarin/>

NativeScript. (s. f.). *Empower JavaScript with native APIs*. <https://nativescript.org/>

Phonegap. (s. f.). *Phonegap descontinuado*. <https://blog.phonegap.com/update-for-customers-using-phonegap-and-phonegap-build-cc701c77502c>

React Native. (s. f.). *React Native*. <https://reactnative.dev/>

Sencha. (s. f.). *Sencha Touch*. <https://www.sencha.com/products/touch/>

Native mobile development: A cross-reference for iOS and android native programming

Lewis, S. y Dunn, M. (2019). *Native mobile development: A cross-reference for iOS and android native programming*. O'Reilly Media.

Guía que muestra cómo realizar las tareas de desarrollo más comunes para iOS y Android. La primera parte contiene una lista de tareas comunes independientes de la plataforma, como escribir un archivo en el almacenamiento local o crear una solicitud HTTP. La segunda parte guía a través de la creación de una aplicación básica en cada plataforma.

Learn simply: Android mobile application development using kotlin

Doush, I. A. (2020). *Learn simply: Android mobile application development using Kotlin*. Leanpub.

En este libro, aprenderás en simples pasos cómo desarrollar aplicaciones móviles usando Kotlin. Primero, expone la programación de Kotlin. Después, muestra cómo usar Kotlin para el desarrollo de apps móviles para Android. Sigue un patrón basado en: comenzar con una idea, diseñar la solución y codificar la solución.

1. Un sitio web que se ve y se comporta como si fuera una aplicación móvil se denomina app:

- A. Nativa.
- B. Reactiva.
- C. Web progresiva.
- D. *Responsive*.

2. Relaciona las características con los diversos tipos de apps:

Web responsive	1	A	App con una sola base de código para múltiples plataformas
App nativa	2	B	App con acceso a todo el dispositivo
App híbrida	3	C	App con actualización inmediata
App web progresiva	4	D	Web que se adapta a una visualización en múltiples dispositivos

3. Marca aquella característica que no es propia de las apps sobre IOS:

- A. Compras en la aplicación.
- B. iCloud.
- C. Procesos monotarea.
- D. Servicios de ubicación.

4. La versión *open source* de la solución comercial Adobe Phonegap es:

- A. Apache Cordova.
- B. Xamarin.
- C. Flutter.
- D. Ionic.

5. Relaciona el componente de Android con su ejemplo:

Actividad	1	A	Notificación nivel de carga de la batería es bajo
Servicio	2	B	Datos privados de la app
Receptor de mensajes	3	C	Reproducción de música
Proveedor de contenido	4	D	Pantalla individual con una interfaz de usuario

6. Selecciona aquel lenguaje de programación que no es válido para desarrollar apps para iOS:

- A. Swift.
- B. Kotlin.
- C. Objective-C.
- D. C++.

7. Selecciona aquel lenguaje de programación que Android Studio no soporta:

- A. Swift.
- B. C++.
- C. Java.
- D. Kotlin.

8. Marca cómo se llama la plataforma de código abierto para compilar aplicaciones modernas y con mejor rendimiento para iOS, Android y Windows con .NET:

- A. Flutter.
- B. Ionic.
- C. Xamarin.
- D. React Native.

9. Marca aquella característica que no es propia de las aplicaciones Android:
- A. Se adaptan a diferentes dispositivos.
 - B. Se ejecutan sobre un Linux multiusuario.
 - C. Se ejecutan bajo el principio de mínimo privilegio.
 - D. Proporcionan un único punto de entrada.
10. Una aplicación móvil que comparte la misma base de código para diversos los dispositivos móviles es una app:
- A. Nativa.
 - B. Híbrida.
 - C. Web progresiva.
 - D. *Responsive*.

Plataformas de Desarrollo de Software

Tema 9. Plataformas para pruebas

Índice

Esquema

Ideas clave

- 9.1. Introducción y objetivos
- 9.2. Contexto de las plataformas para pruebas
- 9.3. Valor y utilidad de las plataformas para pruebas
- 9.4. Plataformas para pruebas unitarias
- 9.5. Plataformas para pruebas de integración
- 9.6. Plataformas para pruebas de sistema
- 9.7. Plataformas para pruebas de aceptación
- 9.8. Referencias bibliográficas

A fondo

- ISO/IEC/IEEE 29119 International Standard — Software and systems engineering —Software testing
- JUnit5 y Test Driven Development
- El probador de software autodidacta

Test

PLATAFORMAS PARA PRUEBAS			
CONCEPTOS	VALOR Y UTILIDAD	HERRAMIENTAS	
Probamos para: Asegurar que el software funciona Buscar posibles fallos	Valor en reducción del coste asociado a: - Diseñar las pruebas - Mantener las pruebas - Ejecutar las pruebas - Analizar el resultado - Documentar las pruebas	Múltiples formas de clasificación de las plataformas (seguimos modelo V)	
Validar vs. verificar Verificar es asegurar que funciona sin fallos Validar es que cumple los requisitos del cliente	Pasos para hacer las pruebas: - Identificar un objetivo a probar - Seleccionar las entradas - Calcular el resultado esperado - Configurar el entorno de ejecución - Ejecutar la prueba - Analizar el resultado	Unitarias - JUnit - NUnit - phpUnit - Mocha Integración - SoapUI - Postman - Mockserver - Mockito	
Resultado de una prueba Pass el resultado obtenido está entre los esperados Fail el resultado obtenido no entra en las reglas de validación (no necesariamente es un fallo)	Criterios para seleccionar plataformas: - Coste de licencia / Soporte - Curva de aprendizaje - Informes de prueba - Compatibilidad integración continua - Tecnologías utilizadas	Sistema - Selenium - Jmeter - K6 - LoadNinja Aceptación - Cucumber - Appium - TestProject - TestComplete Aprendemos a utilizar JUnit5 K6 Selenium Cucumber	

9.1. Introducción y objetivos

En este tema se estudian las principales plataformas que nos ayudan a **automatizar** las pruebas del software.

Existen diversas formas de poder **clasificar** estas herramientas al atender las etapas del ciclo de vida, el alcance o los lenguajes y tecnologías de programación a los que se orienta. En este tema, clasificaremos estas soluciones en las que se centran en las pruebas unitaria, las de integración, las de sistema, las de validación y, finalmente, presentaremos otras herramientas que pueden apoyar en varias de las anteriores.

Con el estudio de este tema pretendemos alcanzar los siguientes objetivos:

- ▶ Comprender el valor y la utilidad que tienen las plataformas para la automatización de las pruebas del software.
- ▶ Identificar los criterios principales que nos ayuden a seleccionar las plataformas en cada caso.
- ▶ Conocer las principales plataformas de pruebas unitarias.
- ▶ Conocer las principales plataformas de pruebas de integración.
- ▶ Conocer las principales plataformas de pruebas de sistema.
- ▶ Conocer las principales plataformas de pruebas de aceptación.
- ▶ Conocer algunas de las plataformas y herramientas de pruebas mejor valoradas.
- ▶ Saber utilizar algunas de las principales plataformas de pruebas.

9.2. Contexto de las plataformas para pruebas

El Instituto de Ingenieros Eléctricos y Electrónicos (IEEE) define las pruebas de software como el proceso para **evaluar un sistema** o sus componentes de forma manual o automática para determinar si cumple con los requisitos del usuario o para encontrar las diferencias entre los resultados reales y los resultados esperados (Zein, Salleh y Grund, 2016).

El desarrollo de pruebas de software tiene persigue estos objetivos (Naik y Tripathy, 2011):

- ▶ **Asegurar que el software funciona:** al implementar una unidad de programa, el programador puede asegurar si el código funciona o no en circunstancias normales. La misma idea se aplica también a un sistema completo: una vez que un sistema se ha integrado, los desarrolladores pueden querer probar si el sistema realiza las funciones de forma correcta en su conjunto.
- ▶ **Buscar posibles fallos:** una vez que el equipo de desarrollo asegura que una unidad o el sistema completo funciona hasta cierto punto, con las pruebas es posible reproducir escenarios que lleven al sistema a situaciones no previstas con el objetivo de encontrar posibles fallos.

A menudo se habla del testeo como la validación y verificación, si bien son conceptos que se aplican a un ámbito distinto (International Organization for Standardization, 2022):

- ▶ **Verificación:** este tipo de actividad nos ayuda a evaluar un sistema de software determinando si el producto de una fase de desarrollo dada satisface los requisitos establecidos antes del inicio de dicha fase. Estas actividades se pueden acotar a un producto intermedio, como la especificación de requisitos, especificación de diseño, código, manual de usuario o incluso el producto final.

- ▶ **Validación:** actividad orientada a confirmar que un producto cumple con su uso previsto, es decir, que un producto cumple con las expectativas de sus clientes. En otras palabras, las actividades de validación se centran en el producto final, que se prueba exhaustivamente desde el punto de vista del cliente. La validación establece si el producto cumple con las expectativas generales de los usuarios.

En el contexto de las pruebas del software, el concepto de verificación hace referencia a que el software funciona sin fallos, mientras que la validación se centra en comprobar que el software cumple las necesidades recogidas en los requisitos.

Las pruebas de *software* pueden realizarse de forma manual o automatizada (Valliammai y Sujatha, 2015):

- ▶ Las pruebas **manuales** se realizan interactuando con el software en función de algunos casos de prueba predefinidos. Los casos de prueba proporcionan explicaciones de las características examinadas y los resultados esperados. El proceso de prueba manual es costoso en tiempo y su mayor problema es que requiere una completa reproducción, con el mismo esfuerzo asociado, cada vez que existen cambios en el software para corregir errores o dentro del mantenimiento evolutivo.
- ▶ Las pruebas **automatizadas** hacen que este proceso sea más eficaz, ya que podemos definir casos de prueba solo una vez y ejecutarlos de forma automática muchas veces. Esta característica es fundamental para poder realizar pruebas de regresión que garanticen que el software sigue funcionando como se esperaba cuando realizamos cambios derivados de mantenimientos correctivos o nuevas funcionalidades.

En lo relativo a las posibles plataformas o herramientas de pruebas, existen muchas formas de poder clasificarlas en función de **diversos criterios** como la fase o ámbito

del testeo, las tecnologías, el alcance técnico, el entorno en el que opera, los lenguajes de programación que son de aplicación, el cumplimiento o no de marcos normativos o las licencias de uso entre otros (Abdulwareth y Al-Shargabi, 2021).

La consultora Gartner, en su *Magic Quadrant for Software Test Automation* (Herschmann, Murphy, Scheibmeir, 2019), posiciona a los principales fabricantes de esta forma:

Figure 1. Magic Quadrant for Software Test Automation



Figura 1. Gartner Magic Quadrant for Software Test Automation. Fuente: Herschmann, Murphy, Scheibmeir, 2019.

- **Líderes:** Tricentis, SmartBear, Eggplant y Micro Focus.

- ▶ **Challengers:** Microsoft, IBM y Worksoft.
- ▶ **Visionaries:** CA Technologies y Parasoft.
- ▶ **Niche players:** Ranorex y Froglogic.

En nuestro caso, vamos a hacer uso del **modelo en V** del testeo de software para establecer una posible clasificación de plataformas, de forma que las pruebas se realicen en diferentes niveles que involucren el sistema completo o partes de a lo largo del ciclo de vida de un producto de software. En el modelo de testeo en V, se plantean **cuatro etapas de pruebas** que son conocidas como pruebas unitarias, de integración, de sistema y de aceptación. Los tres primeros niveles de las pruebas son realizados por una serie de diferentes equipos participantes en el desarrollo organización y las pruebas de aceptación son habitualmente realizadas por los clientes. Por ello, hablaremos de plataformas de pruebas unitarias, de integración, de sistema y de aceptación. Si bien alguna de las herramientas podrá ofrecer funcionalidades en varias fases, tal y como refleja el modelo en la Figura 2:

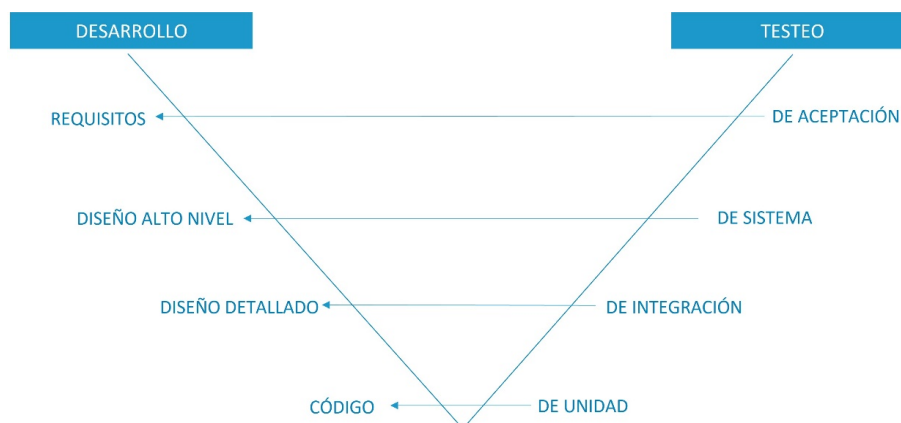


Figura 2. Modelo de testeo en V simple. Fuente: elaboración propia.

9.3. Valor y utilidad de las plataformas para pruebas

El valor y la utilidad de estas herramientas, con carácter general, es **reducir el esfuerzo y coste asociado** a las pruebas, en los que se incluye:

- ▶ El coste de **diseñar** las pruebas: mediante librerías o entornos visuales, podremos definir de una forma más rápida qué queremos probar y generar el código o entorno apropiado para su futura ejecución.
- ▶ El coste de **mantener** las pruebas: en el caso de cambios en el código, podremos redefinir las pruebas y mantenerlas coherentes con cada versión del producto.
- ▶ El coste de **ejecutar** las pruebas: cuando tenemos muchos casos de prueba, las herramientas ayudan a automatizar su ejecución lo que permite establecer prioridades, grupos o incluso dependencias entre ellas.
- ▶ El coste de **analizar el resultado** de cada caso de prueba: una prueba debe tener un resultado esperado y las herramientas nos van a simplificar la forma en la que podemos determinar mediante reglas, expresiones regulares o anotaciones que determinen en qué condiciones daremos la prueba por válida y en cuáles es un posible error.
- ▶ El coste de **documentar** las pruebas: como buena práctica de ingeniería, es necesario documentar las diversas fases del proceso de desarrollo, por lo que las pruebas no son una excepción. Disponer de una documentación que nos permita evidenciar que el software ha sido probado y los resultados obtenidos son una garantía adicional hacia el equipo de desarrollo y el cliente de la calidad esperada del producto.

Para llevar a cabo una prueba, es preciso seguir una serie de pasos como los que se detallan a continuación:

- ▶ Identificar un **objetivo** a probar: el primer paso es definir qué unidad, módulo, sistema, funcionalidad o requisito queremos probar.
- ▶ Seleccionar las **entradas**: el segundo paso, dentro del diseño del caso de prueba, es definir los posibles valores de entrada que serán representativos para llevar a cabo la prueba, basándonos en los requisitos de usuario o técnicos en cada caso.
- ▶ Calcular el **resultado esperado**: el tercer paso es identificar qué valor o conjunto de valores harán que demos por correcta (*pass*) o por errónea (*fail*) una prueba.
- ▶ Configurar el **entorno** de ejecución del programa: como cuarto paso, debemos disponer de un entorno que permita la ejecución de las pruebas.
- ▶ **Ejecutar** la prueba: el quinto paso es la ejecución de la prueba con las entradas seleccionadas y obtener el resultado de la ejecución.
- ▶ **Analizar** el resultado de la prueba: el último paso es analizar el resultado obtenido con el esperado, para ver si es correcta o errónea.

El resultado de una prueba puede ser correcto (*pass*) o erróneo (*fail*).

Que una prueba falle no significa que exista un error en el software, sino que el resultado obtenido en la prueba no satisface las reglas de validación establecidas. Es necesario revisar si en efecto se trata de un error o es un escenario no contemplado en la prueba.

A la hora de seleccionar las plataformas de pruebas, cada organización debe establecer sus propios criterios, basados en múltiples perspectivas (Raulamo-Jurvanen, Mäntylä, y Garousi, 2017), en los que los principales son:

- ▶ El **coste** de licenciamiento y soporte.
- ▶ La curva de **aprendizaje** necesaria, condicionada por la cantidad de documentación o formación existente.
- ▶ La generación de **informes** de prueba.
- ▶ La compatibilidad con entornos de **integración** y despliegue **continuos**.
- ▶ El nivel de conocimiento de **programación** requerido y las tecnologías o lenguajes soportados.

Las plataformas y herramientas de apoyo a la automatización de las pruebas nos pueden reducir el esfuerzo, pero no son suficientes, ya que requieren de un proceso de pruebas que defina qué hay que probar, cómo probar, cuándo probar, cómo medir la calidad y eficiencia de las pruebas y cómo gestionar las incidencias o errores detectados.

Puedes aprender más sobre esto en *ISO/IEC/IEEE 29119-2:2021 Software and systems engineering — Software testing — Part 2: Test processes* en el siguiente enlace <https://www.iso.org/standard/79428.html>

9.4. Plataformas para pruebas unitarias

La prueba unitaria es una parte del proceso de desarrollo de software en el que las **partes comprobables más pequeñas** de una aplicación, llamadas unidades, se analizan de forma individual e independiente para garantizar un funcionamiento adecuado. Esta actividad de pruebas la realizan habitualmente los desarrolladores de software y, a veces, el personal de control de calidad durante el proceso de desarrollo. El objetivo principal de las pruebas unitarias es **aislar el código escrito** para probar y determinar si funciona según lo previsto.

Las pruebas unitarias son un paso importante en el proceso de desarrollo porque, si se realizan correctamente, pueden ayudar a **detectar fallas tempranas** en el código que pueden ser más difíciles de encontrar en etapas de prueba posteriores. Al estar centradas en unidades de código, las plataformas de apoyo a la definición o ejecución de las pruebas son específicas para cada tecnología o lenguaje de programación que estemos utilizando.

Algunas de las herramientas más populares para facilitar la definición y ejecución de pruebas unitarias son:

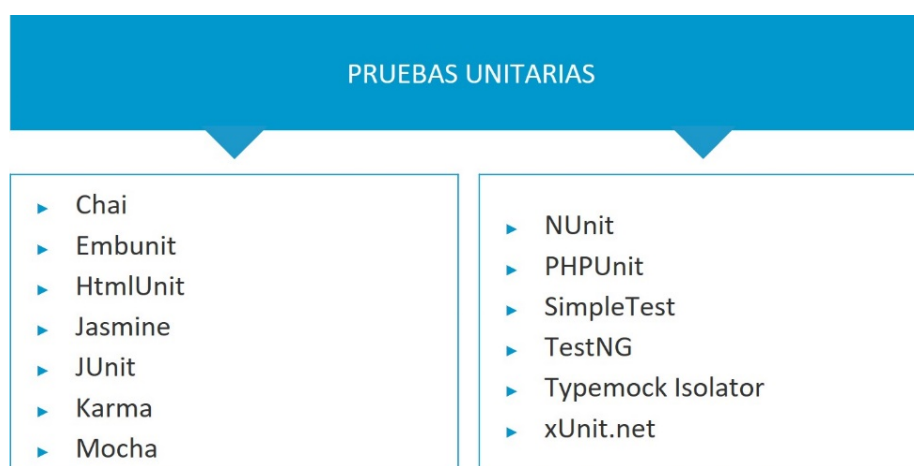


Tabla 1. Plataformas para pruebas unitarias. Fuente: elaboración propia.

Chai

Es una **biblioteca de aserciones** para el Node.js y el navegador que se puede combinar maravillosamente con cualquier marco de prueba de JavaScript. Cuenta con varias interfaces que permiten al desarrollador elegir la más cómoda.

Embunit

Es un marco de prueba de unidad gratuito para software en sistemas embebidos para desarrolladores y para evaluadores de aplicaciones de software escritas en C o C++. Su diseño es algo similar a JUnit, define los casos de prueba en formato estructurado para generar código fuente. **Reduce el retrabajo** de las pruebas unitarias, ya que los casos de prueba relacionados se almacenan en el mismo conjunto de pruebas y el resultado final se genera en formato XML.

HtmlUnit

Es una biblioteca Java de código abierto que contiene un **navegador sin interfaz** de usuario para programas Java. Esta herramienta es compatible con JavaScript y proporciona funciones visuales como formularios, enlaces, tablas, etc. Admite protocolos como HTTP, HTTPS junto con *cookies*, métodos de envío como GET, POST y servidor *proxy*. Permite probar aplicaciones web que se utilizan dentro de plataformas como JUnit y TestNG, usando el motor de JavaScript Mozilla Rhino.

Jasmine

Es un marco de pruebas basadas en el comportamiento (*behavior-driven development*) para JavaScript. No depende de ningún otro marco de JavaScript ni de un DOM y tiene una sintaxis limpia que facilita la escritura de las pruebas. Admite **pruebas de código de interfaz de usuario** a través de una extensión *front-end* de Jasmine llamada Jasmine-jQuery (Hahn, 2013).

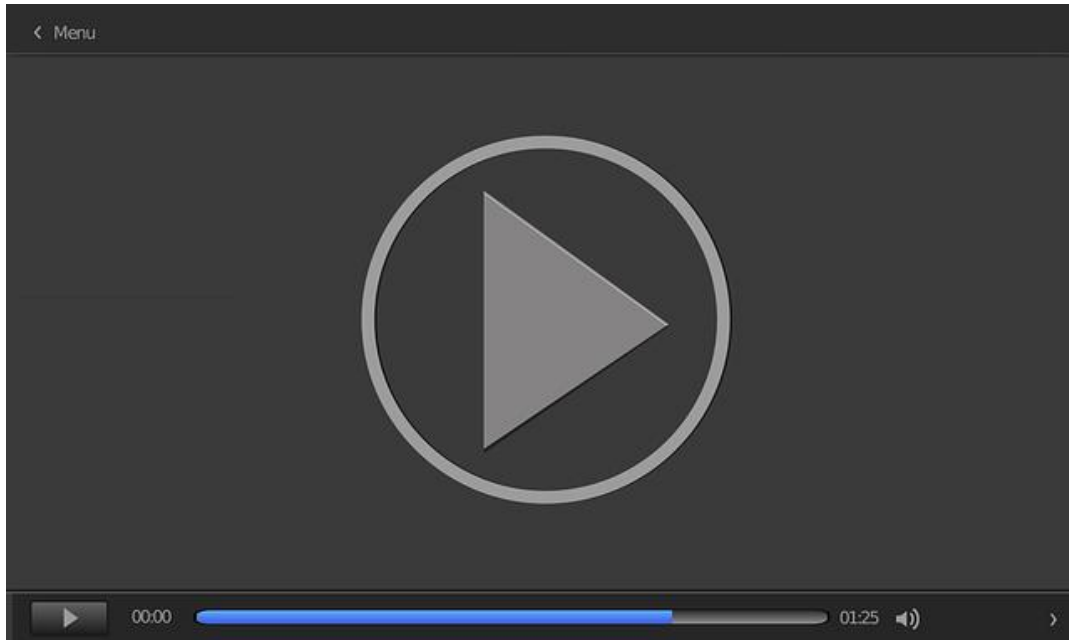
JUnit

Es un marco de pruebas unitarias de código abierto diseñado para el lenguaje de programación Java. Con base en anotaciones, ayuda a escribir código de prueba de manera fácil y rápida (Gulati y Sharma, 2017).

Anotación	Descripción
@Test	Indica que un método es un método de prueba.
@ParameterizedTest	Indica que un método es una prueba parametrizada.
@RepeatedTest	Indica que un método es una plantilla de prueba para una prueba repetida.
@TestFactory	Indica que un método es una fábrica de pruebas para pruebas dinámicas.
@TestTemplate	Indica que un método es una plantilla para casos de prueba diseñada para invocarse varias veces según la cantidad de contextos de invocación devueltos por los proveedores registrados.
@TestClassOrder	Se utiliza para configurar el orden de ejecución de la clase de prueba para las clases de prueba @Nested en la clase de prueba anotada.
@TestMethodOrder	Se utiliza para configurar el orden de ejecución del método de prueba para la clase de prueba anotada (similar a @FixMethodOrder de JUnit 4).
@TestInstance	Se utiliza para configurar el ciclo de vida de la instancia de prueba para la clase de prueba anotada.
@DisplayName	Declara un nombre para mostrar personalizado para la clase de prueba o el método de prueba.
@DisplayNameGeneration	Declara un generador de nombre para mostrar personalizado para la clase de prueba.
@BeforeEach	Indica que el método anotado debe ejecutarse antes de cada método @Test, @RepeatedTest, @ParameterizedTest o @TestFactory en la clase actual (análogo a @Before de JUnit 4).
@AfterEach	Indica que el método anotado debe ejecutarse después de cada método @Test, @RepeatedTest, @ParameterizedTest o @TestFactory en la clase actual (análogo a @After de JUnit 4).
@BeforeAll	Indica que el método anotado debe ejecutarse antes que todos los métodos @Test, @RepeatedTest, @ParameterizedTest y @TestFactory en la clase actual (análogo a @BeforeClass de JUnit 4).
@AfterAll	Indica que el método anotado debe ejecutarse después de todos los métodos @Test, @RepeatedTest, @ParameterizedTest y @TestFactory en la clase actual (análogo a @AfterClass de JUnit 4).
@Nested	Indica que la clase anotada es una clase de prueba anidada no estática. Los métodos @BeforeAll y @AfterAll no se pueden usar directamente en una clase de prueba @Nested a menos que se use el ciclo de vida de la instancia de prueba "por clase".
@Tag	Se utiliza para declarar etiquetas para filtrar pruebas, ya sea a nivel de clase o de método (similar a los grupos de prueba en TestNG o Categorías en JUnit 4).
@Disabled	Se utiliza para deshabilitar una clase de prueba o un método de prueba (análogo a @Ignore de JUnit 4).
@Timeout	Se utiliza para fallar una prueba, una fábrica de pruebas, una plantilla de prueba o un método de ciclo de vida si su ejecución excede una duración determinada.
@ExtendWith	Se utiliza para registrar extensiones de forma declarativa.
@RegisterExtension	Se utiliza para registrar extensiones mediante programación a través de campos.
@TempDir	Se utiliza para proporcionar un directorio temporal a través de la inyección de campo o la inyección de parámetros en un método de ciclo de vida o método de prueba.

Tabla 2. Anotaciones de Junit. Fuente: elaboración propia.

En el vídeo *JUnit5, pruebas sobre la Java Virtual Machine*, aprenderemos a utilizar el entorno JUnit5 para automatizar las pruebas sobre la máquina virtual de Java.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=c77f5deb-e890-4f63-bde8-aec300cd9c14>

Karma

Es un marco de pruebas de código abierto que proporciona un entorno de creación de pruebas productivo para JavaScript y que ejecuta pruebas en **dispositivos reales**. Facilita la depuración fácil y la integración eficiente con Jenkins, Travis o Semaphore.

Mocha

Es un marco de prueba de JavaScript rico en funciones que se ejecuta en Node.js y en el navegador, lo que hace que las pruebas asíncronas sean simples. Las pruebas de Mocha se ejecutan en serie, lo que permite informes flexibles y precisos,

al tiempo que asigna excepciones no detectadas a los casos de prueba correctos.

NUnit

Es un marco de pruebas unitarias para cualquiera de los lenguajes incluidos en la plataforma .NET. Su funcionamiento es similar a JUnit, incluyendo pruebas basadas en datos que se pueden ejecutar en paralelo y cuenta con un Console Runner para cargar y ejecutar pruebas (Thomas, Hunt. y Hargett, 2007).

PHPUnit

Es un marco de prueba orientado al programador para el lenguaje PHP. Es una instancia de la arquitectura xUnit para marcos de pruebas unitarias.

SimpleTest

Es un marco de pruebas unitarias de código abierto dedicado al lenguaje de programación PHP. Admite SSL, formularios, *proxies* y autenticación básica. Las clases de casos de prueba se generan ampliando desde las clases de prueba base junto con los métodos y códigos. Incluye una **funcionalidad para transformar** casos de prueba en *scripts* de prueba ejecutables.

TestNG

Es un marco de prueba de automatización de código abierto para el lenguaje de programación Java. Esta herramienta está fuertemente influenciada por *JUnit* con pruebas simultáneas, soporte de anotaciones, admite pruebas parametrizadas y basadas en datos junto con un modelo de ejecución potente y una configuración de prueba flexible (Jones, 2018).

TypemockIsolator

Es un marco gratuito de código abierto para probar el código del sistema para C y C++ principalmente para Windows. Reduce el consumo de tiempo para la **corrección de errores y la entrega de valor**. Contiene API simples y métodos de entrada sin cambiar el código heredado.

xUnit.net

Es una herramienta gratuita para pruebas unitarias, de código abierto y centrada en la comunidad para .NET Framework. Escrito por el inventor original de NUnit v2, es la última tecnología para pruebas unitarias de C#, F#, VB.NET y otros lenguajes .NET, funciona con extensiones de los entornos de desarrollo integrado (IDE) como ReSharper, CodeRush, TestDriven.NET y Xamarin (Adewole, 2018).

9.5. Plataformas para pruebas de integración

Las pruebas de integración son un tipo de pruebas de software en las que las diferentes unidades, módulos o componentes de una aplicación de software se prueban como un **sistema combinado**. Cada una de sus partes o módulos han podido ser codificados por diferentes programadores. El objetivo de las pruebas de integración es probar las interfaces entre los módulos y exponer cualquier defecto que pueda surgir cuando estos componentes están integrados y necesitan interactuar entre sí.

La principal diferencia entre las pruebas unitarias y las pruebas de integración es que en las pruebas unitarias se prueban módulos individuales. En las pruebas de integración estos módulos se combinan y prueban como **una sola unidad** para verificar la funcionalidad de la aplicación en general.

Si la arquitectura del sistema es monolítica, es posible hacer uso de las plataformas y herramientas de pruebas unitarias para desarrollar pruebas de integración. De esta forma, se modifica el ámbito de la prueba en el sentido de que lo que se cubre no es una pieza de código está bien desarrollada, sino sus interfaces de entradas y salidas con el resto de los módulos con los que interactúa responden al objetivo perseguido.

En arquitecturas distribuidas, los diferentes módulos o subsistemas **interactúan a partir de las API** (*application programming interfaces*) definidos con lenguajes o estándares, por lo que en esta sección nos centraremos en aquellas plataformas que de forma específica nos ayudan a generar pruebas a partir de esta distinción de protocolos o a emular el comportamiento de esos sistemas.

Algunas de las herramientas más populares para facilitar la definición y ejecución de pruebas de integración de API son:

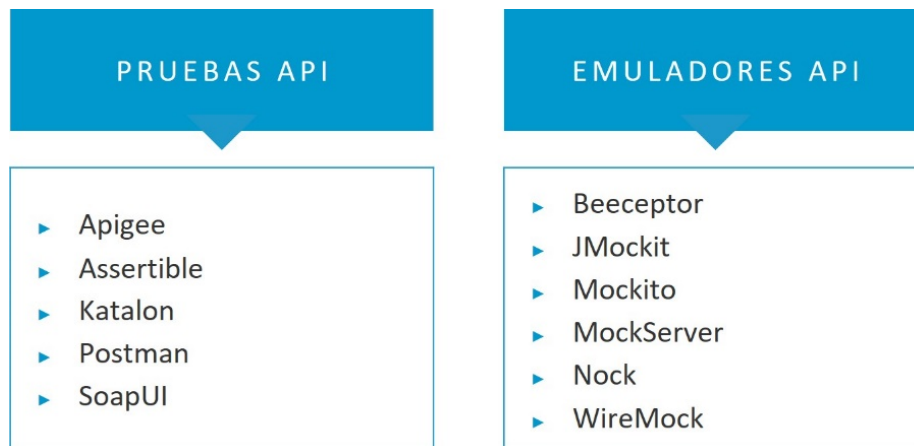


Tabla 3. Plataformas para pruebas de integración. Fuente: elaboración propia.

Pruebas API

Apigee

Es un software de prueba de API del ecosistema de herramienta de Google Cloud, basado en JavaScript que permite a los desarrolladores y evaluadores acceder a sus funciones a través de diferentes editores. Esta herramienta es compatible con API que contienen **grandes cantidades de datos**, por lo que son ideal para negocios digitales robustos y complejos. Soporta pruebas de rendimiento mediante el análisis del tráfico de la API, los tiempos de respuesta y las posibles tasas de error y también permite la creación de *proxies* a partir de OpenAPI Specification.

Assertible

Se trata de una de las principales herramientas de prueba de API con un **alto enfoque en la confiabilidad**, considerando que admite pruebas de API en cada paso que va desde la integración continua hasta la canalización de entrega. Soporta la integración con GitHub, Zapier y Slack, y valida las respuestas HTTP con aserciones.

Katalon

Es una popular herramienta de automatización de pruebas para API, así como aplicaciones web y aplicaciones móviles y de escritorio. Admite solicitudes SOAP y REST y proporciona múltiples funciones y comandos de parametrización. Sigue un **enfoque basado en datos**, admite su vínculo con flujos de integración y despliegue continuos y permite a los probadores ejecutar pruebas exploratorias y automatizadas.

Postman

Aunque inicialmente creada como un complemento para Google Chrome, en la actualidad también se puede usar con Windows y Mac y es una excelente herramienta para probar los servicios de API, incluida la supervisión, la depuración, la ejecución de solicitudes, etc. No requiere que los desarrolladores o evaluadores aprendan un nuevo idioma y además **admite varios formatos** (como Swagger y RAML). Además, también proporciona herramientas integradas, espacios de trabajo y colecciones.

SoapUI

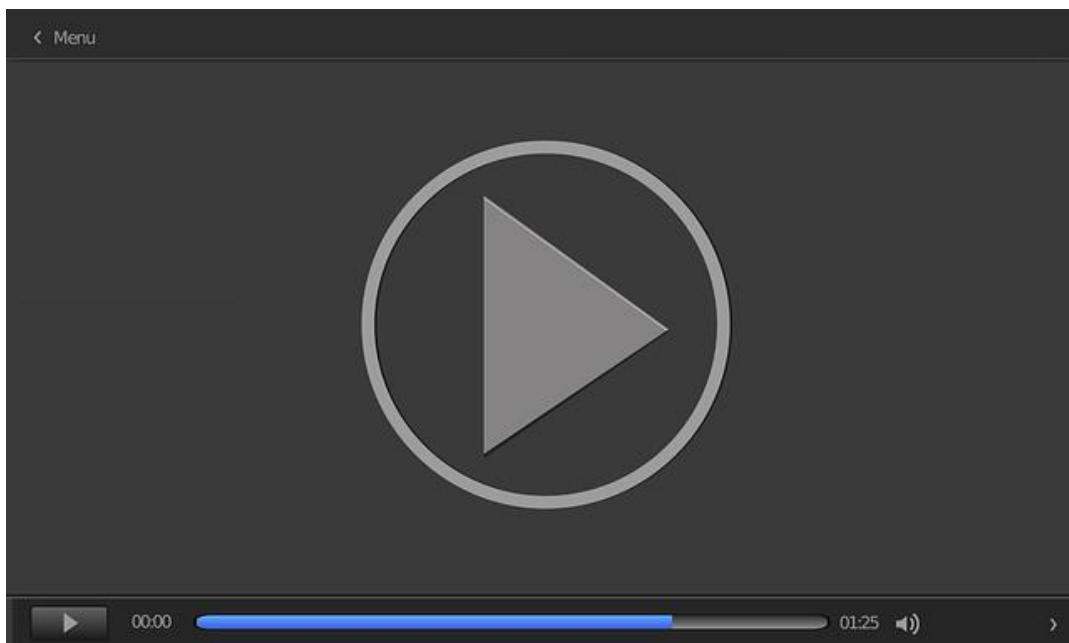
Es una herramienta dedicada a las pruebas de API que permite ejecutar pruebas sobre las API de SOAP y también sobre los servicios web REST, considerando que es *headless*. A menudo, es una de las herramientas preferidas por los equipos de prueba, ya que les permite crear **escenarios complejos** y admite aún más las

pruebas asíncronas. Además, también se destaca por sus excelentes pruebas basadas en datos y su facilidad de uso.

Para automatizar la generación de pruebas sobre un API REST, también podemos hacer uso de la librería Swagger Codegen, que permite generar automáticamente la librería de invocación a un sistema especificado mediante. Puedes verlo en el proyecto Swagger Codegen a continuación.

Accede al contenido a través del siguiente enlace: <https://github.com/swagger-api/swagger-codegen>

En este vídeo, *Cucumber y OpenAPI, testeando servicios distribuidos*, aprenderemos a automatizar las pruebas sobre interfaces descritos con OpenAPI usando Cucumber.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=41e7870d-a308-42a1-b14b-aec300cd9b69>

Emuladores API

Cuando queremos hacer pruebas de integración es posible que no tengamos acceso al código o a un entorno de despliegue de todos los módulos. Este escenario es típico cuando se realiza un **desarrollo en paralelo** al tener equipos que se encargan de la parte del servidor (*back-end*) y otros de la parte del cliente (*front-end*), así como en sistemas que requieren de integración de servicios externos (por ejemplo, conectarnos a un sistema bancario o a una pasarela de pago de un tercero).

Para **evitar los cuellos de botella** del desarrollo (tener que esperar a que el *back-end* esté desarrollado para poder crear el *front-end*) o para poder generar escenarios de prueba específicos (por ejemplo, cómo se comportaría mi sistema si el módulo externo no responde al protocolo establecido), surge el concepto de emular, simular o *mockear* un sistema al crear una **versión falsa** de un servicio externo o interno que pueda reemplazar al real, lo que ayuda a que las pruebas se ejecuten de manera más rápida y confiable.

Algunas de las herramientas más populares para facilitar esta emulación de partes de un sistema son:

Beeceptor

Es una herramienta en línea gratuita para simular una interacción API REST utilizando cualquier solicitud HTTP. Podemos personalizar las respuestas para simular prácticamente cualquier respuesta o situación de falla. También ofrece una excelente funcionalidad para **simular la latencia** en las API posteriores: podemos simular la latencia de transmisión y los tiempos de espera, lo cual es un excelente caso de uso para muchas API y, a menudo, se pasa por alto durante las pruebas.

JMockit

Es una herramienta de código abierto para pruebas unitarias con la colección de herramientas y API. Los desarrolladores pueden usar estas herramientas y API para

escribir pruebas usando TestNG o JUnit. Se plantea como una **alternativa** al uso convencional de objetos simulados (*mock objects*), ya que proporciona tres tipos de cobertura de código como la cobertura de línea, la cobertura de ruta y la cobertura de datos.

Mockito

Se trata de una librería en Java que, combinada con JUnit, facilita el desarrollo de pruebas de integración al emular partes del sistema. A diferencia de las anteriores, muy orientada al testeo de API en sistemas distribuidos, esta herramienta se centra en la **simulación de módulos** para arquitecturas monolíticas (Grzejszczak, 2014).

MockServer

Es una biblioteca que lo ayuda a emular API en cuestión de segundos, con base en la creación de las respuestas HTTP simuladas en una colección de archivos simulados y los atiende como si vinieran de una **API real**. De esta forma, el equipo puede desarrollar el lado del cliente sin necesidad de esperar a que el *back-end* está realmente listo o no.

Nock

Es una biblioteca HTTPS diseñada para **replicar y simular servidores y expectativas** en Node.js. Funcionalmente, Nock hace esto anulando las funciones `http.request` y `http.ClientRequest` al interceptar solicitudes y responder con una respuesta simulada específica mediante el uso de interceptores.

WireMock

Es un simulador de API basadas en HTTP. Puede considerarse como una herramienta de **virtualización de servicios** o un servidor simulado. Permite mantener la productividad del equipo de desarrollo cuando una API de la que depende no existe o no está completa. Admite pruebas de casos extremos y modos de falla que la API real no producirá de manera confiable.

9.6. Plataformas para pruebas de sistema

Las pruebas a nivel de sistema incluyen un **amplio espectro**, como pruebas de funcionalidad, pruebas de seguridad, pruebas de robustez, pruebas de carga, pruebas de estabilidad, pruebas de estrés, pruebas de rendimiento y pruebas de confiabilidad, entre otras.

En entornos en los que tenemos una interfaz con el usuario, ya sea de escritorio, web o aplicación móvil, las pruebas de interacción generalmente se incluyen en este nivel como **pruebas funcionales del sistema**. Algunas de las plataformas que ayudan a realizar pruebas de interfaz de usuario son:

PRUEBAS UI	PRUEBAS CARGA
▶ Selenium ▶ SeleniumIDE ▶ LabdaTest ▶ Parasoft Selenic ▶ Abbot Java GUI Test Framework ▶ Autolt ▶ FlaUI ▶ WinAppDriver	▶ Apache JMeter ▶ SmartMeter.io ▶ LoadNinja ▶ K6 ▶ StresStimulus ▶ WebLOAD ▶ NeoLoad ▶ Kobiton

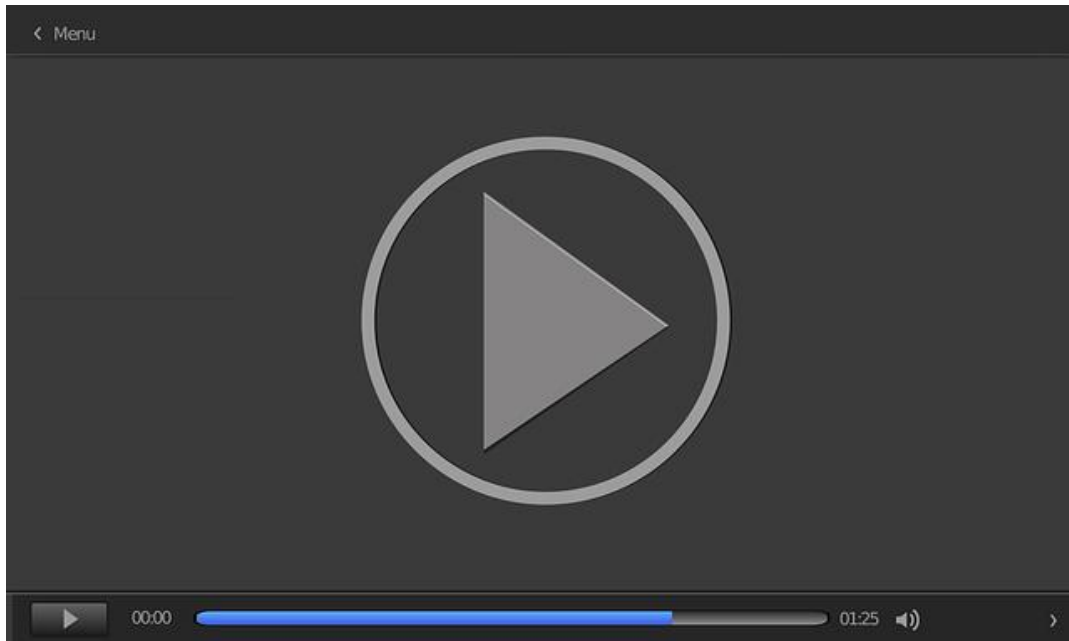
Tabla 4. Plataformas para pruebas de sistema. Fuente: elaboración propia.

Selenium

Es una librería con un conjunto de utilidades que facilita la generación de casos de prueba para aplicaciones web. Nos permite definir **casos de prueba** con un mayor nivel de abstracción y, de forma combinada con JUnit, lleva a cabo la ejecución automática de las mismas con varios navegadores (Gundecha y Avasarala, 2018).

En este vídeo, *Selenium: testeando interfaces de usuario web*, aprenderemos a

automatizar las pruebas sobre interfaces de usuario utilizando Selenium. Accede al vídeo a través del aula virtual



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=831ca218-8d7d-4658-ba38-aec300cd9aa9>

Selenium IDE

Es un proyecto de código abierto que aprovecha el poder de Selenium en un complemento de Chrome y lo pone a disposición de forma gratuita. Comenzar con Selenium IDE no requiere configuración adicional más que instalar la extensión en el navegador, alineándose con la filosofía impulsora del proyecto de proporcionar una herramienta que sea **fácil de usar y brinde retroalimentación instantánea**. Mediante la extensión, podemos reproducir un caso de prueba de forma manual y se podrá exportar en un *script*, lo que la convierte en una herramienta increíblemente poderosa para la creación de pruebas de interfaz de usuario.

LabdaTest

Es una herramienta comercial que permite escalar las pruebas de Selenium en la nube al utilizar miles de navegadores de escritorio y móviles, lo que genera un entorno de ejecución paralelo en el caso de querer **asegurar el funcionamiento** en toda la gama de navegadores y dispositivos.

Parasoft Selenic

Es una solución comercial orientada a usuarios de Selenium que dedican mucho esfuerzo a la hora de **crear y mantener** sus pruebas de interfaz de usuario. Su grabadora inteligente facilita la creación de pruebas de Selenium que utilizan el modelo de objetos de página y se puede integrar en el flujo de integración y distribución continuas (CI/CD).

Abbot Java GUI Test Framework

Es un marco de código abierto que se utiliza para probar la interfaz gráfica de usuario (GUI) de Java proporcionando pruebas unitarias y pruebas funcionales para AWT y SWING.

Autolt

Es un lenguaje de secuencias de comandos gratuito con estructuras básicas que se utilizan para **automatizar la interfaz gráfica** de usuario de Windows y las secuencias de comandos generales.

FlaUI

Es una biblioteca .NET que ayuda con las pruebas de interfaz de usuario automatizadas de las aplicaciones de Windows (Win32, WinForms, WPF, Store Apps, etc.). Se basa en las **bibliotecas nativas de automatización** de la interfaz de usuario de Microsoft y, por lo tanto, es una especie de contenedor que las rodea.

WinAppDriver

Es un servicio similar a Selenium para aplicaciones de Windows. Este servicio admite la prueba de aplicaciones de la plataforma universal de Windows (UWP), Windows Forms (WinForms), Windows Presentation Foundation (WPF) y Classic Windows (Win32) en PC con Windows.

Para poder satisfacer los requisitos no funcionales de rendimiento, es necesario desarrollar pruebas que generen carga en el sistema y lo lleven a situaciones de estrés.

Algunas de las plataformas que ayudan a realizar pruebas de rendimiento son:

Apache JMeter

Es una herramienta de código abierto que se puede utilizar para pruebas de rendimiento y carga para analizar y medir el rendimiento de una variedad de servicios. Es una de las mejores herramientas de prueba de carga que se utiliza principalmente como herramientas de prueba de carga de sitios web para aplicaciones de servicios web (Gomes Rodrigues, Demion y Mouawad, 2019).

SmartMeter.io

Es una alternativa a JMeter y tiene como objetivo **solucionar sus inconvenientes**. Es una de las mejores herramientas de prueba de carga que permite la creación sencilla de escenarios de prueba sin secuencias de comandos utilizando el llamado grabador, pero aún le permite realizar ediciones avanzadas de la prueba. También sobresale en los informes de prueba y hace uso de funciones como la evaluación automática de criterios de prueba, la comparación de ejecuciones de prueba y el análisis de tendencias. Es totalmente compatible con la integración de CI/CD y está disponible para Windows, macOS y Linux.

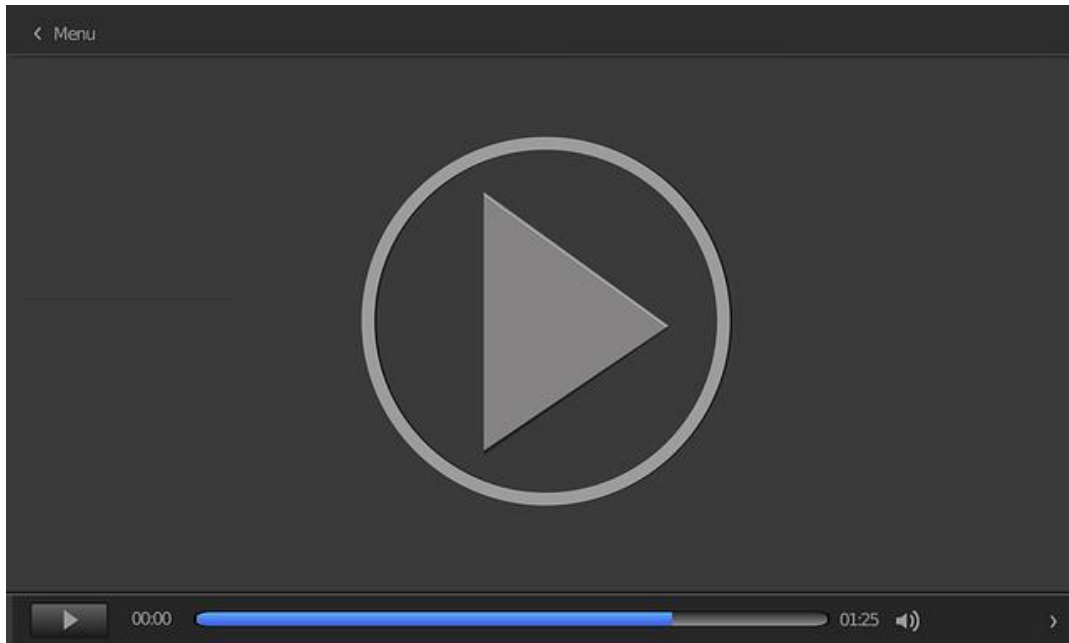
LoadNinja

Permite crear rápidamente pruebas de carga sofisticadas sin secuencias de comandos, reducir el tiempo de prueba en un 50 %, reemplazar los emuladores de carga con navegadores reales y obtener métricas procesables basadas en navegadores, todo a la velocidad de un ninja. Puede capturar fácilmente interacciones del lado del cliente, depurar en tiempo real e identificar problemas de rendimiento de inmediato.

K6

Es una herramienta de prueba de carga gratuita y de código abierto que tiene como objetivo hacer que la prueba de rendimiento sea una **experiencia productiva y agradable**. Se trata de una herramienta de línea de comandos disponible para Windows, Linux y Mac. Alternativamente, hay una imagen oficial de Docker. La herramienta admite secuencias de comandos y las pruebas k6 están escritas en JavaScript, lo que brinda la posibilidad de escribir sus pruebas en uno de los lenguajes de programación más conocidos.

En este vídeo, *K6* y *openAPI*, *pruebas de carga de servicios distribuidos*, aprenderemos a utilizar K6 como plataforma para automatizar pruebas de carga sobre sistemas distribuidos.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=72064c22-07b8-4409-a596-aec300cd9b50>

StresStimulus

Se orienta a escenarios de aplicaciones que son **difíciles de probar** con otras herramientas. Corrige automáticamente los errores de reproducción gracias a su autocorrelación patentada. Registra las acciones del usuario y las reproduce para emular patrones de uso variables. También supervisa el impacto de la carga en la capacidad de respuesta de las aplicaciones y la infraestructura del servidor. El analizador de resultados encuentra errores de concurrencia ocultos que no quedaron expuestos en las pruebas funcionales e incluye métricas detalladas de rendimiento de la aplicación.

WebLOAD

Es una solución de prueba de carga a escala empresarial capaz de generar **escenarios de carga reales y confiables**, incluso para los sistemas más complejos, con análisis inteligentes que brindan información sobre el rendimiento. Se basa en una plataforma flexible con soporte integrado para cientos de tecnologías e integración con muchas herramientas, desde canalizaciones de CI/CD hasta monitoreo.

Neoload

Es una herramienta de prueba de **estrés y carga** de alta eficiencia y bajo precio que se utiliza para medir el rendimiento de las aplicaciones web y móviles. Simula el tráfico a través de usuarios virtuales para determinar el rendimiento de la aplicación bajo carga y analizar los tiempos de respuesta de la transacción e identificar la cantidad de usuarios simultáneos que Internet, la intranet o la aplicación móvil pueden manejar. Una de sus ventajas es que el trabajo del generador y evaluador de las pruebas es más fácil en comparación con otras herramientas.

Kobiton

Ofrece **opciones flexibles** de acceso a dispositivos reales que incluyen una nube pública, una nube privada y local. Los usuarios pueden integrar pruebas de dispositivos reales con soluciones de pruebas de carga y estrés. Esto se puede hacer manualmente, con automatización a través de soluciones basadas en *scripts* como Appium y utilizando el motor sin *scripts* impulsado por su inteligencia artificial (IA). Da soporte para analizar las métricas de rendimiento específicas de la aplicación y el dispositivo y se integra fácilmente con soluciones como NeoLoad o JMeter.

9.7. Plataformas para pruebas de aceptación

Las pruebas de aceptación son aquellas realizadas generalmente por usuarios, clientes o quienes los representan para determinar si el software **generado cumple las especificaciones**. Son las encargadas de validar el sistema y, en ocasiones, determinan el hito de que el cliente acepte o no una entrega del software.

En general, cuando nos referimos a pruebas de aceptación estamos hablando de **validar un sistema**, es decir, comprobar que el software desarrollado es conforme a unos requisitos o necesidades del cliente o usuario. No es posible diseñar pruebas de aceptación sin conocer esas especificaciones, por lo que la tendencia en la ingeniería del software ha sido anticipar la escritura de los casos de prueba de validación antes de diseñar y construir el software y utilizar lenguajes que puedan ser comprensibles por el personal técnico y por el usuario o cliente.

Dentro de las pruebas de aceptación vamos a incluir aquellas que permiten la **ejecución de escenarios de pruebas** a partir de la definición de especificaciones en lenguajes específicos de dominio que pueden ser automáticamente procesados, las que facilitan la automatización de la interacción con el usuario, así como aquellas suites o conjunto de herramientas que dan soporte a todo el ciclo de vida de las pruebas:

PRUEBAS SOBRE GHERKIN	PRUEBAS APPS MÓVILES	PLATAFORMAS TODO CICLO DE VIDA DE PRUEBAS	
▶ Cucumber ▶ SpecFlow ▶ Jbehave	▶ Appium ▶ UI Automator ▶ iOS-driver ▶ Selendroid ▶ Frank ▶ RobusTest	▶ Tricentis ▶ SmartBear TestComplete ▶ Eggplant ▶ QMetry Automation Studio (QAS) ▶ TestProject	▶ Micro Focus UTF ▶ Ranorex Studio ▶ Worksoft Certify ▶ Froglogic Squish GUI Tester

Tabla 5. Plataformas de pruebas de aceptación. Fuente: elaboración propia.

En el contexto de las pruebas del software, las pruebas de aceptación requieren tener unas especificaciones o requisitos que establezcan qué es lo que se esperaba que hiciera el software.

El desarrollo dirigido por comportamiento (*behavioral driven development*, BDD) es un enfoque de desarrollo de software que ha evolucionado desde el desarrollo dirigido por las pruebas (*test driven development*, TDD). Se diferencia por estar **escrito en un lenguaje compartido**, lo que mejora la comunicación entre los equipos tecnológicos y no tecnológicos y las partes interesadas. En ambos enfoques de desarrollo, las pruebas se escriben antes que el código, pero en BDD, las pruebas están más centradas en el usuario y se basan en el comportamiento del sistema (Smart, 2014).

La gran ventaja del enfoque de BDD es el uso de **un lenguaje común** que es fácilmente comprensible por todas las partes. A nivel práctico, los requisitos se convierten en historias de usuario que ayudan a comprender cómo se desarrolla cada funcionalidad. Seguidamente, se definen los escenarios que permiten comprender todas las situaciones en las que se puede desarrollar la historia de usuario, en lugar de un caso concreto (Nagy y Rose, 2018). Para describir un escenario de prueba se utiliza la fórmula *Given-When-Then* (Nagy y Rose, 2021), de esta forma:

- ▶ Ante (*given*) un escenario determinado
- ▶ Cuando (*when*) se lleva a cabo una acción
- ▶ Entonces (*then*) este debería ser el resultado.

El lenguaje más popular para describir estos escenarios es **Gherkin**, que soporta más de setenta lenguajes y se basa en las siguientes anotaciones (Cucumber, s. f.):

Anotación	Descripción
Feature	Proporciona una descripción de alto nivel de una función de software y para agrupar escenarios relacionados. Siempre debe ser la primera palabra en un documento Gherkin seguida de : y un texto breve que describa la característica.
Rule	Representa una regla de negocio que debe implementarse y se usa para proporcionar información adicional para una característica.
Example	Representa un ejemplo concreto que ilustra una regla de negocio y consiste en una lista de pasos. Cada paso comienza con Given, When, Then, And o But.
Given	Pasos que se utilizan para describir el contexto inicial del sistema: la escena del escenario. Por lo general, es algo que sucedió en el pasado.
When	Pasos que se utilizan para describir un evento o una acción. Puede ser una persona que interactúa con el sistema o puede ser un evento desencadenado por otro sistema.
Then	Pasos que se utilizan para describir un resultado o resultado esperado.
And	Paso que añade un elemento más al Given, Then o When anterior.
But	Paso que añade un elemento más, semánticamente queriendo decir lo contrario, al Given, Then o When anterior.
Background	Permite agregar algo de contexto a los escenarios que le siguen. Puede contener uno o más pasos Given, que se ejecutan antes de cada escenario.
Scenario Outline	Se utiliza para ejecutar el mismo escenario varias veces, con diferentes combinaciones de valores. Un Scenario Outline debe contener una o más secciones de ejemplos (o escenarios). Sus pasos se interpretan como una plantilla que nunca se ejecuta directamente. En su lugar, el Scenario Outline se ejecuta una vez para cada fila en la sección Ejemplos debajo de él (sin contar la primera fila del encabezado).
"""	Las cadenas de documentos son útiles para pasar un fragmento de texto más grande a una definición de paso.
	Las tablas de datos son útiles para pasar una lista de valores a una definición de paso:
@	Se usa para etiquetar valores.
#	Se usa para incluir comentarios.

Tabla 6. Palabras clave del lenguaje Gherkin. Fuente: elaboración propia.

Si tomamos el ejemplo de la calculadora, seguidamente tenemos un ejemplo de cómo podríamos especificar pruebas de aceptación para la funcionalidad de sumar números utilizando Gherkin (en su extensión para nuestro idioma).

- ▶ Característica: sumar números con una calculadora
 - Para no aprender matemáticas
 - Como alguien que es malo en matemáticas
 - Quiero poder sumar números usando una calculadora
- ▶ Escenario: sumar dos números positivos
 - Dado que tengo una calculadora
 - Cuando sumo 1 y 1
 - Entonces la suma debe ser 2
- ▶ Escenario: sumar un número positivo y negativo
 - Dado que tengo una calculadora
 - Cuando sumo 1 y -1
 - Entonces la suma debe ser 0
- ▶ Escenario: sumar dos números negativos
 - Dado que tengo una calculadora
 - Cuando sumo -1 y -1
 - Entonces la suma debe ser -2

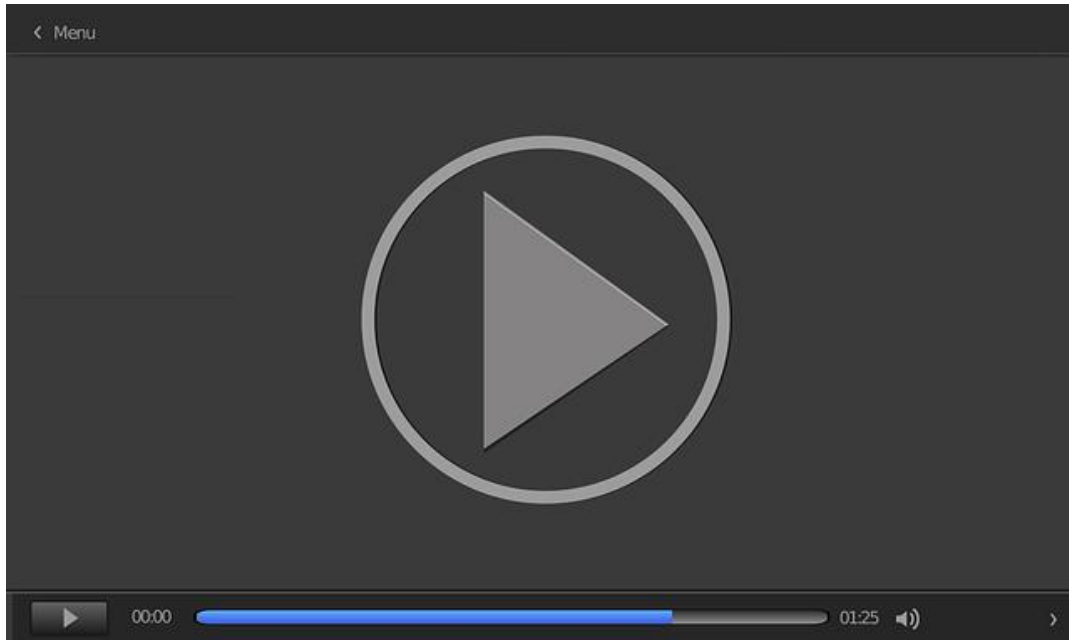
Con Gherkin definimos casos de prueba que definen qué esperamos que haga el software en un determinado contexto, siendo comprensible por el equipo técnico y por el usuario o cliente.

Existen herramientas que nos ayudan a automatizar los casos de prueba a partir de la descripción Gherkin de los escenarios:

Cucumber

Es probablemente la herramienta disponible más reconocida que admite BDD (Chaubal, 2021). Se basa en escenarios de prueba que están escritos en texto y siguen el formato *given/when/then*. Cada escenario es una prueba individual. Los escenarios se agrupan en **características**, lo que es comparable a un conjunto de pruebas. Los escenarios deben estar escritos en la sintaxis de Gherkin para que la herramienta comprenda y ejecute los pasos del escenario. Los pasos legibles por humanos en los escenarios están vinculados a las definiciones de pasos en su código a través de la propia herramienta. Debido a que estas pruebas se ejecutan a partir de las definiciones de los pasos, los informes con los resultados de las pruebas permiten ver **en qué parte del paso ha fallado la prueba**. La herramienta se escribió originalmente en el lenguaje de programación Ruby y se usó exclusivamente para probar Ruby como complemento del marco RSpec BDD. En la actualidad, admite una variedad de diferentes lenguajes de programación, incluidos Java, JavaScript, PHP, Net, Python, Perl, etc. con varias implementaciones.

En este vídeo, *Cucumber, definiendo pruebas con Gherkin*, aprenderemos a automatizar las pruebas sobre interfaces descritos con OpenAPI usando Cucumber



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=d70c8fa9-7ffb-4c62-937b-aec300cd9bfc>

SpecFlow

Se lo considera **el Cucumber para .NET**, surge del anterior y es muy similar en concepto. Se basa en texto con formato *Gherkin* para describir las historias de los usuarios y se orienta a probar soluciones .NET, por lo que puede integrarse con Visual Studio.

JBehave

Es muy similar a Cucumber, se puede usar la descripción de escenarios con Gherkin aunque también **tiene su propia sintaxis**. Frente a Cucumber, tiene más opciones de configuración y sus informes, aunque muy detallados, necesitan más

configuración para obtener retroalimentación de cada paso. Es una herramienta poderosa, pero debido a que se puede **personalizar** más, no es tan fácil comenzar con ella. Los equipos deben preguntarse exactamente qué funciones necesitan y si vale la pena invertir tiempo en aprender las distintas configuraciones de la herramienta.

Plataformas de prueba para aplicaciones móviles

Aunque no son exclusivas para pruebas de aceptación, las plataformas de pruebas de aplicaciones móviles pueden ser utilizadas para este **enfoque de alto nivel**, ya que van más allá de una mera prueba de interacción con el usuario. Con carácter general, el testeo de aplicaciones móviles puede cubrir:

- ▶ Pruebas **funcionales**: tipo básico de prueba utilizado para verificar las funcionalidades de la aplicación según la especificación de requisitos.
- ▶ Pruebas de **rendimiento**: realizadas para probar el rendimiento de la aplicación del cliente, el rendimiento del servidor y el rendimiento de la red.
- ▶ Prueba de **memoria**: los dispositivos móviles vienen con memoria limitada en comparación con las computadoras, este tipo de prueba se realiza para probar el uso de memoria optimizado por parte de una aplicación.
- ▶ Prueba de **interrupción**: se usa para verificar interrupciones debido a llamadas entrantes o SMS, advertencia de memoria baja, advertencia de batería baja, etc. mientras se ejecuta la aplicación.
- ▶ Pruebas de **instalación**: se utilizan para verificar que el proceso de instalación sea fácil y fluido e incluye también la actualización y la desinstalación.
- ▶ Pruebas de **usabilidad**: se utilizan para comprobar la eficiencia, eficacia y satisfacción de la aplicación desde la perspectiva del usuario.

Dentro de este campo de plataformas para pruebas de aplicaciones móviles, encontramos las siguientes plataformas:

Appium

Es una herramienta de código abierto para la automatización de pruebas de aplicaciones nativas, móviles y web, así como híbridas en plataformas iOS y Android. Maneja aplicaciones de iOS, Android y Windows, ya que utiliza el protocolo WebDriver. Admite Safari en iOS y todas las demás aplicaciones de navegador integradas en Android. No es necesario modificar ningún código de aplicación para realizar pruebas, ya que es **adecuado** para ejecutarse en Android o iOS usando el dispositivo o el emulador (Hans, 2015).

UI Automator

Es un marco de código abierto, integrado en Android, que permite probar la interfaz de usuario utilizando **casos de prueba funcionales automatizados**. Facilita el desarrollo de pruebas funcionales entre aplicaciones en todo el sistema y en aplicaciones instaladas.

iOS-driver

Es una herramienta de código abierto para **facilitar** las pruebas de cualquier aplicación web nativa, híbrida o móvil de iOS utilizando la API de Selenium/WebDriver. Se ejecuta de manera eficiente en un emulador en lugar de dispositivos y no es necesario cambiar ningún código de aplicación ni cargar ninguna aplicación adicional para probar la aplicación en el dispositivo.

Selendroid

Selenium para Android es un marco de código abierto que **interactúa simultáneamente** con múltiples dispositivos y emuladores. Está impulsado por la interfaz de usuario de aplicaciones nativas e híbridas y por la web móvil, por lo que la

prueba debe escribirse a través de la API del cliente Selenium. Se puede usar en emuladores y dispositivos reales y se puede integrar como un nodo en Selenium Grid para escalar y realizar pruebas en paralelo.

Frank

Es un marco de prueba exclusivo para iOS de código abierto que representa características combinadas de Cucumber y JSON. Ayuda a escribir pruebas y requisitos de aceptación estructurados y también incluye el inspector de aplicaciones Symbiote. No hay necesidad de modificar el código de la aplicación. El **único desafío** es que es difícil de usar directamente en el dispositivo, pero es más adecuado para aplicaciones nativas y web.

RobusTest

Es una herramienta de prueba móvil gratuita que proporciona pruebas basadas en la nube en dispositivos reales para Android e iOS. Soporta pruebas manuales rápidas y avanzadas automatizadas, pruebas de automatización sin secuencias de comandos, pruebas funcionales y de rendimiento. Ayuda a realizar pruebas de automatización rápidas junto con integración continua y API abierta.

Herramientas destacadas en la actualidad

En esta sección vamos a incluir algunas de las plataformas o herramientas destacadas en la actualidad que cubren varias de las fases anteriormente mencionadas, ya que pueden ser utilizadas por los equipos de desarrollo o los clientes para **validar** un software de **forma completa**.

Tricentis

Valorada por Gartner como la plataforma de pruebas continuas referente del mundo, incluye un **conjunto único de capacidades** que respaldan el ciclo de vida completo de las pruebas en aplicaciones empresariales y personalizadas. Permite acelerar

todas las fases de las pruebas, desde las unitarias hasta las de rendimiento. No es necesario desarrollar código lo que hace que sea fácil de usar para usuarios comerciales no técnicos y miembros del equipo. Es impulsada por inteligencia artificial, por lo que se aprovechan las tecnologías avanzadas de *machine learning* para permitir pruebas más rápidas e inteligentes.

SmartBear TestComplete

Es una aplicación que permite la creación de pruebas de interfaz de usuario automatizadas para entornos de escritorio, web y móvil. Permite la definición de pruebas **basadas en datos y en palabras clave**, que se integran de forma continua en entornos de desarrollo y operaciones (DevOps). Genera informes y análisis de pruebas automatizados, lo que da soporte a pruebas paralelas y con un enfoque de desarrollo dirigido por comportamiento. Con las pruebas entre navegadores y entre dispositivos, podemos registrar fácilmente una prueba de interfaz de usuario de forma automatizada y luego ejecutar la misma prueba en otros navegadores o dispositivos reales localmente o en la nube. Estas pruebas se pueden ejecutar en plataformas Windows, Mac, Linux, Android e iOS en todas las configuraciones de navegador, pudiendo usar la reproducción en JavaScript, Python o VBScript.

Eggplant

Es una solución comercial formada por **diversas herramientas** que permite automatizar las pruebas. A nivel de rendimiento, facilita la creación de *scripts* de prueba, identifica desafíos en la experiencia del usuario, monitorea tiempos de inactividad y analizar resultados. A su vez, permite a los analistas de datos generar automáticamente y simular *scripts* de prueba de viaje del usuario para monitorear el rendimiento general de la aplicación. Les posibilita a los evaluadores colocar inyectores en múltiples regiones geográficas para particionar usuarios virtuales y analizar la experiencia del usuario en función de las ubicaciones de los clientes. A nivel funcional, facilita probar sus aplicaciones mejor, más rápido y con menos

esfuerzo al automatizar la ejecución de sus pruebas funcionales. Destaca su enfoque patentado basado en imágenes para las pruebas de GUI. Permite interactuar con **cualquier dispositivo** (incluidos dispositivos móviles, tabletas, computadoras de escritorio, servidores y dispositivos de Internet de las cosas) de la misma manera que lo hace un usuario, mirando la pantalla, y utiliza sofisticados algoritmos de búsqueda de imágenes y texto para ubicar objetos en la pantalla de una manera completamente independiente de la tecnología.

QMetry Automation Studio (QAS)

Ofrece automatización de pruebas escalable que permite una **fácil transición** de pruebas manuales a automatizadas. Admite pruebas multiplataforma en la web, servicios web/API, dispositivos móviles y escritorio de Windows. QAS también utiliza marcos de automatización de pruebas de código abierto como Selenium y Appium. Admite múltiples navegadores para la ejecución de pruebas, múltiples marcos, integraciones de ALM y complementos para escalar sus pruebas automatizadas. Se integra con herramientas de CI/CD como Jenkins, Bamboo y plataformas web y de pruebas en la nube de dispositivos como Sauce Labs y Perfecto Mobile.

TestProject

Es una herramienta de automatización de **pruebas gratuita de extremo a extremo** para web, móviles y API. Con una comunidad activa que desarrolla más de 200 complementos y posicionada como referente por la consultora Gartner, soporta un uso en modo grabación para personal no técnico, un kit de desarrollo de software (SDK) de comandos avanzados que importa pruebas desde Selenium y Appium, un almacenamiento de las pruebas en la nube, la generación de cuadros de mando y el análisis ejecutivos e integración con herramientas como SauceLabs, BrowserStack, Jenkins o Slack.

Micro Focus UTF

La familia UTF de soluciones de automatización de pruebas **funcionales basadas en IA** se compone de tres herramientas:

- ▶ UFT Uno, que acelera la automatización de pruebas con una solución inteligente para aplicaciones web, móviles, RPA (automatización robótica de procesos), API y empresariales.
- ▶ UTF Desarrollador, que facilita la programación de casos de prueba utilizando lenguajes de programación estándar, entornos de desarrollo integrado (IDE) y marcos de pruebas.
- ▶ Móvil UFT, que aumenta la productividad del equipo con un laboratorio integral de nivel empresarial de emuladores y dispositivos móviles reales.

Ranorex Studio

Es un conjunto de herramientas completo para realizar pruebas de extremo a extremo de aplicaciones de escritorio, web y móviles. Permite **automatizar las pruebas** en un escritorio de Windows y luego **ejecutarlas de forma local o remota** en dispositivos móviles o simuladores/emuladores iOS o Android reales. Soporta la ejecución de pruebas en paralelo con múltiples navegadores como Chrome, Firefox, Safari, Microsoft Edge y más. Se integra con Jira y otras herramientas líderes de DevOps para automatizar el seguimiento de errores y acelerar los ciclos de desarrollo.

Worksoft Certify

Es una *suite* comercial de automatización de pruebas. No es necesario **generar código**, lo que les permite a los usuarios comerciales y a TI crear, mantener, compartir y consumir fácilmente pruebas de procesos comerciales automatizados de extremo a extremo como parte de los ciclos continuos de prueba, integración y

entrega. El marco de acción de objetos basado en modelos actualiza dinámicamente la automatización en todo su entorno de aplicaciones, lo que ofrece una solución verdaderamente escalable para acelerar los tiempos de entrega y reducir el riesgo.

Froglogic Squish GUI Tester

Es una herramienta para la automatización de las **pruebas web, integradas, móviles y de escritorio multiplataforma**. Soporta como tecnologías a probar Qt, QtQuick, QML, Java, AWT, Swing, SWT, RCP, JavaFx, Windows, Mac, Tk, 4Js, iOS, Android, sistemas embebidos (Qt, Windows, Java) y web, HTML5, Flex y JavaApplets. Ejecuta pruebas en Windows, Linux, macOS X, Unix. Soporta pruebas móviles e integradas en Android, iOS, Linux integrado y RTOS como QNX. Dispone de un IDE basado en Eclipse y puede usarse con un enfoque de desarrollo dirigido por el comportamiento, con grabación, reproducción e informes de archivos de funciones.

9.8. Referencias bibliográficas

Abdulwareth, A. J. y Al-Shargabi, A. A. (2021). Toward a Multi-Criteria Framework for Selecting Software Testing Tools. *IEEE Access*, 9, 158872-158891. <https://ieeexplore.ieee.org/abstract/document/9614194>

Adewole, A. (2018). *C# and .NET Core Test-Driven Development: Dive into TDD to create flexible, maintainable, and production-ready .NET Core applications*. Packt Publishing.

Chaubal, P. A. (2021). *Mastering behavior-driven development using cucumber: Practice and implement page object design pattern, test suites in cucumber, POM TestNG integration, cucumber reports, and work with selenium grid*. BPB Publications.

Cucumber. (s. f). *Gherkin Reference*. <https://cucumber.io/docs/gherkin/reference>

Gomes Rodrigues, A., Demion, B. y Mouawad, P. (2019). *Master Apache JMeter - From Load Testing to DevOps: Master performance testing with JMeter*. Packt Publishing.

Grzejszczak, M. (2014). *Mockito Cookbook*. Packt Publishing.

Gulati, S. y Sharma, R. (2017). *Java unit testing with JUnit 5: Test driven development with JUnit 5* (1a ed.). Apress.

Gundecha, U. y Avasarala, S. (2018). *Selenium WebDriver 3 Practical Guide: End-to-end automation testing for web and mobile browsers with Selenium WebDriver* (2a ed.). Packt Publishing.

Hahn, E. (2013). *JavaScript Testing with Jasmine: JavaScript Behavior-Driven Development*. O'Reilly Media.

Hans, M. (2015). *Appium Essentials*. Packt Publishing.

Herschmann, J., Murphy, T. y Scheibmeir J. (2019). *Gartner Magic Quadrant for Software Test Automation*. Gartner. <https://www.gartner.com/en/documents/3975493>

International Organization for Standardization. (2022, enero). *ISO/IEC/IEEE 29119-1:2022: Software and systems engineering — Software testing — Part 1: General concepts*. <https://www.iso.org/standard/81291.html>

Jones, R. A., II. (2018). *Getting Started with Testing: A Java Test Framework*. Createspace Independent Publishing Platform.

Nagy, G. y Rose, S. (2018). *Discovery: Explore behaviour using examples (BDD Books)*. CreateSpace Independent Publishing Platform.

Nagy, G. y Rose, S. (2021). *Formulation: Document examples with Given/When/Then (BDD Books)*.

Naik, K. y Tripathy, P. (2011). *Software testing and quality assurance: theory and practice*. John Wiley & Sons.

Raulamo-Jurvanen, P., Mäntylä, M. y Garousi, V. (2017, junio). *Choosing the Right Test Automation Tool: A Grey Literature Review of Practitioner Sources* [Presentación en papel]. Conferencia Proceedings of the 21st International Conference on Evaluation and Assessment in Software Engineering, Karlskrona, Suecia.

https://www.researchgate.net/publication/317486739_Choosing_the_Right_Test_Automation_Tool_a_Grey_Literature_Review_of_Practitioner_Sources

Smart, J. (2014). *BDD in Action: Behavior-driven development for the whole software lifecycle*. Simon and Schuster.

Thomas, D., Hunt, A. y Hargett, M. (2007). *Pragmatic unit testing in C# with NUnit* (2a ed.). Pragmatic Programmers.

Valliammai, K. y Sujatha, D. P. (2016). Analysis of Efficiency of Automated Software Testing Methods: Direction of Research. Analysis of Efficiency of Automated Software. *International Journal of Science and Research*, 5(12), 34–38.
https://www.ijsr.net/get_abstract.php?paper_id=ART20163174

Zein, S., Salleh, N. y Grund, J. (2016). A systematic mapping study of mobile application testing techniques. *Journal of Systems and Software*, 117, 334–356.
<https://doi.org/10.1016/j.jss.2016.03.065>

ISO/IEC/IEEE 29119 International Standard — Software and systems engineering — Software testing

International Organization for Standardization. (2022, enero). *ISO/IEC/IEEE 29119-1:2022: Software and systems engineering — Software testing — Part 1: General concepts*. <https://www.iso.org/standard/81291.html>

El propósito de la serie de estándares de pruebas de software ISO/IEC/IEEE 29119 es definir un conjunto de normas acordado internacionalmente para las pruebas de software que puedan ser utilizadas por cualquier organización al realizar cualquier tipo de prueba de software. La primera edición de la norma fue creada en 2013 y, durante 2021, se realizó una revisión para crear la segunda edición. La serie está formada por los siguientes documentos:

- ▶ *ISO/IEC/IEEE 29119-1:2022 parte 1: conceptos generales*. Este documento especifica conceptos generales en pruebas de software y presenta conceptos clave para la serie ISO/IEC/IEEE 29119. Es el único documento informativo, el resto pasan a ser normativos. <https://www.iso.org/standard/81291.html>
- ▶ *ISO/IEC/IEEE 29119-2:2021 parte 2: procesos de testing*. Este documento especifica los procesos de prueba que se pueden usar para gobernar, administrar e implementar las pruebas de software para cualquier organización, proyecto o actividad de prueba. <https://www.iso.org/standard/79428.html>
- ▶ *ISO/IEC/IEEE 29119-3:2021 parte 3: documentación del testing*. Este documento especifica plantillas de documentación de pruebas de software que se pueden usar para cualquier organización, proyecto o actividad de prueba, y atiende a los procesos especificados en la parte 2. <https://www.iso.org/standard/79429.html>

- ▶ *ISO/IEC/IEEE 29119-4:2021 parte 4: técnicas de testing*. Este documento define las técnicas de diseño de pruebas que se pueden utilizar durante el proceso de diseño e implementación de pruebas que se define en la parte 2.

<https://www.iso.org/standard/79430.html>

- ▶ *ISO/IEC/IEEE 29119-5:2016 parte 5: testing basado en palabras clave*. Este documento se centra en las técnicas de *keyword-driven testing* que permiten separar la documentación de los casos de test —en el que se incluyen los datos a usar— desde la prescripción de la forma en que los test serán ejecutados.

<https://www.iso.org/standard/62821.html>

- ▶ *ISO/IEC TR 29119-6:2021 parte 6: guías para el uso de las normas ISO/IEC/IEEE 29119 (en todas sus partes) en proyectos ágiles*. Este documento proporciona orientación para la aplicación de las normas a ciclos de vida ágiles.

<https://www.iso.org/standard/81293.html>

JUnit5 y Test Driven Development

Gulati, S. y Sharma, R. (2017). *Java unit testing with JUnit 5: Test driven development with JUnit 5* (1a ed.). Apress.

Libro que proporciona un detalle del uso de la herramienta JUnit5 y su posible uso para seguir un enfoque de desarrollo dirigido por las pruebas.

El probador de software autodidacta

Dosaj, C. R. (2020). *The self-taught software tester: A step by step guide to learn software testing using real-life project the self-taught software tester.*

El libro es una guía paso a paso para aprender a probar software, en el que se explican partes de los fundamentos de las pruebas de software. Posteriormente, utiliza un proyecto de prueba de software del mundo real. Permite aprender cómo se planifican las actividades de prueba mientras obtiene una comprensión del ciclo de vida del proyecto.

1. La validación y verificación son términos asociados a las pruebas, en los que:
 - A. Ambos términos se usan indistintamente para referirse a las pruebas.
 - B. La verificación persigue buscar fallos y la validación comprobar los requisitos del cliente.
 - C. La validación es manual y la verificación es automática.
 - D. La validación persigue buscar fallos y la verificación comprobar los requisitos del cliente.

2. Relaciona las anotaciones de JUnit con su significado:

@BeforeAll	1	A	Indica que un método es una prueba parametrizada
@DisplayName	2	B	Se utiliza para configurar el orden de ejecución del método de prueba para la clase de prueba anotada
@TestMethodOrder	3	C	Declara un nombre para mostrar personalizado para la clase de prueba o el método de prueba
@ParameterizedTest	4	D	Implica que el método anotado debe ejecutarse antes que todos los métodos @Test

3. ¿Por qué se dice que una prueba ha fallado?
 - A. Porque si una prueba falla significa que tenemos un error en el software.
 - B. Porque en una prueba es muy difícil que el resultado obtenido coincida exactamente con el esperado.
 - C. Porque el resultado real de la prueba no se corresponde con los criterios definidos para darla por buena, pero eso no supone en sí un error.
 - D. Porque el resultado real de la prueba no se corresponde con los criterios definidos para darla por buena, por lo que es evidentemente un error.

4. El desarrollo dirigido por comportamiento (*behaviour driven development, BDD*) se diferencia de otros enfoques de pruebas, ya que:

- A. El cliente escribe los requisitos.
- B. Es obligatorio escribir las pruebas antes de diseñar y desarrollar el software.
- C. Se puede usar herramientas que generan completamente las pruebas partiendo desde los requisitos.
- D. Porque los casos de prueba se definen en un lenguaje entendible por usuarios y técnicos.

5. En el modelo de testeo V simple, relaciona qué tipo de pruebas sirven para cada fase:

Pruebas de unidad	1	A	Requisitos
Pruebas de aceptación	2	B	Diseño detallado
Pruebas de integración	3	C	Código fuente
Pruebas de sistema	4	D	Diseño de alto nivel

6. El valor y utilidad de las plataformas para pruebas es:

- A. Reducir el esfuerzo al no tener que escribir los requisitos.
- B. Reducir el coste asociado al diseño, ejecución y documentación de las pruebas.
- C. Agilizar el desarrollo de software con un flujo de integración continua.
- D. Reducir el esfuerzo al no tener que analizar el resultado de las pruebas.

7. Relaciona las herramientas con su fase atendiendo a la clasificación basada en el modelo de testeo V simple:

Pruebas de unidad	1	A	Mockito
Pruebas de aceptación	2	B	PHPUnit
Pruebas de integración	3	C	JMeter
Pruebas de sistema	4	D	Cucumber

8. El concepto de emular, simular o *mockear* un sistema sirve para:

- A. No tener que generar el código de todo el sistema.
- B. Facilitar las pruebas de rendimiento del sistema.
- C. Crear escenarios de prueba en los que se integra una versión falsa de un servicio externo o interno que pueda reemplazar al real.
- D. Tener varias versiones en producción del sistema para desarrollar A/B *testing*.

9. Dado el siguiente fragmento *Gherkin*, marca la respuesta correcta:

Feature: Sumar dos números

Como usuario

Quiero usar una calculadora para sumar números

Para no tener que aprender a sumar

Scenario Outline: Sumar dos números <num1> & <num2>

Given Tengo una calculadora

When sumo <num1> y <num2>

Then el resultado debería ser <total>

Examples:

num1	num2	total
-2	3	1
10	4	14
99	-99	0

- A. No está bien formulado al usar palabras clave no existentes.
- B. Está bien formulado y los ejemplos ayudarán al programador a saber qué datos poner para lanzar las pruebas.
- C. Está bien formulado porque un Scenario Outline sirve para documentar mejor las pruebas.
- D. Está bien formulado porque con un Scenario Outline, Cucumber ejecutará de forma automática la prueba tantas veces como filas tiene la tabla.

10. En las pruebas de aplicaciones sobre dispositivos móviles, relaciona cada prueba con su definición:

Pruebas de interrupción	1	A	Controlar eventos derivados como llamadas entrantes, SMS o batería baja.
Pruebas de instalación	2	B	Controlar el consumo de memoria
Pruebas de memoria	3	C	Verificar el proceso de actualizaciones
Pruebas de rendimiento	4	D	Controlar el consumo de datos y procesador

Plataformas de Desarrollo de Software

Tema 10. Plataformas para el despliegue de sistemas distribuidos

Índice

Esquema

Ideas clave

10.1. Introducción y objetivos

10.2. Contexto de las plataformas para el despliegue de sistemas distribuidos

10.3. Valor y utilidad de las plataformas para el despliegue de sistemas distribuidos

10.4. Plataformas para automatizar el despliegue

10.5. Plataformas para arquitecturas basadas en eventos

10.6. Plataformas de monitorización de sistemas distribuidos

10.7. Referencias bibliográficas

A fondo

Hands-On DevOps for Architects: Implementing continuous delivery through automation.

DevOps Adoption Strategies: Principles, Processes, Tools, and Trends: Embracing DevOps through effective culture, people, and processes.

Test

PLATAFORMAS PARA EL DESPLIEGUE DE SISTEMAS DISTRIBUIDOS				
CONCEPTOS	VALOR Y UTILIDAD	HERRAMIENTAS		
Despliegue DevOps Continuous integration Continuous delivery Monitorización	Valor DevOps y CI/CD: - Menores tamaños en entregas - Aislamiento de errores - Tiempo medio de resolución más rápido - Tasa de liberación más rápida - Equipos dedicados y reducción de costes - Facilidad mantenimiento y actualización	Base ejecución: - Docker - Kubernetes - Apache Mesos - Red Hat OpenShift	Middleware EDA: - Apache ActiveMQ - Apache Kafka - Apache ServiceMix - Microsoft Biztalk - MuleESB - RabbitMq	
Arquitectura basada en eventos (EDA): - Productor de eventos - Consumidor de eventos - Plataforma de procesamiento de eventos - Modelo publicación-suscripción - Modelo de transmisión de eventos	Valor EDA: - Mejor escalabilidad y tolerancia a fallos - Habilita la funcionalidad asíncrona - Manejo más consistente de la concurrencia - Uso de recursos de red optimizado - Información más robusta	Automatizar despliegue: - Argo CD - Gitlab - Octopus Deploy - Red Hat Ansible - Automate		
Despliegue en contenedores: - Encapsular aplicación y dependencias - Más ligeros que las máquinas virtuales - Portables entre sistemas operativos e infraestructuras	Valor monitorización: - Detectar cortes - Generar alertas - Medir la escalabilidad - Facilitar acuerdos de nivel de servicio	Monitorización: - Cacti - Prometheus - ELK: Elasticsearch + Logstash + Kibana		

10.1. Introducción y objetivos

En este tema se estudian algunas de las plataformas que nos ayudan a automatizar el despliegue de sistemas distribuidos.

En primer lugar, estableceremos el contexto de conceptos como integración y distribución continuas (CI/CD), desarrollo y operaciones (DevOps), contenedores y monitorización de sistemas, entre otros. Detallaremos las principales características de las arquitecturas basadas en eventos (*event-driven architecture*, EDA), como una solución de diseño para poder dar respuesta a unas elevadas prestaciones de escalabilidad y rendimiento.

Posteriormente, veremos algunas de las plataformas y herramientas que facilitan el **despliegue y la monitorización** de estos sistemas en diversas infraestructuras computacionales junto con los beneficios que presentan.

Con el estudio de este tema, pretendemos alcanzar los siguientes objetivos:

- ▶ Comprender el contexto, el valor y la utilidad que tiene la automatización del despliegue de sistemas distribuidos.
- ▶ Conocer los principales conceptos y técnicas como CI/CD, DevOps y EDA.
- ▶ Introducir algunas herramientas de automatización para el despliegue de sistemas distribuidos y middleware para EDA.
- ▶ Conocer algunas plataformas de soporte a la monitorización de sistemas distribuidos.

10.2. Contexto de las plataformas para el despliegue de sistemas distribuidos

En el ámbito de la ingeniería del software, una de las **tareas críticas** es integrar los desarrollos realizados, empaquetar la solución y desplegarla para hacerla accesible a los usuarios en un entorno operativo. Existen una serie de conceptos, técnicas, métodos y plataformas que facilitan la automatización de estas actividades para minimizar esfuerzos, errores y agilizar todo el proceso de despliegue de las aplicaciones.

DevOps (acrónimo en inglés de *development and operations*) es una combinación de los términos desarrollo y operaciones, con la intención de representar un enfoque colaborativo o compartido de las tareas realizadas por los equipos de desarrollo de aplicaciones y operaciones de tecnologías de la información (TI) de una empresa. En su significado más amplio, DevOps es una filosofía que promueve una mejor **comunicación y colaboración** entre estos equipos y también en una organización. En su interpretación más estrecha, DevOps describe la adopción de desarrollo de software iterativo, automatización e implementación y mantenimiento de infraestructura programable. El término también cubre los cambios culturales, como generar confianza y cohesión entre los desarrolladores y los administradores de sistemas y alinear los proyectos tecnológicos con los requisitos comerciales. DevOps puede cambiar la cadena de entrega de software, los servicios, los roles laborales, las herramientas TI y las mejores prácticas. Si bien DevOps no es una tecnología, sus entornos generalmente aplican metodologías y tecnologías como (Courtemanche, Mell y Gillis, 2021):

- Herramientas de integración y distribución continuas (*continuous integration/continuous delivery*, CI/CD), con énfasis en la automatización de tareas.

- ▶ Sistemas y herramientas que respaldan la adopción de DevOps, incluido el monitoreo en tiempo real, la gestión de incidentes, la gestión de configuración y las plataformas de colaboración.
- ▶ Computación en la nube, microservicios y contenedores implementados simultáneamente con metodologías DevOps.

La integración y la distribución continuas (CI/CD) es un método para **distribuir las aplicaciones** a los clientes con frecuencia mediante el uso de la automatización en las etapas del desarrollo de aplicaciones. Los principales conceptos que se le atribuyen son la integración, la distribución y la implementación continuas (Red Hat, 2018):

- ▶ CI se refiere a la **integración continua**, que es un proceso de automatización para los desarrolladores. El éxito de la CI implica que se diseñen, prueben y combinen los cambios nuevos en el código de una aplicación con regularidad en un repositorio compartido. Supone una solución al problema de que se desarrollen demasiadas divisiones de una aplicación al mismo tiempo, que luego podrían entrar en conflicto entre sí.
- ▶ CD se refiere a la **distribución o la implementación continuas** y se trata de conceptos relacionados que suelen usarse indistintamente. Ambos se refieren a la automatización de las etapas posteriores del proceso, pero a veces se usan por separado para explicar hasta dónde llega la automatización. Como distribución continua se refiere a que los cambios que implementa un desarrollador en una aplicación se someten a pruebas automáticas de errores y se cargan en un repositorio para que luego el equipo de operaciones pueda implementarlos en un entorno de producción en vivo. Es una solución al problema de la **falta de supervisión y comunicación** entre los equipos comerciales y de desarrollo, así que su propósito es garantizar que la implementación del código nuevo se lleve a cabo con el mínimo esfuerzo. Como implementación continua hace referencia al lanzamiento automático de los cambios que implementa el desarrollador desde el

repositorio hasta la producción, para ponerlos a disposición de los clientes. Así ya no se sobrecarga a los equipos de operaciones con procesos manuales que retrasan la distribución de las aplicaciones.

El ciclo de vida del desarrollo de sistemas finaliza con su **despliegue** en un entorno operacional o productivo, lo que se conoce como *delivery* o puesta en producción. Este proceso ha tenido una evolución en los últimos años:

- ▶ **Despliegue tradicional:** las organizaciones ejecutaban aplicaciones en servidores físicos. Era la única forma de desplegar aplicaciones y no había forma de definir los límites de recursos para las aplicaciones en un servidor físico y esto causaba problemas de asignación de recursos. Por ejemplo, si varias aplicaciones se ejecutan en un servidor físico, puede haber instancias en las que una aplicación consuma la mayoría de los recursos y, como resultado, las otras aplicaciones tengan un rendimiento inferior. Una solución para esto sería ejecutar cada aplicación en un servidor físico diferente, pero esta opción no era factible en muchas ocasiones ya que era costoso para las organizaciones mantener muchos servidores físicos.
- ▶ **Despliegue en entornos virtuales:** como solución a los problemas de los servidores físicos, se introdujo la virtualización. Esto nos permite ejecutar varias máquinas virtuales (*virtual machines*, VM) en la CPU de un solo servidor físico. La virtualización permite que las aplicaciones se aíslen entre VM y proporciona un nivel de seguridad, ya que otra aplicación no puede acceder libremente a la información de una aplicación. La virtualización permite una mejor utilización de los recursos en un servidor físico y permite una mejor escalabilidad porque una aplicación se puede agregar o actualizar fácilmente y reduce los costos de hardware. Con la virtualización, se puede presentar un conjunto de recursos físicos como un clúster de máquinas virtuales desechables. Cada VM es una máquina completa que ejecuta todos los componentes, incluido su propio sistema operativo, sobre el hardware virtualizado.

- **Despliegue en contenedores:** los contenedores son similares a las máquinas virtuales, pero tienen propiedades de aislamiento relajadas para compartir el sistema operativo (SO) entre las aplicaciones. Por lo tanto, los contenedores se consideran ligeros. Al igual que una VM, un contenedor tiene su propio sistema de archivos, cuota de CPU, memoria, espacio de proceso y más. Como están desacoplados de la infraestructura subyacente, son portátiles entre nubes y distribuciones de SO. De esta forma podemos encapsular nuestra aplicación junto con todas las dependencias, librerías, herramientas o sistemas externos que necesite para operar.

En el contexto de los sistemas distribuidos, para poder dar respuesta a las necesidades de escalabilidad y rendimiento, además de las arquitecturas de microservicios se han vuelto populares aquellas basadas en eventos (*event-driven architectures*, EDA).

Una **arquitectura basada en eventos** es una arquitectura de software y un modelo para el diseño de aplicaciones. Con un sistema controlado por eventos, la captura, comunicación, procesamiento y persistencia de eventos son la estructura central de la solución, lo que difiere del modelo tradicional basado en un modelo petición-respuesta cliente-servidor o entre servicios. Su principal ventaja es que facilita un **acoplamiento mínimo**, lo que la convierte en una buena opción para las arquitecturas de aplicaciones modernas y distribuidas, ya que los productores de eventos no saben qué eventos están escuchando los consumidores para un evento, y el evento no sabe cuáles son las consecuencias de su ocurrencia.

En estos modelos de arquitectura, el concepto de evento es la base que coordina los diversos bloques funcionales. Un **evento** es cualquier suceso significativo o cambio de estado del hardware o software del sistema. No es lo mismo que una notificación de evento, que es un mensaje o notificación enviada por el sistema para notificar a otra parte del sistema que se ha producido un evento. Nos referimos, por tanto, a evento como al **contenido del mensaje** que será notificado entre las partes. La fuente de un evento puede ser de entradas internas o externas. Los eventos pueden

generarse a partir de un usuario, como un clic del ratón o una pulsación de tecla, una fuente externa, como la salida de un sensor, o provenir del sistema, como cargar un programa o que se produzca un error.

Los principales actores de una arquitectura basada en eventos son:

- ▶ El **productor** de eventos: detecta un evento y lo representa como un mensaje. No conoce el consumidor del evento o el resultado del procesado de este.
- ▶ El **consumidor** de eventos: generalmente a través de una plataforma de mediación de mensajes o eventos basada en canales, reciben los eventos y realizan las actuaciones pertinentes de forma asíncrona.
- ▶ La **plataforma de procesamiento de eventos**: es el canal de comunicación o bus de mensajes que facilita la interacción desacoplada, distribuida e independiente de la tecnología entre productores y servidores.

Existen dos grandes formas de gestionar una arquitectura basada en eventos:

- ▶ **Modelo publicación-suscripción**: se dispone de una infraestructura de mensajería basada en suscripciones a un flujo de eventos. Con este modelo, después de que ocurre o se publica un evento, se envía a los suscriptores que necesitan ser informados.
- ▶ **Modelo de transmisión de eventos**: en este modelo los eventos se escriben en un registro. Los consumidores de eventos no se suscriben a un flujo de eventos, sino que pueden leer desde cualquier parte de la transmisión y pueden unirse a la transmisión en cualquier momento.

Una vez que hemos desplegado nuestro sistema en un entorno productivo, es necesario **monitorizar** la infraestructura y los componentes lógicas para asegurar que el sistema esté funcionando. En aplicaciones de reducido tamaño que operan en una infraestructura simple es una tarea sencilla, que puede llegar a ser

extremadamente compleja en el caso de sistemas distribuidos desplegados sobre múltiples infraestructuras computacionales, para lo que es necesario disponer de plataformas y herramientas que automaticen la monitorización y lancen alertas.

Dentro del ámbito de la monitorización podemos supervisar diversos aspectos:

- ▶ Supervisión del servidor en tiempo real.
- ▶ Supervisión del rendimiento de la red.
- ▶ Supervisión de contenedores.
- ▶ Supervisión de servicios de infraestructura en la nube, como mediadores de mensajes, sistemas de registro, bases de datos o *API gateways* entre otros.
- ▶ Supervisión de los componentes de la propia aplicación, como servicios o *front-ends*.

10.3. Valor y utilidad de las plataformas para el despliegue de sistemas distribuidos

Las plataformas que nos facilitan la automatización del despliegue de sistemas distribuidos presentan una serie de beneficios, tanto en su visión global como a nivel específico en algunas de las fases, como CI/CD, enfoque DevOps, uso de contenedores, arquitecturas de eventos y monitorización de sistemas.

El uso de plataformas de integración y la entrega continuas presentan una serie de beneficios como:

- ▶ **Cambios de código más pequeños:** permite integrar pequeños fragmentos de código a la vez que son más simples y fáciles de manejar que grandes fragmentos de código y, como tales, tienen menos problemas que deban repararse en una fecha posterior.
- ▶ **Aislamiento de errores:** es la práctica de diseñar sistemas de tal manera que cuando ocurre un error, los resultados negativos tienen un alcance limitado. El diseño de un sistema con CI/CD garantiza que los aislamientos de fallos sean más rápidos de detectar y más fáciles de implementar. Los aislamientos de fallos combinan el monitoreo del sistema, la identificación de cuándo ocurrió el error y la detección de su ubicación. Por lo tanto, las consecuencias de los errores que aparecen en la aplicación tienen un alcance limitado.
- ▶ **Tiempo medio de resolución más rápido:** CI/CD reduce el tiempo de resolución de errores porque los cambios de código son más pequeños y los aislamientos de fallos son más fáciles de detectar.
- ▶ **Más confianza en las pruebas:** al añadir nuevo código con sus pruebas y poder automatizar las pruebas de regresión, aumenta la confianza en que los posibles errores en producción disminuyan.

- ▶ **Tasa de liberación más rápida:** los errores se detectan más rápido y, como tales, se pueden reparar más rápido, lo que lleva a un aumento de las tasas de liberación.
- ▶ **Aumento de la transparencia y responsabilidad del equipo:** CI/CD es una excelente manera de obtener comentarios continuos no solo de sus clientes, sino también de su propio equipo. Esto aumenta la transparencia de cualquier problema en el equipo y fomenta la rendición de cuentas responsable. CI se enfoca principalmente en el equipo de desarrollo, por lo que los comentarios de esta parte de la canalización afectan las fallas de construcción, los problemas de fusión, los contratiempos arquitectónicos, etc. CD se enfoca más en llevar el producto rápidamente a los usuarios finales para obtener el cliente que tanto necesita. comentario. Tanto CI como CD brindan retroalimentación rápida, lo que le permite mejorar el producto de manera constante y continua.
- ▶ **Reducción de costes:** CI/CD reduce la cantidad de errores que pueden ocurrir en los muchos pasos repetitivos, libera tiempo del desarrollador que podría dedicarse al desarrollo del producto, ya que no hay tantos cambios de código para corregir en el futuro si el error se detecta rápidamente.
- ▶ **Mayor facilidad de mantenimiento y actualizaciones:** con CI/CD podemos automatizar las tareas de mantenimiento durante los períodos de inactividad, también conocidos como la hora no crítica.

El uso de un enfoque de DevOps deriva en una serie de ventajas como:

- ▶ **Menores tamaños en entregas:** DevOps se asocia con una entrega rápida y frecuente, así como con ciclos de desarrollo más cortos. La integración y la entrega continuas que son de vital importancia para alcanzar los objetivos de DevOps son las prácticas que permiten pequeños incrementos de entregas y ciclos de retroalimentación rápidos. Los cambios de software se realizan siempre que sea necesario para hacerlo. Básicamente, la adopción de DevOps aumenta la cadencia de los lanzamientos de aplicaciones a producción, lo que influye positivamente en el proceso de desarrollo de software.

- ▶ **Equipos dedicados en lugar de silos centrados en habilidades:** los departamentos de ingeniería y operaciones tradicionalmente aislados ahora pueden comunicarse entre sí sin un gran retraso entre una parte de la organización y la siguiente. Una mayor exposición y transparencia son más productivas y menos disruptivas para sus equipos, lo que permite que un solo equipo de ingenieros trabaje durante todo el ciclo de vida de la aplicación y pueda desarrollar así sus habilidades.
- ▶ **Mejora de la comunicación y la colaboración:** DevOps significa un gran cambio cultural que no solo elimina las barreras de la comunicación, sino que también permite a los empleados colaborar y compartir recursos fácilmente. Para que las operaciones de DevOps tengan éxito, todas las partes interesadas deben participar. Coordinar diferentes equipos para que todos puedan cooperar lo ayudará a evitar señalar con el dedo y habilitará la confianza y la colaboración.
- ▶ **Reducción del tiempo de recuperación:** las prácticas de DevOps son particularmente adecuadas para minimizar el impacto de los cuellos de botella, los retrocesos y los fallos de implementación. Pero cuando se encuentran errores de todos modos, se necesita menos tiempo para recuperarse de ellos. La detección de errores más rápida proporciona un ciclo de retroalimentación eficiente entre las operaciones y los desarrolladores por un lado y los clientes, por el otro.
- ▶ **Mayores eficiencias:** la automatización es el núcleo de la cultura DevOps, ya que ayuda a garantizar que también se observen todos los demás principios clave de DevOps. Tradicionalmente, un entorno de desarrollo se crea desde cero, pero se puede crear un entorno DevOps mediante procesos automatizados para que pueda manejar el rápido crecimiento del producto, las distintas cargas de trabajo y las necesidades cambiantes a lo largo del tiempo. La velocidad en la que se mueven las soluciones DevOps es de suma importancia.
- ▶ **Fiabilidad y escalado con mínimos riesgos:** garantizar la confiabilidad durante las horas pico es fundamental. Sin embargo, cuando los procesos y los sistemas se

enfrentan a una mayor presión, puede resultar más difícil. Es importante desarrollar una estrategia integral que se centre en la confiabilidad, la escalabilidad y el apoyo al crecimiento continuo. DevOps permite a los equipos implementar rápidamente nuevo software mientras protegen los datos locales existentes en un entorno distribuido. Las prácticas de DevOps enfatizan la orquestación de los esfuerzos de un equipo mientras mantienen la integridad del entorno de desarrollo. Por lo tanto, un equipo podrá mantener una excelente experiencia para los usuarios mientras realiza actualizaciones y cambios en un producto.

- ▶ **Ahorro de costes:** las iniciativas DevOps no solo impulsan innovaciones que aumentan el valor comercial, sino que también ahorran costos de mantenimiento y actualizaciones, lo que elimina gastos de capital innecesarios y reduce la complejidad de los proyectos y los excesos de presupuesto. Podría decirse que, cuando existe una estrategia integral de DevOps, una empresa puede reducir significativamente los costos y aumentar su rentabilidad. En particular, se pueden aprovechar las economías de escala integradas en los procesos de producción y prueba. Al automatizar las canalizaciones de lanzamiento, DevOps no solo permite un lanzamiento más rápido, sino que también reduce la mano de obra y, por lo tanto, reduce los costos. En muchos casos, esto puede significar reducir algunos de los gastos generales asociados con las implementaciones, así como reducir el tiempo de inactividad o el costo de las medidas de control de cambios.

El **despliegue** de las aplicaciones **en contenedores** presenta una serie de beneficios, como:

- ▶ **Creación e implementación ágil de aplicaciones:** mayor facilidad y eficiencia en la creación de imágenes de contenedores en comparación con el uso de imágenes de máquinas virtuales.
- ▶ **Desarrollo e integración continua:** proporciona creación e implementación de imágenes de contenedores confiables y frecuentes con reversiones rápidas y eficientes (debido a la inmutabilidad de la imagen).

- ▶ **Separación de preocupaciones de desarrollo y operaciones:** cree imágenes de contenedores de aplicaciones en el momento de la compilación/lanzamiento en lugar del momento de la implementación, para así desvincular las aplicaciones de la infraestructura.
- ▶ **Monitorización:** no solo muestra información y métricas a nivel del sistema operativo, sino también el estado de la aplicación y otras señales.
- ▶ **Portabilidad de distribución en la nube y el sistema operativo:** se ejecuta en Ubuntu, RHEL, CoreOS, en servidores físicos y en las principales nubes públicas.
- ▶ **Gestión centrada en la aplicación:** eleva el nivel de abstracción desde la ejecución de un sistema operativo en hardware virtual hasta la ejecución de una aplicación en un sistema operativo utilizando recursos lógicos.
- ▶ **Microservicios libremente acoplados, distribuidos, elásticos y liberados:** las aplicaciones se dividen en piezas más pequeñas e independientes y se pueden implementar y administrar dinámicamente, no una pila monolítica que se ejecuta en una gran máquina de un solo propósito.
- ▶ **Aislamiento de recursos:** rendimiento predecible de las aplicaciones.
- ▶ **Utilización de recursos:** alta eficiencia y densidad.

El desarrollo de sistemas distribuidos basados en arquitecturas dirigidas por eventos (EDA) presenta una serie de beneficios:

- ▶ **Mejor escalabilidad y tolerancia a fallos:** una EDA es escalable porque se implementa utilizando una arquitectura moderna distribuida y tolerante a fallos. Los nodos informáticos débilmente acoplados trabajan juntos para formar una solución integrada. Si un nodo de procesamiento de eventos falla, otro cobrará vida de inmediato y continuará donde lo dejó el nodo anterior. Este entorno desacoplado y distribuido facilita la capacidad de un sistema de dar soporte a una gran carga de peticiones y recuperarse ante caídas parciales del sistema.

- ▶ **Información más robusta:** dado que los eventos se registran a medida que ocurren, desde fuentes como dispositivos, aplicaciones y redes vinculadas de Internet de las cosas (IoT), las empresas tienen acceso a todos los datos y el contexto que necesitan para tomar las mejores decisiones.
- ▶ **Habilita la funcionalidad asíncrona:** todas las funciones son completamente asíncronas, lo que descarta por completo la naturaleza lineal del diseño de software tradicional. Los desarrolladores ya no están sujetos a un código altamente específico que predetermina cada acción. En cambio, la ocurrencia de un evento determina qué acciones se tomarán a continuación. Si bien esto puede ser más difícil de documentar para los desarrolladores, proporciona una flexibilidad y una apertura sin precedentes en el diseño del sistema.
- ▶ **Manejo más consistente de la concurrencia:** la inmutabilidad de los eventos y el orden de los mensajes facilitan llegar a la concurrencia dentro de los sistemas distribuidos.
- ▶ **Uso de recursos de red optimizado:** cuando tenemos que integrar varios servicios para una funcionalidad, en lugar de tener que realizar múltiples peticiones para comprobar si se ha alcanzado un cierto evento (modo encuesta), es más eficiente no estresar la infraestructura con peticiones no necesarias y esperar la notificación asíncrona cuando el evento en sí tenga lugar.

La **monitorización** de un sistema desplegado sobre una infraestructura tiene, entre otros, estos beneficios:

- ▶ Proporcionar **indicadores** sobre interrupciones y degradación del servicio.
- ▶ **Detectar cortes** de servicio de forma temprana.
- ▶ **Generar alertas en tiempo real** sobre incidencias que agilicen su resolución.
- ▶ Registrar y analizar **actividades no autorizadas o intentos de ataques** que puedan comprometer la seguridad del sistema.

- ▶ **Medir la escalabilidad** del sistema mediante la recopilación de métricas del sistema en tiempo real.
- ▶ Obtener datos de uso del sistema en tiempo real que permitan optimizar aquellas transacciones más comunes.
- ▶ Asegurar la **alta disponibilidad** del sistema.
- ▶ Facilitar el cumplimiento de **acuerdos de nivel de servicio** (*service level agreements*, SLA) con los clientes.

10.4. Plataformas para automatizar el despliegue

A nivel de automatización de despliegue, podemos incluir aquellas plataformas de base que nos permiten ejecutar aplicaciones en sistemas operativos distribuidas o en entornos basados en contenedores y, por otro lado, aquellas herramientas que facilitan la automatización de la fase de despliegue en diversas infraestructuras.

Las principales plataformas que dan soporte a la ejecución de contenedores o sistemas operativos distribuidos y que facilitan la automatización del despliegue se exponen en la siguiente figura:



Tabla 1. Plataformas para automatizar el despliegue. Fuente: elaboración propia.

Plataformas contenedoras o sistemas operativos distribuidos

A nivel de plataformas de contenedores o sistemas operativos distribuidos, las más destacadas se detallan a continuación.

Docker

Es una plataforma abierta para desarrollar, enviar y ejecutar aplicaciones sobre contenedores. Permite separar las aplicaciones de la infraestructura para que pueda entregar software rápidamente. Brinda la capacidad de **empaquetar y ejecutar** una aplicación en un **entorno vagamente aislado llamado contenedor**. El aislamiento y

la seguridad le permiten ejecutar muchos contenedores simultáneamente en un *host* determinado. Los contenedores son livianos y contienen todo lo necesario para ejecutar la aplicación, por lo que no necesita depender de lo que está instalado actualmente en el *host*. Podemos compartir contenedores fácilmente mientras trabaja y asegurarse de que todas las personas con las que comparte obtengan el mismo contenedor que funciona de la misma manera. Proporciona herramientas y una plataforma para administrar el ciclo de vida de los contenedores y utiliza una arquitectura cliente-servidor.

El cliente de Docker se comunica con el demonio de Docker, que hace el trabajo pesado de **crear, ejecutar y distribuir** los contenedores de Docker. El cliente y el demonio de Docker pueden ejecutarse en el mismo sistema o puede conectar un cliente de Docker a un demonio de Docker remoto. Se comunican mediante una API REST, a través de *sockets* UNIX o una interfaz de red. Otro cliente es Docker Compose, que le permite trabajar con aplicaciones que consisten en un conjunto de contenedores. El uso comercial de Docker Desktop en empresas más grandes (más de 250 empleados O más de diez millones de dólares en ingresos anuales) ahora requiere una suscripción (Docker, s. f.).

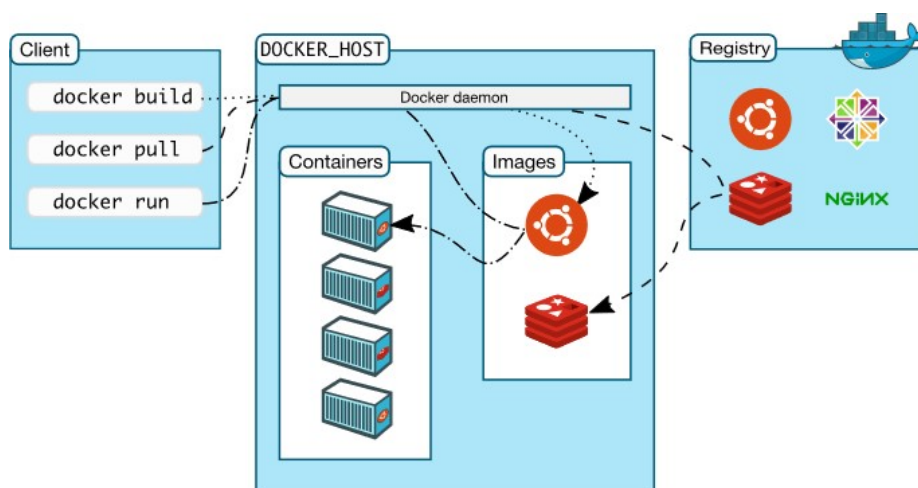


Figura 1. Arquitectura de componentes de Docker. Fuente: Docker Docs, s. f.

Kubernetes (k8s)

Es una plataforma **portátil, extensible y de código abierto** para administrar cargas de trabajo y servicios en contenedores, que facilita tanto la configuración declarativa como la automatización. Los servicios, el soporte y las herramientas de Kubernetes están ampliamente disponibles. Su nombre proviene del griego y significa timonel o piloto. K8s como abreviatura resulta de contar las ocho letras entre la «K» y la «S». Google abrió el código del proyecto Kubernetes en 2014. Kubernetes combina más de quince años de experiencia de Google en la ejecución de cargas de trabajo de producción a escala con las mejores ideas y prácticas de la comunidad. Proporciona un marco para **ejecutar sistemas distribuidos de forma resiliente**. Algunas de sus características son:

- ▶ El descubrimiento de servicios y el balanceo de carga, ya que si el tráfico a un contenedor es alto, Kubernetes puede equilibrar la carga y distribuir el tráfico de la red para que la implementación sea estable.
- ▶ Orquestación de almacenamiento, permite montar automáticamente un sistema de almacenamiento local o en la nube.
- ▶ Implementaciones y reversiones automatizadas.
- ▶ Empaquetado automático en contenedores, gestionando un grupo de nodos que puede usar para ejecutar tareas en contenedores, indicando cuánta CPU y memoria (RAM) necesita cada contenedor.

- ▶ Reparación automática, ya que reinicia los contenedores que fallan, reemplaza los contenedores, elimina los contenedores que no responden a su verificación de estado definida por el usuario y no los anuncia a los clientes hasta que están listos para servir.
- ▶ Administración de configuraciones y secretos, administrando información como contraseñas, *tokens* de OAuth y claves SSH.

Cuando se implementa Kubernetes, se obtiene un **clúster**, que consta de un conjunto de máquinas de trabajo, llamadas **nodos**, que ejecutan aplicaciones en contenedores. Cada clúster tiene al menos un **nodo trabajador**. Los nodos trabajadores alojan los **Pods** que son los componentes de la carga de trabajo de la aplicación. El plano de control administra los nodos trabajadores y los *Pods* en el clúster. En entornos de producción, el plano de control generalmente se ejecuta en varias computadoras y un clúster generalmente ejecuta varios nodos, lo que proporciona tolerancia a fallas y alta disponibilidad (Kubernetes, s. f.).

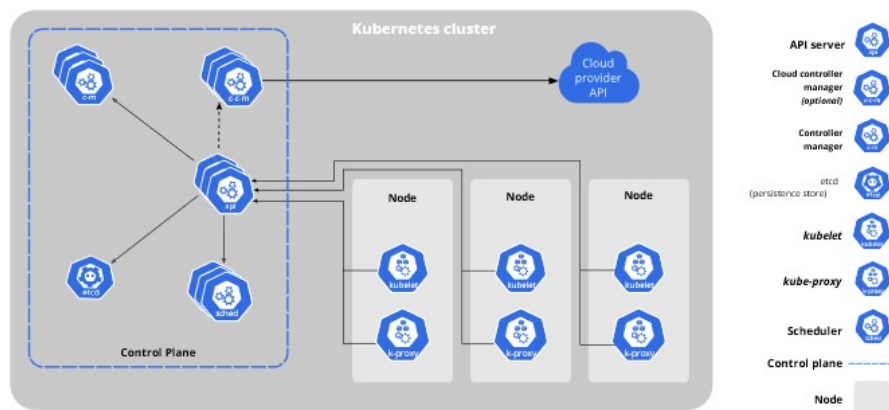


Figura 2. Arquitectura de componentes de un clúster Kubernetes. Fuente: Kubernetes, 2022.

Las dos plataformas más comunes para el despliegue en contenedores son *Docker* y *Kubernetes*. *Docker* se trata de empaquetar aplicaciones en contenedores en un solo nodo y *Kubernetes* está destinado a ejecutarlas en un clúster.

Apache Mesos

Es un núcleo (*kernel*) de sistemas distribuidos. Está construido usando los mismos **principios que el *kernel* de Linux**, solo que en un nivel diferente de abstracción. El *kernel* se ejecuta en cada máquina y proporciona aplicaciones (por ejemplo, Hadoop, Spark, Kafka, Elasticsearch) con API para la gestión y programación de recursos en todo el centro de datos y entornos de nube. Permite abstraer la CPU, la memoria, el almacenamiento y otros recursos informáticos de las máquinas (físicas o virtuales), lo que permite que los sistemas distribuidos elásticos y tolerantes a fallas se construyan fácilmente y se ejecuten de manera efectiva. Los principales componentes principales de Mesos son un demonio maestro que administra los demonios de agente que se ejecutan en cada nodo del clúster y los marcos de trabajo de Mesos que ejecutan tareas en estos agentes (Hindman et al., 2011).

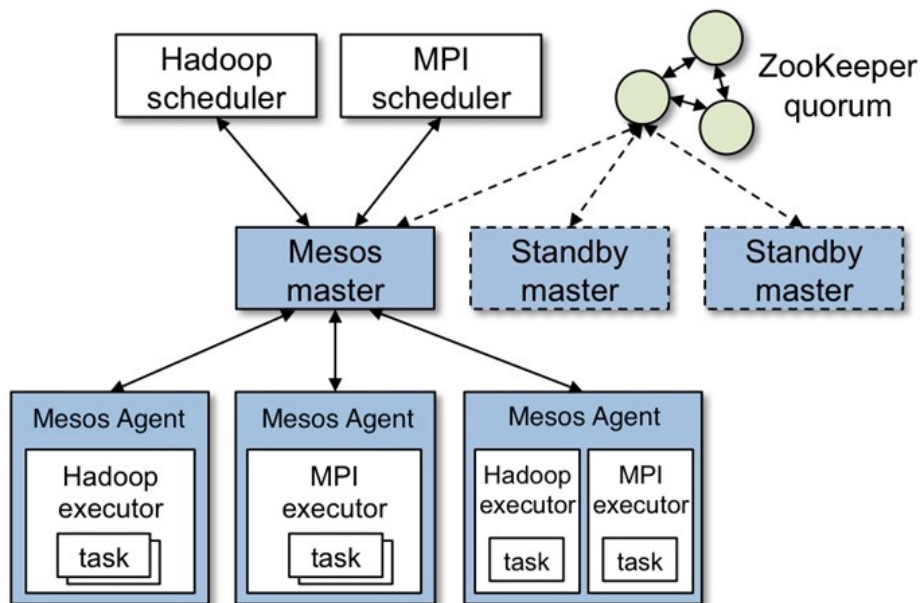


Figura 3. Arquitectura de componentes de Apache Mesos. Fuente: Apache Mesos, s. f.

Red Hat OpenShift

Es una de las principales plataformas de Kubernetes a nivel empresarial, ya que cuenta con el soporte técnico de Red Hat. Permite **diseñar, implementar y ejecutar** las aplicaciones de manera uniforme en la nube o en nuestros propios equipos físicos. Gracias a que ofrece operaciones integrales automatizadas y la implementación de autoservicio para los desarrolladores, los equipos pueden trabajar en conjunto para llevar las ideas de la etapa de desarrollo a la de producción de manera más eficiente. Algunas de sus principales características y beneficios son:

- ▶ Escalabilidad, las aplicaciones pueden escalar a miles de instancias en cientos de nodos en segundos.
- ▶ Flexibilidad: simplifica la implementación y la administración de una infraestructura híbrida, lo que le brinda la flexibilidad de tener un servicio autoadministrado o completamente administrado.
- ▶ Basado en estándares de código abierto, ya que incorpora contenedores *open*

container initiative (OCI) y Kubernetes certificados por Cloud Native Computing Foundation para la orquestación de contenedores; portabilidad de contenedores: las imágenes de contenedores creadas en el estándar de la industria OCI garantizan la portabilidad entre las estaciones de trabajo de los desarrolladores y los entornos de producción de Red Hat OpenShift.

- ▶ Experiencia de desarrollador mejorada: ofrece un conjunto integral de herramientas para desarrolladores, compatibilidad con varios idiomas e integraciones de línea de comandos y entorno de desarrollo integrado (IDE).
- ▶ Instalación y actualizaciones automatizadas: la instalación automatizada y las actualizaciones inalámbricas de la plataforma son compatibles en la nube con Amazon Web Services, Google Cloud Platform, IBM Cloud y Microsoft Azure.
- ▶ Como automatización: incluye compilaciones, implementaciones, escalado, administración de salud y más de contenedores y aplicaciones optimizados y automatizados.
- ▶ Gestión multiclúster: con Red Hat Advanced Cluster Management for Kubernetes puede implementar fácilmente aplicaciones, administrar varios clústeres y aplicar políticas en todos los clústeres a escala (Red Hat, 2022).

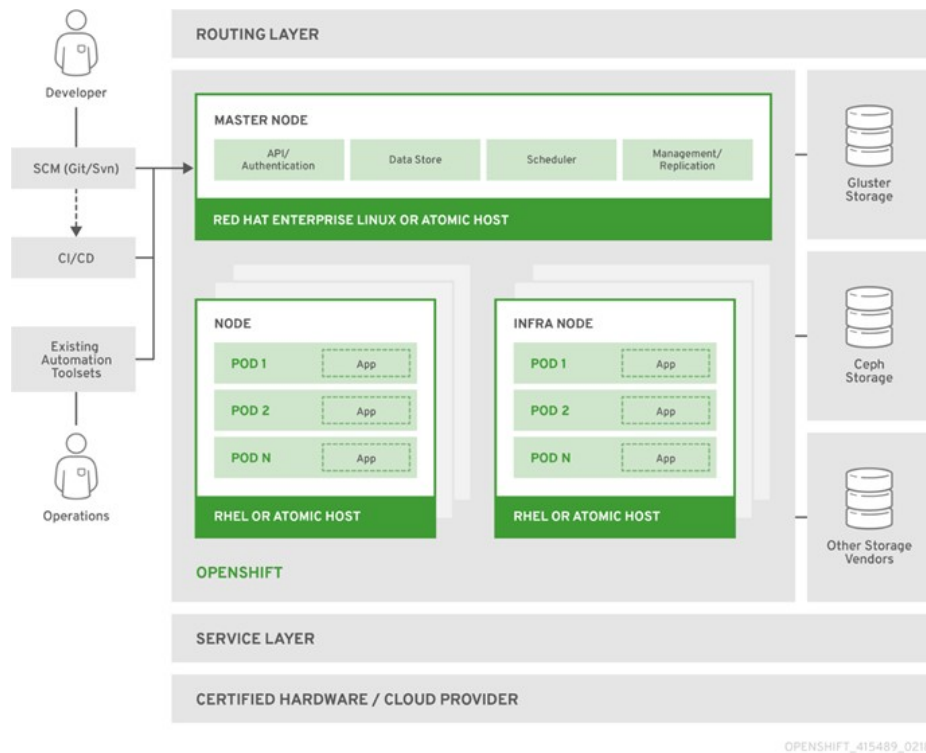


Figura 4. Arquitectura de capas de Red Hat Open Shift. Fuente: Red Hat Open Shift, s. f.

Plataformas que automatizan el despliegue de las aplicaciones

A nivel de herramientas que automatizan el despliegue de las aplicaciones en las **infraestructuras físicas** o en las lógicas anteriormente mencionadas, las más destacadas se detallan a continuación.

Argo Continuos Deployment

Es una solución de **implementación continua** de código abierto que proporciona compatibilidad con Kubernetes primero. Sigue flujos de trabajo basados en Git para automatizar la implementación de servicios dentro de Kubernetes. Como controlador de Kubernetes, garantiza que el estado actual de sus recursos coincida con el estado de recursos de destino deseado o especificado. Se puede encontrar en GitHub bajo argoproj. Argo CD aprovecha lo que se llama Argo GitOps Engine, una biblioteca de código abierto que implementa las funciones principales de GitOps, lo que le permite

administrar, automatizar, auditar y comprender el ciclo de vida de su implementación de software. Argo CD es una de las herramientas de implementación de software más sólidas que existen (Argo CD, s. f.).

Atlassian Bamboo

Bamboo Data Center es una **canalización de entrega continua** que ofrece resistencia, fiabilidad y escalabilidad a equipos de todos los tamaños. Como solución empresarial de Atlassian, Bamboo, Bitbucket y Jira Software están completamente integrados y nos ofrecen total trazabilidad desde el momento en que se solicita una función hasta la implementación. Entre sus funcionalidades destacan (Atlassian, s. f.):

- ▶ Automatización del flujo de trabajo: aprovecha las ventajas de la implementación ágil con flujos de trabajo automatizados desde el código hasta la implementación.
- ▶ Recuperación ante desastres integrada: mantén a los equipos conectados y en la dirección correcta gracias a la resistencia de compilación y la alta disponibilidad.
- ▶ Escala con confianza: aumenta la capacidad y mantén el rendimiento a medida que crece su organización.

AWSCodeDeploy

Es un servicio de despliegue completamente administrado que automatiza las implementaciones de software en una variedad de servicios informáticos, como Amazon EC2, AWS Fargate, AWS Lambda y sus servidores locales. Es **independiente de la plataforma y el lenguaje**, por lo que funciona con cualquier aplicación (aunque existe un requisito para que sus instancias locales se conecten a puntos finales públicos de AWS). Tampoco parece haber ningún soporte para la implementación en otros proveedores de la nube, como GCP y Azure, por lo que en realidad es una solución muy acoplada a la infraestructura AWS (AWS, s. f.).

Azure Automation

Ofrece un servicio de configuración, actualizaciones del sistema operativo y automatización basado en la nube que admite una **administración coherente** en sus entornos Azure y no Azure. Incluye automatización de procesos, gestión de configuración, gestión de actualizaciones, capacidades compartidas y características heterogéneas. Process Automation en Azure Automation le permite automatizar tareas de administración frecuentes, lentas y propensas a errores. Este servicio lo ayuda a concentrarse en el trabajo que agrega valor comercial. Al reducir los errores y aumentar la eficiencia, también ayuda a reducir los costos operativos. El entorno operativo de automatización de procesos se detalla en la ejecución de *Runbook* en Azure Automation (Microsoft, 2022).

Cloud Foundry BOSH

Es un proyecto que **unifica** la ingeniería de lanzamiento, la implementación y la gestión del ciclo de vida del software en la nube a pequeña y gran escala. BOSH puede aprovisionar e implementar software en cientos de máquinas virtuales. También realiza monitoreo, recuperación de fallas y actualizaciones de software con un tiempo de inactividad de cero a mínimo. Si bien BOSH se desarrolló para implementar Cloud Foundry PaaS, también se puede usar para implementar casi cualquier otro software (Hadoop, por ejemplo). BOSH es particularmente adecuado para grandes sistemas distribuidos. Además, BOSH es compatible con múltiples proveedores de infraestructura como servicio (IaaS), como VMware vSphere, Google Cloud Platform, Amazon Web Services EC2, Microsoft Azure, OpenStack y Alibaba Cloud. Hay una interfaz de proveedor de nube (CPI) que permite a los usuarios ampliar BOSH para admitir proveedores de IaaS adicionales, como Apache CloudStack y VirtualBox (Cloud Foundry BOSH, s. f.).

GitLab

Originalmente una herramienta de administración de código fuente basada en Git, introdujo una **solución de CI/CD** en su conjunto de productos. Admiten herramientas de orquestación de contenedores, como Kubernetes y ECS, así como los tres principales proveedores de nube (GCP, Azure, AWS). En términos de infraestructura, GitLab lanzó recientemente una integración de Terraform. La plataforma también ofrece funciones de buen gobierno y cumplimiento disponibles en sus planes de nivel empresarial. En lo que respecta a los idiomas, GitLab tiene como objetivo admitir todos los idiomas, antiguos y nuevos, incluidos PHP, Go, Ruby, .NET, Java, JS y más. Se califica a sí mismo como una plataforma de CI/CD, pero el CD se logra al extender CI con *scripts*, por lo que **no es del todo una plataforma** en el verdadero sentido de la palabra. Adicionalmente, hay una curva de aprendizaje relevante para utilizar la herramienta, no tiene ninguna capacidad de verificación de implementación, pero puede integrarse con herramientas como Prometheus para fines de monitoreo (GitLab, s. f.).

GoCD

Es un servidor de integración y despliegue continuo gratuito y de código abierto, que permite **modelar y visualizar** fácilmente flujos de trabajo complejos. Proporciona visualización de canalización de extremo a extremo, implementaciones nativas de la nube, modelado de flujo de trabajo complejo y trazabilidad avanzada. Algunas de sus funcionalidades son (GoCD, s. f.):

- ▶ Visualización de un extremo a otro: muestra todo el camino desde la creación hasta el compromiso y la producción en una sola vista para permitir que los usuarios naveguen entre trabajos, detecten ineficiencias y optimicen procesos.
- ▶ Implementaciones nativas de la nube: permite optimizar el flujo de trabajo de CD en entornos de nube comunes como Kubernetes, Docker y AWS.

- ▶ Modelado de flujo de trabajo complejo: ejecución paralela, gestión de dependencias, construcciones de modelado de flujo de trabajo de CD.
- ▶ Trazabilidad avanzada: resuelva problemas de tuberías rotas con pistas de auditoría y diferencias.

Google Cloud Deployment Manager

Es un servicio de implementación de infraestructura que automatiza la **creación y la administración** de recursos de Google Cloud. Facilita la escritura de plantillas flexibles y archivos de configuración que sirven de base para crear despliegues que tengan una variedad de servicios de Google Cloud, como Cloud Storage, Compute Engine y Cloud SQL, configurados para funcionar en conjunto (Google Cloud Deployment Manager, s. f.).

Harness

Es una plataforma de entrega de software moderna que ofrece integración e implementación continua, indicadores de funciones y administración de costos en la nube. Esta herramienta se creó para **reducir el tiempo, el esfuerzo y la administración** necesarios para crear nuevos artefactos. Se basa en el uso de aprendizaje automático para optimizar la cantidad de pruebas necesarias durante el proceso de construcción y despliegue. Harness Feature Flags ayuda a administrar los lanzamientos al permitir que los desarrolladores, especialistas en *marketing*, gerentes y otros líderes de la empresa activen y desactiven instantáneamente las funciones en producción. Esto permite pruebas A/B fáciles y puede aumentar drásticamente la velocidad de implementación. Los indicadores de características se están convirtiendo rápidamente en una parte clave del proceso de implementación de software. Harness Cloud Cost Management brinda a los usuarios visibilidad instantánea del coste proveniente del servicio en la nube elegido. También permite a los usuarios controlar o detener automáticamente los recursos que se aprovisionaron en exceso o se dejaron en ejecución accidentalmente (Harness, s. f.).

Heroku Flow

Es un flujo de trabajo de automatización del **despliegue estructurado** que combina integraciones estrechas con GitHub, canalizaciones presentadas visualmente, aplicaciones de revisión desechables y Heroku CI. Está diseñado para optimizar la experiencia de lanzamiento de la aplicación al hacer que la entrega continua sea fácil, visual y eficiente sobre Heroku, la plataforma en la nube como servicio (PaaS) de Salesforce que admite varios lenguajes de programación como Ruby, Java, Node.js, Scala, Clojure, Python, PHP y Go (Heroku, s. f.).

Jenkins

Es la herramienta de código abierto más conocida, que se utiliza tanto para la integración continua como para la implementación continua. Dado que es un servidor de automatización de código abierto, es de uso gratuito para todos y hay una gran comunidad de seguidores, lo que genera **soporte, documentación y funciones adicionales**. Este servidor/herramienta de integración continua es un programa Java independiente que es independiente de la plataforma. Está disponible para la mayoría de los principales sistemas operativos, como Linux, Windows y macOS. Aunque con base en una serie de extensiones y *plugins* los desarrolladores han hecho de Jenkins una herramienta de automatización de despliegue, en realidad no es su objetivo, no es nativo para la nube y como tal requiere de demasiados complementos para cumplir esta función, pero en todo caso lo nombramos al ser habitual en las plataformas de CI/CD (Jenkins, s. f.).

Octopus Deploy

Es una solución de software para **administrar lanzamientos y automatizar implementaciones**. Desde la perspectiva de la gestión de lanzamientos, ayuda a los equipos a comprender qué versiones se lanzan y en qué entornos se implementaron. A nivel de automatización de la compilación y despliegue, incorpora 450 plantillas,

contiene variables para que se pueda usar el mismo proceso en entornos de desarrollo, prueba y producción.

La última gran capacidad de Octopus Deploy es la **automatización de *runbooks***, de forma que es posible automatizar tareas operativas como el mantenimiento de rutina y la recuperación de incidentes de emergencia. Estos *runbooks* incluyen todos los permisos necesarios para que cualquier miembro del equipo pueda obtener permiso para ejecutar el *runbook*. En general, Octopus Deploy es una de las herramientas sólidas de implementación de software que existen, con un sólido conjunto de características y soporte para entornos modernos y heredados. Al igual que con todas las herramientas, hay algunas características importantes que Octopus Deploy no cumple. En particular, faltan por completo la verificación de implementación automatizada y la reversión de implementación automatizada (Octopus Deploy, s. f.).

Progress Chef Automate

Es una herramienta de gestión de configuración escrita en Ruby y Erlang. Utiliza un lenguaje específico de dominio de Ruby puro para escribir recetas de configuración del sistema. Chef se utiliza para agilizar la tarea de configurar y mantener los servidores de una empresa y puede integrarse con plataformas basadas en la nube como Amazon EC2, Google Cloud Platform, Oracle Cloud, OpenStack, IBM Cloud, Microsoft Azure y Rackspace para **aprovisionar y configurar automáticamente** máquinas nuevas. Permite automatizar (Chef, 2022):

- ▶ Aplicaciones: mediante prácticas de entrega continua e implementándolas en entornos nativos de la nube.
- ▶ Seguridad: se detectan problemas de seguridad antes de que entren en producción para reducir el riesgo, aumentar la velocidad y mejorar la eficiencia.

- Infraestructura: la gestión de configuración automatizada permite configuraciones coherentes a escala, lo que garantiza que la política de configuración sea flexible, versionable, comprobable y legible por humanos.

Puppet

Es una herramienta de gestión de sistemas DevOps de código abierto que facilita el **despliegue, configuración y administración** de servidores. Se utiliza para centralizar y automatizar el procedimiento de gestión de la configuración. Esta herramienta está desarrollada utilizando Ruby DSL. Está desarrollado por Puppet Labs y está disponible tanto para versiones de código abierto como para empresas. Se utiliza particularmente para la automatización de la entrega y gestión de infraestructura híbrida. Es compatible con muchas plataformas como Microsoft Windows, Debian/Ubuntu, Red Hat/CentOS/Fedora, macOS X, etc. Utiliza el paradigma cliente-servidor, en el que un sistema de cualquier clúster funciona como servidor, denominado Puppet Master, y el otro funciona como cliente en nodos denominados esclavos (Puppet, s. f.).

Red Hat Ansible Automation Platform

Es una herramienta de despliegue automatizado de aplicaciones, administración de configuración y aprovisionamiento de software de código abierto que habilita la infraestructura como código. Incluye su **propio lenguaje declarativo** para describir la configuración del sistema. Tiene una solución adquirida por Red Hat en 2015, no tiene agente, se conecta temporalmente de forma remota a través de SSH o Windows Remote Management (lo que permite la ejecución remota de PowerShell) para realizar sus tareas. Una de sus ventajas es que permite aprovisionar la infraestructura en la nube, *hosts* virtualizados, dispositivos de red y servidores completos, como primer paso para automatizar el despliegue de las aplicaciones (Red Hat Ansible, s. f.).

10.5. Plataformas para arquitecturas basadas en eventos

Las principales plataformas para dar soporte a arquitecturas basadas en eventos **facilitan la comunicación** entre productores y consumidores de eventos y son conocidas como mediadores de mensajes o eventos. Bajo este término generalista, existen diversas interpretaciones y denominaciones como:

- ▶ **Middleware orientado a mensajes** (*message oriented middleware*, MOM): bibliotecas de software para varios lenguajes de programación con un intermediario (o no) para comunicar mensajes entre aplicaciones. Los mensajes son objetos estructurados o cadenas de texto o datos binarios. Por lo general, tiene confiabilidad adicional sobre TCP/IP o UDP.
- ▶ **Java messaging service** (JMS): es la definición de una API común para MOM, motivado por la ausencia de un estándar para interaccionar con diversas implementaciones de sistemas MOM. Si nuestro sistema está implementado contra JMS, podemos simplemente cambiar las bibliotecas y la misma aplicación que solía funcionar con un MOM (por ejemplo, TIBCO EMS) funcionará con otro (por ejemplo, ActiveMQ)
- ▶ **Enterprise service bus** (ESB): un ESB utiliza un MOM para integrar aplicaciones, bases de datos, intermediarios, etc. Es un MOM con una estructura de datos agregada y administración de definición de estructura. Al conectar un nuevo componente a un ESB, podemos esperar más compatibilidad lista para usar que cuando lo conectamos a un MOM. En un ESB hay estándares más altos sobre lo que debe hacer un componente para conectarse y también incorpora reglas o patrones de transformación entre los mensajes para dar soporte a los patrones de integración de aplicaciones empresariales (*enterprise integration patterns*, EIP) (Hohpe y Woolf, 2003).

Algunas de las plataformas y herramientas que dan soporte a la mensajería de eventos en EDA son:



Tabla 2. Plataformas para arquitecturas basadas en eventos. Fuente: elaboración propia.

Apache ActiveMQ

Es el intermediario de mensajes basado en Java, de código abierto y multiprotocolo más popular. Es compatible con los protocolos estándar de la industria para que los usuarios obtengan los beneficios de las opciones del cliente en una amplia gama de idiomas y plataformas. Es posible conectarse desde clientes escritos en JavaScript, C, C++, Python, .NET y más. Permite integrar sus aplicaciones multiplataforma utilizando el omnipresente protocolo AMQP (*advanced message queuing*), intercambiar mensajes entre aplicaciones web usando STOMP sobre *websockets*, y administrar dispositivos IoT usando MQTT (Christudas, 2019).

Apache Kafka

Es una plataforma distribuida de gestión de colas de eventos, que consta de servidores y clientes que se comunican a través de un **protocolo de red TCP** de alto rendimiento y que se puede implementar en hardware básico, máquinas virtuales y contenedores en entornos locales y en la nube. A nivel de servidores, se ejecuta como un clúster de uno o más servidores que pueden abarcar varios centros de datos o regiones de la nube. Algunos de estos servidores forman la capa de almacenamiento, llamados intermediarios. Otros servidores ejecutan Kafka Connect para importar y exportar datos continuamente como flujos de eventos para integrar Kafka con sus sistemas existentes, como bases de datos relacionales y otros clústeres de Kafka.

Para permitirle implementar casos de uso de misión crítica, un clúster de Kafka es **altamente escalable y tolerante a fallos**. A nivel de clientes, permiten desarrollar aplicaciones distribuidas y microservicios que leen, escriben y procesan flujos de eventos en paralelo, a escala y con tolerancia a fallos, incluso en el caso de problemas de red o fallas de máquinas. Los clientes están disponibles para *Java* y *Scala*, incluida la biblioteca Kafka Streams de nivel superior, para Go, Python, C/C++ y muchos otros lenguajes de programación, así como API REST (Scott, 2022).

Apache ServiceMix

Es un contenedor de integración flexible y de código abierto que unifica las características y la funcionalidad de Apache ActiveMQ, Camel, CXF y Karaf en una **poderosa plataforma de tiempo de ejecución** que podemos usar para crear nuestras propias soluciones de integración. Proporciona un ESB completo y listo para la empresa impulsado exclusivamente por OSGi. Bajo licencia Apache License v2, sus características principales son (ServiceMix, 2022):

- ▶ Mensajería confiable con Apache ActiveMQ.
- ▶ Mensajería, enrutamiento y patrones de integración empresarial con Apache Camel.
- ▶ Servicios web WS-* y RESTful con Apache CXF.
- ▶ Tiempo de ejecución del servidor basado en OSGi con la tecnología de Apache Karaf.

A través de características instalables adicionales, ServiceMix también admite:

- ▶ Motor BPM a través de Activiti.
- ▶ Soporte completo de JPA a través de Apache OpenJPA.
- ▶ Gestión de transacciones XA a través de JTA a través de Apache Aries.

Microsoft Biztalk

Es el *middleware* ESB de Microsoft. Aprovecha .NET y permite a los desarrolladores escribir sus piezas de integración en Visual Studio. Se compone de:

- ▶ MSMQ (*Microsoft message queuing*): implementación de cola de mensajes.
- ▶ Enrutamiento: las especificaciones de enrutamiento/mensaje se implementan a través de XML, pero generalmente este XML se genera utilizando herramientas gráficas.
- ▶ Adaptadores: BizTalk tiene una variedad de adaptadores integrados. Como era de esperar, tiene un excelente soporte de adaptador para las tecnologías de *Microsoft*, como los diversos protocolos WCF.
- ▶ BizTalk Server: BizTalk requiere IIS (servicios de información de Internet) para varias funciones, como HTTP, SOAP, SSL y más. Por lo general, esto se implementa en *Windows Server*.
- ▶ Consola de administración de BizTalk Server: esta es una MMC (*Microsoft management console*) que permite una amplia configuración y administración del servidor.

MuleESB

Es un bus de servicio empresarial basado en Java que ocupa poco espacio. Es de código abierto y, como la mayoría de los ESB, permite la **integración de sistemas** a través de JMS, servicios web, HTTP, JDBC y más. Se compone de (MuleSoft, s. f.):

- ▶ RabbitMQ Java Client para soporta AMQP.
- ▶ Enrutadores: MuleSoft utiliza enrutadores para dividir, combinar, reordenar, evaluar y transmitir mensajes.

- ▶ Anypoint Connectors: conectores de protocolo, base de datos, transporte y base de datos prediseñados. También puedes construir el tuyo propio si es necesario.
- ▶ Mule Runtime Engine: el corazón de la plataforma MuleSoft Anypoint. Implementable en la nube o en las instalaciones.
- ▶ Mule Runtime Manager: permite la implementación, el monitoreo y la resolución de problemas de las instancias de Mule.

Oracle Service Bus

Transforma arquitecturas **complejas y frágiles** en redes de integración ágiles al conectar, virtualizar y administrar interacciones entre servicios y aplicaciones. Ofrece integración basada en estándares y de bajo costo para entornos SOA cruciales donde el máximo rendimiento, la escalabilidad y la fiabilidad son requisitos esenciales. Ya sea en entornos locales, en la nube, en dispositivos móviles y de cara al futuro, permite a las empresas aprovechar sus inversiones existentes de formas totalmente nuevas (Blokdyk, 2018).

RabbitMQ

Es el intermediario de mensajes de código abierto más utilizado. Admite múltiples protocolos de mensajería, colas de mensajes, acuse de recibo de entrega, enrutamiento flexible a colas o tipo de intercambio múltiple. Soporta el desarrollo de intercambio de mensajes con **una gran variedad de lenguajes de programación** como Java, .NET, PHP, Python, JavaScript, Ruby, Go y muchos otros. Soporta el despliegue con herramientas como BOSH, Chef, Docker y Puppet, así como la configuración en clústeres para alta disponibilidad y rendimiento y federarse en varias zonas y regiones de disponibilidad. Dispone de un API HTTP, una utilidad de línea de comandos y un entorno gráfico para gestionar la plataforma (RabbitMQ, 2022).

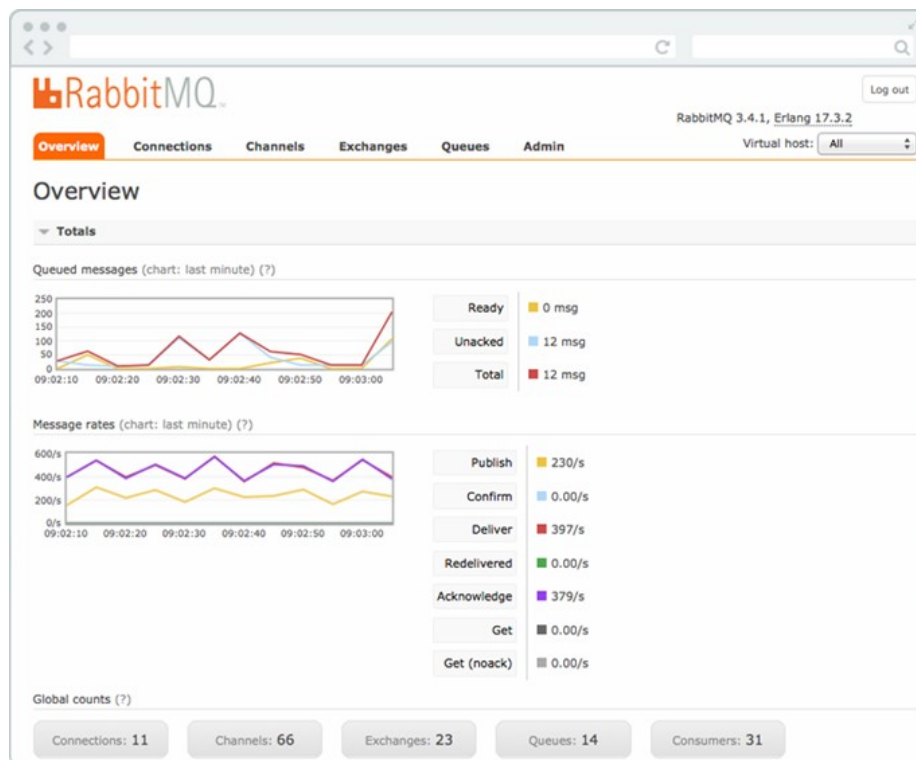
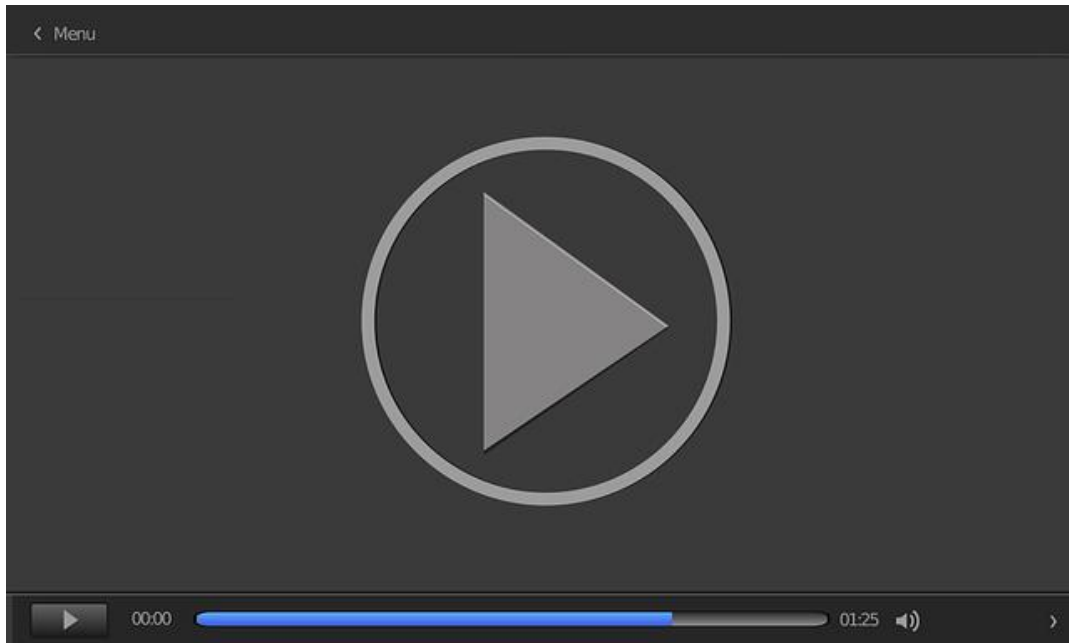


Figura 5. Vista de la consola de administración de RabbitMQ. Fuente: CloudAMQP, 2019.

En este video, *RabbitMQ: arquitecturas distribuidas dirigidas por eventos*, aprenderemos a configurar un proyecto Spring para publicar y recibir mensajes de colas sobre RabbitMQ, configurado sobre Docker.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=7985fd07-ae66-437e-9437-aec300cd9966>

Red Hat Fuse

- ▶ Es más que un ESB, ya que se configura como una plataforma de **integración ligera** de código abierto, basada en soluciones como Apache ServiceMix, que está disponible en las instalaciones o en la nube. Integra (Red Hat, s. f.).
- ▶ Apache ActiveMQ: un *broker* de mensajes rápido y de código abierto que admite JMS y clientes escritos en otros lenguajes como C y Python.
- ▶ Apache Camel: un marco de código abierto que proporciona implementaciones de EIPS (*enterprise integration patterns*).

- ▶ Apache CFX: un marco de servicios web de código abierto que permite la comunicación utilizando varios estándares *como* JAX-WS y JAX-RS, HTTP y FTP, así como diferentes formatos como JSON, XML, CSV, etc.
- ▶ Apache Karaf: un contenedor de tiempo de ejecución OSGi para implementar aplicaciones.
- ▶ Fabric8: una herramienta de orquestación para grandes implementaciones de middleware

10.6. Plataformas de monitorización de sistemas distribuidos

Una vez desplegado un sistema distribuido, el equipo de operaciones de una organización será el **responsable** de hacer la supervisión y monitorización de la infraestructura y los diversos bloques funcionales.

Algunas de las plataformas y herramientas que dan soporte a la supervisión de sistemas distribuidos son:



Tabla 3. Plataformas de monitorización de sistemas distribuidos. Fuente: elaboración propia.

Cacti

Es una solución de monitorización de red de código abierto basada en el sistema de clasificación y trazado de datos con la herramienta RRDTool. La herramienta utiliza la funcionalidad de **recopilación de datos y el sondeo de red** para juntar información sobre varios dispositivos en redes de cualquier alcance. Esto comprende la capacidad de crear *scripts* personalizados para la recopilación de datos junto con la función de sondeo SNMP. Luego, muestra esta información en gráficos fáciles de comprender que se pueden organizar en cualquier jerarquía según la conveniencia de su negocio. Incluye (Cacti, 2022):

- ▶ Un marco de recopilación de datos totalmente distribuido y tolerante a fallos.
- ▶ Funciones avanzadas de automatización basadas en plantillas para dispositivos, gráficos y árboles.
- ▶ Múltiples métodos de adquisición de datos
- ▶ La capacidad de extenderse a través de complementos, funciones de administración de usuarios, grupos y dominios basadas en roles.
- ▶ Un motor de temas y compatibilidad con varios idiomas desde el primer momento.

Device Description	Hostname	ID	Graphs	Data Sources	Status	In State	Uptime	Poll Time	Current (ms)	Average (ms)	Availability	Created
Cacti Server	localhost	1	4	5	Up	N/A	N/A	0.1	0	0	100 %	2020-09-06 21:43:06
Central NAS	192.168.11.105	56	12	19	Up	120	42	0.26	0.35	1.15	99.36 %	2020-09-06 21:43:06
HP Printer	192.168.11.174	55	22	22	Up	137	54	0.65	1.04	1.8	99.81 %	2020-09-06 21:43:06
vhost01	192.168.11.201	46	12	19	Up	120	4	0.38	1.45	1.61	99.99 %	2020-09-06 21:43:06
vhost02	192.168.11.202	45	12	19	Up	120	4	0.34	0.56	0.94	99.99 %	2020-09-06 21:43:06
vhost03	192.168.11.203	44	12	19	Up	120	4	0.24	0.9	2.09	99.98 %	2020-09-06 21:43:06
vhost04	192.168.11.204	43	12	19	Up	120	4	0.26	1.01	0.76	100 %	2020-09-06 21:43:06
vhost05	192.168.11.205	42	12	19	Up	120	4	0.33	0.83	1.25	99.99 %	2020-09-06 21:43:06
vhost06	192.168.11.206	41	12	19	Up	120	4	0.39	0.74	0.79	100 %	2020-09-06 21:43:06
vhost07	192.168.11.207	40	12	19	Up	267	4	0.4	0.52	1.06	98.93 %	2020-09-06 21:43:06
vhost08	192.168.11.208	39	12	19	Up	120	4	0.19	0.89	1.24	99.99 %	2020-09-06 21:43:06
vhost09	192.168.11.209	38	12	19	Up	267	4	0.15	0.7	1.07	98.93 %	2020-09-06 21:43:06
vhost10	192.168.11.210	37	12	19	Up	120	4	0.22	0.77	0.77	100 %	2020-09-06 21:43:06
vhost11	192.168.11.211	36	12	19	Up	120	4	0.09	2.61	1.01	99.98 %	2020-09-06 21:43:06
vhost12	192.168.11.212	35	12	19	Up	120	4	0.32	1.14	1.09	99.99 %	2020-09-06 21:43:06
vhost13	192.168.11.213	34	12	19	Up	120	4	0.25	2.63	1.05	99.98 %	2020-09-06 21:43:06
vhost14	192.168.11.214	33	12	19	Up	267	4	0.26	3.99	1.02	98.93 %	2020-09-06 21:43:06
vhost15	192.168.11.215	32	12	19	Up	120	4	0.31	1.11	0.93	99.99 %	2020-09-06 21:43:06

Figura 6. Vista del supervisor de Cacti. Fuente: Cacti, 2022.

Inciga

Es una herramienta de supervisión de red de código abierto que calcula la disponibilidad y el rendimiento de la red. A través de una interfaz web, podemos observar aplicaciones y *hosts* alrededor de la infraestructura de red completa. La herramienta es **escalable y fácilmente configurable** para funcionar con cada tipo de dispositivo. Existen algunos módulos adicionales para capacidades de monitorización muy específicas, como el del entorno de nube vSphere de VMWare y el modelado de procesos comerciales (Inciga, s. f.).

LibreNMS

Es un sistema de monitorización de red de código abierto que utiliza múltiples protocolos de red para observar cada dispositivo en su red. Su API permite **recuperar, administrar y trazar** los datos que recopila y facilita el escalado horizontal para aumentar sus capacidades de monitoreo junto con su red. La herramienta presenta un sistema de alerta flexible que está hecho a medida para comunicarse con usted por el método que mejor se adapte a su empresa (LibreNMS, s. f.).

Nagios

Es una herramienta de monitorización de código abierto que lleva más de veinte años en disponible para la industria, por lo que existen muchos **complementos disponibles**. Permite monitorizar sistemas Windows, Linux, servidores, aplicaciones, *middlewares*, así como analizar *logs* e integrarse con aplicaciones de terceros para automatizar las gestiones a realizar en el caso de alertas o incidencias (Nagios, s. f. a).

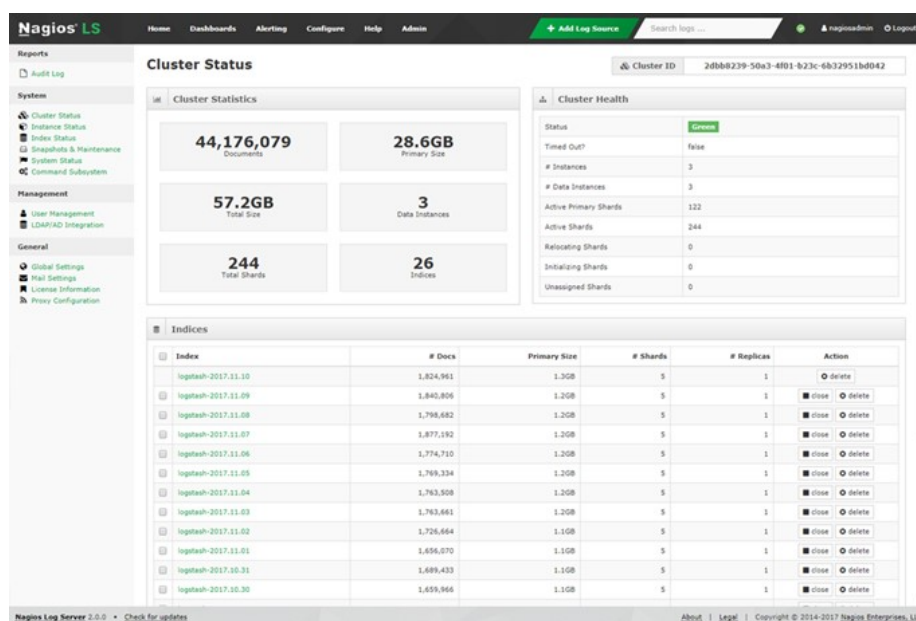


Figura 7. Vista del analizador de Logs de Nagios. Fuente: Nagios, s. f. b.

Pandora FMS

Es una **herramienta de supervisión** de código abierto que ayuda a las empresas a observar toda su infraestructura de TI. No solo presenta capacidades de monitoreo de red, sino también servidores Unix y Windows e interfaces virtuales. Para redes, incluye características de primer nivel como compatibilidad con SNMP, sondeo ICMP, monitorización de la latencia de red y sobrecarga del sistema. Los agentes también se pueden instalar en los dispositivos para observar aspectos como la temperatura y el sobrecalentamiento del dispositivo, así como los sucesos de los archivos de registro (Pandora FMS, s. f.).

Prometheus

Es una solución de monitoreo de código abierto que se centra en la recopilación y el análisis de datos basados en datos de series temporales. Permite a los usuarios configurar **capacidades de monitorización** al utilizar el conjunto de herramientas incorporado. Es ideal para entornos en contenedores como Kubernetes y es considerada como la mejor herramienta de monitorización de servidores de código abierto. Permite unificar métricas de servidores, contenedores y varios dispositivos mediante *pings* SNMP e inspeccionar el uso del ancho de banda de la red desde el punto de vista del dispositivo. Dispone de un administrador de alertas configurable en base a reglas. Se integra con Grafana para disponer de paneles y cuadros de mando de visualización de métricas (Prometheus, s. f. a).



Figura 8. Vista de métricas obtenidas con Prometheus sobre Grafana. Fuente: Prometheus, s. f. b.

Riemann

Es una herramienta de monitorización de código abierto ideal para sistemas distribuidos. Es un sistema de **procesamiento uniforme de baja latencia** capaz de recopilar métricas de una variedad de sistemas distribuidos. Está diseñado para manejar millones de pares por segundo con baja latencia, por lo que es una plataforma ideal para supervisar sistemas escalables altamente distribuidos (Riemann, s. f. a).



Figura 9. Vista de métricas obtenidas con Riemann. Fuente: Riemann, s. f. b.

Sensu

Es una herramienta de monitorización de pila completa. A través de una única plataforma, se puede realizar la supervisión de servicios, aplicaciones, servidores y reportes de KPI (indicador clave de rendimiento) de negocio. Es una solución basada en agentes que instala en la infraestructura de su organización, con un *back-end* que brinda una canalización flexible y automatizada para filtrar, transformar y procesar

alertas y métricas (Sensu, s. f.).

Zabbix

Es un software de supervisión de código abierto con una interfaz fácil de usar que brinda soluciones de clase empresarial a grandes organizaciones. Es un sistema centralizado que almacena los datos en una **base de datos relacional** para un procesamiento eficiente. Entre sus funcionalidades están (Zabbix, 2022).:

- ▶ Supervisión de red.
- ▶ Monitorización del servidor.
- ▶ Supervisión en la nube.
- ▶ Monitorización de aplicaciones.
- ▶ Supervisión del servicio.

Monitorizar diversos entornos

Debido a que es posible que sea necesario monitorizar infraestructuras o aplicaciones heterogéneas, existe una solución global que puede ser aplicada para monitorizar diversos entornos y supervisar múltiples tipos de datos de aplicaciones a través de sus *logs* o registro. En los últimos años ha adquirido una gran popularidad **ELK**, ahora conocido como **Elastic Stack**, que representa la unión de tres proyectos *open source*:

Elasticsearch

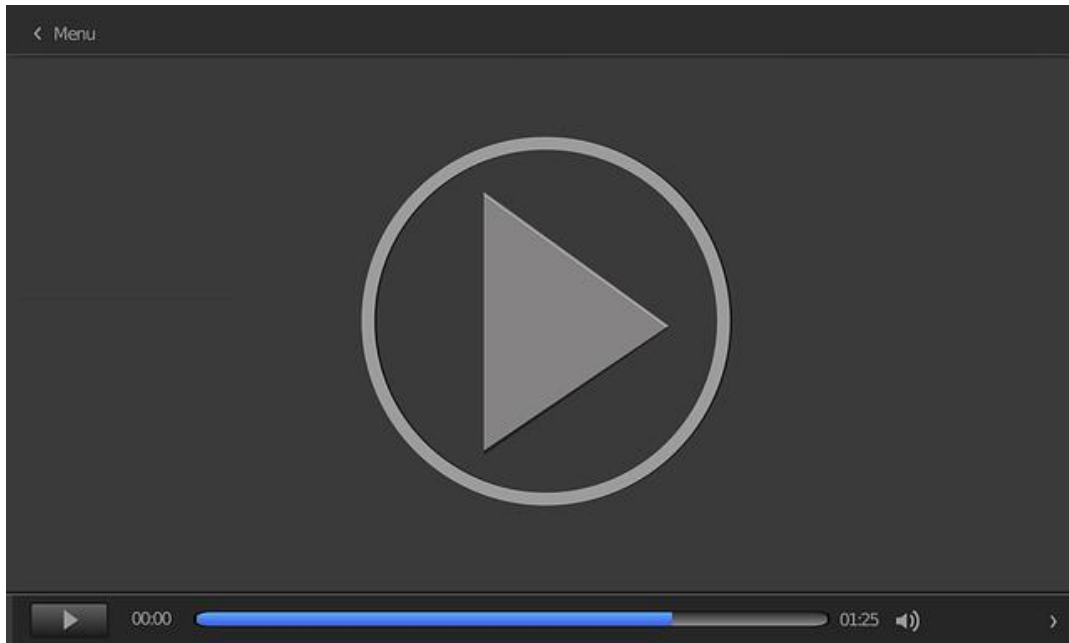
Es un motor de analítica y análisis distribuido, gratuito y abierto para todos los tipos de datos, incluidos textuales, numéricos, geoespaciales, estructurados y no estructurados. Desarrollado a partir de Apache Lucene en 2010, es conocido por sus API REST simples, naturaleza distribuida, velocidad y escalabilidad. Es el **componente principal** del Elastic Stack, un conjunto de herramientas gratuitas y

abiertas para la ingesta, el enriquecimiento, el almacenamiento, el análisis y la visualización de datos (Elastic, s. f. a).

Logstash

Es un *pipeline* de procesamiento de datos del lado del servidor que ingesta datos de una multitud de fuentes simultáneamente, los transforma y luego los envía a un repositorio como Elasticsearch. Básicamente, **ingesta, transforma y envía** de forma dinámica los datos independientemente de su formato o complejidad. Deriva estructura a partir de datos no estructurados con Grok, descifra las coordenadas geográficas de las direcciones IP, anonimiza o excluye los campos sensibles y facilita el procesamiento general (Elastic, s. f. b).

En este video, *Monitorizando sistemas distribuidos con ELK*, aprenderemos a crear un entorno con Beats, Logstash, Elasticsearch y Kibana sobre Docker, para registrar y visualizar los Logs de una aplicación Spring.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=a1efa2cb-a221-46a4-8a61-aec300cd994c>

Kibana

Es una interfaz de usuario gratuita y abierta que te permite visualizar los datos de Elasticsearch y navegar en el Elastic Stack. **Facilita la interacción** con los datos recogidos, desde rastrear la carga de búsqueda hasta comprender la forma en que las solicitudes fluyen por las aplicaciones (Elastic, s. f. c). Dispone de elementos visuales como gráficos, series temporales, mapas, gráficos geoespaciales, tablas de datos, *canvas* y un conjunto de *plugins* para gráficos 3D y más.

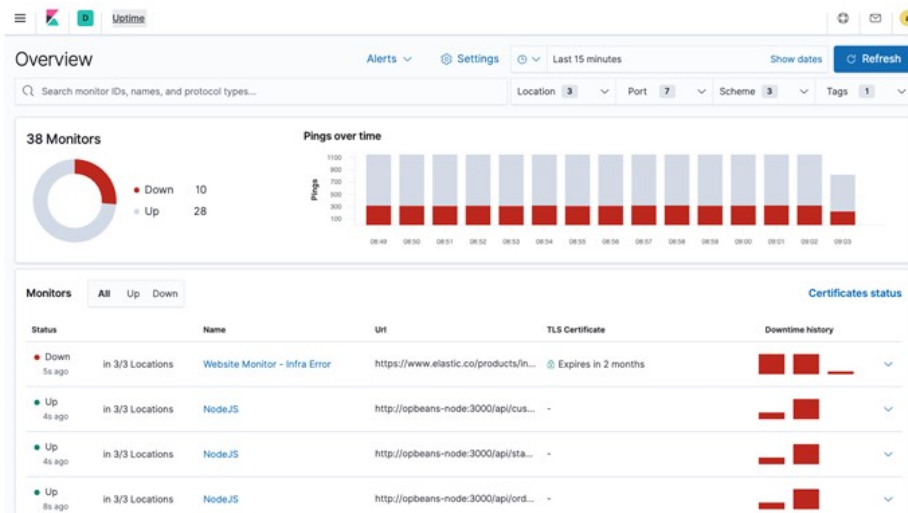


Figura 10. Vista Kibana de una monitorización con Elastic Stack. Fuente: Elastic, s. f. d.

10.7. Referencias bibliográficas

Apache Mesos. (s. f.). *Mesos Architecture*.
<https://mesos.apache.org/documentation/latest/architecture/>

Argo CD. (s. f.) *Overview*. <https://argo-cd.readthedocs.io/en/stable/>

Atlassian. (s. f.). *Compila, prueba e implementa con confianza*.
<https://www.atlassian.com/es/software/bamboo>

AWS (s. f.). *AWS CodeDeploy*. <https://aws.amazon.com/es/codedeploy/>

Blokdyk, G. (2018). *Oracle Service Bus* (3ª ed.). Createspace Independent Publishing Platform.

Cacti. (2022, mayo 18). *About Cacti*. <https://www.cacti.net/>

Christudas, B. (2019). *ActiveMQ*. In *Practical Microservices Architectural Patterns* (pp. 861–867). Apress.

CloudAMPQ. (2019, septiembre 3). *Part 3: The RabbitMQ Management Interface*.
https://www.cloudamqp.com/blog/part3-rabbitmq-for-beginners_the-management-interface.html

Courtemanche, M., Mell, E. y Gillis, A. S. (2021, diciembre). *What is DevOps? The ultimate guide*. TechTarget.
<https://www.techtarget.com/searchitoperations/definition/DevOps>

Docker. (s. f.). *Developers Love Docker. Businesses Trust It*.
<https://www.docker.com/>

Docker Docs. (s. f.). *Docker overview*. <https://docs.docker.com/get-started/overview/>

Elastic. (s. f. a). *¿Qué es Elasticsearch?* <https://www.elastic.co/es/what-is/elasticsearch>

Elastic. (s. f. b). Logstash. <https://www.elastic.co/es/logstash/>

Elastic. (s. f. c). Kibana. <https://www.elastic.co/es/kibana/>

Elastic. (s. f. d). Heartbeat. <https://www.elastic.co/es/beats/heartbeat>

GitLab. (s. f.). *The one DevOps platform.* <https://about.gitlab.com/>

GoCD. (s. f.). Free & Open Source CI/CD Server. <https://www.gocd.org/>

Google Cloud (s. f.). *Documentación de Google Cloud Deployment Manager.* <https://cloud.google.com/deployment-manager/docs>

Harness. (s. f.). *The Modern Software Delivery Platform.* <https://harness.io/>

Heroku. (s. f.). *Continuous Delivery on Heroku.* <https://www.heroku.com/continuous-delivery>

Hindman, B., Konwinski, A., Zaharia, M., Ghodsi, A., Joseph, A. D., Katz, R. y Stoica, I. (2011, marzo). *Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center* [simposio]. 8th USENIX Symposium on Networked Systems Design and Implementation, Boston, Estados Unidos. <https://www.usenix.org/conference/nsdi11/mesos-platform-fine-grained-resource-sharing-data-center>

Hohpe, G. y Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions.* Addison-Wesley Educational.

Icinga. (s. f.). *Monitor Your Entire Infrastructure with.* <https://icinga.com/>

Jenkins. (s. f.). *Jenkins. Build great things at any scale.* <https://www.jenkins.io/>

Kubernetes. (2022, abril 4). *What is Kubernetes?*

<https://kubernetes.io/docs/concepts/overview/what-is-kubernetes/>

Kubernetes. (s. f.) *Production-Grade Container Orchestration*. <https://kubernetes.io/>

LibreNMS. (s. f.). *Welcome to LibreNMS, a fully featured network monitoring system that provides a wealth of features and device support*. <https://www.librenms.org/>

Microsoft. (2022, mayo 26). *What is Azure Automation?*

<https://docs.microsoft.com/en-us/azure/automation/overview>

MuleSoft. (s. f.). *ESB*. <https://www.mulesoft.com/platform/soa/mule-esb-open-source-esb>

Nagios. (s. f. a). *What can Nagios help you do?* <https://www.nagios.org/>

Nagios. (s. f. b). *Centralized Log Management, Monitoring and Analysis Software*. <https://www.nagios.com/products/nagios-log-server>

Octopus Deploy. (s. f.). *Complex deployments made easy*. <https://octopus.com/>

Pandora FMS. (s. f.). *Aloha!, welcome to Pandora FMS Community*. <http://pandorafms.org/>

Página de [Cloud Foundry BOSH](#).

Página de [ServiceMix](#).

Progress Chef. (s. f.). *Chef Automate*. <https://www.chef.io/products/chef-automate>

Prometheus. (s. f. a). *From metrics to insight*. <https://prometheus.io/>

Prometheus. (s. f. b). *Grafana support for Prometheus*. <https://prometheus.io/docs/visualization/grafana/>

Puppet. (s. f.). *Find. Fix. Prevent*. <https://puppet.com/>

RabbitMQ. (s. f.). Quorum queues. <https://www.rabbitmq.com/>

Red Hat. (s. f.). Red Hat Fuse. <https://www.redhat.com/es/technologies/jboss-middleware/fuse>

Red Hat. (2018, enero 31). *¿Qué son la integración/distribución continuas (CI/CD)?* <https://www.redhat.com/es/topics/devops/what-is-ci-cd>

Red Hat Ansible. (s. f.). Red Hat Ansible Platform. <https://www.ansible.com/products/automation-platform>

Red Hat OpenShift. (s. f.). *Overview*. <https://docs.openshift.com/container-platform/3.11/architecture/index.html>

Riemann. (s. f. a). *Riemann monitors distributed systems*. <http://riemann.io/>

Riemann. (s. f. b). *You're clear for launch*. <http://riemann.io/dashboard.html>

Scott, D. (2022). *Kafka in Action*. Manning Publications.

Sensu. (s. f.). *The Observability Pipeline that delivers monitoring as code on any cloud*. <https://sensu.io/>

Zabbix. (s. f.). *Zabbix 6.0 LTS*. <https://www.zabbix.com/>

Hands-On DevOps for Architects: Implementing continuous delivery through automation.

Aiello, B. (2021). *Hands-On DevOps for Architects: Implementing continuous delivery through automation*. Packt Publishing.

En este libro, se aprende a liderar la transformación de DevOps a través de una arquitectura empresarial eficaz. A su vez, se aprende cómo implementar prácticas efectivas, como integración, entrega e implementación continuas, como parte de la creación de la canalización de entrega continua.

DevOps Adoption Strategies: Principles, Processes, Tools, and Trends: Embracing DevOps through effective culture, people, and processes.

Coupland, M. y Ferreira, D. (2021). *DevOps Adoption Strategies: Principles, Processes, Tools, and Trends: Embracing DevOps through effective culture, people, and processes*. Packt Publishing.

Este libro ayuda a comprender los conceptos básicos necesarios para comenzar con DevOps y, a su vez, cómo se conectan los aspectos de la cultura, las personas y los procesos, y que sin ninguno de estos elementos es poco probable que DevOps tenga éxito.

1. Selecciona cuál de los siguientes elementos no forma parte de una arquitectura basada en eventos:
 - A. El productor de eventos.
 - B. El consumidor de eventos.
 - C. La plataforma de monitorización y despliegue de eventos.
 - D. La plataforma de procesamiento de eventos.

2. Selecciona cuál de los siguientes objetivos no es parte de la integración continua:
 - A. Facilitar la integración de código realizado por varias personas a la vez.
 - B. Asegurar que los desarrollos en paralelo se integran y no dan conflictos.
 - C. Facilitar la integración de los cambios de forma frecuente y regular.
 - D. Automatizar las pruebas para garantizar que el paso a producción es correcto y eficiente.

3. Las dos plataformas más comunes para el despliegue en contenedores son Docker y Kubernetes, y sus diferencias son:
 - A. Que una es *open source* y la otra comercial.
 - B. Que una es para contenedores sobre Linux y la otra para cualquier sistema operativo.
 - C. Que Docker es para contenedores en un solo nodo y Kubernetes está destinado a ejecutarlas en un clúster.
 - D. Que solo una de ellas soporta CI/CD.

4. El término DevOps hace referencia a:

- A. Un desarrollo ágil de software directamente en la nube.
- B. Una fase de *scrum* en la que el equipo de *testing* trabaja con el equipo de operaciones y despliegues.
- C. Una forma de representar un trabajo en equipo entre la parte de desarrollo y la de operaciones.
- D. Una disciplina de Ingeniería del software que aplica al mantenimiento de sistemas.

5. Selecciona el modelo de despliegue con su plataforma:

Despliegue en contenedores (un nodo)	1	A	Bare metal
Despliegue en máquinas virtuales	2	B	Docker
Despliegue tradicional	3	C	Red Hat OpenShift
Despliegue en contenedores (varios nodos)	4	D	VMWare

6. Bajo el término de integración y la distribución continuas (CI/CD) nos referimos a:

- A. Una forma de automatizar el testeo de software.
- B. Un método ágil para liberar nuevas versiones cada sprint de dos semanas.
- C. La combinación de metodologías ágiles y DevOps.
- D. Un método para distribuir las aplicaciones a los clientes con frecuencia mediante el uso de la automatización en las etapas del desarrollo de aplicaciones.

7. ELK, ahora conocido como Elastic Stack, representa la unión de tres proyectos *open source*. Selecciona cuál de los siguientes no forma parte de ELK:
- A. ElasticTash.
 - B. Logstash.
 - C. Elasticsearch.
 - D. Kibana.
8. Indica cuál es el intermediario de mensajes de código abierto más utilizado:
- A. Microsoft Biztalk.
 - B. RabbitMQ.
 - C. Apache Kafka.
 - D. Red Hat Fuse.
9. Las plataformas que dan soporte a los patrones de integración de aplicaciones empresariales (*enterprise integration patterns*, EIP) son conocidas como:
- A. Plataformas de gestión de eventos.
 - B. Plataformas middleware orientado a mensajes (*message oriented middleware*, MOM).
 - C. Plataformas *enterprise service bus* (ESB).
 - D. Plataformas *Java messaging service* (JMS).
10. ¿Qué beneficio no está relacionado con la monitorización o supervisión de sistemas?
- A. Detectar cortes de servicio de forma temprana.
 - B. Habilitar la funcionalidad asíncrona.
 - C. Medir la escalabilidad del sistema.
 - D. Obtener datos de uso del sistema en tiempo real.